



OpenShift Container Platform 4.22

Scalability and performance

Scaling your OpenShift Container Platform cluster and tuning performance in production environments

OpenShift Container Platform 4.22 Scalability and performance

Scaling your OpenShift Container Platform cluster and tuning performance in production environments

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

This document provides instructions for scaling your cluster and optimizing the performance of your OpenShift Container Platform environment.

Table of Contents

CHAPTER 1. OPENSIFT CONTAINER PLATFORM SCALABILITY AND PERFORMANCE OVERVIEW	11
1.1. RECOMMENDED PERFORMANCE AND SCALABILITY PRACTICES	11
1.2. TELCO REFERENCE DESIGN SPECIFICATIONS	11
1.3. PLANNING, OPTIMIZATION, AND MEASUREMENT	11
1.4. ADDITIONAL RESOURCES	12
CHAPTER 2. RECOMMENDED PERFORMANCE AND SCALABILITY PRACTICES	13
2.1. RECOMMENDED CONTROL PLANE PRACTICES	13
2.1.1. Recommended practices for scaling the cluster	13
2.1.2. Control plane node sizing	13
2.2. SELECTING A LARGER AWS INSTANCE TYPE FOR CONTROL PLANE MACHINES	16
2.2.1. Additional resources	16
2.2.2. Changing the Amazon Web Services instance type by using a control plane machine set	16
2.2.3. Changing the Amazon Web Services instance type by using the AWS console	17
2.3. RECOMMENDED INFRASTRUCTURE PRACTICES	18
2.3.1. Infrastructure node sizing	18
2.3.2. Scaling the Cluster Monitoring Operator	18
2.3.2.1. Prometheus database storage requirements	19
2.3.2.2. Configuring cluster monitoring	20
2.3.3. Additional resources	21
CHAPTER 3. TELCO CORE REFERENCE DESIGN SPECIFICATIONS	22
3.1. TELCO CORE RDS USE MODEL OVERVIEW	22
3.2. ABOUT THE TELCO CORE CLUSTER USE MODEL	22
3.3. REFERENCE DESIGN SCOPE	23
3.4. DEVIATIONS FROM THE REFERENCE DESIGN	23
3.5. TELCO CORE COMMON BASELINE MODEL	24
3.6. DEPLOYMENT PLANNING	25
3.6.1. Worker Nodes and MachineConfigPools	25
3.6.2. Zones	26
3.7. TELCO CORE CLUSTER COMMON USE MODEL ENGINEERING CONSIDERATIONS	28
3.7.1. Application workloads	29
3.7.2. Signaling workloads	30
3.8. TELCO CORE RDS COMPONENTS	30
3.8.1. CPU partitioning and performance tuning	30
3.8.2. Workloads on schedulable control planes	32
3.8.3. Service Mesh	33
3.8.4. Networking	33
3.8.4.1. Cluster Network Operator	36
3.8.4.2. Load balancer	37
3.8.4.3. SR-IOV	38
3.8.4.4. NMState Operator	39
3.8.5. Logging	40
3.8.6. Power Management	40
3.8.7. Storage	41
3.8.7.1. OpenShift Data Foundation	42
3.8.7.2. Additional storage solutions	43
3.9. TELCO CORE DEPLOYMENT COMPONENTS	43
3.9.1. Red Hat Advanced Cluster Management	43
3.9.2. Topology Aware Lifecycle Manager	44
3.9.3. GitOps Operator and GitOps ZTP plugins	45

3.9.4. Agent-based Installer	46
3.10. MONITORING	46
3.11. SCHEDULING	47
3.12. NODE CONFIGURATION	48
3.12.1. Host firmware and boot loader configuration	49
3.12.2. Kubelet Settings	50
3.13. DISCONNECTED ENVIRONMENT	50
3.14. SECURITY	51
3.15. CERT-MANAGER OPERATOR	53
3.16. SCALABILITY	53
3.17. TELCO CORE REFERENCE CONFIGURATION CRS	54
3.17.1. Extracting the telco core reference design configuration CRs	54
3.17.2. Comparing a cluster with the telco core reference configuration	56
3.17.3. Node configuration reference CRs	59
3.17.4. Cluster infrastructure reference CRs	60
3.17.5. Resource tuning reference CRs	62
3.17.6. Networking reference CRs	62
3.17.7. Scheduling reference CRs	65
3.17.8. Storage reference CRs	66
3.17.9. Security reference CRs	67
3.18. TELCO CORE REFERENCE CONFIGURATION SOFTWARE SPECIFICATIONS	68
CHAPTER 4. TELCO RAN DU REFERENCE DESIGN SPECIFICATION	70
4.1. REFERENCE DESIGN SPECIFICATIONS FOR TELCO RAN DU 5G DEPLOYMENTS	70
4.1.1. Supported CPU architectures for RAN DU	71
4.2. REFERENCE DESIGN SCOPE	71
4.3. DEVIATIONS FROM THE REFERENCE DESIGN	72
4.4. ENGINEERING CONSIDERATIONS FOR THE RAN DU USE MODEL	72
4.5. TELCO RAN DU APPLICATION WORKLOADS	77
4.6. TELCO RAN DU REFERENCE DESIGN COMPONENTS	77
4.6.1. Host firmware tuning	78
4.6.2. Kubelet settings	79
4.6.3. CPU partitioning and performance tuning	80
4.6.4. PTP Operator	83
4.6.5. SR-IOV Operator	85
4.6.6. Logging	86
4.6.7. SRIOV-FEC Operator	86
4.6.8. Lifecycle Agent	87
4.6.9. Local Storage Operator	87
4.6.10. Logical Volume Manager Storage	88
4.6.11. Workload partitioning	89
4.6.12. Cluster tuning	89
4.6.13. Machine configuration	91
4.7. TELCO RAN DU DEPLOYMENT COMPONENTS	92
4.7.1. Red Hat Advanced Cluster Management	92
4.7.2. SiteConfig Operator	93
4.7.3. Topology Aware Lifecycle Manager	94
4.7.4. GitOps Operator and GitOps ZTP	95
4.7.5. Agent-based installer	96
4.8. TELCO RAN DU REFERENCE CONFIGURATION CRS	97
4.8.1. Cluster tuning reference CRs	97
4.8.2. Day 2 Operators reference CRs	99
4.8.3. Machine configuration reference CRs	107

4.9. COMPARING A CLUSTER WITH THE TELCO RAN DU REFERENCE CONFIGURATION	108
4.10. TELCO RAN DU 4.22 VALIDATED SOFTWARE COMPONENTS	111
CHAPTER 5. TELCO HUB REFERENCE DESIGN SPECIFICATION	113
5.1. REFERENCE DESIGN SCOPE	113
5.2. DEVIATIONS FROM THE REFERENCE DESIGN	113
5.3. HUB CLUSTER ARCHITECTURE OVERVIEW	114
5.4. TELCO MANAGEMENT HUB CLUSTER USE MODEL	115
5.5. HUB CLUSTER SCALING TARGET	116
5.6. HUB CLUSTER RESOURCE UTILIZATION	117
5.7. HUB CLUSTER TOPOLOGY	117
5.8. HUB CLUSTER NETWORKING	118
5.9. HUB CLUSTER MEMORY AND CPU REQUIREMENTS	119
5.10. HUB CLUSTER STORAGE REQUIREMENTS	120
5.10.1. Assisted Service	120
5.10.2. RHACM Observability	121
5.10.3. Storage considerations	122
5.11. GIT REPOSITORY	123
5.12. OPENSIFT CONTAINER PLATFORM INSTALLATION ON THE HUB CLUSTER	124
5.13. DAY 2 OPERATORS IN THE HUB CLUSTER	125
5.14. OBSERVABILITY	126
5.15. MANAGED CLUSTER LIFECYCLE MANAGEMENT	127
5.15.1. Managed cluster deployment	128
5.15.2. Managed cluster updates	129
5.15.3. Hub cluster disaster recovery	130
5.16. HUB CLUSTER COMPONENTS	130
5.16.1. Red Hat Advanced Cluster Management (RHACM)	130
5.16.2. Topology Aware Lifecycle Manager	132
5.16.3. GitOps Operator and GitOps ZTP	133
5.16.4. Local Storage Operator	134
5.16.5. Red Hat OpenShift Data Foundation	134
5.16.6. Logging	135
5.16.7. OpenShift API for Data Protection	135
5.16.8. cert-manager Operator	136
5.17. EXTRACTING THE TELCO HUB REFERENCE DESIGN CONFIGURATION CRS	136
5.18. COMPARING A CLUSTER WITH THE TELCO HUB REFERENCE CONFIGURATION	138
5.19. HUB CLUSTER REFERENCE CONFIGURATION CRS	141
5.20. RED HAT ADVANCED CLUSTER MANAGEMENT (RHACM) CRS	141
5.21. STORAGE REFERENCE CRS	144
5.22. GITOPS ZERO TOUCH PROVISIONING (ZTP) REFERENCE CRS	145
5.23. LOGGING REFERENCE CRS	148
5.24. CONTAINER REGISTRY REFERENCE CRS	149
5.25. IMAGE MIRRORING REFERENCE CRS	150
5.26. INSTALLATION REFERENCE CRS	151
5.27. SECURITY REFERENCE CRS	151
5.28. BACKUP RECOVERY REFERENCE CRS	152
5.29. TELCO HUB REFERENCE CONFIGURATION SOFTWARE SPECIFICATIONS	153
CHAPTER 6. COMPARING CLUSTER CONFIGURATIONS	155
6.1. UNDERSTANDING THE CLUSTER-COMPARE PLUGIN	155
6.1.1. Overview of the cluster-compare plugin	155
6.1.2. Understanding a reference configuration	156
6.1.3. Additional resources	157

6.2. INSTALLING THE CLUSTER-COMPARE PLUGIN	157
6.2.1. Installing the cluster-compare plugin	157
6.2.2. Additional resources	158
6.3. USING THE CLUSTER-COMPARE PLUGIN	158
6.3.1. Using the cluster-compare plugin with a live cluster	158
6.3.2. Using the cluster-compare plugin with must-gather data	161
6.3.3. Reference cluster-compare plugin options	163
6.3.4. Example: Comparing a cluster with the telco core reference configuration	164
6.3.5. Additional resources	168
6.4. CREATING A REFERENCE CONFIGURATION	168
6.4.1. Structure of the metadata.yaml file	168
6.4.2. Configuring template relationships	169
6.4.3. Configuring expected variation in a template	170
6.4.3.1. Reference template functions	171
6.4.4. Configuring the metadata.yaml file to exclude template fields	174
6.4.4.1. Excluding all fields not specified in a template	174
6.4.4.2. Excluding specific fields by setting default exclusion fields	175
6.4.4.3. Excluding specific fields	175
6.4.4.4. Excluding specific fields by setting default exclusion groups	176
6.4.5. Configuring inline validation for template fields	177
6.4.5.1. Configuring inline validation with the regex function	177
6.4.5.2. Configuring inline validation with the capturegroups function	178
6.4.6. Configuring descriptions for the output	179
6.5. PERFORMING ADVANCED REFERENCE CONFIGURATION CUSTOMIZATION	179
6.5.1. Configuring manual matching between CRs and templates	180
6.5.2. Patching a reference configuration	181
6.5.2.1. Using the cluster-compare plugin to generate a patch	181
6.5.2.2. Creating a patch file manually	183
6.6. TROUBLESHOOTING CLUSTER COMPARISONS	186
6.6.1. Troubleshooting false positives for missing resources	186
6.6.2. Troubleshooting multiple template matches for the same CR	186
6.6.3. Additional resources	186
CHAPTER 7. PLANNING YOUR ENVIRONMENT ACCORDING TO OBJECT MAXIMUMS	187
7.1. OPENSIFT CONTAINER PLATFORM TESTED CLUSTER MAXIMUMS FOR MAJOR RELEASES	187
7.2. OPENSIFT CONTAINER PLATFORM ENVIRONMENT AND CONFIGURATION ON WHICH THE CLUSTER MAXIMUMS ARE TESTED	191
7.2.1. AWS cloud platform cluster maximums	191
7.2.2. IBM Power platform cluster maximums	192
7.2.3. IBM Z platform cluster maximums	193
7.3. HOW TO PLAN YOUR ENVIRONMENT ACCORDING TO TESTED CLUSTER MAXIMUMS	193
7.4. HOW TO PLAN YOUR ENVIRONMENT ACCORDING TO APPLICATION REQUIREMENTS	194
CHAPTER 8. USING QUOTAS AND LIMIT RANGES	198
8.1. RESOURCES MANAGED BY QUOTA	198
8.2. SETTING RESOURCE QUOTA FOR EXTENDED RESOURCES	200
8.3. QUOTA SCOPES	203
8.4. ADDITIONAL RESOURCES	204
8.5. ADMIN QUOTA USAGE	204
8.5.1. Sample resource quota definitions	204
8.5.2. Creating a quota	209
8.5.3. Creating object count quotas	209
8.5.4. Viewing a quota	210

8.5.5. Configuring quota synchronization period	210
8.5.6. Setting a quota to consume a resource	211
8.6. ADDITIONAL RESOURCES	212
8.7. LIMIT RANGES IN A LIMITRANGE OBJECT	212
8.7.1. Container limits	215
8.7.2. Pod limits	216
8.7.3. Image limits	217
8.7.4. Image stream limits	217
8.7.5. PersistentVolumeClaim limits	218
8.8. ADDITIONAL RESOURCES	219
8.9. LIMIT RANGE OPERATIONS	219
8.10. ADDITIONAL RESOURCES	220
CHAPTER 9. HOST PRACTICES FOR IBM Z AND IBM LINUXONE ENVIRONMENTS	221
9.1. MANAGING CPU OVERCOMMITMENT	221
9.2. DISABLE TRANSPARENT HUGE PAGES	221
9.3. BOOSTING NETWORKING PERFORMANCE WITH RFS	222
9.4. CHOOSE YOUR NETWORKING SETUP	223
9.5. ENSURE HIGH DISK PERFORMANCE WITH HYPERPAV ON Z/VM	223
9.6. RHEL KVM ON IBM Z HOST RECOMMENDATIONS	225
9.6.1. Use I/O threads for your virtual block devices	225
9.6.2. Avoid virtual SCSI devices	226
9.6.3. Configure guest caching for disk	226
9.6.4. Excluding the memory balloon device	226
9.6.5. Tuning the CPU migration algorithm of the host scheduler	227
9.6.6. Disabling the cpuset cgroup controller	227
9.6.7. Tuning the polling period for idle virtual CPUs	228
CHAPTER 10. USING THE NODE TUNING OPERATOR	229
10.1. ABOUT THE NODE TUNING OPERATOR	229
10.2. ACCESSING AN EXAMPLE NODE TUNING OPERATOR SPECIFICATION	229
10.3. DEFAULT PROFILES SET ON A CLUSTER	230
10.4. VERIFYING THAT THE TUNED PROFILES ARE APPLIED	230
10.5. CUSTOM TUNING SPECIFICATION	231
10.6. CUSTOM TUNING EXAMPLES	236
10.7. DEFERRING APPLICATION OF TUNING CHANGES	238
10.7.1. Deferring application of tuning changes: An example	240
10.8. SUPPORTED TUNED DAEMON PLUGINS	244
10.9. CONFIGURING NODE TUNING IN A HOSTED CLUSTER	245
10.10. ADVANCED NODE TUNING FOR HOSTED CLUSTERS BY SETTING KERNEL BOOT PARAMETERS	247
CHAPTER 11. USING CPU MANAGER AND TOPOLOGY MANAGER	251
11.1. SETTING UP CPU MANAGER	251
11.2. TOPOLOGY MANAGER POLICIES	256
11.3. SETTING UP TOPOLOGY MANAGER	257
11.4. POD INTERACTIONS WITH TOPOLOGY MANAGER POLICIES	257
CHAPTER 12. SCHEDULING NUMA-AWARE WORKLOADS	259
12.1. ABOUT NUMA	259
12.2. ABOUT NUMA-AWARE SCHEDULING	259
12.3. NUMA RESOURCE SCHEDULING STRATEGIES	261
12.4. INSTALLING THE NUMA RESOURCES OPERATOR	262
12.4.1. Installing the NUMA Resources Operator using the CLI	262
12.4.2. Installing the NUMA Resources Operator using the web console	263

12.5. CONFIGURING A SINGLE NUMA NODE POLICY	264
12.5.1. Managing high availability (HA) for the NUMA-aware scheduler	264
12.5.1.1. Customizing scheduler replicas	265
12.5.1.2. Disabling NUMA-aware scheduling	266
12.5.1.3. Verifying scheduler high availability (HA) status	266
12.5.2. Sample performance profile	267
12.5.3. Creating a KubeletConfig CR	268
12.6. SCHEDULING NUMA-AWARE WORKLOADS	269
12.6.1. Creating the NUMAResourcesOperator custom resource	270
12.6.2. Creating the NUMAResourcesOperator custom resource for hosted control planes	271
12.6.3. Deploying the NUMA-aware secondary pod scheduler	274
12.6.4. Scheduling workloads with the NUMA-aware scheduler	275
12.7. NUMA RESOURCES OPERATOR SUPPORT FOR SCHEDULABLE CONTROL-PLANE NODES	279
12.7.1. Configuring NUMA Resources Operator on schedulable control plane nodes	280
12.8. CONFIGURING POLLING OPERATIONS FOR NUMA RESOURCES UPDATES	284
12.9. TOPOLOGY-AWARE SCHEDULER SCALABILITY	286
12.9.1. Optimization strategies for large-cluster NUMA-aware scheduling	286
12.9.1.1. NUMAResourcesOperator settings	286
12.9.1.2. NUMAResourcesScheduler settings	287
12.9.1.3. Best practices for large clusters	287
12.10. TROUBLESHOOTING NUMA-AWARE SCHEDULING	287
12.10.1. Reporting more exact resource availability	291
12.10.2. Changing where high-performance workloads run	293
12.10.3. Checking the NUMA-aware scheduler logs	295
12.10.4. Troubleshooting the resource topology exporter	297
12.10.5. Correcting a missing resource topology exporter config map	299
12.10.6. Collecting NUMA Resources Operator data	300
CHAPTER 13. SCALABILITY AND PERFORMANCE OPTIMIZATION	302
13.1. OPTIMIZING STORAGE	302
13.1.1. Available persistent storage options	302
13.1.2. Recommended configurable storage technology	303
13.1.2.1. Specific application storage recommendations	304
13.1.3. Additional resources	306
13.1.4. Data storage management	306
13.1.5. Optimizing storage performance for Microsoft Azure	307
13.2. OPTIMIZING ROUTING	307
13.2.1. Baseline Ingress Controller (router) performance	307
13.2.2. Additional resources	309
13.2.3. Configuring Ingress Controller liveness, readiness, and startup probes	309
13.2.4. Configuring HAProxy reload interval	310
13.3. OPTIMIZING NETWORKING	311
13.3.1. Additional resources	311
13.3.2. Optimizing the MTU for your network	312
13.3.3. Recommended practices for installing large-scale clusters	312
13.3.4. Impact of IPsec	312
13.3.5. Additional resources	313
13.4. OPTIMIZING CPU USAGE WITH MOUNT NAMESPACE ENCAPSULATION	313
13.4.1. Encapsulating mount namespaces	313
13.4.2. Configuring mount namespace encapsulation	315
13.4.3. Inspecting encapsulated namespaces	318
13.4.4. Running additional services in the encapsulated namespace	319
13.4.5. Additional resources	319

CHAPTER 14. MANAGING BARE-METAL HOSTS	320
14.1. ABOUT BARE METAL HOSTS AND NODES	320
14.2. MAINTAINING BARE METAL HOSTS	320
14.2.1. Adding a bare metal host to the cluster using the web console	321
14.2.2. Adding a bare-metal host to the cluster using YAML in the web console	321
14.2.3. Automatically scaling machines to the number of available bare-metal hosts	322
14.2.4. Removing bare-metal hosts from the provisioner node	323
14.2.5. Powering off bare-metal hosts by using the web console	324
14.2.6. Powering off bare-metal hosts by using the CLI	325
CHAPTER 15. OPTIMIZING MEMORY MANAGEMENT FOR WORKLOADS BY USING HUGE PAGES	327
15.1. WHAT HUGE PAGES DO	327
15.2. HOW HUGE PAGES ARE CONSUMED BY APPS	327
15.2.1. Allocating huge pages of a specific size	328
15.2.2. Huge page requirements	328
15.3. CONSUMING HUGE PAGES RESOURCES USING THE DOWNWARD API	328
15.4. CONFIGURING HUGE PAGES AT BOOT TIME	330
15.5. DISABLING TRANSPARENT HUGE PAGES	332
CHAPTER 16. UNDERSTANDING LOW LATENCY TUNING FOR CLUSTER NODES	334
16.1. ABOUT LOW LATENCY	334
16.2. ABOUT HYPER-THREADING FOR LOW LATENCY AND REAL-TIME APPLICATIONS	335
CHAPTER 17. TUNING NODES FOR LOW LATENCY WITH THE PERFORMANCE PROFILE	336
17.1. CREATING A PERFORMANCE PROFILE	336
17.1.1. About the Performance Profile Creator	336
17.1.2. Creating a machine config pool to target nodes for performance tuning	337
17.1.3. Gathering data about your cluster for the PPC	338
17.1.4. Running the Performance Profile Creator using Podman	339
17.1.5. Running the Performance Profile Creator wrapper script	343
17.1.6. Performance Profile Creator arguments	348
17.2. REFERENCE PERFORMANCE PROFILES	351
17.2.1. Performance profile template for clusters that use OVS-DPDK on OpenStack	351
17.2.2. Telco RAN DU reference design performance profile	352
17.2.3. Telco core reference design performance profile	353
17.3. SUPPORTED PERFORMANCE PROFILE API VERSIONS	354
17.4. NODE POWER CONSUMPTION AND REALTIME PROCESSING WITH WORKLOAD HINTS	354
17.5. CONFIGURING POWER SAVING FOR NODES THAT RUN COLOCATED HIGH AND LOW PRIORITY WORKLOADS	356
17.6. CPUS FOR INFRA AND APPLICATION CONTAINERS	357
17.7. PARTITIONING CPUS FOR INFRA AND APPLICATION CONTAINERS	359
17.8. CONFIGURING HYPER-THREADING FOR A CLUSTER	360
17.9. DISABLING HYPER-THREADING FOR LOW LATENCY APPLICATIONS	361
17.10. MANAGING DEVICE INTERRUPT PROCESSING FOR GUARANTEED POD ISOLATED CPUS	362
17.10.1. Finding the effective IRQ affinity setting for a node	362
17.10.2. Configuring node interrupt affinity	364
17.11. CONFIGURING MEMORY PAGE SIZES	365
17.11.1. Configuring kernel page sizes	365
17.11.2. Configuring huge pages	366
17.11.3. Allocating multiple huge page sizes	367
17.12. REDUCING NIC QUEUES USING THE NODE TUNING OPERATOR	368
17.12.1. Adjusting the NIC queues with the performance profile	368
17.12.2. Verifying the queue status	371
17.12.3. Logging associated with adjusting NIC queues	375

CHAPTER 18. TUNING HOSTED CONTROL PLANES FOR LOW LATENCY WITH THE PERFORMANCE PROFILE	376
18.1. CREATING A PERFORMANCE PROFILE FOR HOSTED CONTROL PLANES	376
18.1.1. Gathering data about your hosted control planes cluster for the PPC	376
18.1.2. Running the Performance Profile Creator on a hosted cluster using Podman	378
18.1.3. Configuring low-latency tuning in a hosted cluster	380
CHAPTER 19. PROVISIONING REAL-TIME AND LOW LATENCY WORKLOADS	384
19.1. SCHEDULING A LOW LATENCY WORKLOAD ONTO A COMPUTE NODE	384
19.2. CREATING A POD WITH A GUARANTEED QOS CLASS	387
19.3. DISABLING CPU LOAD BALANCING IN A POD	389
19.4. DISABLING POWER SAVING MODE FOR HIGH PRIORITY PODS	390
19.5. DISABLING CPU CFS QUOTA	391
19.6. CONFIGURING INTERRUPT PROCESSING FOR INDIVIDUAL PODS	392
CHAPTER 20. DEBUGGING LOW LATENCY NODE TUNING STATUS	394
20.1. DEBUGGING LOW LATENCY CNF TUNING STATUS	394
20.2. MACHINE CONFIG POOLS	395
20.3. ABOUT THE MUST-GATHER TOOL	396
20.4. COLLECTING LOW LATENCY TUNING DEBUGGING DATA FOR RED HAT SUPPORT	396
CHAPTER 21. PERFORMING LATENCY TESTS FOR PLATFORM VERIFICATION	399
21.1. PREREQUISITES FOR RUNNING LATENCY TESTS	399
21.2. MEASURING LATENCY	399
21.3. RUNNING THE LATENCY TESTS	401
21.3.1. Running hwlatdetect	402
21.3.2. Example hwlatdetect test results	404
21.3.3. Running cyclicttest	405
21.3.4. Example cyclicttest results	407
21.3.5. Running oslat	408
21.4. GENERATING A LATENCY TEST FAILURE REPORT	410
21.5. GENERATING A JUNIT LATENCY TEST REPORT	410
21.6. RUNNING LATENCY TESTS ON A SINGLE-NODE OPENSIFT CLUSTER	411
21.7. RUNNING LATENCY TESTS IN A DISCONNECTED CLUSTER	412
21.7.1. Mirroring the images to a custom registry accessible from the cluster	412
21.7.2. Configuring the tests to consume images from a custom registry	413
21.7.3. Mirroring images to the cluster OpenShift image registry	413
21.7.4. Mirroring a different set of test images	414
21.8. TROUBLESHOOTING ERRORS WITH THE CNF-TESTS CONTAINER	415
CHAPTER 22. IMPROVING CLUSTER STABILITY IN HIGH LATENCY ENVIRONMENTS USING WORKER LATENCY PROFILES	416
22.1. UNDERSTANDING WORKER LATENCY PROFILES	416
22.2. IMPLEMENTING WORKER LATENCY PROFILES AT CLUSTER CREATION	419
22.3. USING AND CHANGING WORKER LATENCY PROFILES	420
22.4. DISPLAYING RESULTING VALUES OF WORKER LATENCY PROFILE	423
CHAPTER 23. WORKLOAD PARTITIONING	424
23.1. ENABLING WORKLOAD PARTITIONING	424
23.2. PERFORMANCE PROFILES AND WORKLOAD PARTITIONING	425
CHAPTER 24. USING THE NODE OBSERVABILITY OPERATOR	428
24.1. WORKFLOW OF THE NODE OBSERVABILITY OPERATOR	428
24.2. INSTALLING THE NODE OBSERVABILITY OPERATOR	428
24.2.1. Installing the Node Observability Operator using the CLI	428

24.2.2. Installing the Node Observability Operator using the web console	430
24.3. REQUESTING CRI-O AND KUBELET PROFILING DATA USING THE NODE OBSERVABILITY OPERATOR	431
24.3.1. Creating the Node Observability custom resource	431
24.3.2. Running the profiling query	432
24.4. NODE OBSERVABILITY OPERATOR SCRIPTING	434
24.4.1. Creating the Node Observability custom resource for scripting	434
24.4.2. Configuring Node Observability Operator scripting	435
24.5. ADDITIONAL RESOURCES	437

CHAPTER 1. OPENSIFT CONTAINER PLATFORM SCALABILITY AND PERFORMANCE OVERVIEW

OpenShift Container Platform provides best practices and tools to help you optimize the performance and scale of your clusters.

The following documentation provides information on recommended performance and scalability practices, reference design specifications, optimization, and low latency tuning.

To contact Red Hat support, see "Getting support".



NOTE

Some performance and scalability Operators have release cycles that are independent from OpenShift Container Platform release cycles. For more information, see "OpenShift Operators".

1.1. RECOMMENDED PERFORMANCE AND SCALABILITY PRACTICES

Review the recommended performance and scalability practices for OpenShift Container Platform.

[Recommended control plane practices](#)

[Recommended infrastructure practices](#)

1.2. TELCO REFERENCE DESIGN SPECIFICATIONS

Review the telco reference design specifications for OpenShift Container Platform.

[Telco RAN DU reference design specification for OpenShift Container Platform 4.22](#)

[Telco core reference design specification](#)

1.3. PLANNING, OPTIMIZATION, AND MEASUREMENT

Review guidance for planning, optimizing, and measuring the performance and scalability of your OpenShift Container Platform cluster.

[Planning your environment according to object maximums](#)

[Recommended practices for IBM Z and IBM LinuxONE](#)

[Using the Node Tuning Operator](#)

[Using CPU Manager and Topology Manager](#)

[Scheduling NUMA-aware workloads](#)

[Optimizing storage, routing, networking and CPU usage](#)

[Managing bare-metal hosts and events](#)

[What are huge pages and how are they used by apps](#)

[Low latency tuning for improving cluster stability and partitioning workload](#)

[Improving cluster stability in high latency environments using worker latency profiles](#)

[Workload partitioning](#)

[Using the Node Observability Operator](#)

1.4. ADDITIONAL RESOURCES

- [Getting support](#)
- [OpenShift Operators](#)

CHAPTER 2. RECOMMENDED PERFORMANCE AND SCALABILITY PRACTICES

2.1. RECOMMENDED CONTROL PLANE PRACTICES

Review the recommended performance and scalability practices for control planes in OpenShift Container Platform. By doing this task, you can better scale the number of compute machines and set the control plane node sizing for your cluster.

2.1.1. Recommended practices for scaling the cluster

For a cluster installation on a cloud provider, review the recommended practices for scaling the cluster.

Apply the following best practices to scale the number of compute machines in your OpenShift Container Platform cluster. You scale the worker machines by increasing or decreasing the number of replicas that are defined in the compute machine set.

When scaling up the cluster to higher node counts:

- Spread nodes across all of the available zones for higher availability.
- Scale up by no more than 25 to 50 machines at once.
- Consider creating new compute machine sets in each available zone with alternative instance types of similar size to help mitigate any periodic provider capacity constraints. For example, on AWS, use **m5.large** and **m5d.large**.



NOTE

Cloud providers might implement a quota for API services. Therefore, gradually scale the cluster.

The controller might not be able to create the machines if the replicas in the compute machine sets are set to higher numbers all at one time. The number of requests the cloud platform, which OpenShift Container Platform is deployed on top of, is able to handle impacts the process. The controller starts to query more while trying to create, check, and update the machines with the status. The cloud platform on which OpenShift Container Platform is deployed has API request limits; excessive queries might lead to machine creation failures due to cloud platform limitations.

Enable machine health checks when scaling to large node counts. In case of failures, the health checks monitor the condition and automatically repair unhealthy machines.



NOTE

When scaling large and dense clusters to lower node counts, it might take large amounts of time because the process involves draining or evicting the objects running on the nodes being terminated in parallel. Also, the client might start to throttle the requests if there are too many objects to evict. The default client queries per second (QPS) and burst rates are currently set to **50** and **100** respectively. These values cannot be modified in OpenShift Container Platform.

2.1.2. Control plane node sizing

The control plane node resource requirements depend on the number and type of nodes and objects in the cluster. Reference the control plane node size recommendations to better understand your sizing needs.

The following control plane node size recommendations are based on the results of a control plane density focused testing, or *Cluster-density*. This test creates the following objects across a given number of namespaces:

- 1 image stream
- 1 build
- 5 deployments, with 2 pod replicas in a **sleep** state, mounting 4 secrets, 4 config maps, and 1 downward API volume each
- 5 services, each one pointing to the TCP/8080 and TCP/8443 ports of one of the previous deployments
- 1 route pointing to the first of the previous services
- 10 secrets containing 2048 random string characters
- 10 config maps containing 2048 random string characters

Number of compute nodes	Cluster-density (namespaces)	CPU cores	Memory (GB)
24	500	4	16
120	1000	8	32
252	4000	16, but 24 if using the OVN-Kubernetes network plug-in	64, but 128 if using the OVN-Kubernetes network plug-in
501, but untested with the OVN-Kubernetes network plug-in	4000	16	96

The data from the table above is based on an OpenShift Container Platform running on top of AWS, using r5.4xlarge instances as control-plane nodes and m5.2xlarge instances as compute nodes.

On a large and dense cluster with three control plane nodes, the CPU and memory usage will spike up when one of the nodes is stopped, rebooted, or fails. The failures can be due to unexpected issues with power, network, underlying infrastructure, or intentional cases where the cluster is restarted after shutting it down to save costs. The remaining two control plane nodes must handle the load in order to be highly available, which leads to increase in the resource usage. This is also expected during upgrades because the control plane nodes are cordoned, drained, and rebooted serially to apply the operating system updates, as well as the control plane Operators update. To avoid cascading failures, keep the overall CPU and memory resource usage on the control plane nodes to at most 60% of all available capacity to handle the resource usage spikes. Increase the CPU and memory on the control plane nodes accordingly to avoid potential downtime due to lack of resources.

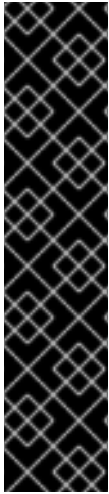


IMPORTANT

The node sizing varies depending on the number of nodes and object counts in the cluster. It also depends on whether the objects are actively being created on the cluster. During object creation, the control plane is more active in terms of resource usage compared to when the objects are in the **Running** phase.

Operator Lifecycle Manager (OLM) runs on the control plane nodes and its memory footprint depends on the number of namespaces and user installed operators that OLM needs to manage on the cluster. Control plane nodes need to be sized accordingly to avoid OOM kills. Following data points are based on the results from cluster maximums testing.

Number of namespaces	OLM memory at idle state (GB)	OLM memory with 5 user operators installed (GB)
500	0.823	1.7
1000	1.2	2.5
1500	1.7	3.2
2000	2	4.4
3000	2.7	5.6
4000	3.8	7.6
5000	4.2	9.02
6000	5.8	11.3
7000	6.6	12.9
8000	6.9	14.8
9000	8	17.7
10,000	9.9	21.6



IMPORTANT

You can modify the control plane node size in a running OpenShift Container Platform 4.22 cluster for the following configurations only:

- Clusters installed with a user-provisioned installation method.
- AWS clusters installed with an installer-provisioned infrastructure installation method.
- Clusters that use a control plane machine set to manage control plane machines.

For all other configurations, you must estimate your total node count and use the suggested control plane node size during installation.



NOTE

In OpenShift Container Platform 4.22, half of a CPU core (500 millicore) is now reserved by the system by default compared to OpenShift Container Platform 3.11 and previous versions. The sizes are determined taking that into consideration.

2.2. SELECTING A LARGER AWS INSTANCE TYPE FOR CONTROL PLANE MACHINES

If the control plane machines in an Amazon Web Services (AWS) cluster require more resources, you can select a larger AWS instance type for the control plane machines to use.



NOTE

The procedure for clusters that use a control plane machine set is different from the procedure for clusters that do not use a control plane machine set.

If you are uncertain about the state of the **ControlPlaneMachineSet** CR in your cluster, you can verify the CR status.

2.2.1. Additional resources

- [Verify the CR status](#)

2.2.2. Changing the Amazon Web Services instance type by using a control plane machine set

You can change the Amazon Web Services (AWS) instance type that your control plane machines use by updating the specification in the control plane machine set custom resource (CR).

Prerequisites

- Your AWS cluster uses a control plane machine set.

Procedure

1. Edit the following line under the **providerSpec** field:

```
providerSpec:
```

```
value:
...
instanceType: <compatible_aws_instance_type>
```

- **<compatible_aws_instance_type>**: Specifies a larger AWS instance type with the same base as the previous selection. For example, you can change **m6i.xlarge** to **m6i.2xlarge** or **m6i.4xlarge**.

2. Save your changes.

Additional resources

- [Managing control plane machines with control plane machine sets](#)

2.2.3. Changing the Amazon Web Services instance type by using the AWS console

You can change the Amazon Web Services (AWS) instance type that your control plane machines use by updating the instance type in the AWS console.

Prerequisites

- You have access to the AWS console with the permissions required to modify the EC2 Instance for your cluster.
- You have access to the OpenShift Container Platform cluster as a user with the **cluster-admin** role.

Procedure

1. Open the AWS console and fetch the instances for the control plane machines.
2. Choose one control plane machine instance.
 - a. For the selected control plane machine, back up the etcd data by creating an etcd snapshot. For more information, see "Backing up etcd".
 - b. In the AWS console, stop the control plane machine instance.
 - c. Select the stopped instance, and click **Actions** → **Instance Settings** → **Change instance type**.
 - d. Change the instance to a larger type, ensuring that the type is the same base as the previous selection, and apply changes. For example, you can change **m6i.xlarge** to **m6i.2xlarge** or **m6i.4xlarge**.
 - e. Start the instance.
 - f. If your OpenShift Container Platform cluster has a corresponding **Machine** object for the instance, update the instance type of the object to match the instance type set in the AWS console.
3. Repeat this process for each control plane machine.

Additional resources

- [Backing up etcd](#)
- [AWS documentation about changing the instance type](#)

2.3. RECOMMENDED INFRASTRUCTURE PRACTICES

This topic provides recommended performance and scalability practices for infrastructure in OpenShift Container Platform.

2.3.1. Infrastructure node sizing

Infrastructure nodes are nodes that are labeled to run pieces of the OpenShift Container Platform environment. The infrastructure node resource requirements depend on the cluster age, nodes, and objects in the cluster, as these factors can lead to an increase in the number of metrics or time series in Prometheus. The following infrastructure node size recommendations are based on the results observed in cluster-density testing detailed in the **Control plane node sizing** section, where the monitoring stack and the default ingress-controller were moved to these nodes.

Number of worker nodes	Cluster density, or number of namespaces	CPU cores	Memory (GB)
27	500	4	24
120	1000	8	48
252	4000	16	128
501	4000	32	128

In general, three infrastructure nodes are recommended per cluster.



IMPORTANT

These sizing recommendations should be used as a guideline. Prometheus is a highly memory intensive application; the resource usage depends on various factors including the number of nodes, objects, the Prometheus metrics scraping interval, metrics or time series, and the age of the cluster. In addition, the router resource usage can also be affected by the number of routes and the amount/type of inbound requests.

These recommendations apply only to infrastructure nodes hosting Monitoring, Ingress and Registry infrastructure components installed during cluster creation.



NOTE

In OpenShift Container Platform 4.22, half of a CPU core (500 millicore) is now reserved by the system by default compared to OpenShift Container Platform 3.11 and previous versions. This influences the stated sizing recommendations.

2.3.2. Scaling the Cluster Monitoring Operator

OpenShift Container Platform exposes metrics that the Cluster Monitoring Operator (CMO) collects

and stores in the Prometheus-based monitoring stack. As an administrator, you can view dashboards for system resources, containers, and components metrics in the OpenShift Container Platform web console by navigating to **Observe → Dashboards**.

2.3.2.1. Prometheus database storage requirements

Red Hat performed various tests for different scale sizes.



NOTE

- The following Prometheus storage requirements are not prescriptive and should be used as a reference. Higher resource consumption might be observed in your cluster depending on workload activity and resource density, including the number of pods, containers, routes, or other resources exposing metrics collected by Prometheus.
- You can configure the size-based data retention policy to suit your storage requirements.

Table 2.1. Prometheus Database storage requirements based on number of nodes/pods in the cluster

Number of nodes	Number of pods (2 containers per pod)	Prometheus storage growth per day	Prometheus storage growth per 15 days	Network (per tsdb chunk)
50	1800	6.3 GB	94 GB	16 MB
100	3600	13 GB	195 GB	26 MB
150	5400	19 GB	283 GB	36 MB
200	7200	25 GB	375 GB	46 MB

Approximately 20 percent of the expected size was added as overhead to ensure that the storage requirements do not exceed the calculated value.

The above calculation is for the default OpenShift Container Platform Cluster Monitoring Operator.



NOTE

CPU utilization has minor impact. The ratio is approximately 1 core out of 40 per 50 nodes and 1800 pods.

Recommendations for OpenShift Container Platform

- Use at least two infrastructure (infra) nodes.
- Use at least three **openshift-container-storage** nodes with non-volatile memory express (SSD or NVMe) drives.

2.3.2.2. Configuring cluster monitoring

You can increase the storage capacity for the Prometheus component in the cluster monitoring stack.

Procedure

1. To increase the storage capacity for Prometheus, create a YAML configuration file, **cluster-monitoring-config.yaml**, as in the following example:

```
apiVersion: v1
kind: ConfigMap
data:
  config.yaml: |
    prometheusK8s:
      retention: <prometheus_retention_period>
      nodeSelector:
        node-role.kubernetes.io/infra: ""
      volumeClaimTemplate:
        spec:
          storageClassName: <storage_class>
          resources:
            requests:
              storage: <prometheus_storage_size>
    alertmanagerMain:
      nodeSelector:
        node-role.kubernetes.io/infra: ""
      volumeClaimTemplate:
        spec:
          storageClassName: <storage_class>
          resources:
            requests:
              storage: <alertmanager_storage_size>
metadata:
  name: cluster-monitoring-config
  namespace: openshift-monitoring
```

- **<prometheus_retention_period>** specifies the Prometheus retention period. The default value is **15d**. Units are measured in time using one of these suffixes: s, m, h, d.
 - **<storage_class>** specifies the storage class for your cluster.
 - **<prometheus_storage_size>** specifies the Prometheus storage size. A typical value is **2000Gi**. Storage values can be a plain integer or a fixed-point integer using one of these suffixes: E, P, T, G, M, K. You can also use the power-of-two equivalents: Ei, Pi, Ti, Gi, Mi, Ki.
 - **<alertmanager_storage_size>** specifies the Alertmanager storage size. A typical value is **20Gi**. Storage values can be a plain integer or a fixed-point integer using one of these suffixes: E, P, T, G, M, K. You can also use the power-of-two equivalents: Ei, Pi, Ti, Gi, Mi, Ki.
2. Add values for the retention period, storage class, and storage sizes.
 3. Save the file.
 4. Apply the changes by running:

```
$ oc create -f cluster-monitoring-config.yaml
```




2.3.3. Additional resources

- [Infrastructure Nodes in OpenShift 4](#)
- [OpenShift Container Platform cluster maximums](#)
- [Creating infrastructure machine sets](#)

CHAPTER 3. TELCO CORE REFERENCE DESIGN SPECIFICATIONS

The telco core reference design specifications (RDS) configures an OpenShift Container Platform cluster running on commodity hardware to host telco core workloads.

3.1. TELCO CORE RDS USE MODEL OVERVIEW

The Telco core reference design specification (RDS) describes a platform that supports large-scale telco applications including control plane functions such as signaling and aggregation. It also includes some centralized data plane functions, for example, user plane functions (UPF). These functions generally require scalability, complex networking support, resilient software-defined storage, and support performance requirements that are less stringent and constrained than far-edge deployments such as RAN.

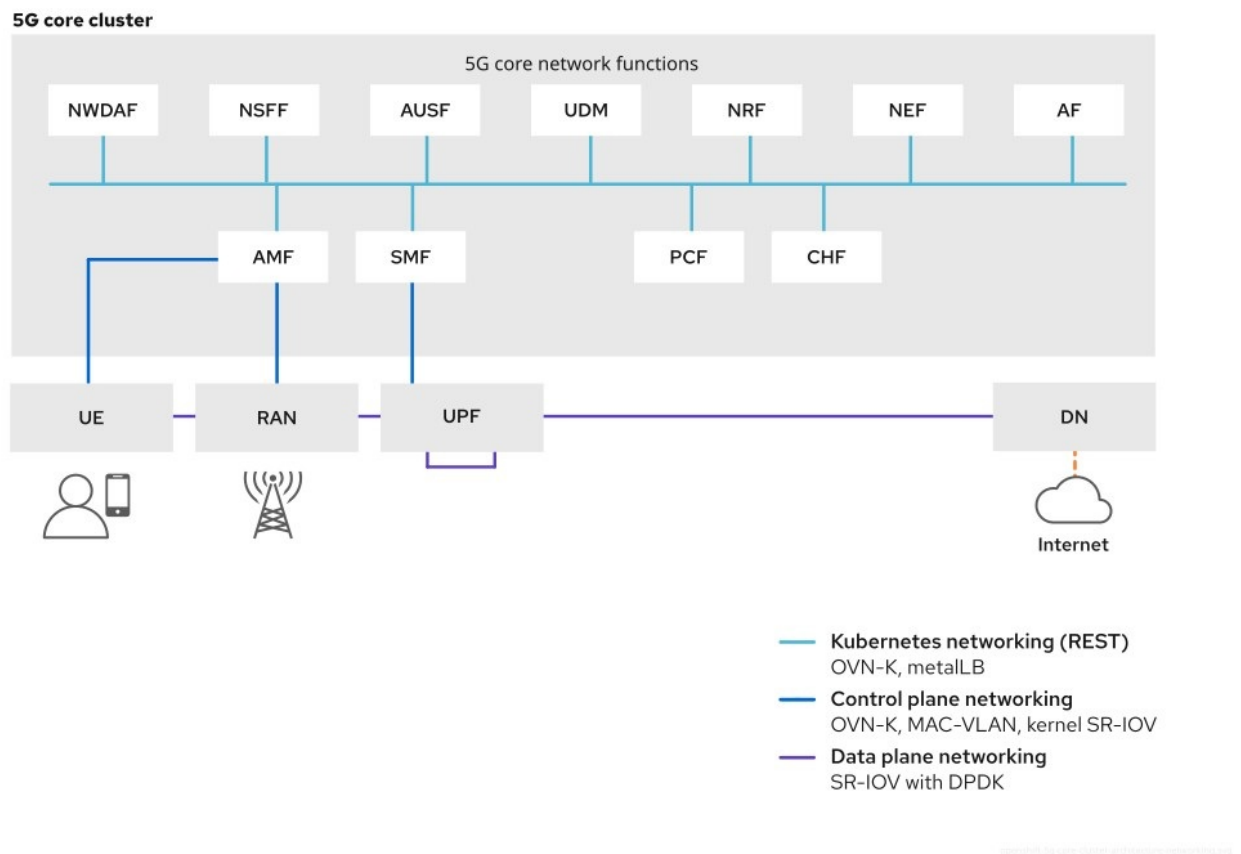
3.2. ABOUT THE TELCO CORE CLUSTER USE MODEL

The telco core cluster use model is designed for clusters running on commodity hardware. Telco core clusters support large scale telco applications including control plane functions like signaling, aggregation, session border controller (SBC), and centralized data plane functions such as 5G user plane functions (UPF). Telco core cluster functions require scalability, complex networking support, resilient software-defined storage, and support performance requirements that are less stringent and constrained than far-edge RAN deployments.

Networking requirements for telco core functions vary widely across a range of networking features and performance points. IPv6 is a requirement and dual-stack is common. Some functions need maximum throughput and transaction rate and require support for user-plane DPDK networking. Other functions use typical cloud-native patterns and can rely on OVN-Kubernetes, kernel networking, and load balancing.

Telco core clusters are configured as standard with three control plane and one or more worker nodes configured with the stock (non-RT) kernel. In support of workloads with varying networking and performance requirements, you can segment worker nodes by using **MachineConfigPool** custom resources (CR), for example, for non-user data plane or high-throughput use cases. In support of required telco operational features, core clusters have a standard set of Day 2 OLM-managed Operators installed.

Figure 3.1. Telco core RDS cluster service-based architecture and networking topology



3.3. REFERENCE DESIGN SCOPE

The telco core, telco RAN and telco hub reference design specifications (RDS) capture the recommended, tested, and supported configurations to get reliable and repeatable performance for clusters running the telco core and telco RAN profiles.

Each RDS includes the released features and supported configurations that are engineered and validated for clusters to run the individual profiles. The configurations provide a baseline OpenShift Container Platform installation that meets feature and KPI targets. Each RDS also describes expected variations for each individual configuration. Validation of each RDS includes many long duration and at-scale tests.

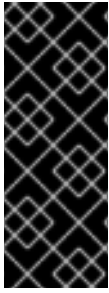


NOTE

The validated reference configurations are updated for each major Y-stream release of OpenShift Container Platform. Z-stream patch releases are periodically re-tested against the reference configurations.

3.4. DEVIATIONS FROM THE REFERENCE DESIGN

Deviating from the validated telco core, telco RAN DU, and telco hub reference design specifications (RDS) can have significant impact beyond the specific component or feature that you change. Deviations require analysis and engineering in the context of the complete solution.



IMPORTANT

All deviations from the RDS should be analyzed and documented with clear action tracking information. Due diligence is expected from partners to understand how to bring deviations into line with the reference design. This might require partners to provide additional resources to engage with Red Hat to work towards enabling their use case to achieve a best in class outcome with the platform. This is critical for the supportability of the solution and ensuring alignment across Red Hat and with partners.

Deviation from the RDS can have some or all of the following consequences:

- It can take longer to resolve issues.
- There is a risk of missing project service-level agreements (SLAs), project deadlines, end provider performance requirements, and so on.
- Unapproved deviations may require escalation at executive levels.



NOTE

Red Hat prioritizes the servicing of requests for deviations based on partner engagement priorities.

3.5. TELCO CORE COMMON BASELINE MODEL

The following configurations and use models are applicable to all telco core use cases. The telco core use cases build on this common baseline of features.

Cluster topology

The telco core reference design supports two distinct cluster configuration variants:

- A non-schedulable control plane variant, where user workloads are strictly prohibited from running on master nodes.
- A schedulable control plane variant, which allows for user workloads to run on master nodes to optimize resource utilization. This variant is only applicable to bare-metal control plane nodes and must be configured at installation time.

All clusters, regardless of the variant, must conform to the following requirements:

- A highly available control plane consisting of three or more nodes.
- The use of multiple machine config pools.

Storage

Telco core use cases require highly available persistent storage as provided by an external storage solution. OpenShift Data Foundation might be used to manage access to the external storage.

Networking

Telco core cluster networking conforms to the following requirements:

- Dual stack IPv4/IPv6 (IPv4 primary).
- Fully disconnected - clusters do not have access to public networking at any point in their lifecycle.

- Supports multiple networks. Segmented networking provides isolation between operations, administration and maintenance (OAM), signaling, and storage traffic.
- Cluster network type is OVN-Kubernetes as required for IPv6 support.
- Telco core clusters have multiple layers of networking supported by underlying RHCOS, SR-IOV Network Operator, Load Balancer and other components. These layers include the following:
 - Cluster networking layer. The cluster network configuration is defined and applied through the installation configuration. Update the configuration during Day 2 operations with the NMState Operator. Use the initial configuration to establish the following:
 - Host interface configuration.
 - Active/active bonding (LACP).
 - Secondary/additional network layer. Configure the OpenShift Container Platform CNI through network **additionalNetwork** or **NetworkAttachmentDefinition** CRs. Use the initial configuration to configure MACVLAN virtual network interfaces.
 - Application workload layer. User plane networking runs in cloud-native network functions (CNFs).

Service Mesh

Telco CNFs can use Service Mesh. Telco core clusters typically include a Service Mesh implementation. The choice of implementation and configuration is outside the scope of this specification.

3.6. DEPLOYMENT PLANNING

Proper deployment planning is essential for telco core clusters to ensure high availability, minimize service disruption during upgrades, and optimize cluster performance.

3.6.1. Worker Nodes and MachineConfigPools

MachineConfigPools (MCPs) custom resources (CR) enable the subdivision of worker nodes in telco core clusters into different node groups based on customer planning parameters. Careful deployment planning using MCPs is crucial to minimize deployment and upgrade time and, more importantly, to minimize interruption of telco-grade services during cluster upgrades.

Description

Telco core clusters can use **MachineConfigPools** (MCPs) to split worker nodes into additional separate roles, for example, due to different hardware profiles. This allows custom tuning for each role and also plays a critical function in speeding up a telco core cluster deployment or upgrade. Multiple MCPs can be used to properly plan cluster upgrades across one or multiple maintenance windows. This is crucial because telco-grade services might otherwise be affected if careful planning is not considered.

During cluster upgrades, you can pause MCPs while you upgrade the control plane. See "Performing a canary rollout update" for more information. This ensures that worker nodes are not rebooted and running workloads remain unaffected until the MCP is unpaused.

Using careful MCP planning, you can control the timing and order of which set of nodes are upgraded at any time. For more information on how to use MCPs to plan telco upgrades, see "Applying MachineConfigPool labels to nodes before the update".

Before beginning the initial deployment, review the following engineering considerations:

PerformanceProfile and Tuned profile association:

When using PerformanceProfiles, remember that each Machine Config Pool (MCP) must be linked to exactly one PerformanceProfile or Tuned profile definition. Consequently, even if the desired configuration is identical for multiple MCPs, each MCP still requires its own dedicated PerformanceProfile definition.

Planning your MCP labeling strategy:

Plan your MCP labeling with an appropriate strategy to split your worker nodes depending on parameters such as:

- The worker node type: identifying a group of nodes with equivalent hardware profile, for example workers for control plane Network Functions (NFs) and workers for user data plane NFs.
- The number of worker nodes per worker node type.
- The minimum number of MCPs required for an equivalent hardware profile is 1, but could be larger for larger clusters. For example, you may design for more MCPs per hardware profile to support a more granular upgrade where a smaller percentage of the cluster capacity is affected with each step.
- The update strategy for nodes within an MCP is by upgrade requirements and the chosen **maxUnavailable** value:
 - Number of maintenance windows allowed
 - Duration of a maintenance window
 - Total number of worker nodes
 - Desired **maxUnavailable** (number of nodes updated concurrently) for the MCP.
- CNF requirements for worker nodes, in terms of:
 - Minimum availability per Pod required during an upgrade, configured with a pod disruption budget (PDB). PDBs are crucial to maintain telco service level Agreements (SLAs) during upgrades. For more information about PDB, see "Understanding how to use pod disruption budgets to specify the number of pods that must be up".
 - Minimum true high availability required per Pod, such that each replica runs on separate hardware.
 - Pod affinity and anti-affinity link: For more information about how to use pod affinity and anti-affinity, see "Placing pods relative to other pods using affinity and anti-affinity rules".
- Duration and number of upgrade maintenance windows during which telco-grade services might be affected.

3.6.2. Zones

Designing the cluster to support disruption of multiple nodes simultaneously is critical for high availability (HA) and reduced upgrade times. OpenShift Container Platform and Kubernetes use the label **topology.kubernetes.io/zone** to create pools of nodes which are subject to a common failure domain. Annotating nodes for topology (availability) zones allows high-availability workloads to spread such that each zone holds only one replica from a set of HA replicated pods. With this spread the loss of a single zone will not violate HA constraints and minimum service availability will be maintained. OpenShift Container Platform and Kubernetes apply a default TopologySpreadConstraint to all replica constructs, such as **Service**, **ReplicaSet**, **StatefulSet** or **ReplicationController** resources, which spreads the replicas based on the **topology.kubernetes.io/zone** label. This default allows zone based spread to apply without any change to your workload pod specs.

Cluster upgrades typically result in node disruption as the underlying OS is updated. In large clusters it is necessary to update multiple nodes concurrently in order to complete upgrades quickly and in as few maintenance windows as possible. By using zones to ensure pod spread, an upgrade can be applied to all nodes in a zone simultaneously (assuming sufficient spare capacity) while maintaining high availability and service availability. The recommended cluster design is to partition nodes into multiple MCPs based on the considerations above and label all nodes in a single MCP as a single zone which is distinct from zones attached to other MCPs. Using this strategy all nodes in an MCP can be updated simultaneously.

Lifecycle hooks (readiness, liveness, startup and pre-stop) play an important role in ensuring application availability. For upgrades in particular the pre-stop hook allows applications to take necessary steps to prepare for disruption prior to being evicted from the node.

Limits and requirements

- The default TopologySpreadConstraints (TSC) only apply when an explicit TSC is not given. If your pods have explicit TSC ensure that spread based on zones is included.
- The cluster must have sufficient spare capacity to tolerate simultaneous update of an MCP. Otherwise the **maxUnavailable** of the MCP must be set to less than 100%.
- The ability to update all nodes in an MCP simultaneously further depends on workload design and ability to maintain required service levels with that level of disruption.

Engineering Considerations

- Pod drain times can significantly impact node update times. Ensure the workload design allows pods to be drained quickly.
- PodDisruptionBudgets (PDB) are used to enforce high availability requirements.
 - With sufficient zones in the cluster design, a zone can be disrupted or lost without violating the PDB. If there are insufficient zones, or other scheduling constraints restrict the set of available nodes, the scheduling of pods might violate the PDB when the zone is disrupted or lost. During upgrades with simultaneous node updates this might lead to partial serialization of updates.
 - PDB with 0 disruptable pods will block node drain and require administrator intervention. This pattern should be avoided for fast and automated upgrades.

Additional resources

- [Performing a canary rollout update](#)
- [Applying MachineConfigPool labels to nodes before the update](#)

- [Understanding how to use pod disruption budgets to specify the number of pods that must be up](#)
- [Placing pods relative to other pods using affinity and anti-affinity rules](#)

3.7. TELCO CORE CLUSTER COMMON USE MODEL ENGINEERING CONSIDERATIONS

The following engineering considerations apply to the telco core cluster common use model.

- Cluster workloads are detailed in "Application workloads".
- Worker nodes should run on either of the following CPUs:
 - Intel 3rd Generation Xeon (IceLake) CPUs or better when supported by OpenShift Container Platform, or CPUs with the silicon security bug (Spectre and similar) mitigations turned off. Skylake and older CPUs can experience 40% transaction performance drops when Spectre and similar mitigations are enabled.
 - AMD EPYC Zen 4 CPUs (Genoa, Bergamo) or AMD EPYC Zen 5 CPUs (Turin) when supported by OpenShift Container Platform.
 - Intel Sierra Forest CPUs when supported by the OpenShift Container Platform.
 - IRQ balancing is enabled on worker nodes. The **PerformanceProfile** CR sets the **globallyDisableIrqLoadBalancing** parameter to a value of **false**. Guaranteed QoS pods are annotated to ensure isolation as described in "CPU partitioning and performance tuning".
- All cluster nodes should have the following features:
 - Have Hyper-Threading enabled
 - Have x86_64 CPU architecture
 - Have the stock (non-realtime) kernel enabled
 - Are not configured for workload partitioning
- The balance between power management and maximum performance varies between machine config pools in the cluster. The following configurations should be consistent for all nodes in a machine config pools group.
 - Cluster scaling. See "Scalability" for more information.
 - Clusters should be able to scale to at least 120 nodes.
- CPU partitioning is configured using a **PerformanceProfile** CR and is applied to nodes on a per **MachineConfigPool** basis. See "CPU partitioning and performance tuning" for additional considerations.
- CPU requirements for OpenShift Container Platform depend on the configured feature set and application workload characteristics. For a cluster configured according to the reference configuration running a simulated workload of 3000 pods as created by the kube-burner node-density test, the following CPU requirements are validated:
 - The minimum number of reserved CPUs for control plane and worker nodes is 2 CPUs (4

hyper-threads) per NUMA node.

- The NICs used for non-DPDK network traffic should be configured to use at most 32 RX/TX queues.
- Nodes with large numbers of pods or other resources might require additional reserved CPUs. The remaining CPUs are available for user workloads.



NOTE

Variations in OpenShift Container Platform configuration, workload size, and workload characteristics require additional analysis to determine the effect on the number of required CPUs for the OpenShift platform.

3.7.1. Application workloads

Application workloads running on telco core clusters can include a mix of high performance cloud-native network functions (CNFs) and traditional best-effort or burstable pod workloads.

Guaranteed QoS scheduling is available to pods that require exclusive or dedicated use of CPUs due to performance or security requirements. Typically, pods that run high performance or latency sensitive CNFs by using user plane networking (for example, DPDK) require exclusive use of dedicated whole CPUs achieved through node tuning and guaranteed QoS scheduling. When creating pod configurations that require exclusive CPUs, be aware of the potential implications of hyper-threaded systems. Pods should request multiples of 2 CPUs when the entire core (2 hyper-threads) must be allocated to the pod.

Pods running network functions that do not require high throughput or low latency networking should be scheduled with best-effort or burstable QoS pods and do not require dedicated or isolated CPU cores.

Engineering considerations

Plan telco core workloads and cluster resources by using the following information:

- As of OpenShift Container Platform 4.19, **cgroup v1** is no longer supported and has been removed. All workloads must now be compatible with **cgroup v2**. For more information, see [Red Hat Enterprise Linux 9 changes in the context of Red Hat OpenShift workloads](#).
- CNF applications should conform to the latest version of [Red Hat Best Practices for Kubernetes](#).
- Use a mix of best-effort and burstable QoS pods as required by your applications.
 - Use guaranteed QoS pods with proper configuration of reserved or isolated CPUs in the **PerformanceProfile** CR that configures the node.
 - Guaranteed QoS Pods must include annotations for fully isolating CPUs.
 - Best effort and burstable pods are not guaranteed exclusive CPU use. Workloads can be preempted by other workloads, operating system daemons, or kernel tasks.
- Use exec probes sparingly and only when no other suitable option is available.
 - Do not use exec probes if a CNF uses CPU pinning. Use other probe implementations, for example, **httpGet** or **tcpSocket**.

- When you need to use exec probes, limit the exec probe frequency and quantity. The maximum number of exec probes must be kept below 10, and the frequency must not be set to less than 10 seconds.
- You can use startup probes, because they do not use significant resources at steady-state operation. The limitation on exec probes applies primarily to liveness and readiness probes. Exec probes cause much higher CPU usage on management cores compared to other probe types because they require process forking.
- Use pre-stop hooks to allow the application workload to perform required actions before pod disruption, such as during an upgrade or node maintenance. The hooks enable a pod to save state to persistent storage, offload traffic from a Service, or signal other Pods.

3.7.2. Signaling workloads

Signaling workloads typically use SCTP, REST, gRPC, or similar TCP or UDP protocols. Signaling workloads support hundreds of thousands of transactions per second (TPS) by using a secondary multus CNI configured as MACVLAN or SR-IOV interface. These workloads can run in pods with either guaranteed or burstable QoS.

3.8. TELCO CORE RDS COMPONENTS

The following sections describe the various OpenShift Container Platform components and configurations that you use to configure and deploy clusters to run telco core workloads.

3.8.1. CPU partitioning and performance tuning

CPU partitioning separates sensitive workloads from general-purpose tasks, interrupts, and driver work queues to improve performance and reduce latency.

New in this release

- There are no reference design updates in this release.

Description

CPU partitioning improves performance and reduces latency by separating sensitive workloads from general-purpose tasks, interrupts, and driver work queues. The CPUs allocated to those auxiliary processes are referred to as **reserved** in the following sections. In a system with Hyper-Threading enabled, a CPU is one hyper-thread.

Limits and requirements

- The operating system needs a certain amount of CPU to perform all the support tasks, including kernel networking.
 - A system with just user plane networking applications (DPDK) needs at least one core (2 hyper-threads when enabled) reserved for the operating system and the infrastructure components.
- In a system with Hyper-Threading enabled, core sibling threads must always be in the same pool of CPUs.
- The set of reserved and isolated cores must include all CPU cores.
- Core 0 of each NUMA node must be included in the reserved CPU set.

- Low latency workloads require special configuration to avoid being affected by interrupts, kernel scheduler, or other parts of the platform.

For more information, see "Creating a performance profile".

Engineering considerations

- As of OpenShift 4.19, **cgroup v1** is no longer supported and has been removed. All workloads must now be compatible with **cgroup v2**. For more information, see [Red Hat Enterprise Linux 9 changes in the context of Red Hat OpenShift workloads](#).
- The minimum reserved capacity (**systemReserved**) required can be found by following the guidance in [Which amount of CPU and memory are recommended to reserve for the system in OCP 4 nodes?](#).
 - The specific values must be custom-tuned for each cluster based on its size and application workload.
 - The minimum recommended **systemReserved** memory is 11Gi for worker nodes and 30Gi for control plane nodes.
- For schedulable control planes, the minimum recommended reserved capacity is at least 16 CPUs.
- The actual required reserved CPU capacity depends on the cluster configuration and workload attributes.
- The reserved CPU value must be rounded up to a full core (2 hyper-threads) alignment.
- Changes to CPU partitioning cause the nodes contained in the relevant machine config pool to be drained and rebooted.
- The reserved CPUs reduce the pod density, because the reserved CPUs are removed from the allocatable capacity of the OpenShift Container Platform node.
- The real-time workload hint should be enabled for real-time capable workloads.
 - Applying the real-time **workloadHint** setting results in the **nohz_full** kernel command line parameter being applied to improve performance of high performance applications. When you apply the **workloadHint** setting, any isolated or burstable pods that do not have the **cpu-quota.crio.io: "disable"** annotation and a proper **runtimeClassName** value, are subject to CRI-O rate limiting. When you set the **workloadHint** parameter, be aware of the tradeoff between increased performance and the potential impact of CRI-O rate limiting. Ensure that required pods are correctly annotated.
- Hardware without IRQ affinity support affects isolated CPUs. All server hardware must support IRQ affinity to ensure that pods with guaranteed CPU QoS can fully use allocated CPUs.
- OVS dynamically manages its **cpuset** entry to adapt to network traffic needs. You do not need to reserve an additional CPU for handling high network throughput on the primary CNI.
- If workloads running on the cluster use kernel level networking, the RX/TX queue count for the participating NICs should be set to 16 or 32 queues if the hardware permits it. Be aware of the default queue count. With no configuration, the default queue count is one RX/TX queue per online CPU; which can result in too many interrupts being allocated.

- The `irdma` kernel module might result in the allocation of too many interrupt vectors on systems with high core counts. To prevent this condition the reference configuration excludes this kernel module from loading through a kernel commandline argument in the **PerformanceProfile** resource. Typically Core workloads do not require this kernel module.
- To enable the **acpi_idle** CPUIdle driver, for example for Intel FlexRAN workloads, add **intel_idle.max_cstate=0** to the **additionalKernelArgs** list in the **PerformanceProfile** resource.
- The **TunedPerformancePatch.yaml** file in the reference configures the **kernel.panic_on_unrecovered_nmi** sysctl parameter to enable triggering a kernel panic through BMC Non-Maskable Interrupt (NMI) on x86_64 architectures. This provides a mechanism to force a kernel panic for system recovery and diagnostic purposes when nodes become unresponsive.

**NOTE**

Some drivers do not deallocate the interrupts even after reducing the queue count.

3.8.2. Workloads on schedulable control planes

You can enable schedulable control planes to run workloads on control plane nodes, utilizing idle CPU capacity on bare-metal machines.

Enabling workloads on control plane nodes

You can enable schedulable control planes to run workloads on control plane nodes, utilizing idle CPU capacity on bare-metal machines for potential cost savings. This feature is only applicable to clusters with bare-metal control plane nodes.

There are two distinct parts to this functionality:

1. Allowing workloads on control plane nodes: This feature can be configured after initial cluster installation, allowing you to enable it when you need to run workloads on those nodes.
2. Enabling workload partitioning: This is a critical isolation measure that protects the control plane from interference by regular workloads, ensuring cluster stability and reliability. Workload partitioning must be configured during the initial "day zero" cluster installation and cannot be enabled later.

If you plan to run workloads on your control plane nodes, you must first enable workload partitioning during the initial setup. You can then enable the schedulable control plane feature at a later time.

Workload characterization and limitations

You must test and verify workloads to ensure that applications do not interfere with core cluster functions. It is recommended that you start with lightweight containers that do not heavily load the CPU or networking.

Certain workloads are not permitted on control plane nodes due to the risk to cluster stability. This includes any workload that reconfigures kernel arguments or system global **sysctls**, as this can lead to unpredictable outcomes for the cluster.

To ensure stability, you must adhere to the following:

- Make sure all non-trivial workloads have memory limits defined. This protects the control plane in case of a memory leak.
- Avoid excessively loading reserved CPUs, for example, by heavy use of exec probes.
- Avoid heavy kernel-based networking usage, as it can increase reserved CPU load through software networking components like OVS.

NUMA Resources Operator support

As part of this enablement, the NUMA Resources Operator is supported on control plane nodes. The functional behavior of the NUMA Resources Operator remains the same, but its use on control plane nodes is explicitly permitted.

Additional resources

- [Creating a performance profile](#)
- [Configuring host firmware for low latency and high performance](#)

3.8.3. Service Mesh

Telco core cloud-native functions (CNFs) typically require a Service Mesh implementation.

Description

Telco core cloud-native functions (CNFs) typically require a Service Mesh implementation. Specific Service Mesh features and performance requirements are dependent on the application. The selection of Service Mesh implementation and configuration is outside the scope of this documentation. The implementation must account for the impact of Service Mesh on cluster resource usage and performance, including additional latency introduced in pod networking.

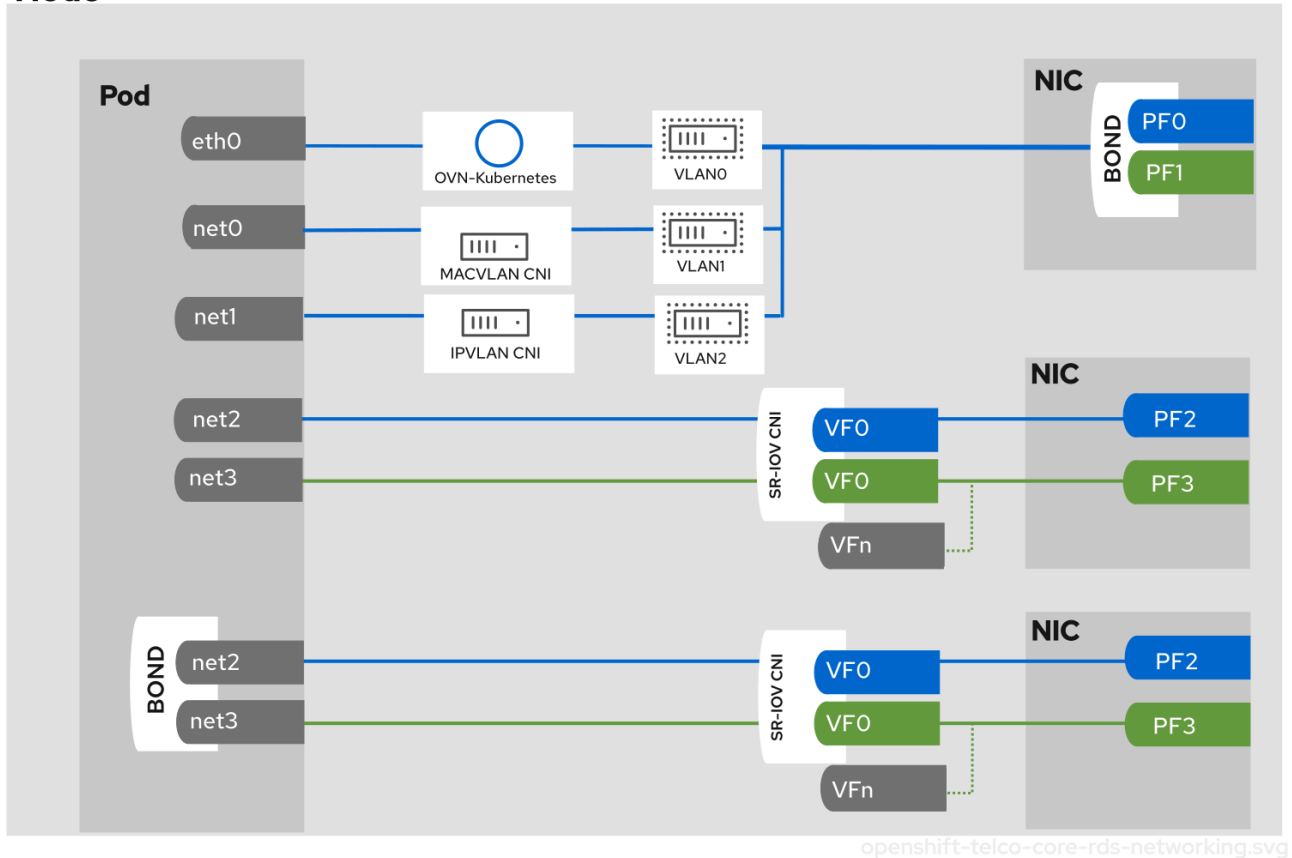
Additional resources

- [About OpenShift Service Mesh](#)

3.8.4. Networking

The following diagram describes the telco core reference design networking configuration.

Figure 3.2. Telco core reference design networking configuration

Node

openshift-telco-core-rds-networking.svg

**NOTE**

The diagram shows the recommended network topology, where bonding uses ports from the same dual-port NIC. Bonding ports across different NICs is not recommended.

New in this release

- Per-interface IP forwarding
- Support for alt-names on network interfaces
- Host firewall rules for multi-node clusters
- MetalLB **ConfigurationStatus** CRD
- Extended BGP/L2 advertisements with service selectors
- Support for Intel E830 Jasper Beach in telco core

**NOTE**

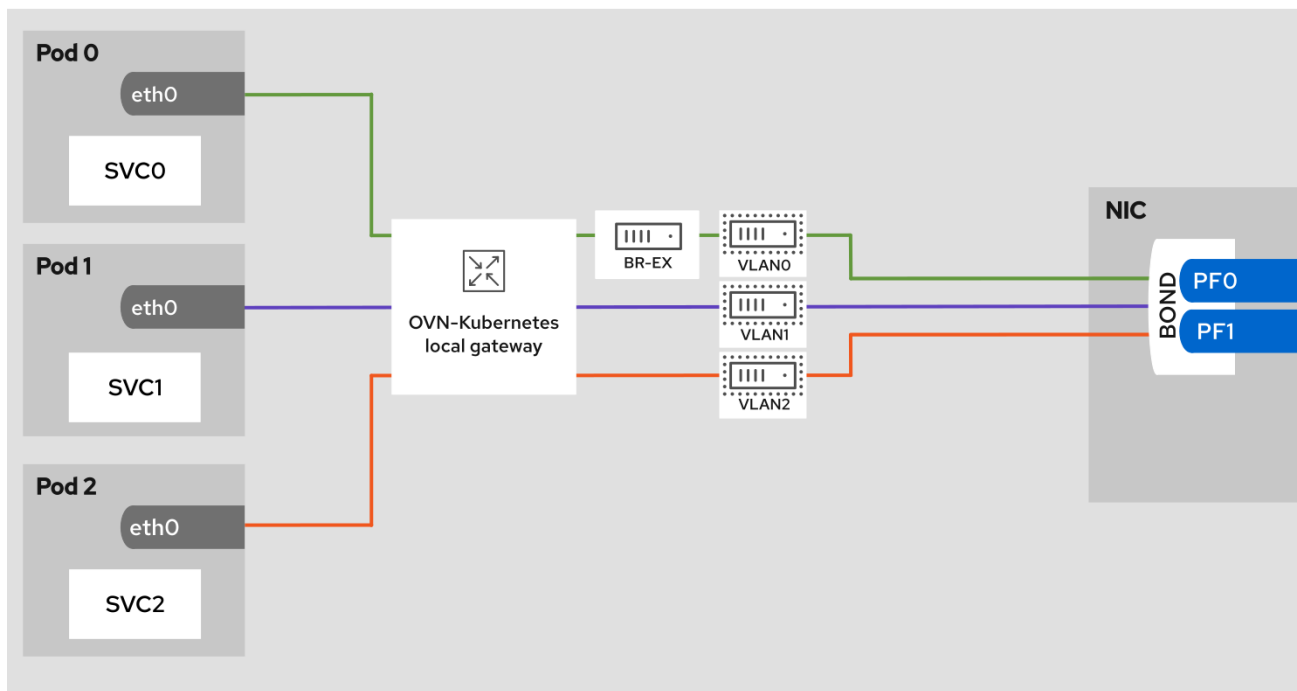
If you have custom **FRRConfiguration** CRs in the **metalb-system** namespace, you must move them under the **openshift-network-operator** namespace.

Description

- The cluster is configured for dual-stack IP (IPv4 and IPv6).

- The validated physical network configuration consists of two dual-port NICs. One NIC is shared among the primary CNI (OVN-Kubernetes) and IPVLAN and MACVLAN traffic, while the second one is dedicated to SR-IOV VF-based pod traffic.
- A Linux bonding interface (**bond0**) is created in active-active IEEE 802.3ad LACP mode with the two NIC ports attached. The top-of-rack networking equipment must support and be configured for multi-chassis link aggregation (mLAG) technology.
- VLAN interfaces are created on top of **bond0**, including for the primary CNI.
- Bond and VLAN interfaces are created at cluster install time during the network configuration stage of the installation. Except for the **vlan0** VLAN used by the primary CNI, all other VLANs can be created during Day 2 activities with the Kubernetes NMstate Operator.
- MACVLAN and IPVLAN interfaces are created with their corresponding CNIs. They do not share the same base interface. For more information, see "Cluster Network Operator".
- SR-IOV VFs are managed by the SR-IOV Network Operator.
- To ensure consistent source IP addresses for pods behind a LoadBalancer Service, configure an **EgressIP** CR and specify the **podSelector** parameter. EgressIP is further discussed in the "Cluster Network Operator" section.
- You can implement service traffic separation by doing the following:
 - a. Configure VLAN interfaces and specific kernel IP routes on the nodes using **NodeNetworkConfigurationPolicy** CRs.
 - b. Create a MetalLB **BGPPeer** CR for each VLAN to establish peering with the remote BGP router.
 - c. Define a MetalLB **BGPAdvertisement** CR to specify which IP address pools should be advertised to a selected list of **BGPPeer** resources. The following diagram illustrates how specific service IP addresses are advertised externally through specific VLAN interfaces. Services routes are defined in **BGPAdvertisement** CRs and configured with values for **IPAddressPool1** and **BGPPeer1** fields.
- Host-level per-interface IP forwarding can be enabled or disabled by using **net.ipv4.conf.<interface>.forwarding** through the install network configuration (Day 1) or the Kubernetes NMState Operator (Day 2).

Figure 3.3. Telco core reference design MetalLB service separation

Node

■ SVC0 is advertised through VLAN0 and the BR-EX bridge.

■ SVC1 is advertised through VLAN1 as defined in the BGPAdvertisement CR with IPAddressPool1 and BGPPeer1 fields configured.

■ SVC2 is advertised through VLAN2 as defined in the BGPAdvertisement CR with IPAddressPool2 and BGPPeer2 fields configured.

openshift-telco-core-rds-metalb-service-separation.svg

Additional resources

- [Understanding networking](#)

3.8.4.1. Cluster Network Operator

The Cluster Network Operator (CNO) deploys and manages the cluster network components including the default OVN-Kubernetes network plugin during cluster installation.

New in this release

- There are no reference design updates in this release.

Description

The Cluster Network Operator (CNO) deploys and manages the cluster network components including the default OVN-Kubernetes network plugin during cluster installation. The CNO allows configuration of primary interface MTU settings, OVN gateway modes to use node routing tables for pod egress, and additional secondary networks such as MACVLAN.

In support of network traffic separation, multiple network interfaces are configured through the CNO. Traffic steering to these interfaces is configured through static routes applied by using the NMState Operator. To ensure that pod traffic is properly routed, OVN-K is configured with the **routingViaHost** option enabled. This setting uses the kernel routing table and the applied static routes rather than OVN for pod egress traffic.

The Whereabouts CNI plugin is used to provide dynamic IPv4 and IPv6 addressing for additional pod network interfaces without the use of a DHCP server.

Limits and requirements

- OVN-Kubernetes is required for IPv6 support.
- Large MTU cluster support requires connected network equipment to be set to the same or larger value. MTU size up to 8900 is supported.
- MACVLAN and IPVLAN cannot co-locate on the same main interface due to their reliance on the same underlying kernel mechanism, specifically the **rx_handler**. This handler allows a third-party module to process incoming packets before the host processes them, and only one such handler can be registered per network interface. Since both MACVLAN and IPVLAN need to register their own **rx_handler** to function, they conflict and cannot coexist on the same interface. Review the source code for more details:
 - [linux/v6.10.2/source/drivers/net/ipvlan/ipvlan_main.c#L82](#)
 - [linux/v6.10.2/source/drivers/net/macvlan.c#L1260](#)
- Alternative NIC configurations include splitting the shared NIC into multiple NICs or using a single dual-port NIC, though they have not been tested and validated.
- Clusters with single-stack IP configuration are not validated.
- EgressIP
 - EgressIP failover time depends on the **reachabilityTotalTimeoutSeconds** parameter in the **Network** CR. This parameter determines the frequency of probes used to detect when the selected egress node is unreachable. The recommended value of this parameter is **1** second.
 - When EgressIP is configured with multiple egress nodes, the failover time is expected to be on the order of seconds or longer.
 - On nodes with additional network interfaces EgressIP traffic will egress through the interface on which the EgressIP address has been assigned. For more information, see "Configuring an egress IP address".
- Pod-level SR-IOV bonding mode must be set to **active-backup** and a value in **miimon** must be set (**100** is recommended).

Engineering considerations

- Pod egress traffic is managed by kernel routing table using the **routingViaHost** option. Appropriate static routes must be configured in the host.

Additional resources

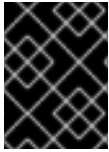
- [Cluster Network Operator](#)
- [Configuring an egress IP address](#)

3.8.4.2. Load balancer

MetalLB is a load-balancer implementation for bare metal Kubernetes clusters that uses standard routing protocols.

New in this release

- There are no reference design updates in this release.



IMPORTANT

If you have custom **FRRConfiguration** CRs in the **metallb-system** namespace, you must move them under the **openshift-network-operator** namespace.

Description

MetalLB is a load-balancer implementation for bare metal Kubernetes clusters that uses standard routing protocols. It enables a Kubernetes service to get an external IP address which is also added to the host network for the cluster. The MetalLB Operator deploys and manages the lifecycle of a MetalLB instance in a cluster. Some use cases might require features not available in MetalLB, such as stateful load balancing. Where necessary, you can use an external third party load balancer. Selection and configuration of an external load balancer is outside the scope of this specification. When an external third-party load balancer is used, the integration effort must include enough analysis to ensure all performance and resource utilization requirements are met.

Limits and requirements

- Stateful load balancing is not supported by MetalLB. An alternate load balancer implementation must be used if this is a requirement for workload CNFs.
- You must ensure that the external IP address is routable from clients to the host network for the cluster.

Engineering considerations

- MetalLB is used in BGP mode only for telco core use models.
- For telco core use models, MetalLB is supported only with the OVN-Kubernetes network provider used in local gateway mode. See **routingViaHost** in "Cluster Network Operator".
- BGP configuration in MetalLB is expected to vary depending on the requirements of the network and peers.
 - You can configure address pools with variations in addresses, aggregation length, auto assignment, and so on.
 - MetalLB uses BGP for announcing routes only. Only the **transmitInterval** and **minimumTtl** parameters are relevant in this mode. Other parameters in the BFD profile should remain close to the defaults as shorter values can lead to false negatives and affect performance.

Additional resources

- [When to use MetalLB](#)

3.8.4.3. SR-IOV

SR-IOV enables physical functions (PFs) to be divided into multiple virtual functions (VFs) for higher throughput performance while keeping pods isolated.

New in this release

- There are no reference design updates in this release.

Description

SR-IOV enables physical functions (PFs) to be divided into multiple virtual functions (VFs). VFs can then be assigned to multiple pods to achieve higher throughput performance while keeping the pods isolated. The SR-IOV Network Operator provisions and manages SR-IOV CNI, network device plugin, and other components of the SR-IOV stack.

Limits and requirements

- Only certain network interfaces are supported. See "Supported devices" for more information.
- Enabling SR-IOV and IOMMU: the SR-IOV Network Operator automatically enables IOMMU on the kernel command line.
- SR-IOV VFs do not receive link state updates from the PF. If a link down detection is required, it must be done at the protocol level.
- **MultiNetworkPolicy** CRs can be applied to **netdevice** networks only. This is because the implementation uses iptables, which cannot manage vfio interfaces.

Engineering considerations

- SR-IOV interfaces in **vfio** mode are typically used to enable additional secondary networks for applications that require high throughput or low latency.
- The **SriovOperatorConfig** CR must be explicitly created. This CR is included in the reference configuration policies, which causes it to be created during initial deployment.
- NICs that do not support firmware updates with UEFI secure boot or kernel lockdown must be preconfigured with sufficient virtual functions (VFs) enabled to support the number of VFs required by the application workload. For Mellanox NICs, you must disable the Mellanox vendor plugin in the SR-IOV Network Operator. For more information see, "Configuring an SR-IOV network device".
- To change the MTU value of a VF after the pod has started, do not configure the **SriovNetworkNodePolicy** MTU field. Instead, use the Kubernetes NMState Operator to set the MTU of the related PF.

Additional resources

- [About Single Root I/O Virtualization \(SR-IOV\) hardware networks](#)
- [Red Hat certified hardware \(Red Hat Ecosystem Catalog\)](#)
- [Configuring the SR-IOV Network Operator on Mellanox cards when Secure Boot is enabled](#)

3.8.4.4. NMState Operator

The Kubernetes NMState Operator provides a Kubernetes API for performing state-driven network configuration across cluster nodes.

New in this release

- There are no reference design updates in this release.

Description

The Kubernetes NMState Operator provides a Kubernetes API for performing state-driven network configuration across cluster nodes. It enables network interface configurations, static IPs and DNS, VLANs, trunks, bonding, static routes, MTU, and enabling promiscuous mode on the secondary interfaces. The cluster nodes periodically report on the state of each node's network interfaces to the API server.

Limits and requirements

Not applicable

Engineering considerations

- Initial networking configuration is applied using **NMStateConfig** content in the installation CRs. The NMState Operator is used only when required for network updates.
- When SR-IOV virtual functions are used for host networking, the NMState Operator (via **nodeNetworkConfigurationPolicy** CRs) is used to configure VF interfaces, such as VLANs and MTU.

Additional resources

- [Kubernetes NMState Operator](#)

3.8.5. Logging

The Cluster Logging Operator enables collection and shipping of logs off the node for remote archival and analysis.

New in this release

- There are no reference design updates in this release.

Description

The Cluster Logging Operator enables collection and shipping of logs off the node for remote archival and analysis. The reference configuration uses Kafka to ship audit and infrastructure logs to a remote archive.

Limits and requirements

Not applicable

Engineering considerations

- The impact of cluster CPU use is based on the number or size of logs generated and the amount of log filtering configured.
- The reference configuration does not include shipping of application logs. The inclusion of application logs in the configuration requires you to evaluate the application logging rate and have sufficient additional CPU resources allocated to the reserved set.

Additional resources

- [Logging 6.0](#)

3.8.6. Power Management

Use the performance profile to configure clusters with high power mode, low power mode, or mixed mode depending on workload latency sensitivity.

New in this release

- There are no reference design updates in this release.

Description

Use the Performance profile to configure clusters with high power mode, low power mode, or mixed mode. The choice of power mode depends on the characteristics of the workloads running on the cluster, particularly how sensitive they are to latency. Configure the maximum latency for a low-latency pod by using the per-pod power management C-states feature.

Limits and requirements

- Power configuration relies on appropriate BIOS configuration, for example, enabling C-states and P-states. Configuration varies between hardware vendors.

Engineering considerations

- Latency: To ensure that latency-sensitive workloads meet requirements, you require a high-power or a per-pod power management configuration. Per-pod power management is only available for Guaranteed QoS pods with dedicated pinned CPUs.

Additional resources

- performance.openshift.io/v2 API reference
- [Configuring power saving for nodes that run colocated high and low priority workloads](#)

3.8.7. Storage

Cloud native storage services can be provided by OpenShift Data Foundation or other third-party solutions for telco core clusters.

New in this release

- There are no reference design updates in this release.

Description

Cloud native storage services can be provided by OpenShift Data Foundation or other third-party solutions.

OpenShift Data Foundation is a Red Hat Ceph Storage based software-defined storage solution for containers. It provides block storage, file system storage, and on-premise object storage, which can be dynamically provisioned for both persistent and non-persistent data requirements. Telco core applications require persistent storage.



NOTE

All storage data might not be encrypted in flight. To reduce risk, isolate the storage network from other cluster networks. The storage network must not be reachable, or routable, from other cluster networks. Only nodes directly attached to the storage network should be allowed to gain access to it.

Additional resources

- [Red Hat OpenShift Data Foundation](#)

3.8.7.1. OpenShift Data Foundation

OpenShift Data Foundation is a software-defined storage service for containers that provides block storage, file system storage, and on-premise object storage.

New in this release

- There are no reference design updates in this release.

Description

OpenShift Data Foundation is a software-defined storage service for containers. OpenShift Data Foundation can be deployed in one of two modes:

- Internal mode, where OpenShift Data Foundation software components are deployed as software containers directly on the OpenShift cluster nodes, together with other containerized applications.
- External mode, where OpenShift Data Foundation is deployed on a dedicated storage cluster, which is usually a separate Red Hat Ceph Storage cluster running on Red Hat Enterprise Linux. These storage services are running externally to the application workload cluster.

For telco core clusters, storage support is provided by OpenShift Data Foundation storage services running in external mode, for several reasons:

- Separating dependencies between OpenShift Container Platform and Ceph operations allows for independent OpenShift Container Platform and OpenShift Data Foundation updates.
- Separation of operations functions for the Storage and OpenShift Container Platform infrastructure layers, is a typical customer requirement for telco core use cases.
- External Red Hat Ceph Storage clusters can be re-used by multiple OpenShift Container Platform clusters deployed in the same region.

OpenShift Data Foundation supports separation of storage traffic using secondary CNI networks.

Limits and requirements

- In an IPv4/IPv6 dual-stack networking environment, OpenShift Data Foundation uses IPv4 addressing. For more information, see [IPv6 support](#).

Engineering considerations

- OpenShift Data Foundation network traffic should be isolated from other traffic on a dedicated network, for example, by using VLAN isolation.
- Workload requirements must be scoped before attaching multiple OpenShift Container Platform clusters to an external OpenShift Data Foundation cluster to ensure enough throughput, bandwidth, and performance KPIs.

3.8.7.2. Additional storage solutions

You can use other storage solutions to provide persistent storage for telco core clusters. The configuration and integration of these solutions is outside the scope of the reference design specifications (RDS).

Integration of the storage solution into the telco core cluster must include proper sizing and performance analysis to ensure the storage meets overall performance and resource usage requirements.

3.9. TELCO CORE DEPLOYMENT COMPONENTS

The following sections describe the various OpenShift Container Platform components and configurations that you use to configure the hub cluster with Red Hat Advanced Cluster Management (RHACM).

3.9.1. Red Hat Advanced Cluster Management

RHACM provides Multi Cluster Engine (MCE) installation and ongoing lifecycle management for deployed clusters.

New in this release

- There are no reference design updates in this release.

Description

RHACM provides Multi Cluster Engine (MCE) installation and ongoing GitOps ZTP lifecycle management for deployed clusters. You manage cluster configuration and upgrades declaratively by applying **Policy** custom resources (CRs) to clusters during maintenance windows.

You apply policies with the RHACM policy controller as managed by TALM. Configuration, upgrades, and cluster status are managed through the policy controller.

When installing managed clusters, RHACM applies labels and initial ignition configuration to individual nodes in support of custom disk partitioning, allocation of roles, and allocation to machine config pools. You define these configurations with **SiteConfig** or **ClusterInstance** CRs.

Limits and requirements

- Hub cluster sizing is discussed in [Sizing your cluster](#).
- RHACM scaling limits are described in [Performance and Scalability](#).

Engineering considerations

- When managing multiple clusters with unique content per installation, site, or deployment, using RHACM hub templating is strongly recommended. With RHACM hub templating, you can apply a consistent set of policies to clusters while providing unique values per installation.

Additional resources

- [Using GitOps ZTP to provision clusters at the network far edge](#)
- [Red Hat Advanced Cluster Management for Kubernetes](#)

3.9.2. Topology Aware Lifecycle Manager

TALM manages how changes are rolled out to managed clusters in the network.

New in this release

- There are no reference design updates in this release.

Description

TALM is an Operator that runs only on the hub cluster. TALM manages how changes including cluster and Operator upgrades, configurations, and so on, are rolled out to managed clusters in the network. TALM has the following core features:

- Provides sequenced updates of cluster configurations and upgrades (OpenShift Container Platform and Operators) as defined by cluster policies.
- Provides for deferred application of cluster updates.
- Supports progressive rollout of policy updates to sets of clusters in user configurable batches.
- Allows for per-cluster actions by adding **ztp-done** or similar user-defined labels to clusters.

Limits and requirements

- Supports concurrent cluster deployments in batches of 400

Engineering considerations

- Only policies with the **ran.openshift.io/ztp-deploy-wave** annotation are applied by TALM during initial cluster installation.
- Any policy can be remediated by TALM under control of a user created **ClusterGroupUpgrade** CR.
- Set the **MachineConfigPool (mcp)** CR **paused** field to true during a cluster upgrade maintenance window and set the **maxUnavailable** field to the maximum tolerable value. This prevents multiple cluster node reboots during upgrade, which results in a shorter overall upgrade. When you unpause the **mcp** CR, all the configuration changes are applied with a single reboot.



NOTE

During installation, custom **mcp** CRs can be paused along with setting **maxUnavailable** to 100% to improve installation times.

- You can orchestrate upgrades for OpenShift Container Platform, Day 2 OLM operators, and custom configurations using a ClusterGroupUpgrade (CGU) CR that defines the required policies.
 - An EUS to EUS upgrade can be orchestrated using chained CGU CRs.
 - Control of MCP pause can be managed through policy in the CGU CRs for a full control plane and worker node rollout of upgrades.

- For more information, see "Performing an EUS-to-EUS update for telco core clusters".

Additional resources

- [Updating managed clusters with the Topology Aware Lifecycle Manager](#)

3.9.3. GitOps Operator and GitOps ZTP plugins

The GitOps Operator provides a GitOps-driven infrastructure for managing cluster deployment and configuration.

New in this release

- There are no reference design updates in this release.

Description

The GitOps Operator provides a GitOps driven infrastructure for managing cluster deployment and configuration. Cluster definitions and configuration are maintained in a Git repository.

The SiteConfig Operator generates installation CRs from **ClusterInstance** CRs.



IMPORTANT

From OpenShift Container Platform 4.21, you must use **ClusterInstance** CRs and the SiteConfig Operator to define managed cluster installations. With this release, support for defining managed cluster installations with the **SiteConfig** CR is removed.

GitOps ZTP plugins provide support for automatically wrapping configuration CRs in policies based on RHACM **PolicyGenerator** CRs.

You should structure the Git repository according to the release version, with all necessary artifacts added to the repository, such as the **ClusterInstance**, **PolicyGenerator**, and supporting reference CRs. This structure enables deploying and managing multiple versions of the OpenShift Container Platform and configuration versions to clusters simultaneously and through upgrades.

The recommended Git structure keeps reference CRs in a directory separate from customer or partner provided content. This means that you can import reference updates by simply overwriting existing content. Customer or partner supplied CRs can be provided in a parallel directory to the reference CRs for easy inclusion in the generated configuration policies.

Limits and requirements

- Each ArgoCD application supports up to 1000 nodes on a hub cluster conforming to the Hub RDS. Multiple ArgoCD applications can be used to achieve the maximum number of clusters supported by a single hub cluster.
- Use the **extraManifestsRefs** field in the **ClusterInstance** CR to reference **ConfigMap** resources that contain additional manifests to apply at install time.

Engineering considerations

- To avoid confusion or unintentional overwriting when updating content, you should use unique and distinguishable names for custom CRs in the **reference-crs/** directory under core-overlay and extra manifests in git.

- The **ClusterInstance** CR allows multiple **ConfigMap** references in the **extraManifestsRefs** field for additional install-time manifests.

Additional resources

- [Preparing the GitOps ZTP site configuration repository for version independence](#)
- [Adding custom content to the GitOps ZTP pipeline](#)

3.9.4. Agent-based Installer

The Agent-based Installer provides an installation method for OpenShift Container Platform in environments without existing deployment infrastructure.

New in this release

- There are no reference design updates in this release.

Description

The recommended method for Telco Core cluster installation is using Red Hat Advanced Cluster Management. The Agent Based Installer (ABI) is a separate installation flow for Openshift in environments without existing infrastructure for running cluster deployments. Use the ABI to install OpenShift Container Platform on bare-metal servers without requiring additional servers or VMs for managing the installation, but does not provide ongoing lifecycle management, monitoring or automations. The ABI can be run on any system for example, from a laptop to generate an ISO installation image. The ISO is used as the installation media for the cluster control plane nodes. You can monitor the progress by using the ABI from any system with network connectivity to the control plane node's API interfaces.

ABI supports the following:

- Installation from declarative CRs
- Installation in disconnected environments
- No additional servers required to support installation, for example, the bastion node is no longer needed

Limits and requirements

- Disconnected installation requires a registry with all required content mirrored and reachable from the installed host.

Engineering considerations

- Networking configuration should be applied as NMState configuration during installation as opposed to Day 2 configuration using the NMState Operator.

Additional resources

- [Installing an OpenShift Container Platform cluster with the Agent-based Installer](#)

3.10. MONITORING

The Cluster Monitoring Operator (CMO) is included by default in OpenShift Container Platform and provides monitoring for the platform components and optionally user projects.

New in this release

- There are no reference design updates in this release.

Description

The Cluster Monitoring Operator (CMO) is included by default in OpenShift Container Platform and provides monitoring (metrics, dashboards, and alerting) for the platform components and optionally user projects. You can customize the default log retention period, custom alert rules, and so on. Configuration of the monitoring stack is done through a single string value in the cluster-monitoring-config ConfigMap. The reference tuning merges content from two requirements:

- Prometheus configuration is extended to forward alerts to the Red Hat Advanced Cluster Management (RHACM) hub cluster for alert aggregation. If required, you can extend this configuration to forward alerts to additional locations.
- Prometheus retention period is reduced from the default. The primary metrics storage is typically external to the cluster. Metrics storage on the core cluster is typically a backup to that central store and available for local troubleshooting purposes. You can add a **remoteWrite** endpoint to the Prometheus configuration to directly forward metrics to the central store.

In addition to the default configuration, telco core clusters typically configure pod CPU and memory metrics and alerts for user workloads.

Engineering considerations

- The Prometheus retention period is specified by the user. The value used is a tradeoff between operational requirements for maintaining historical data on the cluster against CPU and storage resources. Longer retention periods increase the need for storage and require additional CPU to manage the indexing of data.

Additional resources

- [About OpenShift Container Platform monitoring](#)

3.11. SCHEDULING

The scheduler is a cluster-wide component responsible for selecting the right node for a given workload.

New in this release

- There are no reference design updates in this release.

Description

The scheduler is a cluster-wide component responsible for selecting the right node for a given workload. It is a core part of the platform and does not require any specific configuration in the common deployment scenarios. However, there are few specific use cases described in the following section.

NUMA-aware scheduling can be enabled through the NUMA Resources Operator. For more information, see "Scheduling NUMA-aware workloads".

Limits and requirements

- The default scheduler does not understand the NUMA locality of workloads. It only knows about the sum of all free resources on a worker node. This might cause workloads to be rejected when scheduled to a node with the topology manager policy set to single-numa-node or restricted. For more information, see "Topology Manager policies"..
 - For example, consider a pod requesting 6 CPUs and being scheduled to an empty node that has 4 CPUs per NUMA node. The total allocatable capacity of the node is 8 CPUs. The scheduler places the pod on the empty node. The node local admission fails, as there are only 4 CPUs available in each of the NUMA nodes.
- All clusters with multi-NUMA nodes are required to use the NUMA Resources Operator. See "Installing the NUMA Resources Operator" for more information. Use the **machineConfigPoolSelector** field in the **KubeletConfig** CR to select all nodes where NUMA aligned scheduling is required.
- All machine config pools must have consistent hardware configuration. For example, all nodes are expected to have the same NUMA zone count.

Engineering considerations

- Pods might require annotations for correct scheduling and isolation. For more information about annotations, see "CPU partitioning and performance tuning".
- You can configure SR-IOV virtual function NUMA affinity to be ignored during scheduling by using the `excludeTopology` field in **SriovNetworkNodePolicy** CR.

Additional resources

- [Installing the NUMA Resources Operator](#)
- [Scheduling NUMA-aware workloads](#)
- [Topology Manager policies](#)

3.12. NODE CONFIGURATION

Node configuration for telco core clusters includes additional kernel modules, container mount namespace settings, and kdump configuration.

New in this release

- There are no reference design updates in this release.

Limits and requirements

- Analyze additional kernel modules to determine impact on CPU load, system performance, and ability to meet KPIs.

Table 3.1. Additional kernel modules

Feature	Description
Additional kernel modules	Install the following kernel modules by using MachineConfig CRs to provide extended kernel functionality to CNFs. ◦ sctp ◦ ip_gre ◦ nf_tables ◦ nf_conntrack ◦ nft_ct ◦ nft_limit ◦ nft_log ◦ nft_nat ◦ nft_chain_nat ◦ nf_reject_ipv4 ◦ nf_reject_ipv6 ◦ nfnetlink_log
Container mount namespace hiding	Reduce the frequency of kubelet housekeeping and eviction monitoring to reduce CPU usage. Creates a container mount namespace, visible to kubelet/CRI-O, to reduce system mount scanning overhead.
Kdump enable	Optional configuration (enabled by default)

Additional resources

- [Automatic kernel crash dumps with kdump](#)
- [Optimizing CPU usage with mount namespace encapsulation](#)

3.12.1. Host firmware and boot loader configuration

Configure host firmware and boot loader settings to support telco core cluster requirements.

New in this release

- There are no reference design updates in this release.

Engineering considerations

- Enabling secure boot is the recommended configuration.



NOTE

When secure boot is enabled, only signed kernel modules are loaded by the kernel. Out-of-tree drivers are not supported.

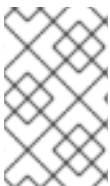
3.12.2. Kubelet Settings

Some CNF workloads make use of sysctls which are not in the list of system-wide safe **sysctls**. Generally network sysctls are namespaced and can be enabled by using the **kubeletconfig.experimental** annotation in the PerformanceProfile as a string of JSON in the form **allowedUnsafeSysctls**.

Additionally, the **systemReserved** memory can be configured through the same **kubeletconfig.experimental** annotation to reserve memory for system daemons and kernel processes.

Example snippet showing allowedUnsafeSysctls and systemReserved

```
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: {{ .metadata.name }}
annotations:
  # allowedUnsafeSysctls: some pods want the kernel stack to ignore IPv6 router Advertisement.
  # systemReserved: when used, it should be tailored for each environment.
  kubeletconfig.experimental: |
    {
      "allowedUnsafeSysctls":["net.ipv6.conf.all.accept_ra"],
      "systemReserved":{"memory":"11Gi"}
    }
```



NOTE

Although these are namespaced they may allow a pod to consume memory or other resources beyond any limits specified in the pod description. You must ensure that these **sysctls** do not exhaust platform resources.

3.13. DISCONNECTED ENVIRONMENT

Telco core clusters are expected to be installed in networks without direct access to the internet.

New in this release

- Added requirement to use the **oc mirror** plugin v2 for mirroring image signatures in disconnected environments.

Description

Telco core clusters are expected to be installed in networks without direct access to the internet. All container images needed to install, configure, and operate the cluster must be available in a

disconnected registry. This includes OpenShift Container Platform images, Day 2 OLM Operator images, and application workload images. The use of a disconnected environment provides multiple benefits, including:

- Security - limiting access to the cluster
- Curated content - the registry is populated based on curated and approved updates for clusters

Limits and requirements

- A unique name is required for all custom **CatalogSource** resources. Do not reuse the default catalog names.

Engineering considerations

- A valid time source must be configured as part of cluster installation.
- In OpenShift Container Platform 4.22 and later, pulling OpenShift images from a disconnected mirror registry requires copying the image signatures into that registry during the mirroring process. The **oc adm mirror** command does not mirror signatures and must not be used. Instead, use the **oc mirror** plugin v2 to ensure signatures are properly mirrored.

Additional resources

- [Mirroring images for a disconnected installation by using the oc-mirror plugin v2](#)
- [About cluster updates in a disconnected environment](#)
- [Using sysctl in containers](#)

3.14. SECURITY

Telco core clusters require hardening against multiple attack vectors through various security features and configurations.

New in this release

- Starting with OpenShift Container Platform 4.22, administrators can use the **oc commatrix** plugin to automatically generate **MachineConfig** CRs with a default set of nftables firewall rules. The generated **MachineConfig** CRs can then be added to the cluster configuration to improve security.

Description

Telco customers are security conscious and require clusters to be hardened against multiple attack vectors. In OpenShift Container Platform, there is no single component or feature responsible for securing a cluster. Described below are various security oriented features and configurations for the use models covered in the telco core RDS.

- SecurityContextConstraints (SCC): All workload pods should be run with **restricted-v2** or **restricted** SCC.
- Seccomp: All pods should run with the **RuntimeDefault** (or stronger) seccomp profile.

- Rootless DPDK pods: Many user-plane networking (DPDK) CNFs require pods to run with root privileges. With this feature, a conformant DPDK pod can be run without requiring root privileges. Rootless DPDK pods create a tap device in a rootless pod that injects traffic from a DPDK application to the kernel.
- Storage: The storage network should be isolated and non-routable to other cluster networks. See the "Storage" section for additional details.

Security-conscious cluster administrators can configure nftables rules to restrict inbound network flows. See [the OpenShift Container Platform documentation](#) for a supported method for implementing custom nftables firewall rules in OpenShift Container Platform cluster nodes with the **oc commatrix** plugin. You can add the generated **MachineConfig** CRs to the cluster configuration at install time as extra manifests. When the cluster is managed under a Hub RDS compliant RHACM cluster the content of the **MachineConfig** resource is maintained across the life of the cluster through a **Policy** resource.

It is crucial to carefully consider the operational implications before deploying this method, including:

- Early application: The rules are applied at boot time, before the network is fully operational. Ensure the rules do not inadvertently block essential services required during the boot process.
- Risk of misconfiguration: Errors in your custom rules can lead to unintended consequences, potentially leading to performance impact or blocking legitimate traffic or isolating nodes. Thoroughly test your rules in a non-production environment before deploying them to your main cluster.
- External endpoints: OpenShift Container Platform requires access to external endpoints to function. For more information about the firewall allowlist, see "Configuring your firewall for OpenShift Container Platform". Ensure that cluster nodes are permitted access to those endpoints.
- Node reboot: Unless node disruption policies are configured, applying the **MachineConfig** CR with the required firewall settings causes a node reboot. Be aware of this impact and schedule a maintenance window accordingly.



NOTE

Node disruption policies are available in OpenShift Container Platform 4.17 and later.

- Network flow matrix: For more information about managing ingress traffic, see "Network flow matrix". You can restrict ingress traffic to essential flows to improve network security. The matrix provides insights into base cluster services but excludes traffic generated by Day 2 Operators.
- Cluster version updates and upgrades: Exercise caution when updating or upgrading OpenShift Container Platform clusters. Recent changes to the platform's firewall requirements might require adjustments to network port permissions. While the documentation provides guidelines, note that these requirements can evolve over time. To minimize disruptions, you should test any updates or upgrades in a staging environment before applying them in production. This helps you to identify and address potential compatibility issues related to firewall configuration changes.

Limits and requirements

- Rootless DPDK pods requires the following additional configuration:
 - Configure the **container_t** SELinux context for the tap plugin.
 - Enable the **container_use_devices** SELinux boolean for the cluster host.

Engineering considerations

- For rootless DPDK pod support, enable the SELinux **container_use_devices** boolean on the host to allow the tap device to be created. This introduces an acceptable security risk.

Additional resources

- [Configuring your firewall for OpenShift Container Platform](#)
- [OpenShift Container Platform network flow matrix](#)
- [Managing security context constraints](#)
- [Using node disruption policies to minimize disruption from machine config changes](#)

3.15. CERT-MANAGER OPERATOR

The cert-manager Operator for OpenShift Container Platform manages the lifecycle of TLS certificates for cluster components and workloads.

New in this release

- The cert-manager Operator is a new optional component in this release.

Description

The cert-manager Operator for OpenShift Container Platform manages the lifecycle of TLS certificates for cluster components and workloads. The cert-manager Operator automates certificate issuance, renewal, and rotation, eliminating manual certificate management. The reference configuration includes the cert-manager Operator to optionally manage certificates for the API server and ingress controller endpoints.

Limits and requirements

- The reference configuration includes only the ACME DNS01 challenge type for platform certificate issuance.

Engineering considerations

- Use RHACM **CertificatePolicy** resources on the hub cluster to monitor certificate expiration and compliance across managed clusters.

3.16. SCALABILITY

Telco core clusters can scale to at least 120 nodes to support large-scale telco application workloads.

New in this release

- There are no reference design updates in this release.

Description

Scale clusters as described in "Limits and requirements". Scaling of workloads is described in "Application workloads".

Limits and requirements

- A telco core cluster can scale to at least 120 nodes.

3.17. TELCO CORE REFERENCE CONFIGURATION CRS

Use the following custom resources (CRs) to configure and deploy OpenShift Container Platform clusters with the telco core profile. Use the CRs to form the common baseline used in all the specific use models unless otherwise indicated.

3.17.1. Extracting the telco core reference design configuration CRs

You can extract the complete set of custom resources (CRs) for the telco core profile from the **telco-core-rds-rhel9** container image. The container image has both the required CRs, and the optional CRs, for the telco core profile.

Prerequisites

- You have installed **podman**.

Procedure

1. Log on to the container image registry with your credentials by running the following command:

```
$ podman login registry.redhat.io
```

2. Extract the content from the **telco-core-rds-rhel9** container image by running the following commands:

```
$ mkdir -p ./out
```

```
$ podman run -it registry.redhat.io/openshift4/openshift-telco-core-rds-rhel9:v4.22 | base64 -d  
| tar xv -C out
```

Verification

- The **out** directory has the following directory structure. You can view the telco core CRs in the **out/telco-core-rds/** directory by running the following command:

```
$ tree -L 4
```

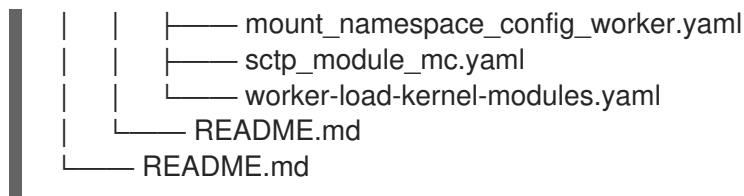
Example output

```
.  
├── configuration  
│   ├── compare.sh  
│   ├── core-baseline.yaml  
│   └── core-finish.yaml
```

```

|—— core-overlay.yaml
|—— core-upgrade.yaml
|—— kustomization.yaml
|—— Makefile
|—— ns.yaml
|—— README.md
|—— reference-crs
|   |—— custom-manifests
|   |   |—— mcp-worker-1.yaml
|   |   |—— mcp-worker-2.yaml
|   |   |—— mcp-worker-3.yaml
|   |   |—— README.md
|   |—— optional
|   |   |—— logging
|   |   |—— networking
|   |   |—— other
|   |   |—— tuning
|   |—— required
|   |   |—— networking
|   |   |—— other
|   |   |—— performance
|   |   |—— scheduling
|   |   |—— storage
|—— reference-crs-kube-compare
|   |—— compare_ignore
|   |—— comparison-overrides.yaml
|   |—— metadata.yaml
|   |—— optional
|   |   |—— logging
|   |   |—— networking
|   |   |—— other
|   |   |—— tuning
|   |—— ReferenceVersionCheck.yaml
|   |—— required
|   |   |—— networking
|   |   |—— other
|   |   |—— performance
|   |   |—— scheduling
|   |   |—— storage
|   |—— unordered_list.tmpl
|   |—— version_match.tmpl
|—— template-values
|   |—— hw-types.yaml
|   |—— regional.yaml
|—— install
|   |—— custom-manifests
|   |   |—— mcp-worker-1.yaml
|   |   |—— mcp-worker-2.yaml
|   |   |—— mcp-worker-3.yaml
|   |—— example-standard.yaml
|   |—— extra-manifests
|   |   |—— control-plane-load-kernel-modules.yaml
|   |   |—— kdump-master.yaml
|   |   |—— kdump-worker.yaml
|   |   |—— mc_rootless_pods_selinux.yaml
|   |   |—— mount_namespace_config_master.yaml

```



3.17.2. Comparing a cluster with the telco core reference configuration

After you deploy a telco core cluster, you can use the **cluster-compare** plugin to assess the cluster's compliance with the telco core reference design specifications (RDS). The **cluster-compare** plugin is an OpenShift CLI (**oc**) plugin. The plugin uses a telco core reference configuration to validate the cluster with the telco core custom resources (CRs).

The plugin-specific reference configuration for telco core is packaged in a container image with the telco core CRs.

For further information about the **cluster-compare** plugin, see "Understanding the cluster-compare plugin".

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have credentials to access the **registry.redhat.io** container image registry.
- You installed the **cluster-compare** plugin.

Procedure

1. Log on to the container image registry with your credentials by running the following command:

```
$ podman login registry.redhat.io
```

2. Extract the content from the **telco-core-rds-rhel9** container image by running the following commands:

```
$ mkdir -p ./out
```

```
$ podman run -it registry.redhat.io/openshift4/openshift-telco-core-rds-rhel9:v4.22 | base64 -d  
| tar xv -C out
```

You can view the reference configuration in the **out/telco-core-rds/configuration/reference-crs-kube-compare** directory by running the following command:

```
$ tree -L 2
```

3. Example output

```
.
├── compare_ignore
├── comparison-overrides.yaml
├── metadata.yaml
└── optional
```

```

|   |—— logging
|   |—— networking
|   |—— other
|   |—— tuning
|—— ReferenceVersionCheck.yaml
|—— required
|   |—— networking
|   |—— other
|   |—— performance
|   |—— scheduling
|   |—— storage
|—— unordered_list.tmpl
|—— version_match.tmpl

```

- **metadata.yaml** is the configuration file for the reference configuration.
 - **optional** is the directory for optional templates.
 - **required** is the directory for required templates.
4. Compare the configuration for your cluster to the telco core reference configuration by running the following command:

```
$ oc cluster-compare -r out/telco-core-rds/configuration/reference-crs-kube-compare/metadata.yaml
```

Example output

```

W1212 14:13:06.281590 36629 compare.go:425] Reference Contains Templates With
Types (kind) Not Supported By Cluster: BFDProfile, BGPPeer, BGPAdvertisement,
ClusterLogForwarder, Community, IPAddressPool, MetalLB, MultiNetworkPolicy, NMState,
NUMAResourcesOperator, NUMAResourcesScheduler, NodeNetworkConfigurationPolicy,
SriovNetwork, SriovNetworkNodePolicy, SriovOperatorConfig, StorageCluster

...

*****

Cluster CR: config.openshift.io/v1_OperatorHub_cluster
Reference File: required/other/operator-hub.yaml
Diff Output: diff -u -N /tmp/MERGED-2801470219/config-openshift-io-v1_operatorhub_cluster
/tmp/LIVE-2569768241/config-openshift-io-v1_operatorhub_cluster
--- /tmp/MERGED-2801470219/config-openshift-io-v1_operatorhub_cluster 2024-12-12
14:13:22.898756462 +0000
+++ /tmp/LIVE-2569768241/config-openshift-io-v1_operatorhub_cluster 2024-12-12
14:13:22.898756462 +0000
@@ -1,6 +1,6 @@
apiVersion: config.openshift.io/v1
kind: OperatorHub
metadata:
+ annotations:
+ include.release.openshift.io/hypershift: "true"
  name: cluster
-spec:
- disableAllDefaultSources: true

```

Summary

CRs with diffs: 3/4

CRs in reference missing from the cluster: 22

other:

other:

Missing CRs:

- optional/other/control-plane-load-kernel-modules.yaml
- optional/other/worker-load-kernel-modules.yaml

required-networking:

networking-root:

Missing CRs:

- required/networking/nodeNetworkConfigurationPolicy.yaml

networking-sriov:

Missing CRs:

- required/networking/sriov/sriovNetwork.yaml
- required/networking/sriov/sriovNetworkNodePolicy.yaml
- required/networking/sriov/SriovOperatorConfig.yaml
- required/networking/sriov/SriovSubscription.yaml
- required/networking/sriov/SriovSubscriptionNS.yaml
- required/networking/sriov/SriovSubscriptionOperGroup.yaml

required-other:

scheduling:

Missing CRs:

- required/other/catalog-source.yaml
- required/other/icsp.yaml

required-performance:

performance:

Missing CRs:

- required/performance/PerformanceProfile.yaml

required-scheduling:

scheduling:

Missing CRs:

- required/scheduling/nrop.yaml
- required/scheduling/NROPSubscription.yaml
- required/scheduling/NROPSubscriptionNS.yaml
- required/scheduling/NROPSubscriptionOperGroup.yaml
- required/scheduling/sched.yaml

required-storage:

storage-odf:

Missing CRs:

- required/storage/odf-external/01-rook-ceph-external-cluster-details.secret.yaml
- required/storage/odf-external/02-ocs-external-storagecluster.yaml
- required/storage/odf-external/odfNS.yaml
- required/storage/odf-external/odfOperGroup.yaml
- required/storage/odf-external/odfSubscription.yaml

No CRs are unmatched to reference CRs

Metadata Hash:

fe41066bac56517be02053d436c815661c9fa35eec5922af25a1be359818f297

No patched CRs

Where:

Cluster CR

The CR under comparison. The plugin displays each CR with a difference from the corresponding template.

Reference File

The template matching with the CR for comparison.

Diff Output

The output in Linux diff format shows the difference between the template and the cluster CR.

Summary

After the plugin reports the line diffs for each CR, the summary of differences are reported.

CRs with diffs

The number of CRs in the comparison with differences from the corresponding templates.

CRs in reference missing from the cluster

The number of CRs represented in the reference configuration, but missing from the live cluster.

Missing CRs

The list of CRs represented in the reference configuration, but missing from the live cluster.

No CRs are unmatched to reference CRs

The CRs that did not match to a corresponding template in the reference configuration.

Metadata Hash

Identifies the reference configuration.

No patched CRs

The list of patched CRs.

Additional resources

- [Understanding the cluster-compare plugin](#)

3.17.3. Node configuration reference CRs

Use the following custom resources (CRs) to configure node settings for the telco core profile.

Table 3.2. Node configuration CRs

Component	Reference CR	Description	Optional
Additional kernel modules	control-plane-load-kernel-modules.yaml	Optional. Configures the kernel modules for control plane nodes.	No
Additional kernel modules	sctp_module_mc.yaml	Optional. Loads the SCTP kernel module in worker nodes.	No
Additional kernel modules	worker-load-kernel-modules.yaml	Optional. Configures kernel modules for worker nodes.	No

Component	Reference CR	Description	Optional
Container mount namespace hiding	mount_namespace_config_master.yaml	Configures a mount namespace for sharing container-specific mounts between kubelet and CRI-O on control plane nodes.	No
Container mount namespace hiding	mount_namespace_config_worker.yaml	Configures a mount namespace for sharing container-specific mounts between kubelet and CRI-O on worker nodes.	No
Kdump enable	kdump-master.yaml	Configures kdump crash reporting on master nodes.	No
Kdump enable	kdump-worker.yaml	Configures kdump crash reporting on worker nodes.	No

3.17.4. Cluster infrastructure reference CRs

Use the following custom resources (CRs) to configure cluster infrastructure components for the telco core profile.

Table 3.3. Cluster infrastructure CRs

Component	Reference CR	Description	Optional
Cluster logging	ClusterLogForwarder.yaml	Configures a log forwarding instance with the specified service account and verifies that the configuration is valid.	Yes
Cluster logging	ClusterLogNS.yaml	Configures the cluster logging namespace.	Yes
Cluster logging	ClusterLogOperatorGroup.yaml	Creates the Operator group in the openshift-logging namespace, allowing the Cluster Logging Operator to watch and manage resources.	Yes

Component	Reference CR	Description	Optional
Cluster logging	ClusterLogServiceAccount.yaml	Configures the cluster logging service account.	Yes
Cluster logging	ClusterLogServiceAccountAuditBinding.yaml	Grants the collect-audit-logs cluster role to the logs collector service account.	Yes
Cluster logging	ClusterLogServiceAccountInfrastructureBinding.yaml	Allows the collector service account to collect logs from infrastructure resources.	Yes
Cluster logging	ClusterLogSubscription.yaml	Creates a subscription resource for the Cluster Logging Operator with manual approval for install plans.	Yes
Disconnected configuration	catalog-source.yaml	Defines a disconnected Red Hat Operators catalog.	No
Disconnected configuration	idms.yaml	Defines a list of mirrored repository digests for the disconnected registry.	No
Disconnected configuration	operator-hub.yaml	Defines an OperatorHub configuration which disables all default sources.	No
Monitoring and observability	monitoring-config-cm.yaml	Configuring storage and retention for Prometheus and Alertmanager.	Yes
Power management	PerformanceProfile.yaml	Defines a performance profile resource, specifying CPU isolation, hugepages configuration, and workload hints for performance optimization on selected nodes.	No

3.17.5. Resource tuning reference CRs

Use the following custom resources (CRs) to configure resource tuning for the telco core profile.

Table 3.4. Resource tuning CRs

Component	Reference CR	Description	Optional
System reserved capacity	control-plane-system-reserved.yaml	Optional. Configures kubelet, enabling auto-sizing reserved resources for the control plane node pool.	No

3.17.6. Networking reference CRs

Use the following custom resources (CRs) to configure networking components for the telco core profile.

Table 3.5. Networking CRs

Component	Reference CR	Description	Optional
Baseline	Network.yaml	Configures the default cluster network, specifying OVN Kubernetes settings like routing via the host. It also allows the definition of additional networks, including custom CNI configurations, and enables the use of MultiNetworkPolicy CRs for network policies across multiple networks.	No
Baseline	networkAttachmentDefinition.yaml	Optional. Defines a NetworkAttachmentDefinition resource specifying network configuration details such as node selector and CNI configuration.	No
Load Balancer	addr-pool.yaml	Configures MetalLB to manage a pool of IP addresses with auto-assign enabled for dynamic allocation of IPs from the specified range.	No

Component	Reference CR	Description	Optional
Load Balancer	bfd-profile.yaml	Configures bidirectional forwarding detection (BFD) with customized intervals, detection multiplier, and modes for quicker network fault detection and load balancing failover.	No
Load Balancer	bgp-advr.yaml	Defines a BGP advertisement resource for MetalLB, specifying how an IP address pool is advertised to BGP peers. This enables fine-grained control over traffic routing and announcements.	No
Load Balancer	bgp-peer.yaml	Defines a BGP peer in MetalLB, representing a BGP neighbor for dynamic routing.	No
Load Balancer	community.yaml	Defines a MetalLB community, which groups one or more BGP communities under a named resource. Communities can be applied to BGP advertisements to control routing policies and change traffic routing.	No
Load Balancer	metallb.yaml	Defines the MetalLB resource in the cluster.	No
Load Balancer	metallbNS.yaml	Defines the metallb-system namespace in the cluster.	No
Load Balancer	metallbOperatorGroup.yaml	Defines the Operator group for the MetalLB Operator.	No

Component	Reference CR	Description	Optional
Load Balancer	metallbSubscription.yaml	Creates a subscription resource for the MetalLB Operator with manual approval for install plans.	No
Multus - Tap CNI for rootless DPDK pods	mc_rootless_pods_selinux.yaml	Configures a MachineConfig resource which sets an SELinux boolean for the tap CNI plugin on worker nodes.	Yes
NMState Operator	NMState.yaml	Defines an NMState resource that is used by the NMState Operator to manage node network configurations.	No
NMState Operator	NMStateNS.yaml	Creates the NMState Operator namespace.	No
NMState Operator	NMStateOperGroup.yaml	Creates the Operator group in the openshift-nmstate namespace, allowing the NMState Operator to watch and manage resources.	No
NMState Operator	NMStateSubscription.yaml	Creates a subscription for the NMState Operator, managed through OLM.	No
SR-IOV Network Operator	sriovNetwork.yaml	Defines an SR-IOV network specifying network capabilities, IP address management (ipam), and the associated network namespace and resource.	No

Component	Reference CR	Description	Optional
SR-IOV Network Operator	sriovNetworkNodePolicy.yaml	Configures network policies for SR-IOV devices on specific nodes, including customization of device selection, VF allocation (numVfs), node-specific settings (nodeSelector), and priorities.	No
SR-IOV Network Operator	SriovOperatorConfig.yaml	Configures various settings for the SR-IOV Operator, including enabling the injector and Operator webhook, disabling pod draining, and defining the node selector for the configuration daemon.	No
SR-IOV Network Operator	SriovSubscription.yaml	Creates a subscription for the SR-IOV Network Operator, managed through OLM.	No
SR-IOV Network Operator	SriovSubscriptionNS.yaml	Creates the SR-IOV Network Operator subscription namespace.	No
SR-IOV Network Operator	SriovSubscriptionOperatorGroup.yaml	Creates the Operator group for the SR-IOV Network Operator, allowing it to watch and manage resources in the target namespace.	No

3.17.7. Scheduling reference CRs

Use the following custom resources (CRs) to configure scheduling for the telco core profile.

Table 3.6. Scheduling CRs

Component	Reference CR	Description	Optional
-----------	--------------	-------------	----------

Component	Reference CR	Description	Optional
NUMA-aware scheduler	nrop.yaml	Enables the NUMA Resources Operator, aligning workloads with specific NUMA node configurations. Required for clusters with multi-NUMA nodes.	No
NUMA-aware scheduler	NROPSubscription.yaml	Creates a subscription for the NUMA Resources Operator, managed through OLM. Required for clusters with multi-NUMA nodes.	No
NUMA-aware scheduler	NROPSubscriptionNamespace.yaml	Creates the NUMA Resources Operator subscription namespace. Required for clusters with multi-NUMA nodes.	No
NUMA-aware scheduler	NROPSubscriptionOperatorGroup.yaml	Creates the Operator group in the numaresources-operator namespace, allowing the NUMA Resources Operator to watch and manage resources. Required for clusters with multi-NUMA nodes.	No
NUMA-aware scheduler	sched.yaml	Configures a topology-aware scheduler in the cluster that can handle NUMA aware scheduling of pods across nodes.	No
NUMA-aware scheduler	Scheduler.yaml	Configures control plane nodes as non-schedulable for workloads.	No

3.17.8. Storage reference CRs

Use the following custom resources (CRs) to configure storage for the telco core profile.

Table 3.7. Storage CRs

Component	Reference CR	Description	Optional
External ODF configuration	01-rook-ceph-external-cluster-details.secret.yaml	Defines a Secret resource containing base64-encoded configuration data for an external Ceph cluster in the openshift-storage namespace.	No
External ODF configuration	02-ocs-external-storagecluster.yaml	Defines an OpenShift Container Storage (OCS) storage resource which configures the cluster to use an external storage back end.	No
External ODF configuration	odfNS.yaml	Creates the monitored openshift-storage namespace for the OpenShift Data Foundation Operator.	No
External ODF configuration	odfOperatorGroup.yaml	Creates the Operator group in the openshift-storage namespace, allowing the OpenShift Data Foundation Operator to watch and manage resources.	No
External ODF configuration	odfSubscription.yaml	Creates the OpenShift Data Foundation Operator subscription, managed through OLM.	No

3.17.9. Security reference CRs

Use the following custom resources (CRs) to configure security components for the telco core profile.

Table 3.8. Security CRs

Component	Reference CR	Description	Optional
Cert-Manager	certManagerNS.yaml	Defines the cert-manager-operator namespace.	Yes

Component	Reference CR	Description	Optional
Cert-Manager	certManagerOperatorGroup.yaml	Defines the OperatorGroup for cert-manager.	Yes
Cert-Manager	certManagerSubscription.yaml	Installs the OpenShift cert-manager operator.	Yes
Cert-Manager	certManagerClusterIssuer.yaml	Configures an ACME ClusterIssuer using Let's Encrypt with DNS-01 challenge.	Yes
Cert-Manager	apiServerCertificate.yaml	Creates a certificate for the API Server endpoint.	Yes
Cert-Manager	ingressCertificate.yaml	Creates a wildcard certificate for the Ingress/Router.	Yes
Cert-Manager	apiServerConfig.yaml	Configures OpenShift to use the cert-manager generated API Server certificate.	Yes
Cert-Manager	ingressControllerConfig.yaml	Configures OpenShift to use the cert-manager generated Ingress certificate.	Yes

3.18. TELCO CORE REFERENCE CONFIGURATION SOFTWARE SPECIFICATIONS

The Red Hat telco core 4.22 solution has been validated using the following Red Hat software products for OpenShift Container Platform clusters.

Table 3.9. Telco core cluster validated software components

Component	Software version
Red Hat Advanced Cluster Management (RHACM)	2.17
Red Hat OpenShift GitOps	1.20
cert-manager Operator	1.19
Cluster Logging Operator	6.5

Component	Software version
OpenShift Data Foundation	4.22
SR-IOV Network Operator	4.22
MetalLB	4.22
NMState Operator	4.22
NUMA-aware scheduler	4.22

- OpenShift Data Foundation is expected to be updated to 4.22 when the aligned OpenShift Data Foundation version is released.
- The Cluster Logging Operator is expected to be updated to 6.6 when the aligned version is released.
- The cert-manager Operator and Red Hat OpenShift GitOps Operator are platform-agnostic operators. The support lifecycle for these operators is independent from the support lifecycle for OpenShift Container Platform. You might need to update to a newer minor version of these operators at the end of an operator lifecycle, or when planning to update the OpenShift Container Platform cluster to continue support. For support lifecycle details for platform-agnostic operators, see [OpenShift Operator Life Cycles](#).

CHAPTER 4. TELCO RAN DU REFERENCE DESIGN SPECIFICATION

The telco RAN DU reference design specifications (RDS) describes the configuration for clusters running on commodity hardware to host 5G workloads in the Radio Access Network (RAN). It captures the recommended, tested, and supported configurations to get reliable and repeatable performance for a cluster running the telco RAN DU profile.

Use the use model and system level information to plan telco RAN DU workloads, cluster resources, and minimum hardware specifications for managed single-node OpenShift clusters.

Specific limits, requirements, and engineering considerations for individual components are described in individual sections.

4.1. REFERENCE DESIGN SPECIFICATIONS FOR TELCO RAN DU 5G DEPLOYMENTS

Red Hat and certified partners offer deep technical expertise and support for networking and operational capabilities required to run telco applications on OpenShift Container Platform 4.22 clusters.

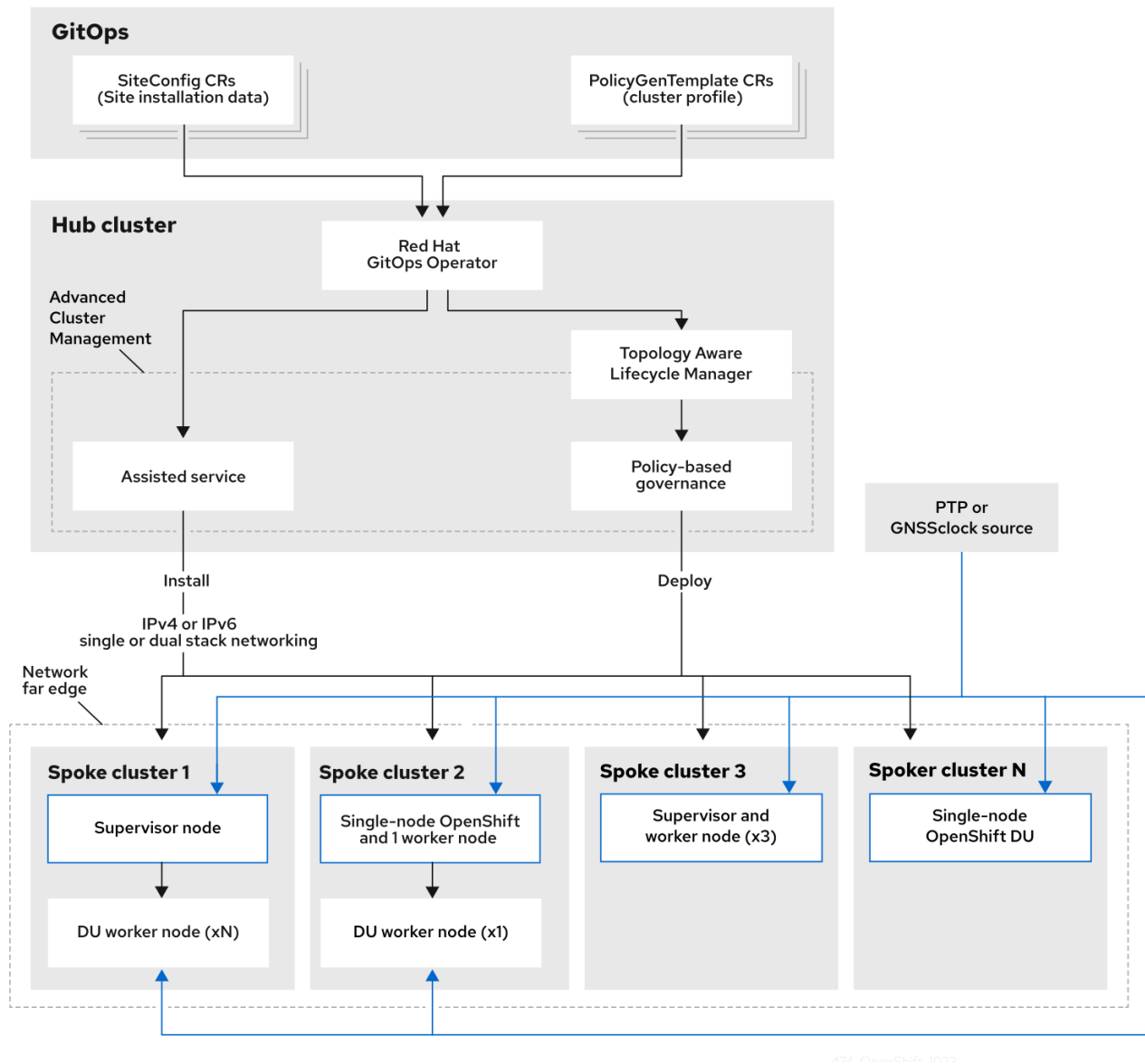
Red Hat's telco partners require a well-integrated, well-tested, and stable environment that can be replicated at scale for enterprise 5G solutions. The telco core and RAN DU reference design specifications (RDS) outline the recommended solution architecture based on a specific version of OpenShift Container Platform. Each RDS describes a tested and validated platform configuration for telco core and RAN DU use models. The RDS ensures an optimal experience when running your applications by defining the set of critical KPIs for telco 5G core and RAN DU. Following the RDS minimizes high severity escalations and improves application stability.

5G use cases are evolving and your workloads are continually changing. Red Hat is committed to iterating over the telco core and RAN DU RDS to support evolving requirements based on customer and partner feedback.

The reference configuration includes the configuration of the far edge clusters and hub cluster components.

The reference configurations in this document are deployed using a centrally managed hub cluster infrastructure as shown in the following image.

Figure 4.1. Telco RAN DU deployment architecture



4.1.1. Supported CPU architectures for RAN DU

Table 4.1. Supported CPU architectures for RAN DU

Architecture	Real-time kernel	Non-real-time kernel
x86_64	Yes	Yes
aarch64	No	Yes

+ * For **aarch64** architecture CPUs, the non-real-time configuration uses the standard kernel with a 64k page size.

4.2. REFERENCE DESIGN SCOPE

The telco core, telco RAN and telco hub reference design specifications (RDS) capture the recommended, tested, and supported configurations to get reliable and repeatable performance for clusters running the telco core and telco RAN profiles.

Each RDS includes the released features and supported configurations that are engineered and validated for clusters to run the individual profiles. The configurations provide a baseline OpenShift Container Platform installation that meets feature and KPI targets. Each RDS also describes expected variations for each individual configuration. Validation of each RDS includes many long duration and at-scale tests.

**NOTE**

The validated reference configurations are updated for each major Y-stream release of OpenShift Container Platform. Z-stream patch releases are periodically re-tested against the reference configurations.

4.3. DEVIATIONS FROM THE REFERENCE DESIGN

Deviating from the validated telco core, telco RAN DU, and telco hub reference design specifications (RDS) can have significant impact beyond the specific component or feature that you change. Deviations require analysis and engineering in the context of the complete solution.

**IMPORTANT**

All deviations from the RDS should be analyzed and documented with clear action tracking information. Due diligence is expected from partners to understand how to bring deviations into line with the reference design. This might require partners to provide additional resources to engage with Red Hat to work towards enabling their use case to achieve a best in class outcome with the platform. This is critical for the supportability of the solution and ensuring alignment across Red Hat and with partners.

Deviation from the RDS can have some or all of the following consequences:

- It can take longer to resolve issues.
- There is a risk of missing project service-level agreements (SLAs), project deadlines, end provider performance requirements, and so on.
- Unapproved deviations may require escalation at executive levels.

**NOTE**

Red Hat prioritizes the servicing of requests for deviations based on partner engagement priorities.

4.4. ENGINEERING CONSIDERATIONS FOR THE RAN DU USE MODEL

The RAN DU use model configures an OpenShift Container Platform cluster running on commodity hardware for hosting RAN distributed unit (DU) workloads. Model and system level considerations are described below. Specific limits, requirements and engineering considerations for individual components are detailed in later sections.

**NOTE**

For details of the telco RAN DU RDS KPI test results, see the [telco RAN DU 4.22 reference design specification KPI test results](#). This information is only available to customers and partners.

Cluster topology

The recommended topology for RAN DU workloads is single-node OpenShift. DU workloads may be run on other cluster topologies such as 3-node compact cluster, high availability (3 control plane + n worker nodes), or SNO+1 as needed. Multiple SNO clusters, or a highly-available 3-node compact cluster, are recommended over the SNO+1 topology.

Under the standard cluster topology case (3+n), a mixed architecture cluster is allowed only if:

- All control plane nodes are x86_64.
- All worker nodes are aarch64.

Remote worker node (RWN) cluster topologies are not recommended or included under this reference design specification. For workloads with high service level agreement requirements such as RAN DU the following drawbacks exclude RWN from consideration:

- No support for Image Based Upgrades and the benefits offered by that feature, such as faster upgrades and rollback capability.
- Updates to Day 2 operators affect all RWNs simultaneously without the ability to perform a rolling update.
- Loss of the control plane (disaster scenario) would have a significantly higher impact on overall service availability due to the greater number of sites served by that control plane.
- Loss of network connectivity between the RWN and the control plane for a period exceeding the monitoring grace period and toleration timeouts might result in pod eviction and lead to a service outage.
- No support for container image pre-caching.
- Additional complexities in workload affinities.

Supported cluster topologies for RAN DU

Table 4.2. Supported cluster topologies for RAN DU

Architecture	SNO	SNO+1	3-node	Standard	RWN
x86_64	Yes	Yes	Yes	Yes	No
aaarch64	Yes	No	No	No	No
mixed	N/A	No	No	Yes	No

- The standard mixed-architecture topology uses **x86_64** control plane nodes and **AArch64** worker nodes.

Workloads

1. DU workloads are described in [Telco RAN DU application workloads](#).
2. DU worker nodes are Intel 3rd Generation Xeon (Ice Lake) 2.20 GHz or newer with host firmware tuned for maximum performance.

Resources

The maximum number of running pods in the system, inclusive of application workload and OpenShift Container Platform pods, is 160.

Resource utilization

OpenShift Container Platform resource utilization varies depending on many factors such as the following application workload characteristics:

- Pod count

- Type and frequency of probes
- Messaging rates on the primary or secondary CNl with kernel networking
- API access rate
- Logging rates
- Storage IOPS

Resource utilization is measured for clusters configured as follows:

1. The cluster is a single host with single-node OpenShift installed.
2. The cluster runs the representative application workload described in "Reference application workload characteristics".
3. The cluster is managed under the constraints detailed in "Hub cluster management characteristics".
4. Components noted as "optional" in the use model configuration are not included.



NOTE

Configuration outside the scope of the RAN DU RDS that do not meet these criteria requires additional analysis to determine the impact on resource utilization and ability to meet KPI targets. You might need to allocate additional cluster resources to meet these requirements.

Reference application workload characteristics

1. Uses 75 pods across 5 namespaces with 4 containers per pod for the vRAN application including its management and control functions
2. Creates 30 **ConfigMap** CRs and 30 **Secret** CRs for each namespace



NOTE

The RDS validates mutable **ConfigMap** CRs. However, use immutable **ConfigMap** CRs where possible. Immutable resources significantly reduce the load on the API server by eliminating resource watches. A high volume of **ConfigMap** CRs, up to the validated 30 for vDU, might increase node recovery time during reboots, as volume mount points are recreated. The maximum size for each **ConfigMap** CR is limited to 1 MB.

3. Uses no exec probes
4. Uses a secondary network

**NOTE**

You can extract CPU load can from the platform metrics. For example:

```
$
query=avg_over_time(pod:container_cpu_usage:sum{namespace="openshift
-kube-apiserver"}[30m])
```

5. Application logs are not collected by the platform log collector.
6. Aggregate traffic on the primary CNI is up to 30 Mbps and up to 5 Gbps on the secondary network

Hub cluster management characteristics

RHACM is the recommended cluster management solution and is configured to these limits:

1. Use a maximum of 10 RHACM configuration policies, comprising 5 Red Hat provided policies and up to 5 custom configuration policies with a compliant evaluation interval of not less than 10 minutes.
2. Use a minimal number (up to 10) of managed cluster templates in cluster policies. Use hub-side templating.
3. Disable RHACM addons with the exception of the **policyController** and configure observability with the default configuration.

The following table describes resource utilization under reference application load.

Table 4.3. Resource utilization under reference application load

Metric	Limits	Notes
OpenShift platform CPU usage	Less than 4000mc – 2 cores (4HT)	Platform CPU is pinned to reserved cores, including both hyper-threads of each reserved core. The system is engineered to 3 CPUs (3000mc) at steady-state to allow for periodic system tasks and spikes.
OpenShift Platform memory	Less than 16G	

Disconnected environment

RAN DU clusters are typically deployed in disconnected environments without direct access to the internet. All container images needed to install, configure, and operate the cluster must be available in a disconnected registry.

In OpenShift Container Platform 4.22 and later, pulling OpenShift images from a disconnected mirror registry requires copying the image signatures into that registry during the mirroring process. The **oc adm mirror** command does not mirror signatures and must not be used. Instead, use the **oc mirror** plugin v2 to ensure signatures are properly mirrored.

Additional resources

- [Mirroring images for a disconnected installation by using the oc-mirror plugin v2](#)

4.5. TELCO RAN DU APPLICATION WORKLOADS

Develop RAN DU applications that are subject to the following requirements and limitations.

Description and limits

- Develop cloud-native network functions (CNFs) that conform to the latest version of [Red Hat best practices for Kubernetes](#).
- Use SR-IOV for high performance networking.
- For information on the decrease in the default maximum open files soft limit for containers, see the OpenShift Container Platform 4.21 release notes.
- Use exec probes sparingly and only when no other suitable options are available.
 - Do not use exec probes if a CNF uses CPU pinning. Use other probe implementations, for example, **httpGet** or **tcpSocket**.
 - When you need to use exec probes, limit the exec probe frequency and quantity. The maximum number of exec probes must be kept below 10, and frequency must not be set to less than 10 seconds. Exec probes cause much higher CPU usage on management cores compared to other probe types because they require process forking.



NOTE

Startup probes require minimal resources during steady-state operation. The limitation on exec probes applies primarily to liveness and readiness probes.



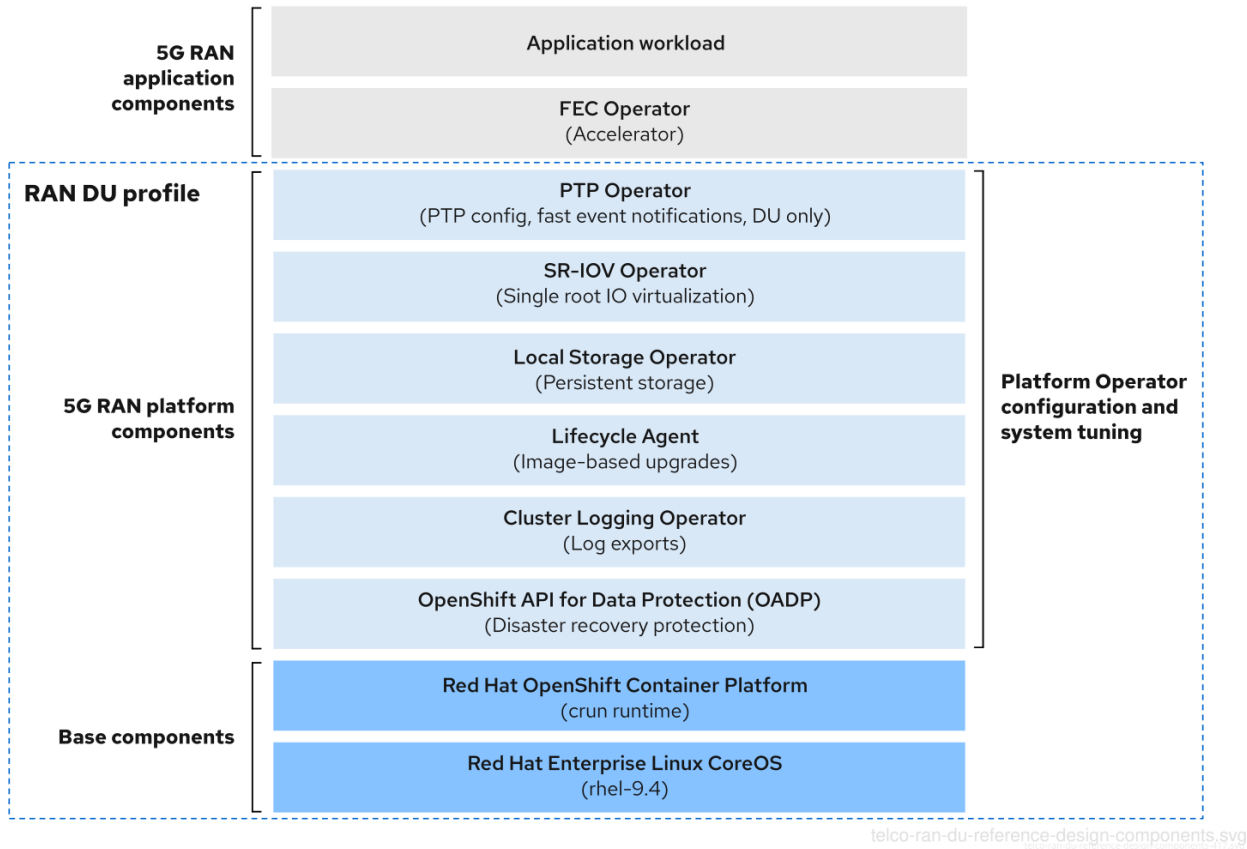
NOTE

A test workload that conforms to the dimensions of the reference DU application workload described in this specification can be found at [openshift-kni/du-test-workloads](#).

4.6. TELCO RAN DU REFERENCE DESIGN COMPONENTS

The following sections describe the various OpenShift Container Platform components and configurations that you use to configure and deploy clusters to run RAN DU workloads.

Figure 4.2. Telco RAN DU reference design components



NOTE

Ensure that additional components you include that are not specified in the telco RAN DU profile do not affect the CPU resources allocated to workload applications.



IMPORTANT

Out of tree drivers are not supported. 5G RAN application components are not included in the RAN DU profile and must be engineered against resources (CPU) allocated to applications.

4.6.1. Host firmware tuning

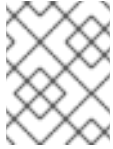
Tune host firmware settings for optimal performance during initial cluster deployment.

New in this release

- No reference design updates in this release

Description

Tune host firmware settings for optimal performance during initial cluster deployment. For more information, see "Recommended single-node OpenShift cluster configuration for vDU application workloads". Apply tuning settings in the host firmware during initial deployment. For more information, see "Managing host firmware settings with GitOps ZTP". The managed cluster host firmware settings are available on the hub cluster as individual **BareMetalHost** custom resources (CRs) that are created when you deploy the managed cluster with the **ClusterInstance** CR and GitOps ZTP.

**NOTE**

Create the **ClusterInstance** CR based on the provided reference **example-sno.yaml** CR.

Limits and requirements

- You must enable Hyper-Threading in the host firmware settings

Engineering considerations

- Tune all firmware settings for maximum performance.
- All settings are expected to be for maximum performance unless tuned for power savings.
- You can tune host firmware for power savings at the expense of performance as required.
- Enable secure boot. When secure boot is enabled, only signed kernel modules are loaded by the kernel. Out-of-tree drivers are not supported.

Additional resources

- [Recommended single-node OpenShift cluster configuration for vDU application workloads](#)
- [Managing host firmware settings with GitOps ZTP](#)
- [Configuring host firmware for low latency and high performance](#)
- [Provisioning real-time and low latency workloads](#)

4.6.2. Kubelet settings

Configure kubelet settings and sysctls for the telco RAN DU use model, including **systemReserved** and unsafe sysctl parameters.

New in this release

- No reference design updates in this release

Some CNF workloads make use of sysctls which are not in the list of system-wide safe sysctls. Generally, network sysctls are namespaced and you can enable them using the **kubeletconfig.experimental** annotation in the **PerformanceProfile** Custom Resource (CR).

Additionally, the **systemReserved** memory can be configured through the same **kubeletconfig.experimental** annotation to reserve memory for system daemons and kernel processes.

An example setting of these parameters as a string of JSON is shown here:

Example snippet showing allowedUnsafeSysctls and systemReserved

```
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: {{ .metadata.name }}
```

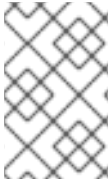
annotations:

```
# allowedUnsafeSysctls: some pods want the kernel stack to ignore IPv6 router Advertisement.
# systemReserved: when used, it should be tailored for each environment.
```

```
kubeletconfig.experimental: |
```

```
{
  "allowedUnsafeSysctls":["net.ipv6.conf.all.accept_ra"],
  "systemReserved":{"memory":"11Gi"}
}
```

```
# ...
```



NOTE

Although these sysctls are namespaced, they may allow a pod to consume memory or other resources beyond any limits specified in the pod description. You must ensure that these sysctls do not exhaust platform resources.

For more information, see "Using sysctls in containers".

Additional resources

- [Using sysctls in containers](#)

4.6.3. CPU partitioning and performance tuning

The RAN DU use model includes cluster performance tuning using **PerformanceProfile** CRs for low-latency performance, and a **TunedPerformancePatch** CR that adds additional RAN-specific tuning.

New in this release

- No reference design updates in this release

Description

The RAN DU use model includes cluster performance tuning using **PerformanceProfile** CRs for low-latency performance, and a **TunedPerformancePatch** CR that adds additional RAN-specific tuning. A reference **PerformanceProfile** is provided for both x86_64 and aarch64 CPU architectures. The single **TunedPerformancePatch** object provided automatically detects the CPU architecture and performs the required additional tuning. The RAN DU use case requires the cluster to be tuned for low-latency performance. The Node Tuning Operator reconciles the **PerformanceProfile** and **TunedPerformancePatch** CRs.

For more information about node tuning with the **PerformanceProfile** CR, see "Tuning nodes for low latency with the performance profile".

Limits and requirements

You must configure the following settings in the telco RAN DU profile **PerformanceProfile** CR:

- Set a reserved **cpuset** of 4 or more, equating to 4 hyper-threads (2 cores) on x86_64, or 4 cores on aarch64 for any of the following CPUs:
 - Intel 3rd Generation Xeon (IceLake) 2.20 GHz, or newer, CPUs with host firmware tuned for maximum performance
 - AMD EPYC Zen 4 CPUs (Genoa, Bergamo)

- ARM CPUs (Neoverse)



NOTE

It is recommended to evaluate features, such as per-pod power management, to determine any potential impact on performance.

- x86_64:
 - Set the reserved **cpuset** to include both hyper-thread siblings for each included core. Unreserved cores are available as allocatable CPU for scheduling workloads.
 - Ensure that hyper-thread siblings are not split across reserved and isolated cores.
 - Ensure that reserved and isolated CPUs include all the threads for all cores in the CPU.
 - Include Core 0 for each NUMA node in the reserved CPU set.
 - Set the hugepage size to 1G.
- aarch64:
 - Use the first 4 cores for the reserved CPU set (or more).
 - Set the hugepage size to 512M.
- Only pin OpenShift Container Platform pods that are by default configured as part of the management workload partition to reserved cores.
- When recommended by the hardware vendor, set the maximum CPU frequency for reserved and isolated CPUs using the **hardwareTuning** section.

Engineering considerations

- RealTime (RT) kernel
 - Under x86_64, to reach the full performance metrics, you must use the RT kernel, which is the default in the **x86_64/PerformanceProfile.yaml** configuration.
 - If required, you can select the non-RT kernel with corresponding impact to performance.
 - Under aarch64, only the 64k-pagesize non-RT kernel is recommended for RAN DU use cases, which is the default in the **aarch64/PerformanceProfile.yaml** configuration.
- The number of hugepages you configure depends on application workload requirements. Variation in this parameter is expected and allowed.
- Variation is expected in the configuration of reserved and isolated CPU sets based on selected hardware and additional components in use on the system. The variation must still meet the specified limits.
- Hardware without IRQ affinity support affects isolated CPUs. To ensure that pods with guaranteed whole CPU QoS have full use of allocated CPUs, all hardware in the server must support IRQ affinity.

- To enable workload partitioning, set **cpuPartitioningMode** to **AllNodes** during deployment, and then use the **PerformanceProfile** CR to allocate enough CPUs to support the operating system, interrupts, and OpenShift Container Platform pods.
- Tailor **systemReserved** memory for each cluster based on its size and application workload. The minimum recommended value is 11Gi.
- Under x86_64, the **PerformanceProfile** may be customized with the following optional arguments in the **additionalKernelargs** list:
 - The **vfio_pci** arguments support devices such as the FEC accelerator. You can omit them if they are not required for your workload.
 - To enable the **acpi_idle** CPUIdle driver, for example, for Intel FlexRAN, add **intel_idle.max_cstate=0**
- Under aarch64, the **PerformanceProfile** must be adjusted depending on the needs of the platform:
 - For Grace Hopper systems, the following kernel commandline arguments are required:
 - **acpi_power_meter.force_cap_on=y**
 - **module_blacklist=nouveau**
 - **pci=realloc=off**
 - **pci=pcie_bus_safe**
 - For other ARM platforms, you may need to enable **iommu.passthrough=1** or **pci=realloc**
- Extending and augmenting **TunedPerformancePatch.yaml**:
 - **TunedPerformancePatch.yaml** introduces a default top-level tuned profile named **ran-du-performance** and an architecture-aware RAN tuning profile named **ran-du-performance-architecture-common**, and additional architecture-specific child policies that are automatically selected by the common policy.
 - By default, the **ran-du-performance** profile is set to **priority** level **18**, and it includes both the PerformanceProfile-created profile **openshift-node-performance-openshift-node-performance-profile** and **ran-du-performance-architecture-common**
 - If you have customized the name of the **PerformanceProfile** object, you must create a new tuned object that includes the name change of the tuned profile created by the **PerformanceProfile** CR, as well as the **ran-du-performance-architecture-common** RAN tuning profile. This must have a **priority** less than 18. For example, if the PerformanceProfile object is named **change-this-name**:

```

apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: custom-performance-profile-override
  namespace: openshift-cluster-node-tuning-operator
spec:
  profile:

```

```

- name: custom-performance-profile-x
  data: |
    [main]
    summary=Override of the default ran-du performance tuning to adjust for our
renamed PerformanceProfile
    include=openshift-node-performance-change-this-name,ran-du-performance-
architecture-common
  recommend:
    - machineConfigLabels:
        machineconfiguration.openshift.io/role: "master"
    priority: 15
    profile: custom-performance-profile-x

```

- To further override, the optional **TunedPowerCustom.yaml** config file exemplifies how to extend the provided **TunedPerformancePatch.yaml** without needing to overlay or edit it directly. Creating an additional tuned profile which includes the top-level tuned profile named **ran-du-performance** and has a lower **priority** number in the **recommend** section allows adding additional settings easily.
- For additional information on the Node Tuning Operator, see "Using the Node Tuning Operator".

Additional resources

- [Finding the effective IRQ affinity setting for a node](#)
- [Tuning nodes for low latency with the performance profile](#)
- [Using the Node Tuning Operator](#)

4.6.4. PTP Operator

Configure Precision Time Protocol (PTP) in cluster nodes to ensure precise timing and reliability in the RAN environment.

New in this release

- Support for PTP boundary clock without holdover on GNR-D hardware.

Description

Configure PTP in cluster nodes. PTP ensures precise timing and reliability in the RAN environment, compared to other clock synchronization protocols, like NTP.

Support includes

- Grandmaster clock (T-GM): use GPS to sync the local clock and provide time synchronization to other devices
- Boundary clock (T-BC): receive time from another PTP source and redistribute it to other devices
- Ordinary clock (T-TSC): synchronize the local clock from another PTP time provider

Configuration variations allow for multiple NIC configurations for greater time distribution and high availability (HA), and optional fast event notification over HTTP.

Limits and requirements

- Supports the PTP G.8275.1 profile for the following telco use-cases:
 - T-GM use-case:
 - Limited to a maximum of 3 Westport channel NICs
 - Requires GNSS input to one NIC card, with SMA connections to synchronize additional NICs
 - HA support N/A
 - GNR-D is not supported for T-GM.
 - T-BC use-case:
 - Limited to a maximum of 2 NICs
 - System clock HA support is optional in 2-NIC configuration.
 - GNR-D is not supported for T-BC.
 - T-TSC use-case:
 - Limited to single NIC only
 - System clock HA support is optional in active/standby 2-port configuration.
 - GNR-D is not supported for T-TSC.
 - T-BC without holdover use-case:
 - GNR-D hardware with 0, 1, or 2 additional Carter Flats, e830 NICs
 - Time receiver port must be one of the onboard NAC ports.
 - Time transmitters may be configured to any combination of NAC and Carter Flats ports, up to a total of 23.
 - Holdover is not enabled, so any failure of the Time receiver will enter **FREERUN** state immediately.
- Log reduction must be enabled with **true** or **enhanced**.

Engineering considerations

- Example RAN DU RDS configurations are provided for:
 - T-GM, T-BC, T-TSC, and BC-without-holdover
 - Variations with and without HA
 - Variations with and without fast event notification
- PTP fast event notifications use **ConfigMap** CRs to persist subscriber details.

- Hierarchical event subscription as described in the O-RAN specification is not supported for PTP events.
- Cluster nodes must have proper NTP configuration to ensure correct time prior to PTP operator taking ownership of node timing.

4.6.5. SR-IOV Operator

The SR-IOV Operator provisions and configures the SR-IOV CNI and device plugins.

New in this release

- No reference design updates in this release

Description

The SR-IOV Operator provisions and configures the SR-IOV CNI and device plugins. Both **netdevice** (kernel VFs) and **vfio** (DPDK) devices are supported and applicable to the RAN DU use models.

Limits and requirements

- Use devices that are supported for OpenShift Container Platform. For more information, see "Supported devices".
- SR-IOV and IOMMU enablement in host firmware settings: The SR-IOV Network Operator automatically enables IOMMU on the kernel command line.
- SR-IOV VFs do not receive link state updates from the PF. If link down detection is required you must configure this at the protocol level.

Engineering considerations

- SR-IOV interfaces with the **vfio** driver type are typically used to enable additional secondary networks for applications that require high throughput or low latency.
- Customer variation on the configuration and number of **SriovNetwork** and **SriovNetworkNodePolicy** custom resources (CRs) is expected.
- IOMMU kernel command line settings are applied with a **MachineConfig** CR at install time. This ensures that the **SriovOperator** CR does not cause a reboot of the node when adding them.
- SR-IOV support for draining nodes in parallel is not applicable in a single-node OpenShift cluster.
- You must include the **SriovOperatorConfig** CR in your deployment; the CR is not created automatically. This CR is included in the reference configuration policies which are applied during initial deployment.
- In scenarios where you pin or restrict workloads to specific nodes, the SR-IOV parallel node drain feature will not result in the rescheduling of pods. In these scenarios, the SR-IOV Operator disables the parallel node drain functionality.
- You must pre-configure NICs which do not support firmware updates under secure boot or kernel lockdown with sufficient virtual functions (VFs) to support the number of VFs needed

by the application workload. For Mellanox NICs, you must disable the Mellanox vendor plugin in the SR-IOV Network Operator. For more information, see "Configuring the SR-IOV Network Operator on Mellanox cards when Secure Boot is enabled".

- To change the MTU value of a virtual function after the pod has started, do not configure the MTU field in the **SriovNetworkNodePolicy** CR. Instead, configure the Network Manager or use a custom **systemd** script to set the MTU of the physical function to an appropriate value. For example:

```
# ip link set dev <physical_function> mtu 9000
```

Additional resources

- [Red Hat certified hardware \(Red Hat Ecosystem Catalog\)](#)
- [Configuring QinQ support for SR-IOV enabled workloads](#)

4.6.6. Logging

Use logging to collect logs from the far edge node for remote analysis.

New in this release

- No reference design updates in this release

Description

Use logging to collect logs from the far edge node for remote analysis. The recommended log collector is Vector.

Engineering considerations

- Handling logs beyond the infrastructure and audit logs, for example, from the application workload requires additional CPU and network bandwidth based on additional logging rate.
- As of OpenShift Container Platform 4.14, Vector is the reference log collector. Use of fluentd in the RAN use models is deprecated.

Additional resources

- [About OpenShift logging](#)

4.6.7. SRIOV-FEC Operator

SRIOV-FEC Operator is an optional 3rd party Certified Operator supporting FEC accelerator hardware.

New in this release

- No reference design updates in this release

Description

SRIOV-FEC Operator is an optional 3rd party Certified Operator supporting FEC accelerator hardware.

Limits and requirements

- Starting with FEC Operator v2.7.0:
 - Secure boot is supported
 - **vfio** drivers for PFs require the usage of a **vfio-token** that is injected into the pods. Applications in the pod can pass the VF token to DPDK by using EAL parameter **--vfio-vf-token**.

Engineering considerations

- The SRIOV-FEC Operator uses CPU cores from the isolated CPU set.
- You can validate FEC readiness as part of the pre-checks for application deployment, for example, by extending the validation policy.

Additional resources

- [SRIOV-FEC Operator for Intel® vRAN Dedicated Accelerator manager container](#)

4.6.8. Lifecycle Agent

The Lifecycle Agent provides local lifecycle management services for image-based upgrade of single-node OpenShift clusters.

New in this release

- No reference design updates in this release

Description

The Lifecycle Agent provides local lifecycle management services for image-based upgrade of single-node OpenShift clusters. Image-based upgrade is the recommended upgrade method for single-node OpenShift clusters.

Limits and requirements

- The Lifecycle Agent is not applicable in multi-node clusters or single-node OpenShift clusters with an additional worker.
- The Lifecycle Agent requires a persistent volume that you create when installing the cluster.

For more information about partition requirements, see "Configuring a shared container directory between ostree stateroots when using GitOps ZTP".

Additional resources

- [Understanding the image-based upgrade for single-node OpenShift clusters](#)
- [Configuring a shared container directory between ostree stateroots when using GitOps ZTP](#)

4.6.9. Local Storage Operator

You can create persistent volumes that can be used as **PVC** resources by applications with the Local Storage Operator.

New in this release

- No reference design updates in this release

Description

You can create persistent volumes that can be used as **PVC** resources by applications with the Local Storage Operator. The number and type of **PV** resources that you create depends on your requirements.

Engineering considerations

- Create backing storage for **PV** CRs before creating the **PV**. This can be a partition, a local volume, LVM volume, or full disk.
- Refer to the device listing in **LocalVolume** CRs by the hardware path used to access each device to ensure correct allocation of disks and partitions, for example, **/dev/disk/by-path/<id>**. Logical names (for example, **/dev/sda**) are not guaranteed to be consistent across node reboots.

4.6.10. Logical Volume Manager Storage

Logical Volume Manager (LVM) Storage provides dynamic provisioning of block and file storage by creating logical volumes from local devices that can be consumed as persistent volume claim (PVC) resources by applications.

New in this release

- No reference design updates in this release

Description

Logical Volume Manager (LVM) Storage is an optional component. It provides dynamic provisioning of both block and file storage by creating logical volumes from local devices that can be consumed as persistent volume claim (PVC) resources by applications. Volume expansion and snapshots are also possible. An example configuration is provided in the RDS with the **StorageLVMCluster.yaml** file.

Limits and requirements

- In single-node OpenShift clusters, persistent storage must be provided by either LVM Storage or local storage, not both.
- Volume snapshots are excluded from the reference configuration.

Engineering considerations

- LVM Storage can be used as the local storage implementation for the RAN DU use case. When LVM Storage is used as the storage solution, it replaces the Local Storage Operator, and the CPU required is assigned to the management partition as platform overhead. The reference configuration must include one of these storage solutions but not both.
- Ensure that sufficient disks or partitions are available for storage requirements.

4.6.11. Workload partitioning

Workload partitioning pins OpenShift Container Platform and Day 2 Operator pods that are part of the DU profile to the reserved CPU set and removes the reserved CPU from node accounting.

New in this release

- No reference design updates in this release

Description

Workload partitioning pins OpenShift Container Platform and Day 2 Operator pods that are part of the DU profile to the reserved CPU set and removes the reserved CPU from node accounting. This leaves all unreserved CPU cores available for user workloads. This leaves all non-reserved CPU cores available for user workloads. Workload partitioning is enabled through a capability set in installation parameters: **cpuPartitioningMode: AllNodes**. The set of management partition cores are set with the reserved CPU set that you configure in the **PerformanceProfile** CR.

Limits and requirements

- **Namespace** and **Pod** CRs must be annotated to allow the pod to be applied to the management partition
- Pods with CPU limits cannot be allocated to the partition. This is because mutation can change the pod QoS.
- For more information about the minimum number of CPUs that can be allocated to the management partition, see "Node Tuning Operator".

Engineering considerations

- Workload partitioning pins all management pods to reserved cores. A sufficient number of cores must be allocated to the reserved set to account for operating system, management pods, and expected spikes in CPU use that occur when the workload starts, the node reboots, or other system events happen.

Additional resources

- [Workload partitioning](#)

4.6.12. Cluster tuning

Configure cluster tuning settings including cluster capabilities and monitoring for the telco RAN DU reference design.

New in this release

- OLM profile collection is removed in OpenShift Container Platform 4.22. The **DisableOLMPprof** CR compliance type is now set to **mustnothave** to remove the previously applied configuration from clusters.

Description

For a full list of components that you can disable using the cluster capabilities feature, see "Cluster capabilities".

Limits and requirements

- Cluster capabilities are not available for installer-provisioned installation methods.

The following table lists the required platform tuning configurations:

Table 4.4. Cluster capabilities configurations

Feature	Description
Remove optional cluster capabilities	Reduce the OpenShift Container Platform footprint by disabling optional cluster Operators on single-node OpenShift clusters only. Remove all optional Operators except the Node Tuning Operator, Operator Lifecycle Manager, and the Ingress Operator.
Configure cluster monitoring	Configure the monitoring stack for reduced footprint by doing the following: - Disable the local alertmanager and telemeter components. - If you use RHACM observability, the CR must be augmented with appropriate additionalAlertManagerConfigs CRs to forward alerts to the hub cluster. - RHACM observability combines its default data values with the monitoring configuration ConfigMap CR provided as part of the cluster tuning reference CRs. This merge results in the policy becoming non-compliant. To ensure that the provided configuration is not overwritten or merged with RHACM data values, you can disable the RHACM management of this ConfigMap CR. This keeps the policy compliant. For more information, see the Observability section of Telco hub reference design specifications. - Reduce the Prometheus retention period to 24h.  NOTE The RHACM hub cluster aggregates managed cluster metrics.
Disable networking diagnostics	Disable networking diagnostics for single-node OpenShift because they are not required.

Feature	Description
Configure a single OperatorHub catalog source	Configure the cluster to use a single catalog source that contains only the Operators required for a RAN DU deployment. Each catalog source increases the CPU use on the cluster. Using a single CatalogSource fits within the platform CPU budget.
Disable the Console Operator	If the cluster was deployed with the console disabled, the Console CR (ConsoleOperatorDisable.yaml) is not needed. If the cluster was deployed with the console enabled, you must apply the Console CR.

Engineering considerations

- As of OpenShift Container Platform 4.19, cgroup v1 is no longer supported and has been removed. All workloads must now be compatible with cgroup v2. For more information, see [Red Hat Enterprise Linux 9 changes in the context of Red Hat OpenShift workloads](#).

Additional resources

- [Cluster capabilities](#)
- [Observability](#)

4.6.13. Machine configuration

Configure machine configuration settings for container runtime, kubelet, SCTP, kdump, and other system-level components in the telco RAN DU reference design.

New in this release

- No reference design updates in this release

Limits and requirements

- To ensure images are static, except during scheduled maintenance in defined maintenance windows, do not set the pod **imagePullPolicy** field to **Always**.
- The configuration CRs in this table are required components unless otherwise noted.

Table 4.5. Machine configuration options

Feature	Description
Container Runtime	Sets the container runtime to crun for all node roles.

Feature	Description
Kubelet config and container mount namespace hiding	Reduces the frequency of kubelet housekeeping and eviction monitoring, which reduces CPU usage
SCTP	Optional configuration (enabled by default)
Kdump	Optional configuration (enabled by default) Enables kdump to capture debug information when a kernel panic occurs. The reference CRs that enable kdump have an increased memory reservation based on the set of drivers and kernel modules included in the reference configuration.
SR-IOV-related kernel arguments	Include additional SR-IOV-related arguments in the kernel command line
Set RCU Normal	Systemd service that sets rcu_normal after the system finishes startup
One-shot time sync	Replaces chrony-wait.service with a one-time NTP system time synchronization job for control plane or worker nodes. This MachineConfig is required for nodes running PTP and should not be included for nodes which use NTP for time synchronization.

Additional resources

- [Recommended cluster install manifests](#)

4.7. TELCO RAN DU DEPLOYMENT COMPONENTS

The following sections describe the various OpenShift Container Platform components and configurations that you use to configure the hub cluster with RHACM.

4.7.1. Red Hat Advanced Cluster Management

RHACM provides Multi Cluster Engine (MCE) installation and ongoing lifecycle management functionality for deployed clusters.

New in this release

- No reference design updates in this release

Description

RHACM provides Multi Cluster Engine (MCE) installation and ongoing lifecycle management functionality for deployed clusters. You manage cluster configuration and upgrades declaratively by applying **Policy** custom resources (CRs) to clusters during maintenance windows.

RHACM provides the following functionality:

- Zero touch provisioning (ZTP) of clusters using the MCE component in RHACM.
- Configuration, upgrades, and cluster status through the RHACM policy controller.

- During managed cluster installation, RHACM can apply labels to individual nodes as configured through the **ClusterInstance** CR.

The recommended method for single-node OpenShift cluster installation is the image-based installation approach, available in MCE, using the **ClusterInstance** CR for cluster definition.

Image-based upgrade is the recommended method for single-node OpenShift cluster upgrade.

Limits and requirements

- A single hub cluster supports up to 3500 deployed single-node OpenShift clusters with 5 **Policy** CRs bound to each cluster.

Engineering considerations

- Use RHACM policy hub-side templating to better scale cluster configuration. You can significantly reduce the number of policies by using a single group policy or small number of general group policies where the group and per-cluster values are substituted into templates.
- Cluster specific configuration: managed clusters typically have some number of configuration values that are specific to the individual cluster. These configurations should be managed using RHACM policy hub-side templating with values pulled from **ConfigMap** CRs based on the cluster name.
- To save CPU resources on managed clusters, policies that apply static configurations should be unbound from managed clusters after GitOps ZTP installation of the cluster.

Additional resources

- [Using GitOps ZTP to provision clusters at the network far edge](#)
- [Red Hat Advanced Cluster Management for Kubernetes](#)

4.7.2. SiteConfig Operator

The SiteConfig Operator is a template-driven solution designed to provision clusters through various installation methods.

New in this release

- No reference design updates in this release

Description

The SiteConfig Operator is a template-driven solution designed to provision clusters through various installation methods. It introduces the unified **ClusterInstance** API, which replaces the deprecated **SiteConfig** API. By leveraging the **ClusterInstance** API, the SiteConfig Operator improves cluster provisioning by providing the following:

- Better isolation of definitions from installation methods
- Unification of Git and non-Git workflows
- Consistent APIs across installation methods

- Enhanced scalability
- Increased flexibility with custom installation templates
- Valuable insights for troubleshooting deployment issues

The SiteConfig Operator provides validated default installation templates to facilitate cluster deployment through both the Assisted Installer and Image-based Installer provisioning methods:

- **Assisted Installer** automates the deployment of OpenShift Container Platform clusters by leveraging predefined configurations and validated host setups. It ensures that the target infrastructure meets OpenShift Container Platform requirements. The Assisted Installer streamlines the installation process while minimizing time and complexity compared to manual setup.
- **Image-based Installer** expedites the deployment of single-node OpenShift clusters by utilizing preconfigured and validated OpenShift Container Platform seed images. Seed images are preinstalled on target hosts, enabling rapid reconfiguration and deployment. The Image-based Installer is particularly well-suited for remote or disconnected environments because it simplifies the cluster creation process and significantly reduces deployment time.

Limits and requirements

- A single hub cluster supports up to 3500 deployed single-node OpenShift clusters.

4.7.3. Topology Aware Lifecycle Manager

TALM manages how changes like cluster upgrades, Operator upgrades, and cluster configuration are rolled out to the network.

New in this release

- No reference design updates in this release

Description

TALM is an Operator that runs only on the hub cluster for managing how changes like cluster upgrades, Operator upgrades, and cluster configuration are rolled out to the network. TALM supports the following features:

- Progressive rollout of policy updates to fleets of clusters in user configurable batches.
- Per-cluster actions add **ztp-done** labels or other user-configurable labels following configuration changes to managed clusters.
- Precaching of single-node OpenShift clusters images: TALM supports optional pre-caching of OpenShift, OLM Operator, and additional user images to single-node OpenShift clusters before initiating an upgrade. The precaching feature is not applicable when using the recommended image-based upgrade method for upgrading single-node OpenShift clusters.
 - Specifying optional pre-caching configurations with **PreCachingConfig** CRs. Review the [sample reference PreCachingConfig CR](#) for more information.
 - Excluding unused images with configurable filtering.

- Enabling before and after pre-caching storage space validations with configurable space-required parameters.

Limits and requirements

- Supports concurrent cluster deployment in batches of 400
- Pre-caching and backup are limited to single-node OpenShift clusters only

Engineering considerations

- The **PreCachingConfig** CR is optional and does not need to be created if you only need to precache platform-related OpenShift and OLM Operator images.
- The **PreCachingConfig** CR must be applied before referencing it in the **ClusterGroupUpgrade** CR.
- Only policies with the **ran.openshift.io/ztp-deploy-wave** annotation are automatically applied by TALM during cluster installation.
- Any policy can be remediated by TALM under control of a user created **ClusterGroupUpgrade** CR.

Additional resources

- [Updating managed clusters with the Topology Aware Lifecycle Manager](#)

4.7.4. GitOps Operator and GitOps ZTP

GitOps Operator and GitOps ZTP provide a GitOps-based infrastructure for managing cluster deployment and configuration.

New in this release

- No reference design updates in this release

Description

GitOps Operator and GitOps ZTP provide a GitOps-based infrastructure for managing cluster deployment and configuration. Cluster definitions and configurations are maintained as a declarative state in Git. You can apply **ClusterInstance** CRs to the hub cluster where the **SiteConfig** Operator renders them as installation CRs. In earlier releases, a GitOps ZTP plugin supported the generation of installation CRs from **SiteConfig** CRs. This plugin is now removed. A separate GitOps ZTP plugin is available to enable automatic wrapping of configuration CRs into policies based on the **PolicyGenerator** or **PolicyGenTemplate** CR.

You can deploy and manage multiple versions of OpenShift Container Platform on managed clusters using the baseline reference configuration CRs. You can use custom CRs alongside the baseline CRs. To maintain multiple per-version policies simultaneously, use Git to manage the versions of the source and policy CRs by using **PolicyGenerator** or **PolicyGenTemplate** CRs. RHACM

PolicyGenerator is the recommended generator plugin starting from OpenShift Container Platform 4.19 release.

Limits and requirements

- 1000 **ClusterInstance** CRs per ArgoCD application on a hub cluster conforming to the Hub RDS. Multiple applications can be used to achieve the maximum number of clusters supported by a single hub cluster
- Content in the **source-crs/** directory in Git overrides content provided in the ZTP plugin container, as Git takes precedence in the search path.
- The **source-crs/** directory must be located in the same directory as the **kustomization.yaml** file, which includes **PolicyGenerator** CRs as a generator. Alternative locations for the **source-crs/** directory are not supported in this context.

Engineering considerations

- For multi-node cluster upgrades, you can pause **MachineConfigPool (MCP)** CRs during maintenance windows by setting the **paused** field to **true**. You can increase the number of simultaneously updated nodes per **MCP** CR by configuring the **maxUnavailable** setting in the **MCP** CR. The **MaxUnavailable** field defines the percentage of nodes in the pool that can be simultaneously unavailable during a **MachineConfig** update. Set **maxUnavailable** to the maximum tolerable value. This reduces the number of reboots in a cluster during upgrades which results in shorter upgrade times. When you finally unpause the **MCP** CR, all the changed configurations are applied with a single reboot.
- During cluster installation, you can pause custom MCP CRs by setting the **paused** field to **true** and setting **maxUnavailable** to 100% to improve installation times.
- Keep reference CRs and custom CRs under different directories. Doing this allows you to patch and update the reference CRs by simple replacement of all directory contents without touching the custom CRs. When managing multiple versions, the following best practices are recommended:
 - Keep all source CRs and policy creation CRs in Git repositories to ensure consistent generation of policies for each OpenShift Container Platform version based solely on the contents in Git.
 - Keep reference source CRs in a separate directory from custom CRs. This facilitates easy update of reference CRs as required.
- To avoid confusion or unintentional overwrites when updating content, it is highly recommended to use unique and distinguishable names for custom CRs in the **source-crs/** directory and extra manifests in Git.
- Extra installation manifests are referenced in the **ClusterInstance** CR through a **ConfigMap** CR. The **ConfigMap** CR should be stored alongside the **ClusterInstance** CR in Git, serving as the single source of truth for the cluster. If needed, you can use a **ConfigMap** generator to create the **ConfigMap** CR.

Additional resources

- [Preparing the GitOps ZTP site configuration repository for version independence](#)
- [Adding custom content to the GitOps ZTP pipeline](#)

4.7.5. Agent-based installer

The optional Agent-based Installer component provides installation capabilities without centralized infrastructure.

New in this release

- No reference design updates in this release

Description

The optional Agent-based Installer component provides installation capabilities without centralized infrastructure. The installation program creates an ISO image that you mount to the server. When the server boots it installs OpenShift Container Platform and supplied extra manifests. The Agent-based Installer allows you to install OpenShift Container Platform without a hub cluster. A container image registry is required for cluster installation.

Limits and requirements

- You can supply a limited set of additional manifests at installation time.
- You must include **MachineConfiguration** CRs that are required by the RAN DU use case.

Engineering considerations

- The Agent-based Installer provides a baseline OpenShift Container Platform installation.
- You install Day 2 Operators and the remainder of the RAN DU use case configurations after installation.

Additional resources

- [Installing a cluster with customizations](#)

4.8. TELCO RAN DU REFERENCE CONFIGURATION CRS

Use the following custom resources (CRs) to configure and deploy OpenShift Container Platform clusters with the telco RAN DU profile. Use the CRs to form the common baseline used in all the specific use models unless otherwise indicated.



NOTE

You can extract the complete set of RAN DU CRs from the **ztp-site-generate** container image. See "Preparing the GitOps ZTP site configuration repository" for more information.

4.8.1. Cluster tuning reference CRs

Use the following custom resources (CRs) to configure cluster tuning for the telco RAN DU reference design.

Table 4.6. Cluster tuning CRs

Component	Reference CR	Description	Optional
-----------	--------------	-------------	----------

Component	Reference CR	Description	Optional
Cluster capabilities	example-sno.yaml	Representative ClusterInstance CR to install single-node OpenShift with the RAN DU profile	No
Console disable	cluster-tuning/console-disable/ConsoleOperatorDisable.yaml	Disables the Console Operator.	No
Disconnected registry	extra-manifest/09-openshift-marketplace-ns.yaml	Defines a dedicated namespace for managing the OpenShift Operator Marketplace.	No
Disconnected registry	disconnected-registry/DefaultCatalog.yaml	Configures the catalog source for the disconnected registry.	No
Disconnected registry	cluster-tuning/DisableOLMPprof.yaml	Removes the obsolete ConfigMap CR that disabled OLM performance profiling in earlier releases. OLM performance profiling collection is removed in OpenShift Container Platform 4.22.	No
Disconnected registry	disconnected-registry/DisconnectedImageContentSourcePolicy.yaml	Configures disconnected registry image content source policy.	No
Disconnected registry	cluster-tuning/operator-hub/OperatorHub.yaml	Optional, for multi-node clusters only. Configures the OperatorHub in OpenShift, disabling all default Operator sources. Not required for single-node OpenShift installs with marketplace capability disabled.	No

Component	Reference CR	Description	Optional
Monitoring configuration	cluster-tuning/monitoring-configuration/ReduceMonitoringFootprint.yaml	Reduces the monitoring footprint by disabling Alertmanager and Telemeter, and sets Prometheus retention to 24 hours	No
Network diagnostics disable	cluster-tuning/disabling-network-diagnostics/DisableSnoNetworkDiagnostic.yaml	Configures the cluster network settings to disable built-in network troubleshooting and diagnostic features.	No

4.8.2. Day 2 Operators reference CRs

Use the following custom resources (CRs) to configure Day 2 Operators for the telco RAN DU reference design.

Table 4.7. Day 2 Operators CRs

Component	Reference CR	Description	Optional
Cluster Logging Operator	cluster-logging/ClusterLogForwarder.yaml	Configures log forwarding for the cluster.	No
Cluster Logging Operator	cluster-logging/ClusterLogNamespace.yaml	Configures the namespace for cluster logging.	No
Cluster Logging Operator	cluster-logging/ClusterLogOperatorGroup.yaml	Configures Operator group for cluster logging.	No
Cluster Logging Operator	cluster-logging/ClusterLogServiceAccount.yaml	New in 4.18. Configures the cluster logging service account.	No
Cluster Logging Operator	cluster-logging/ClusterLogServiceAccountAuditBinding.yaml	New in 4.18. Configures the cluster logging service account.	No

Component	Reference CR	Description	Optional
Cluster Logging Operator	cluster-logging/ClusterLogServiceAccountInfrastructureBinding.yaml	New in 4.18. Configures the cluster logging service account.	No
Cluster Logging Operator	cluster-logging/ClusterLogSubscription.yaml	Manages installation and updates for the Cluster Logging Operator.	No
Lifecycle Agent	ibu/ImageBasedUpgrade.yaml	Manage the image-based upgrade process in OpenShift Container Platform.	Yes
Lifecycle Agent	lca/LcaSubscription.yaml	Manages installation and updates for the LCA Operator.	Yes
Lifecycle Agent	lca/LcaSubscriptionNS.yaml	Configures namespace for LCA subscription.	Yes
Lifecycle Agent	lca/LcaSubscriptionOperatorGroup.yaml	Configures the Operator group for the LCA subscription.	Yes
Local Storage Operator	storage-Iso/StorageClass.yaml	Defines a storage class with a Delete reclaim policy and no dynamic provisioning in the cluster.	No
Local Storage Operator	storage/StorageLV.yaml	Configures local storage devices for the example-storage-class in the openshift-local-storage namespace, specifying device paths and filesystem type.	No
Local Storage Operator	storage-Iso/StorageNS.yaml	Creates the namespace with annotations for workload management and the deployment wave for the Local Storage Operator.	No

Component	Reference CR	Description	Optional
Local Storage Operator	storage-Iso/StorageOperatorGroup.yaml	Creates the Operator group for the Local Storage Operator.	No
Local Storage Operator	storage-Iso/StorageSubscription.yaml	Creates the namespace for the Local Storage Operator with annotations for workload management and deployment wave.	No
LVM Operator	storage-lvm/LVMOperatorStatus.yaml	Verifies the installation or upgrade of the LVM Storage Operator.	Yes
LVM Operator	storage-lvm/StorageLVMCluster.yaml	Defines an LVM cluster configuration, with placeholders for storage device classes and volume group settings. Optional substitute for the Local Storage Operator.	No
LVM Operator	storage-lvm/StorageLVMSubscription.yaml	Manages installation and updates of the LVMS Operator. Optional substitute for the Local Storage Operator.	No
LVM Operator	storage-lvm/StorageLVMSubscriptionNS.yaml	Creates the namespace for the LVMS Operator with labels and annotations for cluster monitoring and workload management. Optional substitute for the Local Storage Operator.	No
LVM Operator	storage-lvm/StorageLVMSubscriptionOperatorGroup.yaml	Defines the target namespace for the LVMS Operator. Optional substitute for the Local Storage Operator.	No

Component	Reference CR	Description	Optional
Node Tuning Operator	node-tuning-operator/aarch64/PerformanceProfile.yaml	Configures node performance settings in an OpenShift Container Platform cluster, optimizing for low latency and real-time workloads for aarch64 CPUs.	No
Node Tuning Operator	node-tuning-operator/x86_64/PerformanceProfile.yaml	Configures node performance settings in an OpenShift Container Platform cluster, optimizing for low latency and real-time workloads for x86_64 CPUs.	No
Node Tuning Operator	node-tuning-operator/TunedPerformancePatch.yaml	Applies performance tuning settings, including scheduler groups and service configurations for nodes in the specific namespace.	No
Node Tuning Operator	node-tuning-operator/TunedPowerCustom.yaml	Applies additional powersave mode tuning as an overlay on top of TunedPerformancePatch.	No
PTP fast event notifications	ptp-operator/configuration/PtpConfigBoundaryForEvent.yaml	Configures PTP settings for PTP boundary clocks with additional options for event synchronization. Dependent on cluster role.	No
PTP fast event notifications	ptp-operator/configuration/PtpConfigForHAFForEvent.yaml	Configures PTP for highly available boundary clocks with additional PTP fast event settings. Dependent on cluster role.	No

Component	Reference CR	Description	Optional
PTP fast event notifications	ptp-operator/configuration/PtpConfigMasterForEvent.yaml	Configures PTP for PTP grandmaster clocks with additional PTP fast event settings. Dependent on cluster role.	No
PTP fast event notifications	ptp-operator/configuration/PtpConfigSlaveForEvent.yaml	Configures PTP for PTP ordinary clocks with additional PTP fast event settings. Dependent on cluster role.	No
PTP fast event notifications	ptp-operator/PtpOperatorConfigForEvent.yaml	Overrides the default OperatorConfig. Configures the PTP Operator specifying node selection criteria for running PTP daemons in the openshift-ptp namespace.	No
PTP Operator	ptp-operator/configuration/PtpConfigBoundary.yaml	Configures PTP settings for PTP boundary clocks. Dependent on cluster role.	No
PTP Operator	ptp-operator/configuration/PtpConfigDualCardGmWpc.yaml	Configures PTP grandmaster clock settings for hosts that have dual NICs. Dependent on cluster role.	No
PTP Operator	ptp-operator/configuration/PtpConfigThreeCardGmWpc.yaml	Configures PTP grandmaster clock settings for hosts that have 3 NICs. Dependent on cluster role.	No
PTP Operator	ptp-operator/configuration/PtpConfigGmWpc.yaml	Configures PTP grandmaster clock settings for hosts that have a single NIC. Dependent on cluster role.	No

Component	Reference CR	Description	Optional
PTP Operator	ptp-operator/configuration/PtpConfigSlave.yaml	Configures PTP settings for a PTP ordinary clock. Dependent on cluster role.	No
PTP Operator	ptp-operator/configuration/PtpConfigDualFollower.yaml	Configures PTP settings for a PTP ordinary clock with 2 interfaces in an active/standby configuration. Dependent on cluster role.	No
PTP Operator	ptp-operator/configuration/PtpConfigTBCWpc.yaml	Configures PTP as a Telecom boundary clock. Dependent on cluster role.	No
PTP Operator	ptp-operator/configuration/PtpConfigDualCardTBCWpc.yaml	Configures PTP as a Telecom boundary clock for hosts that have dual NICs. Dependent on cluster role.	No
PTP Operator	ptp-operator/configuration/PtpConfigThreeCardTBCWpc.yaml	Configures PTP as a Telecom boundary clock for hosts that have 3 NICs. Dependent on cluster role.	No
PTP Operator	ptp-operator/configuration/PtpConfigTTSCWpc.yaml	Configures PTP settings for a PTP Telecom Time Slave Clock with single interface. Dependent on cluster role.	N
PTP Operator	ptp-operator/configuration/PtpConfigGnrdBcNoHoldover.yaml	Configures PTP settings for a PTP Boundary Clock without holdover on GNR-D hardware. Dependent on cluster role.	N

Component	Reference CR	Description	Optional
PTP Operator	ptp-operator/PtpOperatorConfig.yaml	Configures the PTP Operator settings, specifying node selection criteria for running PTP daemons in the openshift-ptp namespace.	No
PTP Operator	ptp-operator/PtpSubscription.yaml	Manages installation and updates of the PTP Operator in the openshift-ptp namespace.	No
PTP Operator	ptp-operator/PtpSubscriptionNS.yaml	Configures the namespace for the PTP Operator.	No
PTP Operator	ptp-operator/PtpSubscriptionOperatorGroup.yaml	Configures the Operator group for the PTP Operator.	No
PTP Operator (high availability)	ptp-operator/configuration/PtpConfigBoundary.yaml	Configures PTP settings for highly available PTP boundary clocks.	No
PTP Operator (high availability)	ptp-operator/configuration/PtpConfigForHA.yaml	Configures PTP settings for highly available PTP boundary clocks.	No
SR-IOV FEC Operator	sriov-fec-operator/AcceleratorSNS.yaml	Configures namespace for the VRAN Acceleration Operator. Optional part of application workload.	Yes
SR-IOV FEC Operator	sriov-fec-operator/AcceleratorSOperatorGroup.yaml	Configures the Operator group for the VRAN Acceleration Operator. Optional part of application workload.	Yes

Component	Reference CR	Description	Optional
SR-IOV FEC Operator	sriov-fec-operator/AcceleratorSubscription.yaml	Manages installation and updates for the VRAN Acceleration Operator. Optional part of application workload.	Yes
SR-IOV FEC Operator	sriov-fec-operator/SriovFecClusterConfig.yaml	Configures SR-IOV FPGA Ethernet Controller (FEC) settings for nodes, specifying drivers, VF amount, and node selection.	Yes
SR-IOV Operator	sriov-operator/SriovNetwork.yaml	Defines an SR-IOV network configuration, with placeholders for various network settings.	No
SR-IOV Operator	sriov-operator/SriovNetworkNodePolicy.yaml	Configures SR-IOV network settings for specific nodes, including device type, RDMA support, physical function names, and the number of virtual functions.	No
SR-IOV Operator	sriov-operator/SriovNetworkOperatorConfig.yaml	Configures SR-IOV Network Operator settings, including node selection, injector, and webhook options.	No
SR-IOV Operator	sriov-operator/SriovNetworkOperatorConfigForSNO.yaml	Configures the SR-IOV Network Operator settings for single-node OpenShift, including node selection, injector, webhook options, and disabling node drain, in the openshift-sriov-network-operator namespace.	No
SR-IOV Operator	sriov-operator/SriovSubscription.yaml	Manages the installation and updates of the SR-IOV Network Operator.	No

Component	Reference CR	Description	Optional
SR-IOV Operator	sriov-operator/SriovSubscriptionNS.yaml	Creates the namespace for the SR-IOV Network Operator with specific annotations for workload management and deployment waves.	No
SR-IOV Operator	sriov-operator/SriovSubscriptionOperatorGroup.yaml	Defines the target namespace for the SR-IOV Network Operators, enabling their management and deployment within this namespace.	No

4.8.3. Machine configuration reference CRs

Use the following custom resources (CRs) to configure machine settings for the telco RAN DU reference design.

Table 4.8. Machine configuration CRs

Component	Reference CR	Description	Optional
Container runtime (crun)	optional-extra-manifest/enable-crun-master.yaml	Configures the container runtime (crun) for control plane nodes.	No
Container runtime (crun)	optional-extra-manifest/enable-crun-worker.yaml	Configures the container runtime (crun) for worker nodes.	No
Kdump enable	extra-manifest/06-kdump-master.yaml	Configures kdump crash reporting on master nodes.	No
Kdump enable	extra-manifest/06-kdump-worker.yaml	Configures kdump crash reporting on worker nodes.	No
Kubelet configuration and container mount hiding	extra-manifest/01-container-mount-ns-and-kubelet-conf-master.yaml	Configures a mount namespace for sharing container-specific mounts between kubelet and CRI-O on control plane nodes.	No

Component	Reference CR	Description	Optional
Kubelet configuration and container mount hiding	extra-manifest/01-container-mount-ns-and-kubelet-conf-worker.yaml	Configures a mount namespace for sharing container-specific mounts between kubelet and CRI-O on worker nodes.	No
One-shot time sync	extra-manifest/99-sync-time-once-master.yaml	Synchronizes time once on master nodes.	No
One-shot time sync	extra-manifest/99-sync-time-once-worker.yaml	Synchronizes time once on worker nodes.	No
SCTP	extra-manifest/03-sctp-machine-config-master.yaml	Loads the SCTP kernel module on master nodes.	Yes
SCTP	extra-manifest/03-sctp-machine-config-worker.yaml	Loads the SCTP kernel module on worker nodes.	Yes
Set RCU normal	extra-manifest/08-set-rcu-normal-master.yaml	Disables rcu_expedited by setting rcu_normal after the control plane node has booted.	No
Set RCU normal	extra-manifest/08-set-rcu-normal-worker.yaml	Disables rcu_expedited by setting rcu_normal after the worker node has booted.	No
SRIOV-related kernel arguments	extra-manifest/07-sriov-related-kernel-args-master.yaml	Enables SR-IOV support on master nodes.	No
SRIOV-related kernel arguments	extra-manifest/07-sriov-related-kernel-args-worker.yaml	Enables SR-IOV support on worker nodes.	No

4.9. COMPARING A CLUSTER WITH THE TELCO RAN DU REFERENCE CONFIGURATION

After you deploy a telco RAN DU cluster, you can use the **cluster-compare** plugin to assess the cluster's

compliance with the telco RAN DU reference design specifications (RDS). The **cluster-compare** plugin is an OpenShift CLI (**oc**) plugin. The plugin uses a telco RAN DU reference configuration to validate the cluster with the telco RAN DU custom resources (CRs).

The plugin-specific reference configuration for telco RAN DU is packaged in a container image with the telco RAN DU CRs.

For further information about the **cluster-compare** plugin, see "Understanding the cluster-compare plugin".

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have credentials to access the **registry.redhat.io** container image registry.
- You installed the **cluster-compare** plugin.

Procedure

1. Log on to the container image registry with your credentials by running the following command:

```
$ podman login registry.redhat.io
```

2. Extract the content from the **ztp-site-generate-rhel8** container image by running the following commands::

```
$ podman pull registry.redhat.io/openshift4/ztp-site-generate-rhel8:v4.22
```

```
$ mkdir -p ./out
```

```
$ podman run --log-driver=none --rm registry.redhat.io/openshift4/ztp-site-generate-rhel8:v4.22 extract /home/ztp --tar | tar x -C ./out
```

3. Compare the configuration for your cluster to the reference configuration by running the following command:

```
$ oc cluster-compare -r out/reference/metadata.yaml
```

Example output

```
...
*****

Cluster CR: config.openshift.io/v1_OperatorHub_cluster
Reference File: required/other/operator-hub.yaml
Diff Output: diff -u -N /tmp/MERGED-2801470219/config-openshift-io-v1_operatorhub_cluster
/tmp/LIVE-2569768241/config-openshift-io-v1_operatorhub_cluster
--- /tmp/MERGED-2801470219/config-openshift-io-v1_operatorhub_cluster 2024-12-12
14:13:22.898756462 +0000
+++ /tmp/LIVE-2569768241/config-openshift-io-v1_operatorhub_cluster 2024-12-12
14:13:22.898756462 +0000
```

```

@@ -1,6 +1,6 @@
apiVersion: config.openshift.io/v1
kind: OperatorHub
metadata:
+ annotations:
+   include.release.openshift.io/hypershift: "true"
+   name: cluster
-spec:
-  disableAllDefaultSources: true

*****

Summary
CRs with diffs: 11/12
CRs in reference missing from the cluster: 40
optional-image-registry:
  image-registry:
    Missing CRs:
    - optional/image-registry/ImageRegistryPV.yaml
optional-ptp-config:
  ptp-config:
    One of the following is required:
    - optional/ptp-config/PtpConfigBoundary.yaml
    - optional/ptp-config/PtpConfigGmWpc.yaml
    - optional/ptp-config/PtpConfigDualCardGmWpc.yaml
    - optional/ptp-config/PtpConfigForHA.yaml
    - optional/ptp-config/PtpConfigMaster.yaml
    - optional/ptp-config/PtpConfigSlave.yaml
    - optional/ptp-config/PtpConfigSlaveForEvent.yaml
    - optional/ptp-config/PtpConfigForHAFOREvent.yaml
    - optional/ptp-config/PtpConfigMasterForEvent.yaml
    - optional/ptp-config/PtpConfigBoundaryForEvent.yaml
  ptp-operator-config:
    One of the following is required:
    - optional/ptp-config/PtpOperatorConfig.yaml
    - optional/ptp-config/PtpOperatorConfigForEvent.yaml
optional-storage:
  storage:
    Missing CRs:
    - optional/local-storage-operator/StorageLV.yaml

...

No CRs are unmatched to reference CRs
Metadata Hash:
09650c31212be9a44b99315ec14d2e7715ee194a5d68fb6d24f65fd5ddbe3c3c
No patched CRs

```

Where:

Cluster CR

The CR under comparison. The plugin displays each CR with a difference from the corresponding template.

Reference File

The template matching with the CR for comparison.

Diff Output

The output in Linux diff format shows the difference between the template and the cluster CR.

Summary

After the plugin reports the line diffs for each CR, the summary of differences are reported.

CRs with diffs

The number of CRs in the comparison with differences from the corresponding templates.

CRs in reference missing from the cluster

The number of CRs represented in the reference configuration, but missing from the live cluster.

Missing CRs

The list of CRs represented in the reference configuration, but missing from the live cluster.

No CRs are unmatched to reference CRs

The CRs that did not match to a corresponding template in the reference configuration.

Metadata Hash

Identifies the reference configuration.

No patched CRs

The list of patched CRs.

Additional resources

- [Understanding the cluster-compare plugin](#)

4.10. TELCO RAN DU 4.22 VALIDATED SOFTWARE COMPONENTS

The Red Hat telco RAN DU 4.22 solution has been validated using the following Red Hat software products for OpenShift Container Platform managed clusters.

Table 4.9. Telco RAN DU managed cluster validated software components

Component	Software version
OpenShift Container Platform	4.22
Cluster Logging Operator	6.5
Local Storage Operator	4.22
OpenShift API for Data Protection (OADP)	1.6
PTP Operator	4.22
SR-IOV Operator	4.22
SRIOV-FEC Operator	2.12

Component	Software version
Lifecycle Agent Operator	4.22

- Cluster Logging Operator will be updated to 6.6 when the aligned Cluster Logging Operator version is released.

Additional resources

- [Telco hub reference configuration software specifications](#)

CHAPTER 5. TELCO HUB REFERENCE DESIGN SPECIFICATION

The telco hub reference design specification (RDS) describes the configuration for a hub cluster that deploys and operates fleets of OpenShift Container Platform clusters in a telco environment.

5.1. REFERENCE DESIGN SCOPE

The telco core, telco RAN and telco hub reference design specifications (RDS) capture the recommended, tested, and supported configurations to get reliable and repeatable performance for clusters running the telco core and telco RAN profiles.

Each RDS includes the released features and supported configurations that are engineered and validated for clusters to run the individual profiles. The configurations provide a baseline OpenShift Container Platform installation that meets feature and KPI targets. Each RDS also describes expected variations for each individual configuration. Validation of each RDS includes many long duration and at-scale tests.



NOTE

The validated reference configurations are updated for each major Y-stream release of OpenShift Container Platform. Z-stream patch releases are periodically re-tested against the reference configurations.

5.2. DEVIATIONS FROM THE REFERENCE DESIGN

Deviating from the validated telco core, telco RAN DU, and telco hub reference design specifications (RDS) can have significant impact beyond the specific component or feature that you change. Deviations require analysis and engineering in the context of the complete solution.



IMPORTANT

All deviations from the RDS should be analyzed and documented with clear action tracking information. Due diligence is expected from partners to understand how to bring deviations into line with the reference design. This might require partners to provide additional resources to engage with Red Hat to work towards enabling their use case to achieve a best in class outcome with the platform. This is critical for the supportability of the solution and ensuring alignment across Red Hat and with partners.

Deviation from the RDS can have some or all of the following consequences:

- It can take longer to resolve issues.
- There is a risk of missing project service-level agreements (SLAs), project deadlines, end provider performance requirements, and so on.
- Unapproved deviations may require escalation at executive levels.



NOTE

Red Hat prioritizes the servicing of requests for deviations based on partner engagement priorities.

5.3. HUB CLUSTER ARCHITECTURE OVERVIEW

Use the features and components running on the management hub cluster to manage many other clusters in a hub-and-spoke topology. The hub cluster provides a highly available and centralized interface for managing the configuration, lifecycle, and observability of the fleet of deployed clusters.



NOTE

All management hub functionality can be deployed on a dedicated OpenShift Container Platform cluster or as applications that are co-resident on an existing cluster.

Managed cluster lifecycle

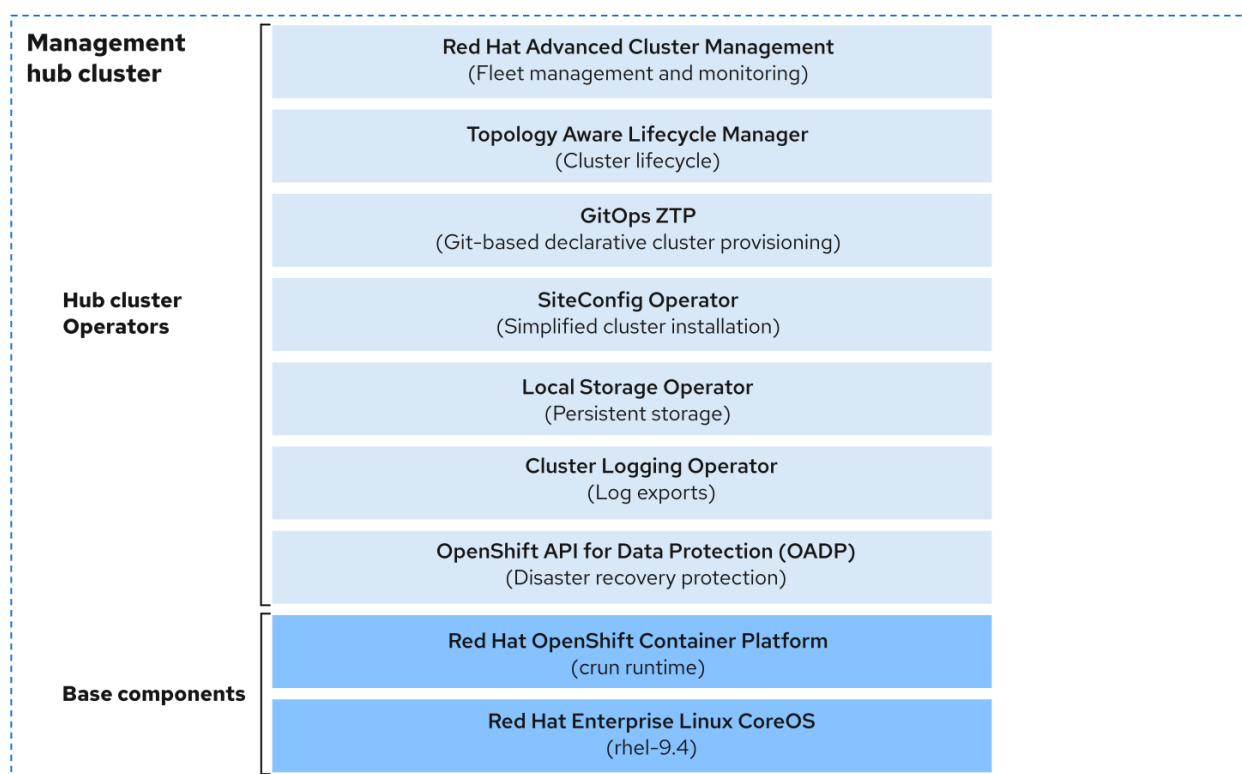
Using a combination of Day 2 Operators, the hub cluster provides the necessary infrastructure to deploy and configure the fleet of clusters by using a GitOps methodology. Over the lifetime of the deployed clusters, further management of upgrades, scaling the number of clusters, node replacement, and other lifecycle management functions can be declaratively defined and rolled out. You can control the timing and progression of the rollout across the fleet.

Monitoring

The hub cluster provides monitoring and status reporting for the managed clusters through the Observability pillar of the RHACM Operator. This includes aggregated metrics, alerts, and compliance monitoring through the Governance policy framework.

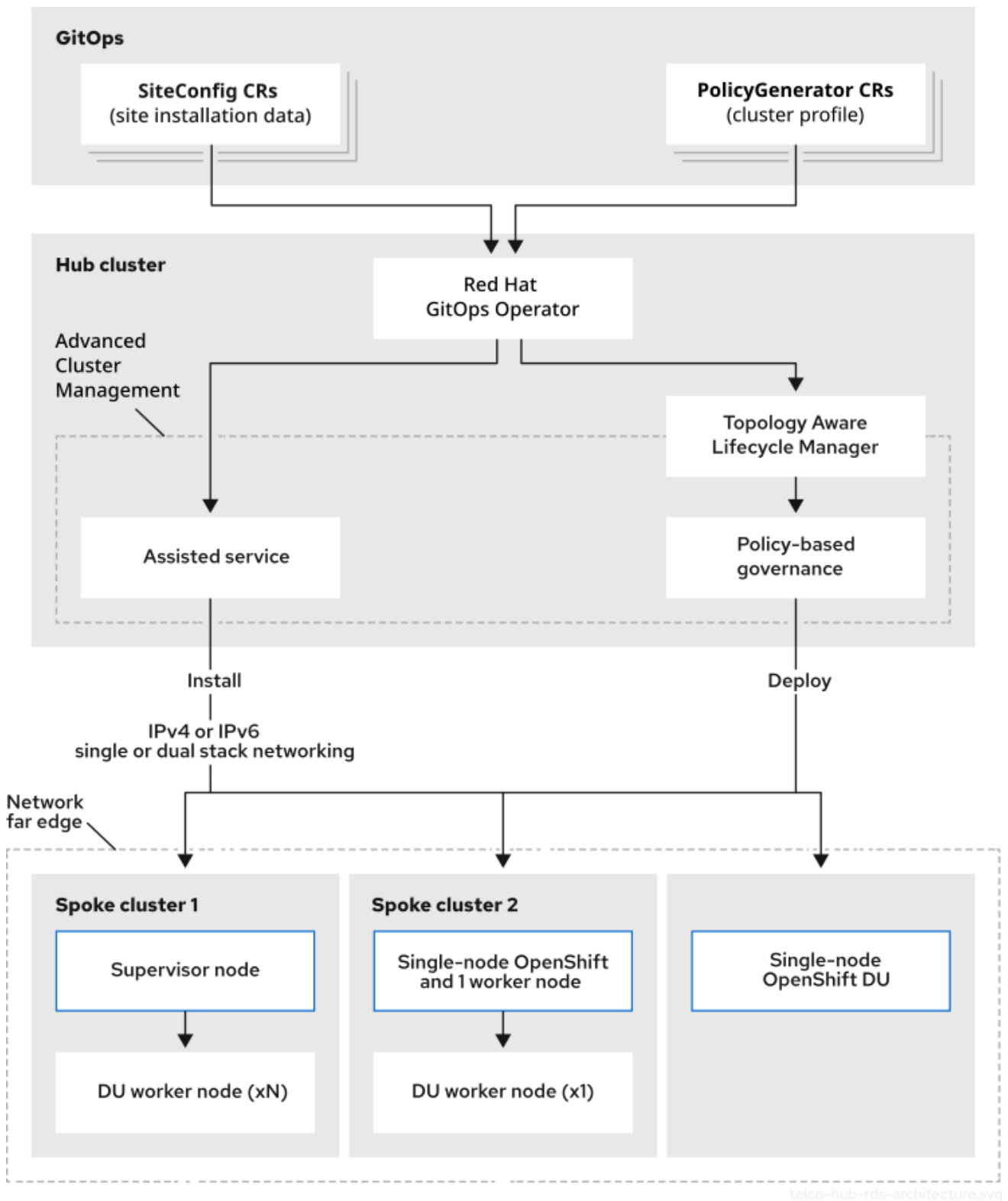
The telco management hub reference design specification (RDS) and the associated reference custom resources (CRs) describe the telco engineering and QE validated method for deploying, configuring and managing the lifecycle of telco managed cluster infrastructure. The reference configuration includes the installation and configuration of the hub cluster components on top of OpenShift Container Platform.

Figure 5.1. Hub cluster reference design components



telco-hub-cluster-reference-design-components.svg

Figure 5.2. Hub cluster reference design architecture



5.4. TELCO MANAGEMENT HUB CLUSTER USE MODEL

The hub cluster provides managed cluster installation, configuration, observability and ongoing lifecycle management for telco application and workload clusters.

Additional resources

- For more information about core clusters or far edge clusters that host RAN distributed unit (DU) workloads, see the following:

- [Telco core RDS](#)
- [Telco RAN DU RDS](#)
- For more information about lifecycle management for the fleet of managed clusters see:
 - [Image-based upgrade for single-node OpenShift clusters](#)
 - [Updating managed clusters with the Topology Aware Lifecycle Manager](#)
 - [Upgrading a telco core CNF cluster](#)
- For more information about declarative cluster provisioning with GitOps ZTP see:
 - [Installing managed clusters with RHACM and SiteConfig resources](#)
- For more information about observability metrics and alerts, see:
 - [Multicluster architecture](#)
 - [Observability](#)

5.5. HUB CLUSTER SCALING TARGET

The resource requirements for the hub cluster are directly dependent on the number of clusters being managed by the hub, the number of policies used for each managed cluster, and the set of features that are configured in Red Hat Advanced Cluster Management (RHACM).

The hub cluster reference configuration can support up to 3500 managed single-node OpenShift clusters under the following conditions:

- 10 RHACM configuration policies (comprising 5 Red Hat-provided policies and up to 5 custom configuration policies) with hub-side templating bound to the 3500 clusters and configured with a 10 minute evaluation interval.
- Only the following RHACM add-ons are enabled:
 - Policy controller
 - Observability with the default configuration
- You deploy managed clusters by using GitOps ZTP in batches of up to 500 clusters at a time.

The reference configuration is also validated for deployment and management of a mix of managed cluster topologies. The specific limits depend on the mix of cluster topologies, enabled RHACM features, and so on. In a mixed topology scenario, the reference hub configuration is validated with a combination of 1200 single-node OpenShift clusters, 400 compact clusters (3 nodes combined control plane and compute nodes), and 230 standard clusters (3 control plane and 2 worker nodes).

A hub cluster conforming to this reference specification can support synchronization of 1000 single node **ClusterInstance** CRs for each ArgoCD application. You can use multiple applications to achieve the maximum number of clusters supported by a single hub cluster.

**NOTE**

Specific dimensioning requirements are highly dependent on the cluster topology and workload. For more information, see "Storage requirements". Adjust cluster dimensions for the specific characteristics of your fleet of managed clusters.

5.6. HUB CLUSTER RESOURCE UTILIZATION

Resource utilization was measured for deploying hub clusters in the following scenario:

- Under reference load managing 3500 single-node OpenShift clusters.
- 3-node compact cluster for management hub running on dual socket bare-metal servers.
- Network impairment of 50 ms round-trip latency, 100 Mbps bandwidth limit and 0.02% packet loss.
- Observability was not enabled.
- Only local storage was used.

Table 5.1. Resource utilization values

Metric	Peak Measurement
OpenShift Platform CPU	106 cores (52 cores peak per node)
OpenShift Platform memory	504 G (168 G peak per node)

Additional resources

- [Comparison of hub cluster and managed cluster templates](#)

5.7. HUB CLUSTER TOPOLOGY

In production environments, the OpenShift Container Platform hub cluster must be highly available to maintain high availability of the management functions.

Limits and requirements

Use a highly available cluster topology for the hub cluster, for example:

- Compact (3 nodes combined control plane and compute nodes)
- Standard (3 control plane nodes + N compute nodes)

Engineering considerations

- In non-production environments, a single-node OpenShift cluster can be used for limited hub cluster functionality.
- Certain capabilities, for example Red Hat OpenShift Data Foundation, are not supported on single-node OpenShift. In this configuration, some hub cluster features might not be available.

- The number of optional compute nodes can vary depending on the scale of the specific use case.
- Compute nodes can be added later as required.
- Consult the 4.21 release notes regarding the decrease in the default maximum open files soft limit for containers in this release.

Additional resources

- [OpenShift Container Platform architecture](#)
- [Postinstallation node tasks](#)

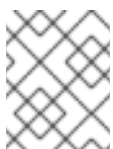
5.8. HUB CLUSTER NETWORKING

The reference hub cluster is designed to operate in a disconnected networking environment where direct access to the internet is not possible. As with all OpenShift Container Platform clusters, the hub cluster requires access to an image registry hosting all OpenShift and Day 2 Operator Lifecycle Manager (OLM) images.

The hub cluster supports dual-stack networking support for IPv6 and IPv4 networks. IPv6 is typical in edge or far-edge network segments, while IPv4 is more prevalent for use with legacy equipment in the data center.

Limits and requirements

- Regardless of the installation method, you must configure the following network types for the hub cluster:
 - **clusterNetwork**
 - **serviceNetwork**
 - **machineNetwork**
- You must configure the following IP addresses for the hub cluster:
 - **apiVIP**
 - **ingressVIP**



NOTE

For the above networking configurations, some values are required, or can be auto-assigned, depending on the chosen architecture and DHCP configuration.

- You must use the default OpenShift Container Platform network provider OVN-Kubernetes.
- Networking between the managed cluster and hub cluster must meet the networking requirements in the Red Hat Advanced Cluster Management (RHACM) documentation, for example:
 - Hub cluster access to managed cluster API service, Ironi Python agent, and baseboard management controller (BMC) port.

- Managed cluster access to hub cluster API service, ingress IP and control plane node IP addresses.
- Managed cluster BMC access to hub cluster control plane node IP addresses.
- An image registry must be accessible throughout the lifetime of the hub cluster.
 - All required container images must be mirrored to the disconnected registry. All OpenShift Container Platform releases and OLM Operator release images needed in your deployment must be mirrored to the registry. You can see example mirroring configuration in the **imageset-config.yaml** resource. You must update this example YAML to include your required versions. For deploying clusters, you can only use **ClusterImageSet** CRs that reference mirrored versions.
 - The hub cluster must be configured to use a disconnected registry.
 - The hub cluster cannot host its own image registry. For example, the registry must be available in a scenario where a power failure affects all cluster nodes.

Engineering considerations

- When deploying a hub cluster, ensure you define appropriately sized CIDR range definitions.
- In OpenShift Container Platform 4.22 and later, pulling OpenShift images from a disconnected mirror registry requires copying the image signatures into that registry during the mirroring process. The **oc adm mirror** command does not mirror signatures and must not be used. Instead, use the **oc mirror** plugin v2 to ensure signatures are properly mirrored.

Additional resources

- [Mirroring images for a disconnected installation by using the oc-mirror plugin v2](#)
- [Installing a cluster in a disconnected environment](#)
- [Using Operator Lifecycle Manager on restricted networks](#)
- [Configuring the hub cluster to use a disconnected mirror registry](#)
- [CIDR range definitions](#)
- [Installing OpenShift Container Platform](#)
- [Networking in OpenShift Container Platform](#)
- [Networking in RHACM](#)
- [Network configuration in RHACM](#)

5.9. HUB CLUSTER MEMORY AND CPU REQUIREMENTS

The memory and CPU requirements of the hub cluster vary depending on the configuration of the hub cluster, the number of resources on the cluster, and the number of managed clusters.

Limits and requirements

- Ensure that the hub cluster meets the underlying memory and CPU requirements for OpenShift Container Platform and Red Hat Advanced Cluster Management (RHACM).

Engineering considerations

- Before deploying a telco hub cluster, ensure that your cluster host meets cluster requirements.

For more information about scaling the number of managed clusters, see "Hub cluster scaling target".

Additional resources

- [Scaling your OpenShift Container Platform cluster and tuning performance in production environments](#)
- [Sizing your cluster](#)

5.10. HUB CLUSTER STORAGE REQUIREMENTS

The total amount of storage required by the management hub cluster is dependant on the storage requirements for each of the applications deployed on the cluster. The main components that require storage through highly available **PersistentVolume** resources are described in the following sections.



NOTE

The storage required for the underlying OpenShift Container Platform installation is separate to these requirements.

5.10.1. Assisted Service

The Assisted Service is deployed with the multicluster engine and Red Hat Advanced Cluster Management (RHACM).



NOTE

The following numbers are estimates. Tune the values for more accurate results. Add an engineering margin, for example +20%, to the results to account for potential estimation inaccuracies.

Table 5.2. Assisted Service storage requirements

Persistent volume resource	Size (GB)
imageStorage ^[1]	30
filesystemStorage ^[2]	709
dataBaseStorage ^[3]	0.7

[1][2] For more information, refer to the multicluster engine Operator documentation [About enabling central infrastructure management](#).

[3] The **databaseStorage** value is an empirical estimate based on cluster topology, number of installation events, hardware profile, and configuration complexity. Based on empirical testing, estimate approximately 200 KB per host.

Additional resources

- [Enabling central infrastructure management in disconnected environments](#)

5.10.2. RHACM Observability

Cluster Observability is provided by the multicluster engine and Red Hat Advanced Cluster Management (RHACM).

- Observability storage needs several **PV** resources and an S3 compatible bucket storage for long-term retention of the metrics.
- Storage requirements calculation is complex and dependent on the specific workloads and characteristics of managed clusters. Requirements for **PV** resources and the S3 bucket depend on many aspects including data retention, the number of managed clusters, managed cluster workloads, and so on.
- Estimate the required storage for observability by using the observability sizing calculator in the RHACM capacity planning repository. See the Red Hat Knowledgebase article [Calculating storage need for MultiClusterHub Observability on telco environments](#) for an explanation of using the calculator to estimate observability storage requirements. The below table uses inputs derived from the telco RAN DU RDS and the hub cluster RDS as representative values.

The following numbers are estimates. Tune the values for more accurate results. Add an engineering margin, for example +20%, to the results to account for potential estimation inaccuracies.

Storage resources depend on the number of replicas for each component. You can configure the sizing for the Observability stack in the **MultiClusterObservability** custom resource. The number of replicas scales with the sizing configuration. The sizing values in this specification use the default size.

Table 5.3. Cluster requirements

Capacity planner input	Data source	Example value
Number of control plane nodes	Hub cluster RDS (scale) and telco RAN DU RDS (topology)	3500
Number of additional worker nodes	Hub cluster RDS (scale) and telco RAN DU RDS (topology)	0
Days for storage of data	Hub cluster RDS	15
Total number of pods per cluster	Telco RAN DU RDS	120

Capacity planner input	Data source	Example value
Number of namespaces (excluding OpenShift Container Platform)	Telco RAN DU RDS	4
Number of metric samples per hour	Default value	12
Number of hours of retention in receiver persistent volume (PV)	Default value	24

With these input values, the sizing calculator as described in the Red Hat Knowledgebase article [Calculating storage need for MultiClusterHub Observability on telco environments](#) indicates the following storage needs:

Table 5.4. Storage requirements

alertmanager PV		thanos receive PV		thanos compact PV
Per replica	Total	Per replica	Total	Total
10 GiB	30 GiB	10 GiB	30 GiB	100 GiB

Table 5.5. Storage requirements

thanos rule PV	thanos store PV	Object bucket ^[1]		
Per replica	Total	Per replica	Total	Total
30 GiB	90 GiB	100 GiB	300 GiB	310 GiB

[1] This value assumes downsampling is enabled. You cannot configure the object bucket size in the **MultiClusterObservability** CR. Ensure this storage capacity is available in your environment.

5.10.3. Storage considerations

Limits and requirements

- Minimum OpenShift Container Platform and Red Hat Advanced Cluster Management (RHACM) limits apply
- High availability should be provided through a storage backend. The hub cluster reference configuration provides storage through Red Hat OpenShift Data Foundation.
- Object bucket storage is provided through OpenShift Data Foundation.

Engineering considerations

- Use SSD or NVMe disks with low latency and high throughput for etcd storage.

- You must use clean storage disks with OpenShift Data Foundation, including before a re-install procedure. See "ODF disks cleaning procedure" for further information.
- The storage solution for telco hub clusters is OpenShift Data Foundation.
 - Local Storage Operator supports the storage class used by OpenShift Data Foundation to provide block, file, and object storage as needed by other components on the hub cluster.
- The Local Storage Operator **LocalVolume** configuration includes setting **forceWipeDevicesAndDestroyAllData: true** to support the reinstallation of hub cluster nodes where OpenShift Data Foundation has previously been used.

Additional resources

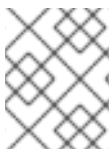
- [ODF disks cleaning procedure](#)
- [Persistent storage overview](#)
- [OpenShift Data Foundation architecture](#)
- [Persistent storage using local volumes](#)
- [Recommended etcd practices](#)

5.11. GIT REPOSITORY

The telco management hub cluster supports a GitOps-driven methodology for installing and managing the configuration of OpenShift clusters for various telco applications. This methodology requires an accessible Git repository that serves as the authoritative source of truth for cluster definitions and configuration artifacts.

Red Hat does not offer a commercially supported Git server. An existing Git server provided in the production environment can be used. Gitea and Gogs are examples of self-hosted Git servers that you can use.

The Git repository is typically provided in the production network external to the hub cluster. In a large-scale deployment, multiple hub clusters can use the same Git repository for maintaining the definitions of managed clusters. Using this approach, you can easily review the state of the complete network. As the source of truth for cluster definitions, the Git repository should be highly available and recoverable in disaster scenarios.



NOTE

For disaster recovery and multi-hub considerations, run the Git repository separately from the hub cluster.

Limits and requirements

- A Git repository is required to support the GitOps ZTP functions of the hub cluster, including installation, configuration, and lifecycle management of the managed clusters.
- The Git repository must be accessible from the management cluster.

Engineering considerations

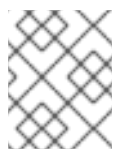
- The Git repository is used by the GitOps Operator to ensure continuous deployment and a single source of truth for the applied configuration.

5.12. OPENSIFT CONTAINER PLATFORM INSTALLATION ON THE HUB CLUSTER

Description

The reference method for installing OpenShift Container Platform for the hub cluster is through the Agent-based Installer.

Agent-based Installer provides installation capabilities without additional centralized infrastructure. The Agent-based Installer creates an ISO image, which you mount to the server to be installed. When you boot the server, OpenShift Container Platform is installed alongside optionally supplied extra manifests, such as Red Hat OpenShift GitOps custom resources.



NOTE

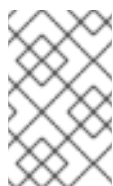
You can also install OpenShift Container Platform in the hub cluster by using other installation methods.

If hub cluster functions are being applied to an existing OpenShift Container Platform cluster, the Agent-based Installer installation is not required. The remaining steps to install Day 2 Operators and configure the cluster for these functions remains the same. When OpenShift Container Platform installation is complete, the set of additional Operators and their configuration must be installed on the hub cluster.

The reference configuration includes all of these custom resources (CRs), which you can apply manually, for example:

```
$ oc apply -f <reference_cr>
```

You can also add the reference configuration to the Git repository and apply it using ArgoCD.



NOTE

If you apply the CRs manually, ensure you apply the CRs in the order of their dependencies. For example, apply namespaces before Operators and apply Operators before configurations.

Limits and requirements

- Agent-based Installer requires an accessible image repository containing all required OpenShift Container Platform and Day 2 Operator images.
- Agent-based Installer builds ISO images based on a specific OpenShift releases and specific cluster details. Installation of a second hub requires a separate ISO image to be built.

Engineering considerations

- Agent-based Installer provides a baseline OpenShift Container Platform installation. You apply Day 2 Operators and other configuration CRs after the cluster is installed.

- The reference configuration supports Agent-based Installer installation in a disconnected environment.
- A limited set of additional manifests can be supplied at installation time.

Additional resources

- [OpenShift Container Platform installation overview](#)
- [Installing a cluster with customizations](#)
- [Preparing to install with the Agent-based Installer](#)

5.13. DAY 2 OPERATORS IN THE HUB CLUSTER

The management hub cluster relies on a set of Day 2 Operators to provide critical management services and infrastructure. Use Operator versions that match the set of managed cluster versions in your fleet.

Install Day 2 Operators using Operator Lifecycle Manager (OLM) and **Subscription** custom resources (CRs). **Subscription** CRs identify the specific Day 2 Operator to install, the catalog in which the Operator is found, and the appropriate version channel for the Operator. By default OLM installs and attempt to keep Operators updated with the latest z-stream version available in the channel. By default all Subscriptions are set with an **installPlanApproval: Automatic** value. In this mode, OLM automatically installs new Operator versions when they are available in the catalog and channel.



NOTE

Setting **installPlanApproval** to automatic exposes the risk of the Operator being updated outside of defined maintenance windows if the catalog index is updated to include newer Operator versions. In a disconnected environment where you are building and maintaining a curated set of Operators and versions in the catalog, and if you follow a strategy of creating a new catalog index for updated versions, the risk of the Operators being inadvertently updated is largely removed. However, if you want to further close this risk, the **Subscription** CRs can be set to **installPlanApproval: Manual** which prevents Operators from being updated without explicit administrator approval.

Limits and requirements

- When upgrading a telco hub cluster, the versions of OpenShift Container Platform and Operators must meet the requirements of all relevant compatibility matrixes.

Additional resources

- [Red Hat Advanced Cluster Management for Kubernetes 2.11 Support Matrix](#)
- [OpenShift Operator lifecycles](#)
- For more information about telco hub cluster update requirements, see:
 - [Recommended hub cluster specifications and managed cluster limits for GitOps ZTP](#) .
 - [Red Hat Advanced Cluster Management for Kubernetes 2.11 Support Matrix](#)

- [OpenShift Operator Life Cycles](#)
- For more information about updating the hub cluster, see:
 - [Introduction to OpenShift updates](#)
 - [Upgrading your hub cluster](#)
 - [Updating GitOps ZTP](#)

5.14. OBSERVABILITY

The Red Hat Advanced Cluster Management (RHACM) multicluster engine Observability component provides centralized aggregation and visualization of metrics and alerts for all managed clusters. To balance performance and data analysis, the monitoring service maintains a subset list of aggregated metrics that are collected at a downsampled interval. The metrics can be accessed on the hub through a set of different preconfigured dashboards.

Observability installation

The primary custom resource (CR) to enable and configure the observability service is the **MulticlusterObservability** CR, which defines the following settings:

- Configurable retention settings.
- Storage for the different components: **thanos receive**, **thanos compact**, **thanos rule**, **thanos store** sharding, **alertmanager**.
- The **metadata.annotations.mco-disable-alerting="true"** annotation that enables tuning for the monitoring configuration on managed clusters.



NOTE

Without this setting the Observability component attempts to configure the managed cluster monitoring configuration. With this value set you can merge your desired configuration with the necessary Observability configuration of alert forwarding into the managed cluster monitoring **ConfigMap** object. When the Observability service is enabled RHACM will deploy to each managed cluster a workload to push metrics and alerts generated by local Monitoring to the hub cluster. The metrics and alerts to be forwarded from the managed cluster to the hub, are defined by a **ConfigMap** CR in the **open-cluster-management-addon-observability** namespace. You can also specify custom metrics. For more information, see [Adding custom metrics](#).

Alertmanager configuration

- The hub cluster provides an Observability Alertmanager that can be configured to push alerts to external systems, for example, email. The Alertmanager is enabled by default.
- You must configure alert forwarding.
- When the Alertmanager is enabled but not configured, the hub Alertmanager does not forward alerts externally.
- When Observability is enabled, the managed clusters can be configured to send alerts to any endpoint including the hub Alertmanager.

- When a managed cluster is configured to forward alerts to external sources, alerts are not routed through the hub cluster Alertmanager.
- Alert state is available as a metric.
- When observability is enabled, the managed cluster alert states are included in the subset of metrics forwarded to the hub cluster and are available through Observability dashboards.

Limits and requirements

- Observability requires persistent object storage for long-term metrics. For more information, see "Storage requirements".

Engineering considerations

- Forwarding of metrics is a subset of the full metric data. It includes only the metrics defined in the **observability-metrics-allowlist** config map and any custom metrics added by the user.
- Metrics are forwarded at a downsampled rate. Metrics are forwarded by taking the latest datapoint at a 5 minute interval (or as defined by the **MultiClusterObservability** CR configuration).
- A network outage may lead to a loss of metrics forwarded to the hub cluster during that interval. This can be mitigated if metrics are also forwarded directly from managed clusters to an external metrics collector in the providers network. Full resolution metrics are available on the managed cluster.
- In addition to default metrics dashboards on the hub, users may define custom dashboards.
- The reference configuration is sized based on 15 days of metrics storage by the hub cluster for 3500 single-node OpenShift clusters. If longer retention or other managed cluster topology or sizing is required, the storage calculations must be updated and sufficient storage capacity be maintained. For more information about calculating new values, see "Storage requirements".

Additional resources

- For more information about observability, see:
 - [Exporting metrics to external endpoints](#)
 - [Enabling the Observability service](#)
- For more information about custom metrics, see [Adding custom metrics](#)
- For more information about forwarding alerts to other external systems, see [Forwarding alerts](#)
- For more information about CPU and memory requirements see: [Observability pod capacity requests](#)
- For more information about custom dashboards, see [Using Grafana dashboards](#)

5.15. MANAGED CLUSTER LIFECYCLE MANAGEMENT

To provision and manage sites at the far edge of the network, use GitOps ZTP in a hub-and-spoke architecture, where a single hub cluster manages many managed clusters.

Lifecycle management for spoke clusters can be divided into two different stages: cluster deployment, including OpenShift Container Platform installation, and cluster configuration.

Additional resources

- [Challenges of the network far edge](#)

5.15.1. Managed cluster deployment

Description

As of Red Hat Advanced Cluster Management (RHACM) 2.12, using the SiteConfig Operator is the recommended method for deploying managed clusters. The SiteConfig Operator introduces a unified ClusterInstance API that decouples the parameters that define the cluster from the manner in which it is deployed. The SiteConfig Operator uses a set of cluster templates that are instantiated using the data from a **ClusterInstance** custom resource (CR) to dynamically generate installation manifests. Following the GitOps methodology, the **ClusterInstance** CR is sourced from a Git repository through ArgoCD. The **ClusterInstance** CR can be used to initiate cluster installation by using either Assisted Installer, or the image-based installation available in multicluster engine.

Limits and requirements

- The **SiteConfig** ArgoCD plugin which handles **SiteConfig** CRs is removed from OpenShift Container Platform 4.21. From this release, use **ClusterInstance** CRs to define managed cluster deployments.
- An HTTP server hosting the root filesystem and RHCOS live ISO images is required for cluster deployment. These images are release specific. ISO images for each OpenShift Container Platform release to be deployed must be reachable by the hub cluster and each deployed spoke cluster. Only include ISO images which exist on the HTTP server in the **AgentServiceConfig** CR.
- A container registry hosting all OpenShift Container Platform and day-2 OLM operator images must be reachable from all deployed spoke clusters. The hub configuration includes Kustomize overlays which you can use to provide the TLS certificates and credentials for a disconnected container registry.

Engineering considerations

- You must create a **Secret** CR with the login information for the cluster baseboard management controller (BMC). This **Secret** CR is then referenced in the **SiteConfig** CR. Integration with a secret store, such as Vault, can be used to manage the secrets.
- Besides offering deployment method isolation and unification of Git and non-Git workflows, the SiteConfig Operator provides better scalability, greater flexibility with the use of custom templates, and an enhanced troubleshooting experience.

Additional resources

- [SiteConfig](#)
- [ClusterInstance](#)

- [Creating the managed bare-metal host secrets](#)

5.15.2. Managed cluster updates

Description

You can upgrade versions of OpenShift Container Platform, Day 2 Operators, and managed cluster configurations, by declaring the required version in the **Policy** custom resources (CRs) that target the clusters to be upgraded.

Policy controllers periodically check for policy compliance. If the result is negative, a violation report is created. If the policy remediation action is set to **enforce** the violations are remediated according to the updated policy. If the policy remediation action is set to **inform**, the process ends with a non-compliant status report and responsibility to initiate the upgrade is left to the user to perform during an appropriate maintenance window.

The Topology Aware Lifecycle Manager (TALM) extends Red Hat Advanced Cluster Management (RHACM) with features to manage the rollout of upgrades or configuration throughout the lifecycle of the fleet of clusters. It operates in progressive, limited size batches of clusters. When upgrades to OpenShift Container Platform or the Day 2 Operators are required, TALM progressively rolls out the updates by stepping through the set of policies and switching them to an "enforce" policy to push the configuration to the managed cluster.

The custom resource (CR) that TALM uses to build the remediation plan is the **ClusterGroupUpgrade** CR.

You can use image-based upgrade (IBU) with the Lifecycle Agent as an alternative upgrade path for the single-node OpenShift cluster platform version. IBU uses an OCI image generated from a dedicated seed cluster to install single-node OpenShift on the target cluster.

TALM uses the **ImageBasedGroupUpgrade** CR to roll out image-based upgrades to a set of identified clusters.

Limits and requirements

- You can perform direct upgrades for single-node OpenShift clusters using image-based upgrade for OpenShift Container Platform **<4.y>** to **<4.y+2>**, and **<4.y.z>** to **<4.y.z+n>**.
- Image-based upgrade uses custom images that are specific to the hardware platform that the clusters are running on. Different hardware platforms require separate seed images.

Engineering considerations

- In edge deployments, you can minimize the disruption to managed clusters by managing the timing and rollout of changes. Set all policies to **inform** to monitor compliance without triggering automatic enforcement. Similarly, configure Day 2 Operator subscriptions to manual to prevent updates from occurring outside of scheduled maintenance windows.
- The recommended upgrade approach for single-node OpenShift clusters is the image-based upgrade.
- For multi-node cluster upgrades, consider the following **MachineConfigPool** CR configurations to reduce upgrade times:
 - Pause configuration deployments to nodes during a maintenance window by setting the **paused** field to **true**.

- Adjust the **maxUnavailable** field to control how many nodes in the pool can be updated simultaneously. The **MaxUnavailable** field defines the percentage of nodes in the pool that can be simultaneously unavailable during a **MachineConfig** object update. Set **maxUnavailable** to the maximum tolerable value. This reduces the number of reboots in a cluster during upgrades which results in shorter upgrade times.
- Resume configuration deployments by setting the **paused** field to **false**. The configuration changes are applied in a single reboot.
- During cluster installation, you can pause **MachineConfigPool** CRs by setting the **paused** field to **true** and setting **maxUnavailable** to 100% to improve installation times.

Additional resources

- [Configuration policy YAML structure](#)
- [About the ClusterGroupUpgrade CR](#)
- [Understanding the image-based upgrade for single-node OpenShift clusters](#)
- [Performing an image-based upgrade for single-node OpenShift clusters using GitOps ZTP](#)

5.15.3. Hub cluster disaster recovery

Note that loss of the hub cluster does not typically create a service outage on the managed clusters. Functions provided by the hub cluster will be lost, such as observability, configuration, lifecycle management updates being driven through the hub cluster, and so on.

Limits and requirements

- Backup, restore and disaster recovery are offered by the cluster backup and restore Operator, which depends on the OpenShift API for Data Protection (OADP) Operator.

Engineering considerations

- You can extend the cluster backup and restore operator to third party resources of the hub cluster based on your configuration.
- The cluster backup and restore operator is not enabled by default in Red Hat Advanced Cluster Management (RHACM). The reference configuration enables this feature.

Additional resources

- [Business continuity](#)

5.16. HUB CLUSTER COMPONENTS

5.16.1. Red Hat Advanced Cluster Management (RHACM)

New in this release

- No reference design updates in this release.

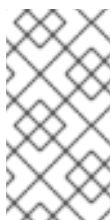
Description

Red Hat Advanced Cluster Management (RHACM) provides multicluster engine installation and ongoing lifecycle management functionality for deployed clusters. You can manage cluster configuration and upgrades declaratively by applying **Policy** custom resources (CRs) to clusters during maintenance windows.

RHACM provides functionality such as the following:

- Zero touch provisioning (ZTP) and ongoing scaling of clusters using the multicluster engine component in RHACM.
- Configuration, upgrades, and cluster status through the RHACM policy controller.
- During managed cluster installation, RHACM can apply labels to individual nodes as configured through the **ClusterInstance** CR.
- The Topology Aware Lifecycle Manager component of RHACM provides phased rollout of configuration changes to managed clusters.
- The RHACM multicluster engine Observability component provides selective monitoring, dashboards, alerts, and metrics. The recommended method for single-node OpenShift cluster installation is the image-based installation method in multicluster engine, which uses the **ClusterInstance** CR for cluster definition.

The recommended method for single-node OpenShift upgrade is the image-based upgrade method.



NOTE

The RHACM multicluster engine Observability component brings you a centralized view of the health and status of all the managed clusters. By default, every managed cluster is enabled to send metrics and alerts, created by their Cluster Monitoring Operator (CMO), back to Observability. For more information, see "Observability".

Limits and requirements

- For more information about limits on number of clusters managed by a single hub cluster, see "Telco management hub cluster use model".
- The number of managed clusters that can be effectively managed by the hub depends on various factors, including:
 - Resource availability at each managed cluster
 - Policy complexity and cluster size
 - Network utilization
 - Workload demands and distribution
- The hub and managed clusters must maintain sufficient bidirectional connectivity. Refer to the RHACM Hub Network Configuration for further details.

Engineering considerations

- You can configure the cluster backup and restore Operator to include third-party resources.

- The use of RHACM hub side templating when defining configuration through policy is strongly recommended. This feature reduces the number of policies needed to manage the fleet by enabling for each cluster or for each group. For example, regional or hardware type content to be templated in a policy and substituted on cluster or group basis.
- Managed clusters typically have some number of configuration values which are specific to an individual cluster. These should be managed using RHACM policy hub side templating with values pulled from **ConfigMap** CRs based on the cluster name.

Additional resources

- * [Hub Network Configuration](#)
- [Multi Cluster Engine](#)
- [Governance](#)
- [Topology Aware Lifecycle Manager](#)
- [MultiClusterHub Observability](#)
- [Business continuity](#)
- [Performance and scalability](#)
- [Network configuration](#)

5.16.2. Topology Aware Lifecycle Manager

New in this release

- No reference design updates in this release.

Description

TALM is an Operator that runs only on the hub cluster for managing how changes like cluster upgrades, Operator upgrades, and cluster configuration are rolled out to the network. TALM supports the following features:

- Progressive rollout of policy updates to fleets of clusters in user configurable batches.
- Per-cluster actions add **ztp-done** labels or other user-configurable labels following configuration changes to managed clusters.
- TALM supports optional pre-caching of OpenShift Container Platform, OLM Operator, and additional images to single-node OpenShift clusters before initiating an upgrade. The pre-caching feature is not applicable when using the recommended image-based upgrade method for upgrading single-node OpenShift clusters.
 - Specifying optional pre-caching configurations with **PreCachingConfig** CRs.
 - Configurable image filtering to exclude unused content.
 - Storage validation before and after pre-caching, using defined space requirement parameters.

Limits and requirements

- TALM supports concurrent cluster upgrades in batches of 500.
- Pre-caching is limited to single-node OpenShift cluster topology.

Engineering considerations

- The **PreCachingConfig** custom resource (CR) is optional. You do not need to create it if you want to pre-cache platform-related images only, such as OpenShift Container Platform and OLM.
- TALM supports the use of hub-side templating with Red Hat Advanced Cluster Management policies.

5.16.3. GitOps Operator and GitOps ZTP

New in this release

- No reference design updates in this release

Description

GitOps Operator and GitOps ZTP provide a GitOps-based infrastructure for managing cluster deployment and configuration. Cluster definitions and configurations are maintained as a declarative state in Git. You can apply **ClusterInstance** custom resources (CRs) to the hub cluster where the **SiteConfig** Operator renders them as installation CRs. In earlier releases, a GitOps ZTP plugin supported the generation of installation CRs from **SiteConfig** CRs. This plugin is now deprecated. A separate GitOps ZTP plugin is available to enable automatic wrapping of configuration CRs into policies based on the **PolicyGenerator** or the **PolicyGenTemplate** CRs.

You can deploy and manage multiple versions of OpenShift Container Platform on managed clusters by using the baseline reference configuration CRs. You can use custom CRs alongside the baseline CRs. To maintain multiple per-version policies simultaneously, use Git to manage the versions of the source and policy CRs by using the **PolicyGenerator** or the **PolicyGenTemplate** CRs.

Limits and requirements

- To ensure consistent and complete cleanup of managed clusters and their associated resources during cluster or node deletion, you must configure ArgoCD to use background deletion mode.

Engineering considerations

- To avoid confusion or unintentional overwrite when updating content, use unique and distinguishable names for custom CRs in the **source-crs** directory and extra manifests.
- Keep reference source CRs in a separate directory from custom CRs. This facilitates easy update of reference CRs as required.
- To help with multiple versions, keep all source CRs and policy creation CRs in versioned Git repositories to ensure consistent generation of policies for each OpenShift Container Platform version.

Additional resources

- [ClusterInstance CR](#)
- [PolicyGenTemplate CRs](#)
- [GitOps ZTP version independence](#)

5.16.4. Local Storage Operator

New in this release

- No reference design updates in this release

Description

You can create persistent volumes that can be used as **PVC** resources by applications with the Local Storage Operator. The number and type of **PV** resources that you create depends on your requirements.

Engineering considerations

- Create backing storage for **PV** CRs before creating the persistent volume. This can be a partition, a local volume, LVM volume, or full disk.
- Refer to the device listing in **LocalVolume** CRs by the hardware path used to access each device to ensure correct allocation of disks and partitions, for example, **/dev/disk/by-path/<id>**. Logical names (for example, **/dev/sda**) are not guaranteed to be consistent across node reboots.

5.16.5. Red Hat OpenShift Data Foundation

New in this release

- No reference design updates in this release

Description

Red Hat OpenShift Data Foundation provides file, block, and object storage services to the hub cluster.

Limits and requirements

- Red Hat OpenShift Data Foundation (ODF) in internal mode requires the Local Storage Operator to define a storage class which will provide the necessary underlying storage.
- When doing the planning for a telco management cluster, consider the ODF infrastructure and networking requirements.
- Dual stack support is limited. ODF IPv4 is supported on dual-stack clusters.

Engineering considerations

- Address capacity warnings promptly as recovery can be difficult in case of storage capacity exhaustion, see [Capacity planning](#).

Additional resources

- [Support OpenShift dual stack with OpenShift Data Foundation using IPv4](#)
- [Infrastructure requirements](#)
- [Network requirements](#)
- [Storage cluster deployment approaches](#)

5.16.6. Logging

New in this release

- No reference design updates in this release

Description

Use the Cluster Logging Operator to collect and ship logs off the node for remote archival and analysis. The reference configuration uses Kafka to ship audit and infrastructure logs to a remote archive.

Limits and requirements

- The reference configuration does not include local log storage.
- The reference configuration does not include aggregation of managed cluster logs at the hub cluster.

Engineering considerations

- The impact of cluster CPU use is based on the number or size of logs generated and the amount of log filtering configured.
- The reference configuration does not include shipping of application logs. The inclusion of application logs in the configuration requires you to evaluate the application logging rate and have sufficient additional CPU resources allocated to the reserved set.

5.16.7. OpenShift API for Data Protection

New in this release

- No reference design updates in this release

Description

The OpenShift API for Data Protection (OADP) Operator is automatically installed and managed by Red Hat Advanced Cluster Management (RHACM) when the backup feature is enabled.

The OADP Operator facilitates the backup and restore of workloads in OpenShift Container Platform clusters. Based on the upstream open source project Velero, it allows you to backup and restore all Kubernetes resources for a given project, including persistent volumes.

While it is not mandatory to have it on the hub cluster, it is highly recommended for cluster backup, disaster recovery and high availability architecture for the hub cluster. The OADP Operator must be enabled to use the disaster recovery solutions for RHACM. The reference configuration enables backup (OADP) through the **MultiClusterHub** custom resource (CR) provided by the RHACM Operator.

Limits and requirements

- Only one version of OADP can be installed on a cluster. The version installed by RHACM must be used for RHACM disaster recovery features.

Engineering considerations

- No engineering consideration updates in this release.

5.16.8. cert-manager Operator

The cert-manager Operator for OpenShift Container Platform manages the lifecycle of TLS certificates for hub cluster components and workloads.

New in this release

- No reference design updates in this release.

Description

The cert-manager Operator for OpenShift Container Platform manages the lifecycle of TLS certificates for cluster components and workloads. The cert-manager Operator automates certificate issuance, renewal, and rotation, eliminating manual certificate management. The reference configuration includes the cert-manager Operator to optionally manage certificates for the API server and ingress controller endpoints.

You can use RHACM **CertificatePolicy** resources to monitor certificate health across all managed clusters.

Limits and requirements

- The reference configuration includes only the ACME DNS01 challenge type for platform certificate issuance.

Engineering considerations

- Use RHACM **CertificatePolicy** resources on the hub cluster to monitor certificate expiration and compliance across managed clusters.
- Optionally, configure a **PrometheusRule** on the hub cluster to generate alerts based on policy compliance status.

5.17. EXTRACTING THE TELCO HUB REFERENCE DESIGN CONFIGURATION CRS

You can extract the complete set of custom resources (CRs) for the telco hub profile from the **openshift-telco-hub-rds-rhel9** container image. The container image has both the required CRs, and the optional CRs, for the telco hub profile.

Prerequisites

- You have installed **podman**.

Procedure

1. Log on to the container image registry with your credentials by running the following command:

```
$ podman login registry.redhat.io
```

2. Extract the content from the **openshift-telco-hub-rds-rhel9** container image by running the following commands:

```
$ mkdir -p ./out
```

```
$ podman run -it registry.redhat.io/openshift4/openshift-telco-hub-rds-rhel9:v4.22 | base64 -d  
| tar xv -C out
```

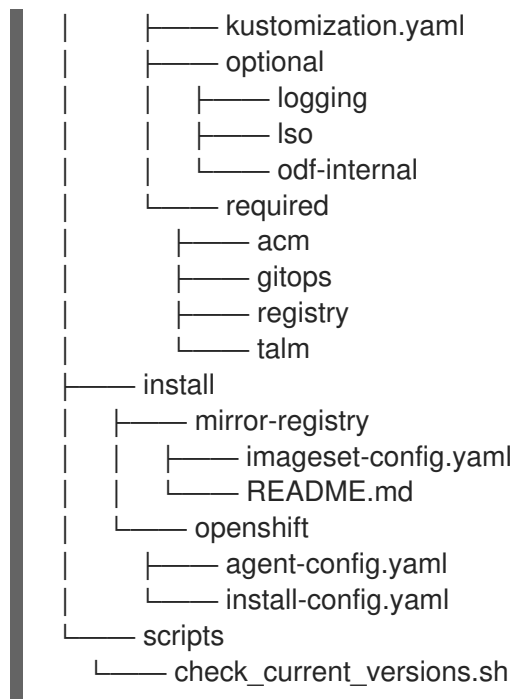
Verification

- The **out** directory has the following directory structure. You can view the telco hub CRs in the **out/telco-hub-rds/** directory by running the following command:

```
$ tree -L 4 out/telco-hub-rds/
```

Example output

```
out/telco-hub-rds/
├── configuration
│   ├── example-overlays-config
│   │   ├── acm
│   │   │   ├── acmMirrorRegistryCM-patch.yaml
│   │   │   ├── kustomization.yaml
│   │   │   ├── options-agentserviceconfig-patch.yaml
│   │   │   └── storage-mco-patch.yaml
│   │   ├── gitops
│   │   │   ├── argocd-tls-certs-cm-patch.yaml
│   │   │   ├── init-argocd-app.yaml
│   │   │   └── kustomization.yaml
│   │   ├── logging
│   │   │   ├── cluster-log-forwarder-patch.yaml
│   │   │   ├── kustomization.yaml
│   │   │   └── README.md
│   │   ├── lso
│   │   │   ├── kustomization.yaml
│   │   │   └── local-storage-disks-patch.yaml
│   │   ├── odf
│   │   │   ├── kustomization.yaml
│   │   │   └── options-storage-cluster.yaml
│   │   └── registry
│   │       ├── catalog-source-image-patch.yaml
│   │       ├── idms-operator-mirrors-patch.yaml
│   │       ├── idms-release-mirrors-patch.yaml
│   │       ├── itms-generic-mirrors-patch.yaml
│   │       ├── itms-release-mirrors-patch.yaml
│   │       ├── kustomization.yaml
│   │       └── registry-ca-patch.yaml
│   ├── kustomization.yaml
│   ├── README.md
│   └── reference-crs
```



5.18. COMPARING A CLUSTER WITH THE TELCO HUB REFERENCE CONFIGURATION

After you deploy a telco hub cluster, you can use the **cluster-compare** plugin to assess the cluster's compliance with the telco hub reference design specifications (RDS). The **cluster-compare** plugin is an OpenShift CLI (**oc**) plugin. The plugin uses a telco hub reference configuration to validate the cluster with the telco hub custom resources (CRs).

The plugin-specific reference configuration for telco hub is packaged in a container image with the telco hub CRs.

For further information about the **cluster-compare** plugin, see "Understanding the cluster-compare plugin".

The following example shows how to compare the configuration of a cluster to the telco hub reference configuration by using **must-gather** data.



NOTE

When comparing a cluster to the telco hub reference configuration by using **must-gather** data, you must use the **--all-images** flag when generating the **must-gather** data. You must also collect cluster-scoped resource information, as well as Operator and registry configurations. Without this data, the plugin might report false positives.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have credentials to access the **registry.redhat.io** container image registry.
- You installed the **cluster-compare** plugin.
- You extracted the telco hub reference configuration from the **openshift-telco-hub-rds-rhel9** container image.

Procedure

1. Collect data about your cluster by running the **must-gather** command with the **--all-images** flag:

```
$ oc adm must-gather --all-images
```

- The **--all-images** flag ensures that the **must-gather** command collects all the data required by the telco hub reference configuration.

2. Collect cluster-scoped resource information by running the following command:

```
$ oc adm inspect clusterroles,clusterrolebindings,namespaces,nodes --dest-dir=./cluster-scoped
```

3. Collect Operator and registry configurations by running the following command:

```
$ oc adm inspect
imagedigestmirrorset,imagetagmirrorset,catalogsource,clusterserviceversion,customresourcedefinition,operatorhub --dest-dir=./cluster-config
```

4. Compare the collected data to a reference configuration by running the following command:

```
$ oc cluster-compare -r <path_to_reference_config>/metadata.yaml -f "must-gather*/cluster-scoped-resources","must-gather*/namespaces","cluster-scoped","cluster-config" -R
```

- **-r** specifies a path to the **metadata.yaml** file of the reference configuration. You can specify a local directory or a URL.
- **-f** specifies the path to the **must-gather** data directory. You can specify a local directory or a URL. This example restricts the comparison to the relevant cluster configuration directories in the **must-gather** data, and also the **cluster-config** and **cluster-scoped** directories you created.
- **-R** searches the target directories recursively.

Example output

```
W0309 13:08:01.564387 29400 compare.go:476] Reference Contains Templates With
Types (kind) Not Supported By Cluster: AgentServiceConfig, AppProject, Application,
Certificate, ClusterIssuer, ClusterLogForwarder, LocalVolume,
ManagedClusterSetBinding, MultiClusterEngine, MultiClusterHub,
MultiClusterObservability, ObjectBucketClaim, Placement, PlacementBinding, Policy,
StorageCluster
```

```
...
```

```
*****
```

```
Cluster CR: operator.openshift.io/v1_IngressController_openshift-ingress-
operator_default
```

```
Reference File: optional/cert-manager/ingressControllerConfig.yaml
```

```
Diff Output: diff -u -N /tmp/MERGED-3542158379/operator-openshift-io-
v1_ingresscontroller_openshift-ingress-operator_default /tmp/LIVE-285048405/operator-
openshift-io-v1_ingresscontroller_openshift-ingress-operator_default
```

```

--- /tmp/MERGED-3542158379/operator-openshift-io-v1_ingresscontroller_openshift-
ingress-operator_default 2026-03-09 13:09:32.985703558 +0000
+++ /tmp/LIVE-285048405/operator-openshift-io-v1_ingresscontroller_openshift-ingress-
operator_default 2026-03-09 13:09:32.985703558 +0000
@@ -4,5 +4,17 @@
    name: default
    namespace: openshift-ingress-operator
  spec:
  - defaultCertificate:
  -   name: ingress-wildcard-cert
+ clientTLS:
+   clientCA:
+   name: ""
+   clientCertificatePolicy: ""
+   closedClientConnectionPolicy: Continue
+   httpCompression: {}
+   httpEmptyRequestsPolicy: Respond
+   httpErrorCodePages:
+   name: ""
+   idleConnectionTerminationPolicy: Immediate
+   replicas: 2
+   tuningOptions:
+   reloadInterval: 0s
+   unsupportedConfigOverrides: null

*****

Summary
CRs with diffs: 5/5
CRs in reference missing from the cluster: 43
optional-cert-manager:
  cert-manager-apiserver:
    Missing CRs:
    - optional/cert-manager/apiServerCertificate.yaml
  cert-manager-ingress:
    Missing CRs:
    - optional/cert-manager/ingressCertificate.yaml

...

No CRs are unmatched to reference CRs
Metadata Hash:
6297bc738df2373467cc6f5acc3a6aa23f3c3d0b0ce2ac23887d7914a6241d92
No patched CRs

```

- **Cluster CR** shows the CR with a difference from the corresponding template.
- **Reference File** shows the template file that the tool uses in its comparison with the cluster CR. The output in Linux diff format shows the difference between the template and the cluster CR.
- **CRs with diffs** shows the number of CRs in the comparison with differences from the corresponding templates.
- **CRs in reference missing from the cluster** shows the number of CRs represented in the reference configuration, but missing from the live cluster.

- **Missing CRs** shows the list of CRs represented in the reference configuration, but missing from the live cluster.
- **No CRs are unmatched to reference CRs** indicates that all CRs in the cluster matched to a corresponding template in the reference configuration.
- **Metadata Hash** shows the metadata hash that identifies the reference configuration.
- **No patched CRs** indicates that there are no patched CRs in the cluster.

5.19. HUB CLUSTER REFERENCE CONFIGURATION CRS

The following sections briefly describe each custom resource (CR) for the telco management hub reference configuration in 4.21.

5.20. RED HAT ADVANCED CLUSTER MANAGEMENT (RHACM) CRS

The following custom resources (CRs) configure Red Hat Advanced Cluster Management (RHACM) for the telco hub cluster.

Table 5.6. RHACM CRs

Component	Reference CR	Description	Optional
RHACM	acmAgentServiceConfig.yaml	Creates a policy to manage copying data from an object bucket claim into a secret for Observability to connect to Thanos.	No
RHACM	acmMCE.yaml	Defines the MultiCluster Engine configuration required by ACM.	No
RHACM	acmMCH.yaml	Configures a MultiClusterHub CR with high availability, enabling various components and specifying installation settings.	No

Component	Reference CR	Description	Optional
RHACM	acmMirrorRegistryCM.yaml	Defines the SSL certificates and mirror registry configuration for various Red Hat and OpenShift Container Platform registries used by the multicluster-engine in the multicluster-engine namespace.	No
RHACM	acmNS.yaml	Defines the open-cluster-management namespace with a label to enable cluster monitoring.	No
RHACM	acmOperGroup.yaml	Defines an OperatorGroup for the open-cluster-management namespace, targeting the same namespace.	No
RHACM	acmPerfSearch.yaml	Configures search for Open Cluster Management by defining various parameters and API settings.	No
RHACM	acmProvisioning.yaml	Configures a provisioning resource in the metal3.io/v1alpha1 API version to watch all namespaces.	No
RHACM	acmSubscription.yaml	Subscribes to the RHACM Operator using automatic install plan approval.	No
RHACM	observabilityMCO.yaml	Configures MultiClusterObservability for managing observability and alerting across multiple clusters.	No

Component	Reference CR	Description	Optional
RHACM	observabilityNS.yaml	Creates an open-cluster-management-observability namespace.	No
RHACM	observabilityOBC.yaml	Creates an ObjectBucketClaim CR in the open-cluster-management-observability namespace.	No
RHACM	observabilitySecret.yaml	Creates a Secret CR in the open-cluster-management-observability namespace for storing Docker configuration details.	No
RHACM	observabilityRoutePolicy.yaml	Policy to propagate RHACM observability route to the managed cluster.	No
RHACM	pullSecretMCSB.yaml	Creates a ManagedClusterSet Binding CR for the pull secret policy.	No
RHACM	pullSecretPlacementBinding.yaml	Creates the PlacementBinding CR needed for the pull secret policy.	No
RHACM	pullSecretPlacement.yaml	Creates the Placement CR against local cluster needed for the pull secret policy.	No
RHACM	pullSecretPolicy.yaml	Creates a policy to copy the global pull secret into observability namespaces.	No

Component	Reference CR	Description	Optional
RHACM	thanosSecretPlacementBinding.yaml	Creates the PlacementBinding CR needed for the thanos secret policy.	No
RHACM	thanosSecretPlacement.yaml	Creates the Placement CR against local cluster needed for the thanos secret policy.	No
RHACM	thanosSecretPolicy.yaml	Creates a policy to copy data from an object bucket claim into a secret for observability to connect to Thanos.	No
TALM	talmSubscription.yaml	Creates a Subscription CR for TALM.	No

5.21. STORAGE REFERENCE CRS

The following custom resources (CRs) configure storage for the telco hub cluster.

Table 5.7. Storage CRs

Component	Reference CR	Description	Optional
Local Storage Operator	IsoLocalVolume.yaml	Defines a LocalVolume CR specifying local storage configuration and node selection criteria.	Yes
Local Storage Operator	IsoNS.yaml	Defines the openshift-local-storage namespace.	Yes
Local Storage Operator	IsoOperatorGroup.yaml	Defines an OperatorGroup for the openshift-local-storage namespace.	Yes
Local Storage Operator	IsoSubscription.yaml	Defines a Subscription CR for the Local Storage Operator.	Yes

Component	Reference CR	Description	Optional
OpenShift Data Foundation	odfNS.yaml	Defines the openshift-storage namespace with specific annotations and labels for workload management and cluster monitoring.	Yes
OpenShift Data Foundation	odfOperatorGroup.yaml	Defines an OperatorGroup for the openshift-storage namespace .	Yes
OpenShift Data Foundation	odfReady.yaml	Defines a resource to verify readiness of the ODF deployment.	Yes
OpenShift Data Foundation	odfSubscription.yaml	Configures an OpenShift Container Platform subscription to the Red Hat OpenShift Data Foundation Operator, specifying installation details such as the Operator's name, namespace, channel, and approval strategy.	Yes
OpenShift Data Foundation	storageCluster.yaml	Defines a StorageCluster CR with specific resource requests and limits, storage device sets, and annotations for Argo CD synchronization.	No

5.22. GITOPS ZERO TOUCH PROVISIONING (ZTP) REFERENCE CRS

The following custom resources (CRs) configure GitOps Zero Touch Provisioning (ZTP) for the telco hub cluster.

Table 5.8. GitOps ZTP CRs

Component	Reference CR	Description	Optional
-----------	--------------	-------------	----------

Component	Reference CR	Description	Optional
GitOps Operator	argocd-ssh-known-hosts-cm.yaml	Defines a ConfigMap CR to store SSH known hosts used by ArgoCD in a disconnected environment.	No
GitOps Operator	addPluginsMCSB.yaml	Defines the ManagedClusterSet Binding CR for policy used to patch GitOps operator.	No
GitOps Operator	addPluginsPolicyNS.yaml	Namespace for GitOps plugin policy.	No
GitOps Operator	addPluginsPolicyPlacementBinding.yaml	Defines the PlacementBinding CR for the GitOps plugin policy.	No
GitOps Operator	addPluginsPolicyPlacement.yaml	Defines the Placement CR against local cluster for the GitOps plugin policy.	No
GitOps Operator	addPluginsPolicy.yaml	Defines a policy to add ArgoCD custom plugins to the GitOps controller.	No
GitOps Operator	argocd-application.yaml	Defines the ArgoCD Application for GitOps management.	No
GitOps Operator	argocd-tls-certs-cm.yaml	Defines a ConfigMap CR for ArgoCD TLS certificate management.	No
GitOps Operator	clusterrole.yaml	Defines the ClusterRole CR that grants permissions to the GitOps Operator.	No
GitOps Operator	clusterrolebinding.yaml	Binds the ClusterRole CR to the ArgoCD controller ServiceAccount CR.	No

Component	Reference CR	Description	Optional
GitOps Operator	gitopsNS.yaml	Defines an openshift-gitops-operator namespace with a label for cluster monitoring.	No
GitOps Operator	gitopsOperatorGroup.yaml	Defines an OperatorGroup in the openshift-gitops-operator namespace with a default upgrade strategy.	No
GitOps Operator	gitopsSubscription.yaml	Defines a subscription for the OpenShift Container Platform GitOps Operator, specifying automatic install plan approval and source details.	No
GitOps Operator	ztp-repo.yaml	Defines the Git repository for ZTP manifests and configurations.	No
GitOps applications	app-project.yaml	Defines an ArgoCD AppProject CR specifying resource whitelists and destination rules for cluster and namespace resources.	No
GitOps applications	clusters-app.yaml	Defines a namespace and an ArgoCD application for managing the deployment of cluster configurations from the specified Git repository.	No

Component	Reference CR	Description	Optional
GitOps applications	gitops-cluster-rolebinding.yaml	Defines a ClusterRoleBinding CR that grants the cluster-admin role to the openshift-gitops-argocd-application-controller service account in the openshift-gitops namespace.	No
GitOps applications	gitops-policy-rolebinding.yaml	Binds the cluster-manager-admin cluster role to the ArgoCD application controller ServiceAccount CR.	No
GitOps applications	kustomization.yaml	Defines a Kustomization configuration for the GitOps ZTP application installations, listing various YAML resources to be included.	No
GitOps applications	policies-app-project.yaml	Defines an Argo CD AppProject resource, specifying cluster and namespace resource whitelists and destinations.	No
GitOps applications	policies-app.yaml	Defines the ArgoCD Application CR for policy management.	No
GitOps applications	extra-manifests-policy.yaml	Defines policy to manage extra configuration manifests provided during Day 0 installation.	No

5.23. LOGGING REFERENCE CRS

The following custom resources (CRs) configure logging for the telco hub cluster.

Table 5.9. Logging CRs

Component	Reference CR	Description	Optional
Cluster Logging Operator	clusterLogForwarder.yaml	Defines the ClusterLogForwarder CR to send logs to configured outputs.	Yes
Cluster Logging Operator	clusterLogNS.yaml	Configures a namespace for the Cluster Logging Operator.	Yes
Cluster Logging Operator	clusterLogOperatorGroup.yaml	Configures an Operator group for the Cluster Logging Operator.	Yes
Cluster Logging Operator	clusterLogServiceAccount.yaml	Defines the ServiceAccount CR used by Cluster Logging Operator components.	Yes
Cluster Logging Operator	clusterLogServiceAccountAuditBinding.yaml	Binds the Cluster Logging ServiceAccount CR to audit log roles.	Yes
Cluster Logging Operator	clusterLogServiceAccountInfrastructureBinding.yaml	Binds the Cluster Logging ServiceAccount CR to infrastructure log roles.	Yes
Cluster Logging Operator	clusterLogSubscription.yaml	Defines a subscription for installing and managing the Cluster Logging Operator.	Yes

5.24. CONTAINER REGISTRY REFERENCE CRS

The following custom resources (CRs) configure the container registry for the telco hub cluster.

Table 5.10. Container registry CRs

Component	Reference CR	Description	Optional
Registry	catalog-source.yaml	Defines a CatalogSource CR for mirrored Operator catalogs.	No

Component	Reference CR	Description	Optional
Registry	idms-operator.yaml	Defines an image digest MirrorSet Operator CR for mirrored Operator images.	No
Registry	idms-release.yaml	Defines an image digest MirrorSet CR for OpenShift Container Platform release images.	No
Registry	image-config.yaml	Defines an image configuration CR to manage image registries and policies.	No
Registry	itms-generic.yaml	Defines an image tag MirrorSet CR for mirrored images in a disconnected registry.	No
Registry	itms-release.yaml	Defines an image tag MirrorSet CR for OpenShift Container Platform release images.	No
Registry	kustomization.yaml	Defines a Kustomization manifest for registry-related CRs.	No
Registry	operator-hub.yaml	Configures the OperatorHub CR for offline catalog sources.	No
Registry	registry-ca.yaml	Defines a ConfigMap CR containing registry CA certificates.	No

5.25. IMAGE MIRRORING REFERENCE CRS

The following custom resources (CRs) configure image mirroring for the telco hub cluster.

Table 5.11. Image mirroring CRs

Component	Reference CR	Description	Optional
Mirroring configuration CRs	imageset-config.yaml	Defines an ImageSetConfiguration CR for mirroring OpenShift Container Platform channels and Operator packages, specifying versions and target catalogs.	No

5.26. INSTALLATION REFERENCE CRS

The following custom resources (CRs) configure the installation for the telco hub cluster.

Table 5.12. Installation CRs

Component	Reference CR	Description	Optional
Agent-based install	agent-config.yaml	Configures the Agent-based installer, specifying network and device settings for the hosts to be installed.	No
Agent-based install	install-config.yaml	Configures the hub cluster installation for networking, control plane, compute nodes, mirror registries, and so on.	No

5.27. SECURITY REFERENCE CRS

The following table lists the security reference configuration custom resources (CRs) for the hub cluster.

Table 5.13. Security CRs

Component	Reference CR	Description	Optional
Cert-Manager	certManagerNS.yaml	Defines the cert-manager-operator namespace.	Yes
Cert-Manager	certManagerOperatorgroup.yaml	Defines the OperatorGroup for cert-manager.	Yes

Component	Reference CR	Description	Optional
Cert-Manager	certManagerSubscription.yaml	Installs the OpenShift cert-manager operator.	Yes
Cert-Manager	certManagerClusterIssuer.yaml	Configures an ACME ClusterIssuer using Let's Encrypt with DNS-01 challenge.	Yes
Cert-Manager	apiServerCertificate.yaml	Creates a certificate for the API Server endpoint.	Yes
Cert-Manager	ingressCertificate.yaml	Creates a wildcard certificate for the Ingress/Router.	Yes
Cert-Manager	apiServerConfig.yaml	Configures OpenShift to use the cert-manager generated API Server certificate.	Yes
Cert-Manager	ingressControllerConfig.yaml	Configures OpenShift to use the cert-manager generated Ingress certificate.	Yes
Cert-Manager	certManagerCertificatePolicy.yaml	Defines CertificatePolicy for monitoring certificate expiration and compliance across managed clusters.	Yes
Cert-Manager	certManagerCertificatePolicyPlacement.yaml	Defines Placement for CertificatePolicy targeting clusters with the common label.	Yes
Cert-Manager	certManagerCertificatePolicyPlacementBinding.yaml	Binds the CertificatePolicy to the Placement for policy distribution.	Yes

5.28. BACKUP RECOVERY REFERENCE CRS

The following table lists the backup and recovery reference configuration custom resources (CRs) for the hub cluster.

Table 5.14. Backup Recovery CRs

Component	Reference CR	Description	Optional
OADP	backupSchedule.yaml	Defines a BackupSchedule CR.	Yes
OADP	dataProtectionApplication.yaml	Defines the data protection application with backup storage and configuration parameters.	Yes
OADP	objectBucketClaim.yaml	Defines the object bucket used by backup.	Yes
OADP	policy-backup.yaml	Defines a policy to ensure BareMetalHost CRs are correctly annotated for backup.	Yes
OADP	restore.yaml	Example Restore CR.	Yes

5.29. TELCO HUB REFERENCE CONFIGURATION SOFTWARE SPECIFICATIONS

The following y-stream versions were used in validation of the telco hub solution for OpenShift Container Platform clusters.

Hub Cluster Component	Software Version (y-stream)
OpenShift Container Platform	4.22
Red Hat Advanced Cluster Management (RHACM)	2.17
Local Storage Operator	4.22
cert-manager Operator	1.19
Red Hat OpenShift Data Foundation (ODF)	4.21
Red Hat OpenShift GitOps	1.20
GitOps Zero Touch Provisioning (ZTP) plugins	4.22
multicluster engine Operator PolicyGenerator plugin	2.17
Topology Aware Lifecycle Manager (TALM)	4.22

Hub Cluster Component	Software Version (y-stream)
Cluster Logging Operator	6.5
OpenShift API for Data Protection (OADP)	The version aligned with the RHACM release.

- ODF will be updated to 4.22 when the aligned ODF version is released.
- Cluster Logging Operator will be updated to 6.6 when the aligned Cluster Logging Operator version is released.
- The cert-manager Operator and Red Hat OpenShift GitOps Operator are platform agnostic operators. The support lifecycle for these operators is independent from the support lifecycle for OpenShift Container Platform. You might need to update to a newer minor version of these operators at the end of an operator lifecycle, or when planning to update the OpenShift Container Platform cluster to continue support. For support lifecycle details for platform agnostic operators, see [OpenShift Operator Life Cycles](#).

CHAPTER 6. COMPARING CLUSTER CONFIGURATIONS

6.1. UNDERSTANDING THE CLUSTER-COMPARE PLUGIN

The **cluster-compare** plugin is an OpenShift CLI (**oc**) plugin that compares a cluster configuration with a reference configuration. The plugin reports configuration differences while suppressing expected variations by using configurable validation rules and templates.

Use the **cluster-compare** plugin in development, production, and support scenarios to ensure cluster compliance with a reference configuration, and to quickly identify and troubleshoot relevant configuration differences.

6.1.1. Overview of the cluster-compare plugin

Clusters deployed at scale typically use a validated set of baseline custom resources (CRs) to configure clusters to meet use-case requirements and ensure consistency when deploying across different environments.

In live clusters, some variation from the validated set of CRs is expected. For example, configurations might differ because of variable substitution, optional components, or hardware-specific fields. This variation makes it difficult to accurately assess if a cluster is compliant with the baseline configuration.

Using the **cluster-compare** plugin with the **oc** command, you can compare the configuration from a live cluster with a reference configuration. A reference configuration represents the baseline configuration but uses the various plugin features to suppresses expected variation during a comparison. For example, you can apply validation rules, specify optional and required resources, and define relationships between resources. By reducing irrelevant differences, the plugin makes it easier to assess cluster compliance with baseline configurations, and across environments.

The ability to intelligently compare a configuration from a cluster with a reference configuration has the following example use-cases:

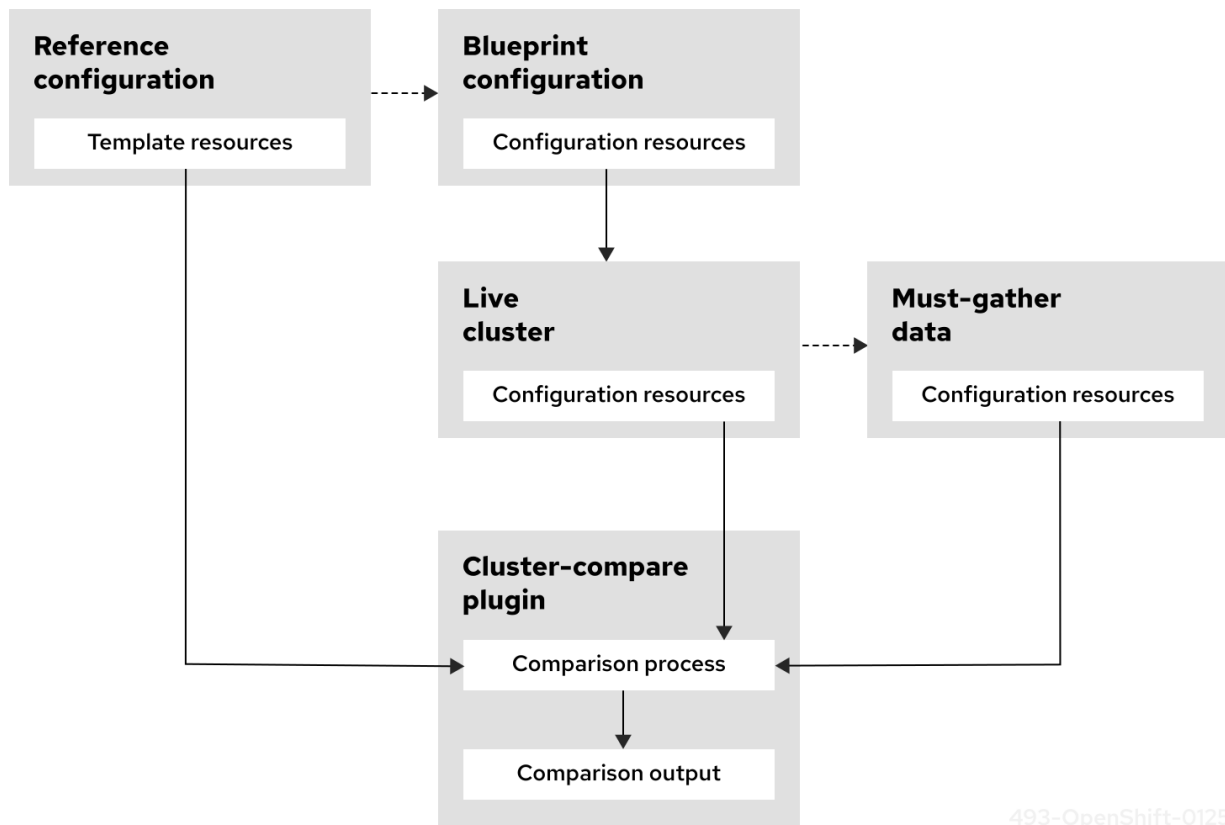
Production: Ensure compliance with a reference configuration across service updates, upgrades and changes to the reference configuration.

Development: Ensure compliance with a reference configuration in test pipelines.

Design: Compare configurations with a partner lab reference configuration to ensure consistency.

Support: Compare the reference configuration to **must-gather** data from a live cluster to troubleshoot configuration issues.

Figure 6.1. Cluster-compare plugin overview



493-OpenShift-0125

6.1.2. Understanding a reference configuration

The **cluster-compare** plugin uses a reference configuration to validate a configuration from a live cluster. The reference configuration consists of a YAML file called **metadata.yaml**, which references a set of templates that represent the baseline configuration.

Example directory structure for a reference configuration

```

├── metadata.yaml
├── optional
│   ├── optionalTemplate1.yaml
│   └── optionalTemplate2.yaml
├── required
│   ├── requiredTemplate3.yaml
│   └── requiredTemplate4.yaml
└── baselineClusterResources
    ├── clusterResource1.yaml
    ├── clusterResource2.yaml
    ├── clusterResource3.yaml
    └── clusterResource4.yaml
  
```

where:

metadata.yaml

The reference configuration consists of the **metadata.yaml** file and a set of templates.

optional

This example uses an optional and required directory structure for templates that are referenced by the **metadata.yaml** file.

baselineClusterResources

The configuration CRs to use as a baseline configuration for clusters.

During a comparison, the plugin matches each template to a configuration resource from the cluster. The plugin evaluates optional or required fields in the template using features such as Golang templating syntax and inline regular expression validation. The **metadata.yaml** file applies additional validation rules to decide whether a template is optional or required and assesses template dependency relationships.

Using these features, the plugin identifies relevant configuration differences between the cluster and the reference configuration. For example, the plugin can highlight mismatched field values, missing resources, extra resources, field type mismatches, or version discrepancies.

For further information about configuring a reference configuration, see "Creating a reference configuration".

6.1.3. Additional resources

- [Telco RAN DU reference design specification for OpenShift Container Platform](#)

6.2. INSTALLING THE CLUSTER-COMPARE PLUGIN

You can extract the **cluster-compare** plugin from a container image in the Red Hat container catalog and use it as a plugin to the **oc** command.

6.2.1. Installing the cluster-compare plugin

Install the **cluster-compare** plugin to compare a reference configuration with a cluster configuration from a live cluster or **must-gather** data.

Prerequisites

1. You have installed the OpenShift CLI (**oc**).
2. You installed **podman**.
3. You have access to the Red Hat container catalog.

Procedure

1. Log in to the Red Hat container catalog by running the following command:

```
$ podman login registry.redhat.io
```

2. Create a container for the **cluster-compare** image by running the following command:

```
$ podman create --name cca registry.redhat.io/openshift4/kube-compare-artifacts-rhel9:latest
```

3. Copy the **cluster-compare** plugin to a directory that is included in your **PATH** environment variable by running the following command:

■

```
$ podman cp cca:/usr/share/openshift/<arch>/kube-compare.<rhel_version>
<directory_on_path>/kubectl-cluster_compare
```

- **arch** is the architecture for your machine. Valid values are:
 - **linux_amd64**
 - **linux_arm64**
 - **linux_ppc64le**
 - **linux_s390x**
- **<rhel_version>** is the version of RHEL on your machine. Valid values are **rhel8** or **rhel9**.
- **<directory_on_path>** is the path to a directory included in your **PATH** environment variable.

Verification

- View the help for the plugin by running the following command:

```
$ oc cluster-compare -h
```

The following is example output:

```
Compare a known valid reference configuration and a set of specific cluster configuration
CRs.

...

Usage:
  compare -r <Reference File>

Examples:
  # Compare a known valid reference configuration with a live cluster:
  kubectl cluster-compare -r ./reference/metadata.yaml

...
```

6.2.2. Additional resources

- [Extending the OpenShift CLI with plugins](#)

6.3. USING THE CLUSTER-COMPARE PLUGIN

You can use the **cluster-compare** plugin to compare a reference configuration with a configuration from a live cluster or **must-gather** data.

6.3.1. Using the cluster-compare plugin with a live cluster

You can use the **cluster-compare** plugin to compare a reference configuration with configuration custom resources (CRs) from a live cluster.

Validate live cluster configurations to ensure compliance with reference configurations during design, development, or testing scenarios.



NOTE

Use the **cluster-compare** plugin with live clusters in non-production environments only. For production environments, use the plugin with **must-gather** data.

Prerequisites

- You installed the OpenShift CLI (**oc**).
- You have access to the cluster as a user with the **cluster-admin** role.
- You downloaded the **cluster-compare** plugin and include it in your **PATH** environment variable.
- You have access to a reference configuration.

Procedure

- Run the **cluster-compare** plugin by using the following command:

```
$ oc cluster-compare -r <path_to_reference_config>/metadata.yaml
```

- **-r** specifies a path to the **metadata.yaml** file of the reference configuration. You can specify a local directory or a URI.

The following is example output:

```
...
*****

Cluster CR: operator.openshift.io/v1_Console_cluster
Reference File: optional/console-disable/ConsoleOperatorDisable.yaml
Diff Output: diff -u -N /tmp/MERGED-622469311/operator-openshift-io-v1_console_cluster /tmp/LIVE-
2358803347/operator-openshift-io-v1_console_cluster
/tmp/MERGED-622469311/operator-openshift-io-v1_console_cluster 2024-11-20
15:43:42.888633602 +0000
+++ /tmp/LIVE-2358803347/operator-openshift-io-v1_console_cluster 2024-11-20
15:43:42.888633602 +0000
@@ -4,5 +4,5 @@
   name: cluster
spec:
  logLevel: Normal
- managementState: Removed
+ managementState: Managed
  operatorLogLevel: Normal

*****
...

Summary
CRs with diffs: 5/49
```

```

CRs in reference missing from the cluster: 1
required-cluster-tuning:
  cluster-tuning:
    Missing CRs:
      - required/cluster-tuning/disabling-network-diagnostics/DisableSnoNetworkDiag.yaml
No CRs are unmatched to reference CRs
Metadata Hash: 512a9bf2e57fd5a5c44bbdea7abb3ffd7739d4a1f14ef9021f6793d5cdf868f0
No patched CRs

```

where:

Cluster CR

The CR under comparison. The plugin displays each CR with a difference from the corresponding template.

Reference File

The template matching with the CR for comparison.

Diff Output

The output in Linux diff format shows the difference between the template and the cluster CR.

Summary

After the plugin reports the line diffs for each CR, the summary of differences are reported.

CRs with diffs

The number of CRs in the comparison with differences from the corresponding templates.

CRs in reference missing from the cluster

The number of CRs represented in the reference configuration, but missing from the live cluster.

Missing CRs

The list of CRs represented in the reference configuration, but missing from the live cluster.

No CRs are unmatched to reference CRs

The CRs that did not match to a corresponding template in the reference configuration.

Metadata Hash

The metadata hash identifies the reference configuration.

No patched CRs

The list of patched CRs.

NOTE

Get the output in the **junit** format by adding **-o junit** to the command. For example:

```
$ oc cluster-compare -r <path_to_reference_config>/metadata.yaml -o junit
```

The **junit** output includes the following result types:

- Passed results for each fully matched template.
- Failed results for differences found or missing required custom resources (CRs).
- Skipped results for differences patched using the user override mechanism.

6.3.2. Using the cluster-compare plugin with must-gather data

You can use the **cluster-compare** plugin to compare a reference configuration with configuration custom resources (CRs) from **must-gather** data.

Validate cluster configurations by using **must-gather** data to troubleshoot configuration issues in production environments.



NOTE

For production environments, use the **cluster-compare** plugin with **must-gather** data only.

- You have access to **must-gather** data from a target cluster.
- You installed the OpenShift CLI (**oc**).
- You have downloaded the **cluster-compare** plugin and included it in your **PATH** environment variable.
- You have access to a reference configuration.

Procedure

- Compare the **must-gather** data to a reference configuration by running the following command:

```
$ oc cluster-compare -r <path_to_reference_config>/metadata.yaml -f "must-gather*/*/cluster-scoped-resources","must-gather*/*/namespaces" -R
```

- **-r** specifies a path to the **metadata.yaml** file of the reference configuration. You can specify a local directory or a URI.
 - **-f** specifies the path to the **must-gather** data directory. You can specify a local directory or a URI. This example restricts the comparison to the relevant cluster configuration directories.
 - **-R** searches the target directories recursively.
- The following is example output:

```
...
```

```
*****
```

```
Cluster CR: operator.openshift.io/v1_Console_cluster
Reference File: optional/console-disable/ConsoleOperatorDisable.yaml
Diff Output: diff -u -N /tmp/MERGED-622469311/operator-openshift-io-
v1_console_cluster /tmp/LIVE-2358803347/operator-openshift-io-v1_console_cluster
/tmp/MERGED-622469311/operator-openshift-io-v1_console_cluster 2024-11-20
15:43:42.888633602 +0000
+++ /tmp/LIVE-2358803347/operator-openshift-io-v1_console_cluster 2024-11-20
15:43:42.888633602 +0000
@@ -4,5 +4,5 @@
    name: cluster
```

```

spec:
  logLevel: Normal
- managementState: Removed
+ managementState: Managed
  operatorLogLevel: Normal

*****

...

Summary
CRs with diffs: 5/49
CRs in reference missing from the cluster: 1
required-cluster-tuning:
  cluster-tuning:
    Missing CRs:
      - required/cluster-tuning/disabling-network-diagnostics/DisableSnoNetworkDiag.yaml
No CRs are unmatched to reference CRs
Metadata Hash:
512a9bf2e57fd5a5c44bbdea7abb3ffd7739d4a1f14ef9021f6793d5cdf868f0
No patched CRs

```

where:

Cluster CR

The CR under comparison. The plugin displays each CR with a difference from the corresponding template.

Reference File

The template matching with the CR for comparison.

Diff Output

The output in Linux diff format shows the difference between the template and the cluster CR.

Summary

After the plugin reports the line diffs for each CR, the summary of differences are reported.

CRs with diffs

The number of CRs in the comparison with differences from the corresponding templates.

CRs in reference missing from the cluster

The number of CRs represented in the reference configuration, but missing from the live cluster.

Missing CRs

The list of CRs represented in the reference configuration, but missing from the live cluster.

No CRs are unmatched to reference CRs

The CRs that did not match to a corresponding template in the reference configuration.

Metadata Hash

The metadata hash identifies the reference configuration.

No patched CRs

The list of patched CRs.



NOTE

Get the output in the **junit** format by adding **-o junit** to the command. For example:

```
$ oc cluster-compare -r <path_to_reference_config>/metadata.yaml -f "must-gather*/*/cluster-scoped-resources","must-gather*/*/namespaces" -R -o junit
```

The **junit** output includes the following result types:

- Passed results for each fully matched template.
- Failed results for differences found or missing required custom resources (CRs).
- Skipped results for differences patched using the user override mechanism.

Additional resources

- [Gathering data about your cluster](#)

6.3.3. Reference cluster-compare plugin options

The following content describes the options for the **cluster-compare** plugin.

Table 6.1. Cluster-compare plugin options

Option	Description
-A, --all-resources	When used with a live cluster, attempts to match all resources in the cluster that match a type in the reference configuration. When used with local files, attempts to match all resources in the local files that match a type in the reference configuration.
--concurrency	Specify an integer value for the number of templates to process in parallel when comparing with resources from the live version. A larger number increases speed but also memory, I/O, and CPU usage during that period. The default value is 4 .
-c, --diff-config	Specify the path to the user configuration file.
-f, --filename	Specify a filename, directory, or URL for the configuration custom resources that you want to use for a comparison with a reference configuration.
--generate-override-for	Specify the path for templates that requires a patch.

Option	Description
--show-template-functions	Displays the available template functions.  NOTE You must use a file path for the target template that is relative to the metadata.yaml file. For example, if the file path for the metadata.yaml file is ./compare/metadata.yaml, a relative file path for the template might be optional/my-template.yaml.
-h, --help	Display help information.
-k, --kustomize	Specify a path to process the kustomization directory. This flag cannot be used together with -f or -R .
-o, --output	Specify the output format. Options include json , yaml , junit , or generate-patches .
--override-reason	Specify a reason for generating the override.
-p, --overrides	Specify a path to a patch override file for the reference configuration.
-R, --recursive	Processes the directory specified in -f, --filename recursively.
-r, --reference	Specify the path to the reference configuration metadata.yaml file.
--show-managed-fields	Specify true to include managed fields in the comparison.
-v, --verbose	Increases the verbosity of the plugin output.

6.3.4. Example: Comparing a cluster with the telco core reference configuration

You can use the **cluster-compare** plugin to compare a reference configuration with a configuration from a live cluster or **must-gather** data.

This example compares a configuration from a live cluster with the telco core reference configuration. The telco core reference configuration is derived from the telco core reference design specifications (RDS). The telco core RDS is designed for clusters to support large scale telco applications including control plane and some centralized data plane functions.

The reference configuration is packaged in a container image with the telco core RDS.

For further examples of using the **cluster-compare** plugin with the telco core and telco RAN distributed unit (DU) profiles, see the "Additional resources" section.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have credentials to access the **registry.redhat.io** container image registry.
- You installed the **cluster-compare** plugin.

Procedure

1. Log on to the container image registry with your credentials by running the following command:

```
$ podman login registry.redhat.io
```

2. Extract the content from the **telco-core-rds-rhel9** container image by running the following commands:

```
$ mkdir -p ./out
```

```
$ podman run -it registry.redhat.io/openshift4/openshift-telco-core-rds-rhel9:v4.18 | base64 -d  
| tar xv -C out
```

You can view the reference configuration in the **reference-crs-kube-compare/** directory.

```
out/telco-core-rds/configuration/reference-crs-kube-compare/  
├── metadata.yaml  
├── optional  
│   ├── logging  
│   ├── networking  
│   ├── other  
│   └── tuning  
├── required  
│   ├── networking  
│   ├── other  
│   ├── performance  
│   ├── scheduling  
│   └── storage
```

where:

metadata.yaml

Configuration file for the reference configuration.

optional

Directory for optional templates.

required

Directory for required templates.

3. Compare the configuration for your cluster to the telco core reference configuration by running the following command:

```
$ oc cluster-compare -r out/telco-core-rds/configuration/reference-crs-kube-
compare/metadata.yaml
```

The following is example output:

```
W1212 14:13:06.281590 36629 compare.go:425] Reference Contains Templates With
Types (kind) Not Supported By Cluster: BFDProfile, BGPAdvertisement, BGPPeer,
ClusterLogForwarder, Community, IPAddressPool, MetalLB, MultiNetworkPolicy, NMState,
NUMAResourcesOperator, NUMAResourcesScheduler, NodeNetworkConfigurationPolicy,
SriovNetwork, SriovNetworkNodePolicy, SriovOperatorConfig, StorageCluster

...

*****

Cluster CR: config.openshift.io/v1_OperatorHub_cluster
Reference File: required/other/operator-hub.yaml
Diff Output: diff -u -N /tmp/MERGED-2801470219/config-openshift-io-v1_operatorhub_cluster
/tmp/LIVE-2569768241/config-openshift-io-v1_operatorhub_cluster
--- /tmp/MERGED-2801470219/config-openshift-io-v1_operatorhub_cluster 2024-12-12
14:13:22.898756462 +0000
+++ /tmp/LIVE-2569768241/config-openshift-io-v1_operatorhub_cluster 2024-12-12
14:13:22.898756462 +0000
@@ -1,6 +1,6 @@
  apiVersion: config.openshift.io/v1
  kind: OperatorHub
  metadata:
+ annotations:
+   include.release.openshift.io/hypershift: "true"
+   name: cluster
- spec:
-   disableAllDefaultSources: true

*****

Summary
CRs with diffs: 3/4
CRs in reference missing from the cluster: 22
other:
  other:
    Missing CRs:
    - optional/other/control-plane-load-kernel-modules.yaml
    - optional/other/worker-load-kernel-modules.yaml
required-networking:
  networking-root:
    Missing CRs:
    - required/networking/nodeNetworkConfigurationPolicy.yaml
  networking-sriov:
    Missing CRs:
    - required/networking/sriov/sriovNetwork.yaml
    - required/networking/sriov/sriovNetworkNodePolicy.yaml
    - required/networking/sriov/SriovOperatorConfig.yaml
    - required/networking/sriov/SriovSubscription.yaml
    - required/networking/sriov/SriovSubscriptionNS.yaml
    - required/networking/sriov/SriovSubscriptionOperGroup.yaml
required-other:
```

```

scheduling:
  Missing CRs:
  - required/other/catalog-source.yaml
  - required/other/icsp.yaml
required-performance:
  performance:
    Missing CRs:
    - required/performance/PerformanceProfile.yaml
required-scheduling:
  scheduling:
    Missing CRs:
    - required/scheduling/nrop.yaml
    - required/scheduling/NROPSubscription.yaml
    - required/scheduling/NROPSubscriptionNS.yaml
    - required/scheduling/NROPSubscriptionOperGroup.yaml
    - required/scheduling/sched.yaml
required-storage:
  storage-odf:
    Missing CRs:
    - required/storage/odf-external/01-rook-ceph-external-cluster-details.secret.yaml
    - required/storage/odf-external/02-ocs-external-storagecluster.yaml
    - required/storage/odf-external/odfNS.yaml
    - required/storage/odf-external/odfOperGroup.yaml
    - required/storage/odf-external/odfSubscription.yaml
No CRs are unmatched to reference CRs
Metadata Hash:
fe41066bac56517be02053d436c815661c9fa35eec5922af25a1be359818f297
No patched CRs

```

where:

Cluster CR

The CR under comparison. The plugin displays each CR with a difference from the corresponding template.

Reference File

The template matching with the CR for comparison.

Diff Output

The output in Linux diff format shows the difference between the template and the cluster CR.

Summary

After the plugin reports the line diffs for each CR, the summary of differences are reported.

CRs with diffs

The number of CRs in the comparison with differences from the corresponding templates.

CRs in reference missing from the cluster

The number of CRs represented in the reference configuration, but missing from the live cluster.

Missing CRs

The list of CRs represented in the reference configuration, but missing from the live cluster.

No CRs are unmatched to reference CRs

The CRs that did not match to a corresponding template in the reference configuration.

Metadata Hash

The metadata hash identifies the reference configuration.

No patched CRs

The list of patched CRs.



NOTE

Get the output in the **junit** format by adding **-o junit** to the command. For example:

```
$ oc cluster-compare -r out/telco-core-rds/configuration/reference-crs-kube-compare/metadata.yaml -o junit
```

The **junit** output includes the following result types:

- Passed results for each fully matched template.
- Failed results for differences found or missing required custom resources (CRs).
- Skipped results for differences patched using the user override mechanism.

6.3.5. Additional resources

- [Comparing a cluster with the telco RAN DU reference configuration](#)
- [Comparing a cluster with the telco core reference configuration](#)

6.4. CREATING A REFERENCE CONFIGURATION

Configure a reference configuration to validate configuration resources from a cluster.

6.4.1. Structure of the metadata.yaml file

The **metadata.yaml** file provides a central configuration point to define and configure the templates in a reference configuration. The file features a hierarchy of **parts** and **components**. **parts** are groups of **components** and **components** are groups of templates. Under each component, you can configure template dependencies, validation rules, and add descriptive metadata.

Example metadata.yaml file

```
apiVersion: v2
parts:
  - name: <part_name>
    components:
      - name: <component_name>
        <component_configuration>
      - name: <part_name>
        - name: <component_name>
          <component_configuration>
```

where:

<part_name>

Specify a **part** name. Every **part** typically describes a workload or a set of workloads.

<component_name>

Specify a **component** name.

<component_configuration>

Specify the configuration for a template. For example, define template relationships or configure what fields to use in a comparison.

6.4.2. Configuring template relationships

By defining relationships between templates in your reference configuration, you can support use-cases with complex dependencies. For example, you can configure a component to require specific templates, require one template from a group, or allow any template from a group, and so on.

Procedure

- Create a **metadata.yaml** file to match your use case. Use the following structure as an example:

Example metadata.yaml file

```
apiVersion: v2
parts:
  - name: Part1
    components:
      - name: Component1
        allOf:
          - path: RequiredTemplate1.yaml
          - path: RequiredTemplate2.yaml
      - name: Component2
        allOrNoneOf:
          - path: OptionalBlockTemplate1.yaml
          - path: OptionalBlockTemplate2.yaml
      - name: Component3
        anyOf:
          - path: OptionalTemplate1.yaml
          - path: OptionalTemplate2.yaml
      - name: Component4
        noneOf:
          - path: BannedTemplate1.yaml
          - path: BannedTemplate2.yaml
      - name: Component5
        oneOf:
          - path: RequiredExclusiveTemplate1.yaml
          - path: RequiredExclusiveTemplate2.yaml
      - name: Component6
        anyOneOf:
          - path: OptionalExclusiveTemplate1.yaml
          - path: OptionalExclusiveTemplate2.yaml
#...
```

where:

allOf

Specifies required templates.

allOrNoneOf

Specifies a group of templates that are either all required or all optional. If one corresponding custom resource (CR) is present in the cluster, then all corresponding CRs must be present in the cluster.

anyOf

Specifies optional templates.

noneOf

Specifies templates to exclude. If a corresponding CR is present in the cluster, the plugin returns a validation error.

oneOf

Specifies templates where only one can be present. If none, or more than one of the corresponding CRs are present in the cluster, the plugin returns a validation error.

anyOneOf

Specifies templates where only one can be present in the cluster. If more than one of the corresponding CRs are present in the cluster, the plugin returns a validation error.

6.4.3. Configuring expected variation in a template

You can handle variable content within a template by using Golang templating syntax. Using this syntax, you can configure validation logic that handles optional, required, and conditional content within the template.

**NOTE**

- The **cluster-compare** plugin requires all templates to render as valid YAML. To avoid parsing errors for missing fields, use conditional templating syntax such as `{{- if .spec.<optional_field> }}` when implementing templating syntax. This conditional logic ensures templates process missing fields gracefully and maintains valid YAML formatting.
- You can use the Golang templating syntax with custom and built-in functions for complex use cases. All Golang built-in functions are supported including the functions in the Sprig library.

Procedure

- Create a **metadata.yaml** file to match your use case. Use the following structure as an example:

```
apiVersion: v2
kind: Service
metadata:
  name: frontend
  namespace: {{ .metadata.namespace }}
  labels:
    app: guestbook
    tier: frontend
spec:
  {{- if and .spec.type (eq (.spec.type) "NodePort" "LoadBalancer") }}
  type: {{.spec.type}}
  {{- else }}
  type: should be NodePort or LoadBalancer
```

```

{{- end }}
ports:
- port: 80
selector:
  app: guestbook
  {{- if .spec.selector.tier }}
  tier: frontend
  {{- end }}

```

where:

name: frontend

Configures a required field that must match the specified value.

namespace: \{{ .metadata.namespace }}

Configures a required field that can have any value.

type: \{{.spec.type }}

Configures validation for the **.spec.type** field.

\{{- if .spec.selector.tier }}

Configures an optional field.

6.4.3.1. Reference template functions

The **cluster-compare** plugin supports all **sprig** library functions, except for the **env** and **expandenv** functions. For the full list of **sprig** library functions, see "Sprig Function Documentation".

The following table describes the additional template functions for the **cluster-compare** plugin:

Table 6.2. Additional cluster-compare template functions

Function	Description	Example
fromJson	Parses the incoming string as a structured JSON object.	value: {{ obj := spec.jsoncontext fromJson }}{{ obj.field }}
fromJsonArray	Parses the incoming string as a structured JSON array.	value: {{ obj := spec.jsoncontext fromJson }}{{ index \$obj 0 }}
fromYaml	Parses the incoming string as a structured YAML object.	value: {{ obj := spec.yamltext fromYaml }}{{ obj.field }}
fromYamlArray	Parses the incoming string as a structured YAML array.	value: {{ obj := spec.yamltext fromYaml }}{{ index \$obj 0 }}
toJson	Renders incoming data as JSON while preserving object types.	jsonstring: {{ \$variable toJson }}
toToml	Renders the incoming string as structured TOML data.	tomlstring: {{ \$variable toToml }}

Function	Description	Example
toYaml	Renders incoming data as YAML while preserving object types.	For simple scalar values: value: {{ \$data toYaml }} For lists or dictionaries: value: {{ \$dict toYaml nindent 2 }}
doNotMatch	Prevents a template from matching a cluster resource, even if it would normally match. You can use this function inside a template to conditionally exclude certain resources from correlation. The specified reason is logged when running with the --verbose flag. Templates excluded due to doNotMatch are not considered comparison failures. This function is especially useful when your template does not specify a fixed name or namespace. In these cases, you can use the doNotMatch function to exclude specific resources based on other fields, such as labels or annotations .	{{ if \$condition }}{{ doNotMatch \$reason }}{{ end }}
lookupCRs	Returns an array of objects that match the specified parameters. For example: lookupCRs \$apiVersion \$kind \$namespace \$name . If the \$namespace parameter is an empty string (""), or * , the function matches all namespaces. For cluster-scoped objects, the function matches objects with no namespace. If the \$name is an empty string or * , the function matches any named object.	-

Function	Description	Example
lookupCR	Returns a single object that matches the parameters. If multiple objects match, the function returns nothing. This function takes the same arguments as the lookupCRs function.	-

The following example shows how to use the **lookupCRs** function to retrieve and render values from multiple matching resources:

Config map example using lookupCRs

```
kind: ConfigMap
apiVersion: v1
metadata:
  labels:
    k8s-app: kubernetes-dashboard
  name: kubernetes-dashboard-settings
  namespace: kubernetes-dashboard
data:
  dashboard: {{ index (lookupCR "apps/v1" "Deployment" "kubernetes-dashboard" "kubernetes-
  dashboard") "metadata" "name" \} toYaml }}
  metrics: {{ (lookupCR "apps/v1" "Deployment" "kubernetes-dashboard" "dashboard-metrics-
  scraper").metadata.name \} toYaml }}
```

The following example shows how to use the **lookupCR** function to retrieve and use specific values from a single matching resource:

Config map example using lookupCR

```
kind: ConfigMap
apiVersion: v1
metadata:
  labels:
    k8s-app: kubernetes-dashboard
  name: kubernetes-dashboard-settings
  namespace: kubernetes-dashboard
data:
  {{- $objlist := lookupCRs "apps/v1" "Deployment" "kubernetes-dashboard" "" }}
  {{- $dashboardName := "unknown" }}
  {{- $metricsName := "unknown" }}
  {{- range $obj := $objlist }}
    {{- $appname := index $obj "metadata" "labels" "k8s-app" }}
    {{- if contains "metrics" $appname }}
      {{- $metricsName = $obj.metadata.name }}
    {{- end }}
    {{- if eq "kubernetes-dashboard" $appname }}
      {{- $dashboardName = $obj.metadata.name }}
    {{- end }}
  {{- end }}
```

```
{{- end }}
dashboard: {{ $dashboardName }}
metrics: {{ $metricsName }}
```

Additional resources

- [Sprig Function Documentation](#)

6.4.4. Configuring the `metadata.yaml` file to exclude template fields

You can configure the **`metadata.yaml`** file to exclude fields from a comparison. Exclude fields that are irrelevant to a comparison, for example annotations or labels that are inconsequential to a cluster configuration.

You can configure exclusions in the **`metadata.yaml`** file in the following ways:

- Exclude all fields in a custom resource not specified in a template.
- Exclude specific fields that you define using the **`pathToKey`** field.



NOTE

`pathToKey` is a dot separated path. Use quotes to escape key values featuring a period.

6.4.4.1. Excluding all fields not specified in a template

During the comparison process, the **`cluster-compare`** plugin renders a template by merging fields from the corresponding custom resource (CR). If you configure the **`ignore-unspecified-fields`** to **`true`**, all fields that are present in the CR, but not in the template, are excluded from the merge. Use this approach when you want to focus the comparison on the fields specified in the template only.

Procedure

- Create a **`metadata.yaml`** file to match your use case. Use the following structure as an example:

```
apiVersion: v2
parts:
  - name: Part1
    components:
      - name: Namespace
        allOf:
          - path: namespace.yaml
            config:
              ignore-unspecified-fields: true
#...
```

where:

`ignore-unspecified-fields: true`

Specify **`true`** to exclude from the comparison all fields in a CR that are not explicitly configured in the corresponding **`namespace.yaml`** template.

6.4.4.2. Excluding specific fields by setting default exclusion fields

You can exclude fields by defining a default value for **fieldsToOmitRefs** in the **defaultOmitRef** field. This default exclusion applies to all templates, unless overridden by the **config.fieldsToOmitRefs** field for a specific template.

Procedure

- Create a **metadata.yaml** file to match your use case. Use the following structure as an example:

Example metadata.yaml file

```
apiVersion: v2
parts:

#...

fieldsToOmit:
  defaultOmitRef: default
  items:
    default:
      - pathToKey: a.custom.default."k8s.io"
```

where:

defaultOmitRef: default

Sets the default exclusion for all templates, unless overridden by the **config.fieldsToOmitRefs** field for a specific template.

pathToKey: a.custom.default."k8s.io"

The value is excluded for all templates.

6.4.4.3. Excluding specific fields

You can specify fields to exclude by defining the path to the field, and then referencing the definition in the **config** section for a template.

Procedure

- Create a **metadata.yaml** file to match your use case. Use the following structure as an example:

Example metadata.yaml file

```
apiVersion: v2
parts:
  - name: Part1
    components:
      - name: Component1
        path: deployment.yaml
        config:
          fieldsToOmitRefs:
            - deployments

#...
```

```

fieldsToOmit:
  items:
    deployments:
      - pathToKey: spec.selector.matchLabels.k8s-app

```

where:

deployments

References the **fieldsToOmit.items.deployments** item for the **deployment.yaml** template.

pathToKey: spec.selector.matchLabels.k8s-app

Excludes the **spec.selector.matchLabels.k8s-app** field from the comparison.



NOTE

Setting **fieldsToOmitRefs** replaces the default value.

6.4.4.4. Excluding specific fields by setting default exclusion groups

You can create default groups of fields to exclude. A group of exclusions can reference another group to avoid duplication when defining exclusions.

Procedure

- Create a **metadata.yaml** file to match your use case. Use the following structure as an example:

Example metadata.yaml file

```

apiVersion: v2
parts:

#...

fieldsToOmit:
  defaultOmitRef: default
  items:
    common:
      - pathToKey: metadata.annotations."kubernetes.io/metadata.name"
      - pathToKey: metadata.annotations."kubernetes.io/metadata.name"
      - pathToKey: metadata.annotations."kubectl.kubernetes.io/last-applied-configuration"
      - pathToKey: metadata.creationTimestamp
      - pathToKey: metadata.generation
      - pathToKey: spec.ownerReferences
      - pathToKey: metadata.ownerReferences
  default:
    - include: common
    - pathToKey: status

```

where:

include: common

The **common** group is included in the default group.

6.4.5. Configuring inline validation for template fields

You can enable inline regular expressions to validate template fields, especially in scenarios where Golang templating syntax is difficult to maintain or overly complex. Using inline regular expressions simplifies templates, improves readability, and allows for more advanced validation logic.

The **cluster-compare** plugin provides two functions for inline validation:

- **regex**: Validates content in a field using a regular expression.
- **capturegroups**: Enhances multi-line text comparisons by processing non-capture group text as exact matches, applying regular expression matching only within named capture groups, and ensuring consistency for repeated capture groups.

When you use either the **regex** or **capturegroups** function for inline validation, the **cluster-compare** plugin enforces that identically named capture groups have the same values across multiple fields within a template. This means that if a named capture group, such as **(?<username>[a-z0-9]+)**, appears in multiple fields, the values for that group must be consistent throughout the template.

6.4.5.1. Configuring inline validation with the regex function

Use the **regex** inline function to validate fields using regular expressions.

Procedure

1. Create a **metadata.yaml** file to match your use case. Use the following structure as an example:

```
apiVersion: v2
parts:
- name: Part1
  components:
  - name: Example
    allOf:
    - path: example.yaml
      config:
        perField:
        - pathToKey: spec.bigTextBlock
          inlineDiffFunc: regex
```

where:

pathToKey: spec.bigTextBlock

Specifies the field for inline validation.

inlineDiffFunc: regex

Enables inline validation using regular expressions.

2. Use a regular expression to validate the field in the associated template:

```
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: dashboard
```

```
data:
  username: "(?<username>[a-z0-9]+)"
  bigTextBlock: |-
    This is a big text block with some static content, like this line.
    It also has a place where (?<username>[a-z0-9]+) would put in their own name. (?
    <username>[a-z0-9]+) would put in their own name.
```

6.4.5.2. Configuring inline validation with the `capturegroups` function

Use the **`capturegroups`** inline function for more precise validation of fields featuring multi-line strings. This function also ensures that identically named capture groups have the same values across multiple fields.

Procedure

1. Create a **`metadata.yaml`** file to match your use case. Use the following structure as an example:

```
apiVersion: v2
parts:
- name: Part1
  components:
  - name: Example
    allOf:
    - path: example.yaml
      config:
        perField:
        - pathToKey: data.username
          inlineDiffFunc: regex
        - pathToKey: spec.bigTextBlock
          inlineDiffFunc: capturegroups
```

where:

`pathToKey: data.username`

Specifies the field for inline validation.

`inlineDiffFunc: regex`

Enables inline validation using capture groups.

`pathToKey: spec.bigTextBlock`

Specifies the multi-line field for capture-group validation.

`inlineDiffFunc: capturegroups`

Enables inline validation using capture groups.

2. Use a regular expression to validate the field in the associated template:

```
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: dashboard
data:
  username: "(?<username>[a-z0-9]+)"
  bigTextBlock: |-
```

This static content outside of a capture group should match exactly.
 Here is a username capture group: (?<username>[a-z0-9]+).
 It should match this capture group: (?<username>[a-z0-9]+).

If the username value captured in the **data.username** field does not match the value captured in **bigTextBlock**, the **cluster-compare** plugin warns you about the inconsistent matching.

The following is example output with a warning about the inconsistent matching:

```
WARNING: Capturegroup (?<username>...) matched multiple values: « mismatchuser | exampleuser
»
```

6.4.6. Configuring descriptions for the output

Each part, component, or template can include descriptions to provide additional context, instructions, or documentation links. These descriptions are helpful to convey why a specific template or structure is required.

Procedure

- Create a **metadata.yaml** file to match your use case. Use the following structure as an example:

```
apiVersion: v2
parts:
  - name: Part1
    description: |-
      General text for every template under this part, unless overridden.
    components:
      - name: Component1
        # With no description set, this inherits the description from the part above.
        OneOf:
          - path: Template1.yaml
            # This inherits the component description, if set.
          - path: Template2.yaml
          - path: Template3.yaml
          description: |-
            This template has special instructions that don't apply to the others.
      - name: Component2
        description: |-
          This overrides the part text with something more specific.
          Multi-line text is supported, at all levels.
        allOf:
          - path: RequiredTemplate1.yaml
          - path: RequiredTemplate2.yaml
          description: |-
            Required for important reasons.
          - path: RequiredTemplate3.yaml
```

6.5. PERFORMING ADVANCED REFERENCE CONFIGURATION CUSTOMIZATION

For scenarios where you want to allow temporary deviations from the reference design, you can apply more advanced customizations.

**WARNING**

These customizations override the default matching process that the **cluster-compare** plugin uses during a comparison. Use caution when applying these advanced customizations as it can lead to unintended consequences, such as excluding consequential information from a cluster comparison.

Some advanced tasks to dynamically customize your reference configuration include the following:

- **Manual matching:** Configure a user configuration file to manually match a custom resource from the cluster to a template in the reference configuration.
- **Patching the reference:** Patch a reference to configure a reference configuration by using a patch option with the **cluster-compare** command.

6.5.1. Configuring manual matching between CRs and templates

In some cases, the **cluster-compare** plugin's default matching might not work as expected. You can manually define how a custom resource (CR) maps to a template by using a user configuration file.

By default, the plugin maps a CR to a template based on the **apiversion**, **kind**, **name**, and **namespace** fields. However, multiple templates might match a single CR. For example, this can occur in the following scenarios:

- Multiple templates exist with the same **apiversion**, **kind**, **name**, and **namespace** fields.
- Templates match any CR with a specific **apiversion** and **kind**, regardless of its **namespace** or **name**.

When a CR matches multiple templates, the plugin uses a tie-breaking mechanism that selects the template with the fewest differences. To explicitly control which template the plugin chooses, you can create a user configuration YAML file that defines manual matching rules. You can pass this configuration file to the **cluster-compare** command to enforce the required template selection.

Procedure

1. Create a user configuration file to define the manual matching criteria:

Example user-config.yaml file

```
correlationSettings:
  manualCorrelation:
    correlationPairs:
      ptp.openshift.io/v1_PtpConfig_openshift-ptp_grandmaster: optional/ptp-
config/PtpOperatorConfig.yaml
      ptp.openshift.io/v1_PtpOperatorConfig_openshift-ptp_default: optional/ptp-
config/PtpOperatorConfig.yaml
```

where:

correlationSettings

The **correlationSettings** section contains the manual correlation settings.

manualCorrelation

The **manualCorrelation** section specifies that manual correlation is enabled.

correlationPairs

The **correlationPairs** section lists the CR and template pairs to manually match.

ptp.openshift.io/v1_PtpConfig_openshift-ptp_grandmaster: optional/ptp-config/PtpOperatorConfig.yaml

Specifies the CR and template pair to match. The CR specification uses the following format: **<apiversion>_<kind>_<namespace>_<name>**. For cluster-scoped CRs that do not have a namespace, use the following format: **<apiversion>_<kind>_<name>**. The path to the template must be relative to the **metadata.yaml** file.

2. Reference the user configuration file in a **cluster-compare** command by running the following command:

```
$ oc cluster-compare -r <path_to_reference_config>/metadata.yaml -c
<path_to_user_config>/user-config.yaml
```

where:

-c <path_to_user_config>/user-config.yaml

Specify the **user-config.yaml** file by using the **-c** option.

6.5.2. Patching a reference configuration

In certain scenarios, you might need to patch the reference configuration to handle expected deviations in a cluster configuration. The plugin applies the patch during the comparison process, modifying the specified resource fields as defined in the patch file.

For example, you might need to temporarily patch a template because a cluster uses a deprecated field that is out-of-date with the latest reference configuration. Patched files are reported in the comparison output summary.

You can create a patch file in two ways:

- Use the **cluster-compare** plugin to generate a patch YAML file.
- Create your own patch file.

6.5.2.1. Using the cluster-compare plugin to generate a patch

You can use the **cluster-compare** plugin to generate a patch for specific template files. The plugin adjusts the template to ensure it matches with the cluster custom resource (CR). Any previously valid differences in the patched template are not reported. The plugin highlights the patched files in the output.

Procedure

1. Generate patches for templates by running the following command:

```
$ oc cluster-compare -r <path_to_reference_config>/metadata.yaml -o 'generate-patches' --
override-reason "A valid reason for the override" --generate-override-for "<template1_path>"
--generate-override-for "<template2_path>" > <path_to_patches_file>
```

- **-r** specifies the path to the metadata.yaml file of the reference configuration.
- **-o** specifies the output format. To generate a patch output, you must use the **generate-patches** value.
- **--override-reason** describes the reason for the patch.
- **--generate-override-for** specifies a path to the template that requires a patch.



NOTE

You must use a file path for the target template that is relative to the **metadata.yaml** file. For example, if the file path for the **metadata.yaml** file is **./compare/metadata.yaml**, a relative file path for the template might be **optional/my-template.yaml**.

- **<path_to_patches_file>** specifies the filename and path for your patch.

2. Optional: Review the patch file before applying to the reference configuration:

Example patch-config file

```
- apiVersion: storage.k8s.io/v1
  kind: StorageClass
  name: crc-csi-hostpath-provisioner
  patch: '{"provisioner":"kubevirt.io.hostpath-provisioner"}'
  reason: A valid reason for the override
  templatePath: optional/local-storage-operator/StorageClass.yaml
  type: mergepatch
```

where:

patch

The plugin patches the fields in the template to match the CR.

templatePath

The path to the template.

type: mergepatch

The **mergepatch** option merges the JSON into the target template. Unspecified fields remain unchanged.

3. Apply the patch to the reference configuration by running the following command:

```
$ oc cluster-compare -r <referenceConfigurationDirectory> -p <path_to_patches_file>
```

- **-r** specifies the path to the metadata.yaml file of the reference configuration.
- **-p** specifies the path to the patch file.

The following is example output:

```
...

Cluster CR: storage.k8s.io/v1_StorageClass_crc-csi-hostpath-provisioner
Reference File: optional/local-storage-operator/StorageClass.yaml
Description: Component description
Diff Output: None
Patched with patch
Patch Reasons:
- A valid reason for the override

...

No CRs are unmatched to reference CRs
Metadata Hash: bb2165004c496b32e0c8509428fb99c653c3cf4fba41196ea6821bd05c3083ab
Cluster CRs with patches applied: 1
```

6.5.2.2. Creating a patch file manually

You can write a patch file to handle expected deviations in a cluster configuration.

NOTE

Patches have three possible values for the **type** field:

- **mergepatch** - Merges the JSON into the target template. Unspecified fields remain unchanged.
- **rfc6902** - Merges the JSON in the target template using **add**, **remove**, **replace**, **move**, and **copy** operations. Each operation targets a specific path.
- **go-template** - Defines a Golang template. The plugin renders the template using the cluster custom resource (CR) as input and generates either a **mergepatch** or **rfc6902** patch for the target template.

The following example shows the same patch using all three different formats.

Procedure

1. Create a patch file to match your use case. Use the following structure as an example:

Example patch-config

```
- apiVersion: v1
  kind: Namespace
  name: openshift-storage
  reason: known deviation
  templatePath: namespace.yaml
  type: mergepatch
  patch: '{"metadata":{"annotations":
{"openshift.io/sa.scc.mcs":"s0:c29,c14","openshift.io/sa.scc.supplemental-
groups":"1000840000/10000","openshift.io/sa.scc.uid-
range":"1000840000/10000","reclaimspace.csiaddons.openshift.io/schedule":"@weekly","worklc
```

```

ad.openshift.io/allowed":null},"labels":{"kubernetes.io/metadata.name":"openshift-
storage","olm.operatorgroup.uid/ffcf3f2d-3e37-4772-97bc-
983cdfce128b":"","openshift.io/cluster-monitoring":"false","pod-
security.kubernetes.io/audit":"privileged","pod-security.kubernetes.io/audit-
version":"v1.24","pod-security.kubernetes.io/warn":"privileged","pod-
security.kubernetes.io/warn-
version":"v1.24","security.openshift.io/scc.podSecurityLabelSync":"true"},"spec":{"finalizers":
["kubernetes"]}}'
- name: openshift-storage
  apiVersion: v1
  kind: Namespace
  templatePath: namespace.yaml
  type: rfc6902
  reason: known deviation
  patch: '[
    {"op": "add", "path": "/metadata/annotations/openshift.io~1sa.scc.mcs", "value":
"s0:c29,c14"},
    {"op": "add", "path": "/metadata/annotations/openshift.io~1sa.scc.supplemental-groups",
"value": "1000840000/10000"},
    {"op": "add", "path": "/metadata/annotations/openshift.io~1sa.scc.uid-range", "value":
"1000840000/10000"},
    {"op": "add", "path":
"/metadata/annotations/reclaimspace.csiaddons.openshift.io~1schedule", "value":
"@weekly"},
    {"op": "remove", "path": "/metadata/annotations/workload.openshift.io~1allowed"},
    {"op": "add", "path": "/metadata/labels/kubernetes.io~1metadata.name", "value": "openshift-
storage"},
    {"op": "add", "path": "/metadata/labels/olm.operatorgroup.uid~1ffcf3f2d-3e37-4772-97bc-
983cdfce128b", "value": ""},
    {"op": "add", "path": "/metadata/labels/openshift.io~1cluster-monitoring", "value": "false"},
    {"op": "add", "path": "/metadata/labels/pod-security.kubernetes.io~1audit", "value":
"privileged"},
    {"op": "add", "path": "/metadata/labels/pod-security.kubernetes.io~1audit-version", "value":
"v1.24"},
    {"op": "add", "path": "/metadata/labels/pod-security.kubernetes.io~1warn", "value":
"privileged"},
    {"op": "add", "path": "/metadata/labels/pod-security.kubernetes.io~1warn-version", "value":
"v1.24"},
    {"op": "add", "path": "/metadata/labels/security.openshift.io~1scc.podSecurityLabelSync",
"value": "true"},
    {"op": "add", "path": "/spec", "value": {"finalizers": ["kubernetes"]}}
  ]'
- apiVersion: v1
  kind: Namespace
  name: openshift-storage
  reason: "known deviation"
  templatePath: namespace.yaml
  type: go-template
  patch: |
    {
      "type": "rfc6902",
      "patch": '[
        {"op": "add", "path": "/metadata/annotations/openshift.io~1sa.scc.mcs", "value":
"s0:c29,c14"},
        {"op": "add", "path": "/metadata/annotations/openshift.io~1sa.scc.supplemental-
groups", "value": "1000840000/10000"},

```

```

      {"op": "add", "path": "/metadata/annotations/openshift.io~1sa.scc.uid-range", "value":
"1000840000/10000"},
      {"op": "add", "path":
"/metadata/annotations/reclaimspace.csiaddons.openshift.io~1schedule", "value":
"@weekly"},
      {"op": "remove", "path": "/metadata/annotations/workload.openshift.io~1allowed"},
      {"op": "add", "path": "/metadata/labels/kubernetes.io~1metadata.name", "value":
"openshift-storage"},
      {"op": "add", "path": "/metadata/labels/olm.operatorgroup.uid~1ffc3f2d-3e37-4772-
97bc-983cdfce128b", "value": ""},
      {"op": "add", "path": "/metadata/labels/openshift.io~1cluster-monitoring", "value":
"false"},
      {"op": "add", "path": "/metadata/labels/pod-security.kubernetes.io~1audit", "value":
"privileged"},
      {"op": "add", "path": "/metadata/labels/pod-security.kubernetes.io~1audit-version",
"value": "v1.24"},
      {"op": "add", "path": "/metadata/labels/pod-security.kubernetes.io~1warn", "value":
"privileged"},
      {"op": "add", "path": "/metadata/labels/pod-security.kubernetes.io~1warn-version",
"value": "v1.24"},
      {"op": "add", "path":
"/metadata/labels/security.openshift.io~1scc.podSecurityLabelSync", "value": "true"},
      {"op": "add", "path": "/spec", "value": {"finalizers": {{ .spec.finalizers | toJson }} }}
    ]
  }

```

The patches use the **kind**, **apiVersion**, **name**, and **namespace** fields to match the patch with the correct cluster CR.

2. Apply the patch to the reference configuration by running the following command:

```
$ oc cluster-compare -r <referenceConfigurationDirectory> -p <path_to_patches_file>
```

- **-r** specifies the path to the metadata.yaml file of the reference configuration.
- **p** specifies the path to the patch file.

The following is example output:

...

Cluster CR: storage.k8s.io/v1_StorageClass_crc-csi-hostpath-provisioner

Reference File: namespace.yaml

Description: Component description

Diff Output: None

Patched with patch

Patch Reasons:

- known deviation
- known deviation
- known deviation

...

No CRs are unmatched to reference CRs

Metadata Hash: bb2165004c496b32e0c8509428fb99c653c3cf4fba41196ea6821bd05c3083ab

Cluster CRs with patches applied: 1



6.6. TROUBLESHOOTING CLUSTER COMPARISONS

When using the **cluster-compare** plugin, you might see unexpected results, such as false positives or conflicts when multiple cluster custom resources (CRs) exist.

6.6.1. Troubleshooting false positives for missing resources

The plugin might report a missing resource even though the cluster custom resource (CR) is present in the cluster.

Procedure

1. Ensure you are using the latest version of the **cluster-compare** plugin. For more information, see "Installing the cluster-compare plugin".
2. Ensure you are using the most up-to-date version of the reference configuration.
3. Ensure that template has the same **apiVersion**, **kind**, **name**, and **namespace** fields as the cluster CR.

6.6.2. Troubleshooting multiple template matches for the same CR

In some cases, more than one cluster CR can match a template because they feature the same **apiVersion**, **namespace**, and **kind**. The plugin's default matching compares the CR that features the least differences.

You can optionally configure your reference configuration to avoid this situation.

Procedure

1. Ensure the templates feature distinct **apiVersion**, **namespace**, and **kind** values to ensure no duplicate template matching.
2. Use a user configuration file to manually match a template to a CR. For more information, see "Configuring manual matching between CRs and templates".

6.6.3. Additional resources

- [Installing the cluster-compare plugin](#)
- [Configuring manual matching between CRs and templates](#)

CHAPTER 7. PLANNING YOUR ENVIRONMENT ACCORDING TO OBJECT MAXIMUMS

To ensure your cluster meets performance and scalability requirements, plan your environment according to tested object maximums. By reviewing these limits, you can design an OpenShift Container Platform deployment that operates reliably within supported boundaries.

The example guidelines are based on the largest possible cluster. For smaller clusters, the maximums are lower. There are many factors that influence the stated thresholds, including the etcd version or storage data format. In most cases, exceeding these numbers results in lower overall performance but might not cause your cluster to fail.



WARNING

Clusters that experience rapid change, such as those with many starting and stopping pods, can have a lower practical maximum size than documented.

7.1. OPENSIFT CONTAINER PLATFORM TESTED CLUSTER MAXIMUMS FOR MAJOR RELEASES

To ensure your deployment remains supported, plan your cluster configuration by using tested cluster maximums. OpenShift Container Platform validates these specific limits for major releases rather than theoretical absolute cluster maximums, ensuring stability for your environment.



NOTE

Red Hat does not provide direct guidance on sizing your OpenShift Container Platform cluster. This is because determining whether your cluster is within the supported bounds of OpenShift Container Platform requires careful consideration of all the multidimensional factors that limit the cluster scale.

OpenShift Container Platform supports tested cluster maximums rather than absolute cluster maximums. Not every combination of OpenShift Container Platform version, control plane workload, and network plugin are tested, so the following table does not represent an absolute expectation of scale for all deployments. Scaling to a maximum on all dimensions simultaneously might not be possible. The table contains tested maximums for specific workloads and deployments, and serves as a scale guide as to what can be expected with similar deployments.

Maximum type	4.x tested maximum	Notes
--------------	--------------------	-------

Maximum type	4.x tested maximum	Notes
Number of nodes	2,000	Pause pods were deployed to stress the control plane components of OpenShift Container Platform at 2000 node scale. The ability to scale to similar numbers will vary depending upon specific deployment and workload parameters.
Number of pods	150,000	The pod count displayed here is the number of test pods. The actual number of pods depends on the application's memory, CPU, and storage requirements.
Number of pods per node	2,500	This was tested on a cluster with 31 servers: 3 control planes, 2 infrastructure nodes, and 26 compute nodes. If you need 2,500 user pods, you need both a hostPrefix of 20 , which allocates a network large enough for each node to contain more than 2000 pods, and a custom kubelet config with maxPods set to 2500 . For more information, see Running 2500 pods per node on OCP 4.13 .
Number of namespaces	10,000	When there are a large number of active projects, etcd might suffer from poor performance if the keyspace grows excessively large and exceeds the space quota. Periodic maintenance of etcd, including defragmentation, is highly recommended to free etcd storage.
Number of builds	10,000 (Default pod RAM 512 Mi) - Source-to-Image (S2I) build strategy	-

Maximum type	4.x tested maximum	Notes
Number of pods per namespace	25,000	There are several control loops in the system that must iterate over all objects in a given namespace as a reaction to some changes in state. Having a large number of objects of a given type in a single namespace can make those loops expensive and slow down processing given state changes. The limit assumes that the system has enough CPU, memory, and disk to satisfy the application requirements.
Number of routes per default 2-router deployment	9,000	-
Number of secrets	80,000	-
Number of config maps	90,000	-
Number of services	10,000	Each service port and each service back-end has a corresponding entry in iptables . The number of back-ends of a given service impact the size of the Endpoints objects, which impacts the size of data that is being sent all over the system.
Number of services per namespace	5,000	-
Number of back-ends per service	5,000	-
Number of deployments per namespace	2,000	-
Number of build configs	12,000	-

Maximum type	4.x tested maximum	Notes
Number of custom resource definitions (CRD)	1,024	Tested on a cluster with 29 servers: 3 control planes, 2 infrastructure nodes, and 24 compute nodes. The cluster had 500 namespaces. OpenShift Container Platform has a limit of 1,024 total custom resource definitions (CRD), including those installed by OpenShift Container Platform, products integrating with OpenShift Container Platform, and user-created CRDs. If there are more than 1,024 CRDs created, then there is a possibility that oc command requests might be throttled.

Example scenario

As an example, 500 compute nodes (m5.2xl) were tested, and are supported, by using OpenShift Container Platform 4.22, the OVN-Kubernetes network plugin, and the following workload objects:

- 200 namespaces, in addition to the defaults
- 60 pods per node; 30 server and 30 client pods (30k total)
- 57 image streams/ns (11.4k total)
- 15 services/ns backed by the server pods (3k total)
- 15 routes/ns backed by the previous services (3k total)
- 20 secrets/ns (4k total)
- 10 config maps/ns (2k total)
- 6 network policies/ns, including deny-all, allow-from ingress and intra-namespaces rules
- 57 builds/ns

The following factors are known to affect cluster workload scaling, positively or negatively, and should be factored into the scale numbers when planning a deployment. For additional information and guidance, contact your sales representative or [Red Hat support](#).

- Number of pods per node
- Number of containers per pod
- Type of probes used (for example, liveness/readiness, exec/http)
- Number of network policies
- Number of projects, or namespaces

- Number of image streams per project
- Number of builds per project
- Number of services/endpoints and type
- Number of routes
- Number of shards
- Number of secrets
- Number of config maps
- Rate of API calls, or the cluster “churn”, which is an estimation of how quickly things change in the cluster configuration.
 - Prometheus query for pod creation requests per second over 5 minute windows:
`sum(irate(apiserver_request_count{resource="pods",verb="POST"}[5m]))`
 - Prometheus query for all API requests per second over 5 minute windows:
`sum(irate(apiserver_request_count{}[5m]))`
- Cluster node resource consumption of CPU
- Cluster node resource consumption of memory

7.2. OPENSIFT CONTAINER PLATFORM ENVIRONMENT AND CONFIGURATION ON WHICH THE CLUSTER MAXIMUMS ARE TESTED

To validate your deployment limits, review the environment and configuration details for the cloud platforms on which OpenShift Container Platform cluster maximums are tested. This reference ensures your infrastructure aligns with the specific scenarios used to validate scalability limits.

7.2.1. AWS cloud platform cluster maximums

Node	Flavor	vCPU	RAM (GiB)	Disk type	Disk size (GiB) or IOS	Count	Region
Control plane/etc d	r5.4xlarge	16	128	gp3	220	3	us-west-2
Infra	m5.12xlarge	48	192	gp3	100	3	us-west-2
Workload	m5.4xlarge	16	64	gp3	500	1	us-west-2
Compute	m5.2xlarge	8	32	gp3	100	3/25/250/500	us-west-2

where:

Control plane/etcd

Control plane/etcd nodes use gp3 disks with a baseline performance of 3000 IOPS and 125 MiB per second because etcd is latency sensitive. The gp3 volumes do not use burst performance.

Infra

Infra nodes are used to host Monitoring, Ingress, and Registry components to ensure they have enough resources to run at large scale.

Workload

The workload node is dedicated to run performance and scalability workload generators. Using a larger disk size of 500 GiB ensures that there is enough space to store the large amounts of data that is collected during the performance and scalability test run.

Compute

The cluster is scaled in iterations of 3, 25, 250, and 500 compute nodes. Performance and scalability tests are executed at the specified node counts.

7.2.2. IBM Power platform cluster maximums

Node	vCPU	RAM (GiB)	Disk type	Disk size (GiB) or IOS	Count
Control plane/etcd	16	32	io1	120 / 10 IOPS per GiB	3
Infra	16	64	gp2	120	2
Workload	16	256	gp2	120	1
Compute	16	64	gp2	120	2 to 100

where:

Control plane/etcd

io1 disks with 120 / 10 IOPS per GiB are used for control plane/etcd nodes as etcd is I/O intensive and latency sensitive.

Infra

Infra nodes are used to host Monitoring, Ingress, and Registry components to ensure they have enough resources to run at large scale.

Workload

Workload node is dedicated to run performance and scalability workload generators.

Workload.120

Larger disk size is used so that there is enough space to store the large amounts of data that is collected during the performance and scalability test run.

Compute.2 to 100

Cluster is scaled in iterations.

7.2.3. IBM Z platform cluster maximums

Node	vCPU	RAM (GiB)	Disk type	Disk size (GiB) or IOS	Count or IOS
Control plane/etcd	8	32	ds8k	300 / LCU 1	3
Compute	8	32	ds8k	150 / LCU 2	4 nodes (scaled to 100/250/500 pods per node)

where:

Control plane/etcd

Nodes are distributed between two logical control units (LCUs) to optimize disk I/O load of the control plane/etcd nodes as etcd is I/O intensive and latency sensitive. Etcd I/O demand should not interfere with other workloads. Four compute nodes are used for the tests running several iterations with 100/250/500 pods at the same time. First, idling pods were used to evaluate if pods can be instanced. Next, a network and CPU demanding client/server workload were used to evaluate the stability of the system under stress. Client and server pods were pairwise deployed and each pair was spread over two compute nodes.

Compute

No separate workload node was used. The workload simulates a microservice workload between two compute nodes.

vCPU

Physical number of processors used is six Integrated Facilities for Linux (IFLs).

RAM (GiB)

Total physical memory used is 512 GiB.

7.3. HOW TO PLAN YOUR ENVIRONMENT ACCORDING TO TESTED CLUSTER MAXIMUMS

To ensure your infrastructure meets operational requirements, plan your OpenShift Container Platform environment according to tested cluster maximums. Designing your cluster within these validated limits ensures that you can maintain stability and ensures your deployment remains supported



IMPORTANT

Oversubscribing the physical resources on a node affects resource guarantees the Kubernetes scheduler makes during pod placement. Learn what measures you can take to avoid memory swapping.

Some of the tested maximums are stretched only in a single dimension. They will vary when many objects are running on the cluster.

The numbers noted in this documentation are based on Red Hat's test methodology, setup, configuration, and tunings. These numbers can vary based on your own individual setup and environments.

While planning your environment, determine how many pods are expected to fit per node by using the following formula:

$$\text{required pods per cluster} / \text{pods per node} = \text{total number of nodes needed}$$

The default maximum number of pods per node is 250. However, the number of pods that fit on a node is dependent on the application itself. Consider the application's memory, CPU, and storage requirements, as described in "How to plan your environment according to application requirements".

Example scenario

If you want to scope your cluster for 2200 pods per cluster, you would need at least five nodes, assuming that there are 500 maximum pods per node. The following formula shows the calculation:

$$2200 / 500 = 4.4$$

If you increase the number of nodes to 20, then the pod distribution changes to 110 pods per node. The following formula shows the calculation:

$$2200 / 20 = 110$$

Where:

$$\text{required pods per cluster} / \text{total number of nodes} = \text{expected pods per node}$$

OpenShift Container Platform includes several system pods, such as OVN-Kubernetes, DNS, Operators, and others, which run across every compute node by default. Therefore, the result of the above formula can vary.

7.4. HOW TO PLAN YOUR ENVIRONMENT ACCORDING TO APPLICATION REQUIREMENTS

To ensure your infrastructure handles workload demands efficiently, plan your environment according to application requirements. By planning in this way, you can determine the necessary compute, storage, and networking resources to maintain performance and stability.

Consider an example application environment:

Pod type	Pod quantity	Max memory	CPU cores	Persistent storage
apache	100	500 MB	0.5	1 GB
node.js	200	1 GB	1	1 GB
postgresql	100	1 GB	2	10 GB
JBoss EAP	100	1 GB	1	1 GB

Extrapolated requirements: 550 CPU cores, 450GB RAM, and 1.4TB storage.

Instance size for nodes can be modulated up or down, depending on your preference. Nodes are often resource overcommitted. In this deployment scenario, you can choose to run additional smaller nodes or fewer larger nodes to provide the same amount of resources. Factors such as operational agility and cost-per-instance should be considered.

Node type	Quantity	CPUs	RAM (GB)
Nodes (option 1)	100	4	16
Nodes (option 2)	50	8	32
Nodes (option 3)	25	16	64

Some applications lend themselves well to overcommitted environments, and some do not. Most Java applications and applications that use huge pages are examples of applications that would not allow for overcommitment. That memory can not be used for other applications. In the example above, the environment would be roughly 30 percent overcommitted, a common ratio.

The application pods can access a service either by using environment variables or DNS. If using environment variables, for each active service the variables are injected by the kubelet when a pod is run on a node. A cluster-aware DNS server watches the Kubernetes API for new services and creates a set of DNS records for each one.

If DNS is enabled throughout your cluster, then all pods should automatically be able to resolve services by their DNS name. Service discovery using DNS can be used in case you must go beyond 5000 services. When using environment variables for service discovery, the argument list exceeds the allowed length after 5000 services in a namespace, then the pods and deployments will start failing. Disable the service links in the deployment's service specification file to overcome this:

```
apiVersion: template.openshift.io/v1
kind: Template
metadata:
  name: deployment-config-template
  creationTimestamp:
  annotations:
    description: This template will create a deploymentConfig with 1 replica, 4 env vars and a service.
    tags: "
objects:
```

```

- apiVersion: apps.openshift.io/v1
  kind: DeploymentConfig
  metadata:
    name: deploymentconfig${IDENTIFIER}
  spec:
    template:
      metadata:
        labels:
          name: replicationcontroller${IDENTIFIER}
      spec:
        enableServiceLinks: false
        containers:
        - name: pause${IDENTIFIER}
          image: "${IMAGE}"
          ports:
            - containerPort: 8080
              protocol: TCP
          env:
            - name: ENVVAR1_${IDENTIFIER}
              value: "${ENV_VALUE}"
            - name: ENVVAR2_${IDENTIFIER}
              value: "${ENV_VALUE}"
            - name: ENVVAR3_${IDENTIFIER}
              value: "${ENV_VALUE}"
            - name: ENVVAR4_${IDENTIFIER}
              value: "${ENV_VALUE}"
          resources: {}
          imagePullPolicy: IfNotPresent
          capabilities: {}
          securityContext:
            capabilities: {}
            privileged: false
          restartPolicy: Always
          serviceAccount: ""
        replicas: 1
        selector:
          name: replicationcontroller${IDENTIFIER}
        triggers:
        - type: ConfigChange
        strategy:
          type: Rolling
- apiVersion: v1
  kind: Service
  metadata:
    name: service${IDENTIFIER}
  spec:
    selector:
      name: replicationcontroller${IDENTIFIER}
    ports:
      - name: serviceport${IDENTIFIER}
        protocol: TCP
        port: 80
        targetPort: 8080
    clusterIP: ""
    type: ClusterIP
    sessionAffinity: None

```



```

status:
  loadBalancer: {}
parameters:
- name: IDENTIFIER
  description: Number to append to the name of resources
  value: '1'
  required: true
- name: IMAGE
  description: Image to use for deploymentConfig
  value: gcr.io/google-containers/pause-amd64:3.0
  required: false
- name: ENV_VALUE
  description: Value to use for environment variables
  generate: expression
  from: "[A-Za-z0-9]{255}"
  required: false
labels:
  template: deployment-config-template

```

The number of application pods that can run in a namespace is dependent on the number of services and the length of the service name when the environment variables are used for service discovery.

ARG_MAX on the system defines the maximum argument length for a new process and the variable is set to 2097152 bytes (2 MiB) by default. The Kubelet injects environment variables in to each pod scheduled to run in the namespace including the following variables:

- **<SERVICE_NAME>_SERVICE_HOST=<IP>**
- **<SERVICE_NAME>_SERVICE_PORT=<PORT>**
- **<SERVICE_NAME>_PORT=tcp://<IP>:<PORT>**
- **<SERVICE_NAME>_PORT_<PORT>_TCP=tcp://<IP>:<PORT>**
- **<SERVICE_NAME>_PORT_<PORT>_TCP_PROTO=tcp**
- **<SERVICE_NAME>_PORT_<PORT>_TCP_PORT=<PORT>**
- **<SERVICE_NAME>_PORT_<PORT>_TCP_ADDR=<ADDR>**

The pods in the namespace will start to fail if the argument length exceeds the allowed value and the number of characters in a service name impacts it. For example, in a namespace with 5000 services, the limit on the service name is 33 characters, which enables you to run 5000 pods in the namespace.

CHAPTER 8. USING QUOTAS AND LIMIT RANGES

As a cluster administrator, you can use quotas and limit ranges to set constraints. These constraints limit the number of objects or the amount of compute resources that are used in your project.

By using quotes and limits, you can better manage and allocate resources across all projects. You can also ensure that no projects use more resources than is appropriate for the cluster size.

A resource quota, defined by a **ResourceQuota** object, provides constraints that limit aggregate resource consumption per project. The quota can limit the quantity of objects that can be created in a project by type. Additionally, the quota can limit the total amount of compute resources and storage that might be consumed by resources in that project.



IMPORTANT

Quotas are set by cluster administrators and are scoped to a given project. OpenShift Container Platform project owners can change quotas for their project, but not limit ranges. OpenShift Container Platform users cannot modify quotas or limit ranges.

8.1. RESOURCES MANAGED BY QUOTA

To limit aggregate resource consumption per project, define a **ResourceQuota** object. By using this object, you can restrict the number of created objects by type. You can also restrict the total amount of compute resources and storage consumed within the project.

The following tables describe the set of compute resources and object types that a quota might manage.



NOTE

A pod is in a terminal state if **status.phase** is **Failed** or **Succeeded**.

Table 8.1. Compute resources managed by quota

Resource Name	Description
cpu	The sum of CPU requests across all pods in a non-terminal state cannot exceed this value. cpu and requests.cpu are the same value and can be used interchangeably.
memory	The sum of memory requests across all pods in a non-terminal state cannot exceed this value. memory and requests.memory are the same value and can be used interchangeably.
ephemeral-storage	The sum of local ephemeral storage requests across all pods in a non-terminal state cannot exceed this value. ephemeral-storage and requests.ephemeral-storage are the same value and can be used interchangeably. This resource is available only if you enabled the ephemeral storage technology preview. This feature is disabled by default.

Resource Name	Description
requests.cpu	The sum of CPU requests across all pods in a non-terminal state cannot exceed this value. cpu and requests.cpu are the same value and can be used interchangeably.
requests.memory	The sum of memory requests across all pods in a non-terminal state cannot exceed this value. memory and requests.memory are the same value and can be used interchangeably.
requests.ephemeral-storage	The sum of ephemeral storage requests across all pods in a non-terminal state cannot exceed this value. ephemeral-storage and requests.ephemeral-storage are the same value and can be used interchangeably. This resource is available only if you enabled the ephemeral storage technology preview. This feature is disabled by default.
limits.cpu	The sum of CPU limits across all pods in a non-terminal state cannot exceed this value.
limits.memory	The sum of memory limits across all pods in a non-terminal state cannot exceed this value.
limits.ephemeral-storage	The sum of ephemeral storage limits across all pods in a non-terminal state cannot exceed this value. This resource is available only if you enabled the ephemeral storage technology preview. This feature is disabled by default.

Table 8.2. Storage resources managed by quota

Resource Name	Description
requests.storage	The sum of storage requests across all persistent volume claims in any state cannot exceed this value.
persistentvolumeclaims	The total number of persistent volume claims that can exist in the project.
<storage-class-name>.storageclass.storage.k8s.io/requests.storage	The sum of storage requests across all persistent volume claims in any state that have a matching storage class, cannot exceed this value.
<storage-class-name>.storageclass.storage.k8s.io/persistentvolumeclaims	The total number of persistent volume claims with a matching storage class that can exist in the project.

Table 8.3. Object counts managed by quota

Resource Name	Description
pods	The total number of pods in a non-terminal state that can exist in the project.
replicationcontrollers	The total number of replication controllers that can exist in the project.
resourcequotas	The total number of resource quotas that can exist in the project.
services	The total number of services that can exist in the project.
secrets	The total number of secrets that can exist in the project.
configmaps	The total number of ConfigMap objects that can exist in the project.
persistentvolumeclaims	The total number of persistent volume claims that can exist in the project.
openshift.io/imagestreams	The total number of image streams that can exist in the project.

You can configure an object count quota for these standard namespaced resource types using the **count/<resource>.<group>** syntax.

```
$ oc create quota <name> --hard=count/<resource>.<group>=<quota>
```

where:

<resource>

Specifies the name of the resource.

<group>

Specifies the API group, if applicable. You can use the **kubectl api-resources** command for a list of resources and their associated API groups.

8.2. SETTING RESOURCE QUOTA FOR EXTENDED RESOURCES

To manage the consumption of extended resources, such as **nvidia.com/gpu**, define a resource quota by using the **requests** prefix. Since overcommitment is prohibited for these resources, you must explicitly specify both requests and limits to ensure valid configuration.

Procedure

1. To determine how many GPUs are available on a node in your cluster, use the following command:

```
$ oc describe node ip-172-31-27-209.us-west-2.compute.internal | egrep
'Capacity|Allocatable|gpu'
```

Example output

```

openshift.com/gpu-accelerator=true
Capacity:
  nvidia.com/gpu: 2
Allocatable:
  nvidia.com/gpu: 2
  nvidia.com/gpu: 0      0

```

In this example, 2 GPUs are available.

2. Use this command to set a quota in the namespace **nvidia**. In this example, the quota is **1**:

```
$ cat gpu-quota.yaml
```

Example output

```

apiVersion: v1
kind: ResourceQuota
metadata:
  name: gpu-quota
  namespace: nvidia
spec:
  hard:
    requests.nvidia.com/gpu: 1

```

3. Create the quota with the following command:

```
$ oc create -f gpu-quota.yaml
```

Example output

```
resourcequota/gpu-quota created
```

4. Verify that the namespace has the correct quota set using the following command:

```
$ oc describe quota gpu-quota -n nvidia
```

Example output

```

Name:          gpu-quota
Namespace:     nvidia
Resource       Used  Hard
-----
requests.nvidia.com/gpu 0   1

```

5. Run a pod that asks for a single GPU with the following command:

```
$ oc create pod gpu-pod.yaml
```

Example output

```
■
```

```

apiVersion: v1
kind: Pod
metadata:
  generateName: gpu-pod-s46h7
  namespace: nvidia
spec:
  restartPolicy: OnFailure
  containers:
  - name: rhel7-gpu-pod
    image: rhel7
    env:
      - name: NVIDIA_VISIBLE_DEVICES
        value: all
      - name: NVIDIA_DRIVER_CAPABILITIES
        value: "compute,utility"
      - name: NVIDIA_REQUIRE_CUDA
        value: "cuda>=5.0"

    command: ["sleep"]
    args: ["infinity"]

  resources:
    limits:
      nvidia.com/gpu: 1

```

6. Verify that the pod is running with the following command:

```
$ oc get pods
```

Example output

```

NAME          READY   STATUS    RESTARTS   AGE
gpu-pod-s46h7  1/1     Running   0           1m

```

7. Verify that the quota **Used** counter is correct by running the following command:

```
$ oc describe quota gpu-quota -n nvidia
```

Example output

```

Name:          gpu-quota
Namespace:     nvidia
Resource       Used  Hard
-----
requests.nvidia.com/gpu  1    1

```

8. Using the following command, attempt to create a second GPU pod in the **nvidia** namespace. This is technically available on the node because it has 2 GPUs:

```
$ oc create -f gpu-pod.yaml
```

Example output

Error from server (Forbidden): error when creating "gpu-pod.yaml": pods "gpu-pod-f7z2w" is forbidden: exceeded quota: gpu-quota, requested: requests.nvidia.com/gpu=1, used: requests.nvidia.com/gpu=1, limited: requests.nvidia.com/gpu=1

You receive this **Forbidden** error message because you have a quota of 1 GPU and the pod tried to allocate a second GPU, which exceeds the allowed quota.

8.3. QUOTA SCOPES

To restrict the set of resources that a quota applies to, add associated scopes. This configuration limits usage measurement to the intersection of the enumerated scopes, ensuring that specifying a resource outside the allowed set results in a validation error.

Scope	Description
Terminating	Match pods where spec.activeDeadlineSeconds >= 0 .
NotTerminating	Match pods where spec.activeDeadlineSeconds is nil .
BestEffort	Match pods that have best effort quality of service for either cpu or memory .
NotBestEffort	Match pods that do not have best effort quality of service for cpu and memory .

A **BestEffort** scope restricts a quota to limiting the following resources:

- pods

A **Terminating**, **NotTerminating**, and **NotBestEffort** scope restricts a quota to tracking the following resources:

- pods
- memory
- requests.memory
- limits.memory
- cpu
- requests.cpu
- limits.cpu
- ephemeral-storage
- requests.ephemeral-storage
- limits.ephemeral-storage

**NOTE**

Ephemeral storage requests and limits apply only if you enabled the ephemeral storage technology preview. This feature is disabled by default.

8.4. ADDITIONAL RESOURCES

- [Resources managed by quotas](#)
- [Resource requests and overcommitment](#)

8.5. ADMIN QUOTA USAGE

To ensure projects remain within defined constraints, monitor admin quota usage. After a resource quota for a project is first created, the project restricts the ability to create any new resources that can violate a quota constraint until it has calculated updated usage statistics.

Quota enforcement

After a resource quota for a project is first created, the project restricts the ability to create any new resources that can violate a quota constraint until the quota has calculated updated usage statistics. After a quota is created and usage statistics are updated, the project accepts the creation of new content. When you create or modify resources, your quota usage is incremented immediately upon the request to create or modify the resource.

When you delete a resource, your quota use is decremented during the next full recalculation of quota statistics for the project.

A configurable amount of time determines how long the quota takes to reduce quota usage statistics to their current observed system value.

If project modifications exceed a quota usage limit, the server denies the action and returns an appropriate error message to the user. The error message explains the quota constraint violated and what their currently observed usage statistics are in the system.

Requests compared to limits

When allocating compute resources by quota, each container can specify a request and a limit value each for CPU, memory, and ephemeral storage. Quotas can restrict any of these values.

If the quota has a value specified for **requests.cpu** or **requests.memory**, then the quota requires that every incoming container makes an explicit request for those resources. If the quota has a value specified for **limits.cpu** or **limits.memory**, the quota requires that every incoming container specify an explicit limit for those resources.

8.5.1. Sample resource quota definitions

To properly structure your quota configurations, reference these sample **ResourceQuota** definitions. These YAML examples demonstrate how to specify hard limits for compute resources, storage, and object counts to ensure your project complies with cluster policies.

Example core-object-counts.yaml

```
apiVersion: v1
kind: ResourceQuota
metadata:
```



```

    name: core-object-counts
  spec:
    hard:
      configmaps: "10"
      persistentvolumeclaims: "4"
      replicationcontrollers: "20"
      secrets: "10"
      services: "10"
  # ...

```

where:

configmaps

Specifies the total number of **ConfigMap** objects that can exist in the project.

persistentvolumeclaims

Specifies the total number of persistent volume claims (PVCs) that can exist in the project.

replicationcontrollers

Specifies the total number of replication controllers that can exist in the project.

secrets

Specifies the total number of secrets that can exist in the project.

services

Specifies the total number of services that can exist in the project.

Example openshift-object-counts.yaml

```

apiVersion: v1
kind: ResourceQuota
metadata:
  name: openshift-object-counts
spec:
  hard:
    openshift.io/imagestreams: "10"
  # ...

```

where:

openshift.io/imagestreams

Specifies the total number of image streams that can exist in the project.

Example compute-resources.yaml

```

apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-resources
spec:
  hard:
    pods: "4"
    requests.cpu: "1"
    requests.memory: 1Gi
    requests.ephemeral-storage: 2Gi

```

```
limits.cpu: "2"  
limits.memory: 2Gi  
limits.ephemeral-storage: 4Gi  
# ...
```

where:

pods

Specifies the total number of pods in a non-terminal state that can exist in the project.

requests.cpu

Specifies that across all pods in a non-terminal state, the sum of CPU requests cannot exceed 1 core.

requests.memory

Specifies that across all pods in a non-terminal state, the sum of memory requests cannot exceed 1 Gi.

requests.ephemeral-storage

Specifies that across all pods in a non-terminal state, the sum of ephemeral storage requests cannot exceed 2 Gi.

limits.cpu

Specifies that across all pods in a non-terminal state, the sum of CPU limits cannot exceed 2 cores.

limits.memory

Specifies that across all pods in a non-terminal state, the sum of memory limits cannot exceed 2 Gi.

limits.ephemeral-storage

Specifies that across all pods in a non-terminal state, the sum of ephemeral storage limits cannot exceed 4 Gi.

Example besteffort.yaml

```
apiVersion: v1  
kind: ResourceQuota  
metadata:  
  name: besteffort  
spec:  
  hard:  
    pods: "1"  
  scopes:  
    - BestEffort  
# ...
```

where:

pods

Specifies the total number of pods in a non-terminal state with **BestEffort** quality of service that can exist in the project.

scopes

Specifies a restriction on the quota to only match pods that have **BestEffort** quality of service for either memory or CPU.

Example compute-resources-long-running.yaml

```

apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-resources-long-running
spec:
  hard:
    pods: "4"
    limits.cpu: "4"
    limits.memory: "2Gi"
    limits.ephemeral-storage: "4Gi"
  scopes:
    - NotTerminating
# ...

```

where:

pods

Specifies the total number of pods in a non-terminal state.

limits.cpu

Specifies that across all pods in a non-terminal state, the sum of CPU limits cannot exceed this value.

limits.memory

Specifies that across all pods in a non-terminal state, the sum of memory limits cannot exceed this value.

limits.ephemeral-storage

Specifies that across all pods in a non-terminal state, the sum of ephemeral storage limits cannot exceed this value.

scopes

Specifies a restriction on the quota that only matches pods where **spec.activeDeadlineSeconds** is set to **nil**. Build pods fall under **NotTerminating** unless the **RestartNever** policy is applied.

Example compute-resources-time-bound.yaml

```

apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-resources-time-bound
spec:
  hard:
    pods: "2"
    limits.cpu: "1"
    limits.memory: "1Gi"
    limits.ephemeral-storage: "1Gi"
  scopes:
    - Terminating
# ...

```

where:

pods

Specifies the total number of pods in a non-terminal state.

limits.cpu

Specifies that across all pods in a non-terminal state, the sum of CPU limits cannot exceed this value.

limits.memory

Specifies that across all pods in a non-terminal state, the sum of memory limits cannot exceed this value.

limits.ephemeral-storage

Specifies that across all pods in a non-terminal state, the sum of ephemeral storage limits cannot exceed this value.

scopes

Specifies a restriction on the quota that only matches pods where

spec.activeDeadlineSeconds ≥ 0 . For example, this quota would charge for build pods, but not long running pods such as a web server or database.

Example storage-consumption.yaml

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: storage-consumption
spec:
  hard:
    persistentvolumeclaims: "10"
    requests.storage: "50Gi"
    gold.storageclass.storage.k8s.io/requests.storage: "10Gi"
    silver.storageclass.storage.k8s.io/requests.storage: "20Gi"
    silver.storageclass.storage.k8s.io/persistentvolumeclaims: "5"
    bronze.storageclass.storage.k8s.io/requests.storage: "0"
    bronze.storageclass.storage.k8s.io/persistentvolumeclaims: "0"
  # ...
```

where:

persistentvolumeclaims

Specifies the total number of PVCs in a project.

requests.storage

Specifies that across all PVCs in a project, the sum of storage requested cannot exceed this value.

gold.storageclass.storage.k8s.io/requests.storage

Specifies that across all PVCs in a project, the sum of storage requested in the gold storage class cannot exceed this value.

silver.storageclass.storage.k8s.io/requests.storage

Specifies that across all PVCs in a project, the sum of storage requested in the silver storage class cannot exceed this value.

silver.storageclass.storage.k8s.io/persistentvolumeclaims

Specifies that across PVCs in a project, the total number of claims in the silver storage class cannot exceed this value.

bronze.storageclass.storage.k8s.io/requests.storage

Specifies that across all PVCs in a project, the sum of storage requested in the bronze storage class cannot exceed this value. When this is set to **0**, the bronze storage class cannot request storage.

bronze.storageclass.storage.k8s.io/persistentvolumeclaims

Specifies that across all PVCs in a project, the sum of storage requested in the bronze storage class cannot exceed this value. When this is set to **0**, the bronze storage class cannot create claims.

8.5.2. Creating a quota

To create a quota, define a **ResourceQuota** object in a file and apply the file to a project. By doing this task, you can restrict aggregate resource consumption and object counts within the project to ensure the project complies with cluster policies.

Procedure

- To apply resource constraints to a specific project, create a **ResourceQuota** object by using the OpenShift CLI (**oc**). Run the following **oc create** command with your definition file to enforce the limits on aggregate resource consumption and object counts specified for that namespace:

```
$ oc create -f <resource_quota_definition> [-n <project_name>]
```

Example command to create a ResourceQuota object

```
$ oc create -f core-object-counts.yaml -n demoproject
```

8.5.3. Creating object count quotas

To manage the consumption of standard namespaced resource types, create an object count quota. By creating an object count quota within a OpenShift Container Platform project, you can set defined limits on the number of objects, such as **BuildConfig** and **DeploymentConfig** objects.

When you use a resource quota, OpenShift Container Platform charges an object against the quota if the object exists in server storage. These quotas protect against exhaustion of storage resources.

Procedure

- To configure an object count quota for a resource, run the following command:

```
$ oc create quota <name> --hard=count/<resource>.<group>=<quota>,count/<resource>.<group>=<quota>
```

Example showing object count quota

```
$ oc create quota test --
hard=count/deployments.extensions=2,count/replicasets.extensions=4,count/pods=3,count/secrets=4
resourcequota "test" created
```

- To inspect the detailed status of the object count quota, use the following **oc describe** command:

```
$ oc describe quota test
```

Example output

```
Name: test
```

```

Namespace:      quota
Resource        Used Hard
-----
count/deployments.extensions 0 2
count/pods          0 3
count/replicasets.extensions 0 4
count/secrets       0 4

```

This example limits the listed resources to the hard limit in each project in the cluster.

8.5.4. Viewing a quota

To monitor usage statistics against defined hard limits, navigate to the **Quota** page in the web console. Alternatively, you can use the CLI to view detailed quota information for the project.

Procedure

1. Get the list of quotas defined in the project by entering the following command:

Example command with a project called demoproject

```
$ oc get quota -n demoproject
```

Example output

```

NAME          AGE
besteffort    11m
compute-resources 2m
core-object-counts 29m

```

2. Describe the target quota by entering the following command:

Example command for the core-object-counts quota

```
$ oc describe quota core-object-counts -n demoproject
```

Example output

```

Name: core-object-counts
Namespace: demoproject
Resource Used Hard
-----
configmaps 3 10
persistentvolumeclaims 0 4
replicationcontrollers 3 20
secrets 9 10
services 2 10

```

8.5.5. Configuring quota synchronization period

When a set of resources are deleted, the synchronization time frame of resources is determined by the **resource-quota-sync-period** setting in the **/etc/origin/master/master-config.yaml** file. You can

change the **resource-quota-sync-period** setting to have the set of resources regenerate in the needed amount of time (in seconds) for the resources to be once again available.



NOTE

Before quota usage is restored, you might encounter problems when attempting to reuse the resources.

Adjusting the regeneration time can be helpful for creating resources and determining resource usage when automation is used.



NOTE

The **resource-quota-sync-period** setting balances system performance. Reducing the sync period can result in a heavy load on the controller.

Procedure

1. To specify the time required for resources to regenerate and become available again, edit the **resource-quota-sync-period** setting. With this configuration, you can set the synchronization interval in seconds.

Example of the **resource-quota-sync-period** setting

```
kubernetesMasterConfig:
  apiLevels:
    - v1beta3
    - v1
  apiServerArguments: null
  controllerArguments:
    resource-quota-sync-period:
      - "10s"
  # ...
```

2. Restart the controller services to apply them to your cluster by entering the following commands:

```
$ master-restart api
```

```
$ master-restart controllers
```

8.5.6. Setting a quota to consume a resource

To restrict the amount of a resource that a user can consume, set a quota. By doing this task, you can prevent unbounded usage of resources, such as storage classes, ensuring that project consumption remains within defined limits.

If a quota does not manage a resource, a user has no restriction on the amount of that resource that can be consumed. For example, if there is no quota on storage related to the gold storage class, the amount of gold storage a project can create is unbounded.

For high-cost compute or storage resources, administrators can require an explicit quota be granted to consume a resource. For example, if a project was not explicitly given quota for storage related to the gold storage class, users of that project would not be able to create any storage of that type.

The example in the procedure shows how the quota system intercepts every operation that creates or updates a **PersistentVolumeClaim** resource. The quota system checks what resources controlled by quota would be consumed. If there is no covering quota for those resources in the project, the request is denied. In this example, if a user creates a **PersistentVolumeClaim** resource that uses storage associated with the gold storage class and there is no matching quota in the project, the request is denied.

Procedure

- Add the following stanza to the **master-config.yaml** file. This stanza requires explicit quota to consume a particular resource.

```
admissionConfig:
  pluginConfig:
    ResourceQuota:
      configuration:
        apiVersion: resourcequota.admission.k8s.io/v1alpha1
        kind: Configuration
        limitedResources:
          - resource: persistentvolumeclaims
        matchContains:
          - gold.storageclass.storage.k8s.io/requests.storage
# ...
```

where:

configuration.resource

Specifies the group or resource whose consumption is limited by default.

configuration.matchContains

Specifies the name of the resource tracked by quota associated with the group or resource to limit by default.

8.6. ADDITIONAL RESOURCES

- [Resources managed by quotas](#)
- [Working with projects](#)
- [Understanding deployments](#)

8.7. LIMIT RANGES IN A LIMITRANGE OBJECT

To define compute resource constraints at the object level, create a **LimitRange** object. By creating this object, you can specify the exact amount of resources that an individual pod, container, image, image stream, or persistent volume claim can consume.

All requests to create and modify resources are evaluated against each **LimitRange** object in the project. If the resource violates any of the enumerated constraints, the resource is rejected. If the resource does not set an explicit value, and if the constraint supports a default value, the default value is applied to the resource.

For CPU and memory limits, if you specify a maximum value but do not specify a minimum limit, the resource can consume more CPU and memory resources than the maximum value.

Core limit range object definition

```
apiVersion: "v1"
kind: "LimitRange"
metadata:
  name: "core-resource-limits"
spec:
  limits:
    - type: "Pod"
      max:
        cpu: "2"
        memory: "1Gi"
      min:
        cpu: "200m"
        memory: "6Mi"
    - type: "Container"
      max:
        cpu: "2"
        memory: "1Gi"
      min:
        cpu: "100m"
        memory: "4Mi"
      default:
        cpu: "300m"
        memory: "200Mi"
      defaultRequest:
        cpu: "200m"
        memory: "100Mi"
      maxLimitRequestRatio:
        cpu: "10"
# ...
```

where:

metadata.name

Specifies the name of the limit range object.

max.cpu

Specifies the maximum amount of CPU that a pod can request on a node across all containers.

max.memory

Specifies the maximum amount of memory that a pod can request on a node across all containers.

min.cpu

Specifies the minimum amount of CPU that a pod can request on a node across all containers. If you do not set a **min** value or you set **min** to **0**, the result is no limit and the pod can consume more than the **max** CPU value.

min.memory

Specifies the minimum amount of memory that a pod can request on a node across all containers. If you do not set a **min** value or you set **min** to **0**, the result is no limit and the pod can consume more than the **max** memory value.

max.cpu

Specifies the maximum amount of CPU that a single container in a pod can request.

max.memory

Specifies the maximum amount of memory that a single container in a pod can request.

min.cpu

Specifies the minimum amount of CPU that a single container in a pod can request. If you do not set a **min** value or you set **min** to **0**, the result is no limit and the pod can consume more than the **max** CPU value.

max.memory

Specifies the minimum amount of memory that a single container in a pod can request. If you do not set a **min** value or you set **min** to **0**, the result is no limit and the pod can consume more than the **max** memory value.

default.cpu

Specifies the default CPU limit for a container if you do not specify a limit in the pod specification.

default.memory

Specifies the default memory limit for a container if you do not specify a limit in the pod specification.

defaultRequest.cpu

Specifies the default CPU request for a container if you do not specify a request in the pod specification.

defaultRequest.memory

Specifies the default memory request for a container if you do not specify a request in the pod specification.

maxLimitRequestRatio.cpu

Specifies the maximum limit-to-request ratio for a container.

OpenShift Container Platform Limit range object definition

```
apiVersion: "v1"
kind: "LimitRange"
metadata:
  name: "openshift-resource-limits"
spec:
  limits:
    - type: openshift.io/Image
      max:
        storage: 1Gi
    - type: openshift.io/ImageStream
      max:
        openshift.io/image-tags: 20
        openshift.io/images: 30
    - type: "Pod"
      max:
        cpu: "2"
        memory: "1Gi"
        ephemeral-storage: "1Gi"
      min:
        cpu: "1"
        memory: "1Gi"
# ...
```

where:

limits.max.storage

Specifies the maximum size of an image that can be pushed to an internal registry.

limits.max.openshift.io/image-tags

Specifies the maximum number of unique references counted from tag definitions in the **imagestream.spec.tags** resource.

limits.max.openshift.io/images

Specifies the maximum number of unique image identities, or digests, recorded in the **imagestream.status.tags** resource.

type.max.cpu

Specifies the maximum amount of CPU that a pod can request on a node across all containers.

type.max.memory

Specifies the maximum amount of memory that a pod can request on a node across all containers.

type.max.ephemeral-storage

Specifies the maximum amount of ephemeral storage that a pod can request on a node across all containers.

min.cpu

Specifies the minimum amount of CPU that a pod can request on a node across all containers. See the Supported Constraints table for important information.

min.memory

Specifies the minimum amount of memory that a pod can request on a node across all containers. If you do not set a **min** value or you set **min** to **0**, the result is no limit and the pod can consume more than the **max** memory value.

You can specify both core and OpenShift Container Platform resources in one limit range object.

8.7.1. Container limits

After you create the **LimitRange** object, you can specify the exact amount of resources that a container can consume.

The following list shows resources that a container can consume:

- CPU
- Memory

The following table shows the supported constraints for a container. If specified, the constraints must hold true for each container.

Table 8.4. Supported constraints

Constraint	Behavior
------------	----------

Constraint	Behavior
Min	Min[<resource>] less than or equal to container.resources.requests[<resource>] (required) less than or equal to container/resources.limits[<resource>] (optional) If the configuration defines a min CPU, the request value must be greater than the CPU value. If you do not set a min value or you set min to 0, the result is no limit and the pod can consume more of the resource than the max value.
Max	container.resources.limits[<resource>] (required) less than or equal to Max[<resource>] If the configuration defines a max CPU, you do not need to define a CPU request value. However, you must set a limit that satisfies the maximum CPU constraint that is specified in the limit range.
MaxLimitRequestRatio	MaxLimitRequestRatio[<resource>] less than or equal to $(\text{container.resources.limits[<resource>]} / \text{container.resources.requests[<resource>}])$ If the limit range defines a maxLimitRequestRatio constraint, any new containers must have both a request and a limit value. Additionally, OpenShift Container Platform calculates a limit-to-request ratio by dividing the limit by the request. The result should be an integer greater than 1. For example, if a container has cpu: 500 in the limit value, and cpu: 100 in the request value, the limit-to-request ratio for cpu is 5. This ratio must be less than or equal to the maxLimitRequestRatio.

The following list shows default resources that a container can consume:

- **Default[<resource>]**: Defaults **container.resources.limit[<resource>]** to specified value if none.
- **Default Requests[<resource>]**: Defaults **container.resources.requests[<resource>]** to specified value if none.

8.7.2. Pod limits

After you create the **LimitRange** object, you can specify the exact amount of resources that a pod can consume.

A pod can consume the following resources:

- CPU
- Memory

The following table shows the supported constraints for a pod. Across all pods, the following behavior must hold true:

Constraint	Enforced behavior
Min	Min[<resource>] less than or equal to container.resources.requests[<resource>] (required) less than or equal to container.resources.limits[<resource>] . If you do not set amin value or you set min to 0 , the result is no limit and the pod can consume more of the resource than the max value.
Max	container.resources.limits[<resource>] (required) less than or equal to Max[<resource>] .
MaxLimitRequestRatio	MaxLimitRequestRatio[<resource>] less than or equal to $(\text{container.resources.limits[<resource>]} / \text{container.resources.requests[<resource>}])$.

8.7.3. Image limits

After you create the **LimitRange** object, you can specify the exact amount of resources that an image can consume.

An image can consume the following resources:

- Storage
- **openshift.io/Image**

The following table shows the supported constraints for an image. If specified, the constraints must hold true for each image.

Table 8.5. Image limits

Constraint	Behavior
Max	image.dockerimagemetadata.size less than or equal to Max[<resource>]



NOTE

To prevent blobs that exceed the limit from being uploaded to the registry, you must configure the registry to enforce quota. The **REGISTRY_MIDDLEWARE_REPOSITORY_OPENSHIFT_ENFORCEQUOTA** environment variable must be set to **true**. By default, the environment variable is set to **true** for new deployments.

8.7.4. Image stream limits

After you create the **LimitRange** object, you can specify the exact amount of resources that an image stream can consume.

An image stream can consume the following resources:

- **openshift.io/image-tags**
- **openshift.io/images**
- **openshift.io/ImageStream**

The **openshift.io/image-tags** limit bounds unique references derived from tag definitions in the **imagestream.spec.tags** resource. A reference can be an **ImageStreamTag**, an **ImageStreamImage**, or a **DockerImage**. You can use the **oc tag** and **oc import-image** commands to create tags. Internal and external references are not distinguished, and each unique reference in the spec is counted once. Updates that would exceed the limit are rejected, including updates from pushes to the internal registry that add or change tag definitions.

The **openshift.io/images** limit bounds unique image identities recorded in **imagestream.status.tags**. The name is equivalent to the digest for the image. It limits how many distinct images the stream can reference in status, including from registry pushes. Internal and external references are not distinguished.



IMPORTANT

Do not read **openshift.io/image-tags** and **openshift.io/images** as "tag names versus images per tag." The first limit is computed from the **ImageStream spec.tags** resource. The second is computed from the **imagestream.status.tags** resource. Both limits can cause image stream updates to fail when a push or other operation would exceed them.

The following table shows the supported constraints for an image stream. If specified, the constraints must hold true for each image stream.

Constraint	Behavior
Max[openshift.io/image-tags]	length(uniqueimagetags(imagestream.spec.tags)) less than or equal to Max[openshift.io/image-tags] uniqueimagetags returns unique references to images of given spec tags.
Max[openshift.io/images]	length(uniqueimages(imagestream.status.tags)) less than or equal to Max[openshift.io/images] uniqueimages returns unique image names found in status tags. The name is equal to the digest for the image.

8.7.5. PersistentVolumeClaim limits

After you create the **LimitRange** object, you can specify the exact amount of resources that a **PersistentVolumeClaim** resource can consume.

A **PersistentVolumeClaim** resource can consume storage resources.

The following table shows the supported constraints for a persistent volume claim. If specified, the constraints must hold true for each persistent volume claim.

Table 8.6. PersistentVolumeClaim resource limits

Constraint	Enforced behavior
Min	$\text{Min}[\text{<resource>}] \leq \text{claim.spec.resources.requests}[\text{<resource>}]$ (required)
Max	$\text{claim.spec.resources.requests}[\text{<resource>}]$ (required) $\leq \text{Max}[\text{<resource>}]$

Limit range object definition example

```
{
  "apiVersion": "v1",
  "kind": "LimitRange",
  "metadata": {
    "name": "pvcs"
  },
  "spec": {
    "limits": [{
      "type": "PersistentVolumeClaim",
      "min": {
        "storage": "2Gi"
      },
      "max": {
        "storage": "50Gi"
      }
    }]
  }
}
```

where:

metadata.name

Specifies the name of the limit range object.

limits.min.storage

Specifies the minimum amount of storage that can be requested in a persistent volume claim.

limits.max.storage

Specifies the maximum amount of storage that can be requested in a persistent volume claim.

8.8. ADDITIONAL RESOURCES

- [Managing images streams](#)
- [Restrict resource consumption with limit ranges](#)
- [About limit ranges](#)
- [Recommended control plane practices](#)
- [Understanding ephemeral storage](#)

8.9. LIMIT RANGE OPERATIONS

You can create, view, and delete limit ranges in a project.

You can view any limit ranges that are defined in a project by navigating in the web console to the **Quota** page for the project. You can also use the CLI to view limit range details.

Procedure

- To create the object, enter the following command:

```
$ oc create -f <limit_range_file> -n <project>
```

- To view the list of limit range objects that exist in a project, enter the following command:

Example command with a project called **demoproject**

```
$ oc get limits -n demoproject
```

Example output

```
NAME          AGE
resource-limits 6d
```

- To describe a limit range, enter the following command:

Example command with a limit range called **resource-limits**

```
$ oc describe limits resource-limits -n demoproject
```

Example output

```
Name:                resource-limits
Namespace:           demoproject
Type                Resource      Min    Max    Default Request Default Limit  Max
Limit/Request Ratio
-----
Pod                  cpu          200m   2      -      -      -
Pod                  memory       6Mi    1Gi    -      -      -
Container            cpu          100m   2      200m   300m   10
Container            memory       4Mi    1Gi    100Mi   200Mi   -
openshift.io/Image   storage      -      1Gi    -      -      -
openshift.io/ImageStream openshift.io/image -      12     -      -      -
openshift.io/ImageStream openshift.io/image-tags -      10     -      -      -
```

- To delete a limit range, enter the following command:

```
$ oc delete limits <limit_name>
```

8.10. ADDITIONAL RESOURCES

- [Resource quotas per projects](#)

CHAPTER 9. HOST PRACTICES FOR IBM Z AND IBM LINUXONE ENVIRONMENTS

You can apply host practices for IBM Z and IBM® LinuxONE environments to ensure your s390x architecture meets specific operational requirements.

The s390x architecture is unique in many aspects. Some host practice recommendations might not apply to other platforms.



NOTE

Unless stated otherwise, the host practices apply to both z/VM and Red Hat Enterprise Linux (RHEL) KVM installations on IBM Z® and IBM® LinuxONE.

9.1. MANAGING CPU OVERCOMMITMENT

To optimize infrastructure sizing in a highly virtualized IBM Z environment, manage CPU overcommitment. By adopting this strategy, you can allocate more resources to virtual machines than are physically available at the hypervisor level. This capability requires that you plan carefully for specific workload dependencies.

Depending on your setup, consider the following best practices regarding CPU overcommitment:

- Avoid over-allocating physical cores, Integrated Facilities for Linux (IFLs), at the Logical Partition (LPAR) level (PR/SM hypervisor). If your system has 4 physical IFLs, do not configure multiple LPARs with 4 logical IFLs each.
- Check and understand LPAR shares and weights.
- An excessive number of virtual CPUs can adversely affect performance. Do not define more virtual processors to a guest than logical processors are defined to the LPAR.
- Configure the number of virtual processors per guest for peak workload.
- Start small and monitor the workload. If required, increase the vCPU number incrementally.
- Not all workloads are suitable for high overcommitment ratios. If the workload is CPU intensive, you might experience performance problems with high overcommitment ratios. Workloads that are more I/O intensive can keep consistent performance even with high overcommitment ratios.

Additional resources

- [z/VM Common Performance Problems and Solutions](#)
- [z/VM overcommitment considerations](#)
- [LPAR CPU management](#)

9.2. DISABLE TRANSPARENT HUGE PAGES

To prevent the operating system from automatically managing memory segments, disable Transparent Huge Pages (THP).

Transparent Huge Pages (THP) tries to automate most aspects of creating, managing, and using huge

pages. Since THP automatically manages the huge pages, THP does not always handle optimally for all types of workloads. THP can lead to performance regressions, since many applications handle huge pages on their own.

9.3. BOOSTING NETWORKING PERFORMANCE WITH RFS

To boost networking performance, activate Receive Flow Steering (RFS) by using the Machine Config Operator (MCO). This configuration improves packet processing efficiency.

RFS extends Receive Packet Steering (RPS) by further reducing network latency. RFS is technically based on RPS, and improves the efficiency of packet processing by increasing the CPU cache hit rate. RFS achieves this, while considering queue length, by determining the most convenient CPU for computation so that cache hits are more likely to occur within the CPU. This means that the CPU cache is invalidated less and requires fewer cycles to rebuild the cache, which reduces packet processing run time.

Procedure

1. Copy the following MCO sample profile into a YAML file. For example, **enable-rfs.yaml**:

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: worker
  name: 50-enable-rfs
spec:
  config:
    ignition:
      version: 2.2.0
    storage:
      files:
        - contents:
            source: data:text/plain;charset=US-
ASCII,%23%20turn%20on%20Receive%20Flow%20Steering%20%28RFS%29%20for%20all
%20network%20interfaces%0ASUBSYSTEM%3D%3D%22net%22%2C%20ACTION%3D%
3D%22add%22%2C%20RUN%7Bprogram%7D%2B%3D%22/bin/bash%20-
c%20%27for%20x%20in%20/sys/%24DEVPATH/queues/rx-
%2A%3B%20do%20echo%208192%20%3E%20%24x/rps_flow_cnt%3B%20%20done%27
%22%0A
            filesystem: root
            mode: 0644
            path: /etc/udev/rules.d/70-persistent-net.rules
        - contents:
            source: data:text/plain;charset=US-
ASCII,%23%20define%20sock%20flow%20enbtried%20for%20%20Receive%20Flow%20Ste
ering%20%28RFS%29%0Anet.core.rps_sock_flow_entries%3D8192%0A
            filesystem: root
            mode: 0644
            path: /etc/sysctl.d/95-enable-rps.conf
```

2. Create the MCO profile by entering the following command:

```
$ oc create -f enable-rfs.yaml
```

3. Verify that an entry named **50-enable-rfs** is listed by entering the following command:

```
$ oc get mc
```

4. To deactivate the MCO profile, enter the following command:

```
$ oc delete mc 50-enable-rfs
```

Additional resources

- [OpenShift Container Platform on IBM Z®: Tune your network performance with RFS](#)
- [Configuring Receive Flow Steering \(RFS\)](#)
- [Scaling in the Linux Networking Stack](#)

9.4. CHOOSE YOUR NETWORKING SETUP

For IBM Z® setups, the networking setup depends on the hypervisor of your choice. Depending on the workload and the application, the best fit usually changes with the use case and the traffic pattern.

The networking stack is one of the most important components for a Kubernetes-based product like OpenShift Container Platform.

Depending on your setup, consider these best practices:

- Consider all options regarding networking devices to optimize your traffic pattern. Explore the advantages of OSA-Express, RoCE Express, HiperSockets, z/VM VSwitch, Linux Bridge (KVM), and others to decide which option leads to the greatest benefit for your setup.
- Always use the latest available NIC version. For example, OSA Express 7S 10 GbE shows great improvement compared to OSA Express 6S 10 GbE with transactional workload types, although both are 10 GbE adapters.
- Each virtual switch adds an additional layer of latency.
- The load balancer plays an important role for network communication outside the cluster. Consider using a production-grade hardware load balancer if this is critical for your application.
- OpenShift Container Platform OVN-Kubernetes network plugin introduces flows and rules, which impact the networking performance. Make sure to consider pod affinities and placements, to benefit from the locality of services where communication is critical.
- Balance the trade-off between performance and functionality.

Additional resources

- [OpenShift Container Platform on IBM Z® - Performance Experiences, Hints and Tips](#)
- [OpenShift Container Platform on IBM Z® Networking Performance](#)
- [Controlling pod placement on nodes using node affinity rules](#)

9.5. ENSURE HIGH DISK PERFORMANCE WITH HYPERPAV ON Z/VM

To improve I/O performance for Direct Access Storage Devices (DASD) disks in z/VM environments, configure HyperPAV alias devices. To increase throughput for both control plane nodes and compute nodes, add YAML configurations with full-pack minidisks to the Machine Config Operator (MCO) profiles for IBM Z clusters.

DASD and Extended Count Key Data (ECKD) devices are commonly used disk types in IBM Z® environments. In a typical OpenShift Container Platform setup in z/VM environments, DASD disks are commonly used to support the local storage for the nodes. You can set up HyperPAV alias devices to provide more throughput and overall better I/O performance for the DASD disks that support the z/VM guests.

Using HyperPAV for the local storage devices leads to a significant performance benefit. However, be aware of the trade-off between throughput and CPU costs.

Procedure

1. Copy the following MCO sample profile into a YAML file for the control plane node. For example, **05-master-kernelarg-hpav.yaml**:

```
$ cat 05-master-kernelarg-hpav.yaml
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: master
  name: 05-master-kernelarg-hpav
spec:
  config:
    ignition:
      version: 3.1.0
    kernelArguments:
      - rd.dasd=800-805
  # ...
```

2. Copy the following MCO sample profile into a YAML file for the compute node. For example, **05-worker-kernelarg-hpav.yaml**:

```
$ cat 05-worker-kernelarg-hpav.yaml
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: worker
  name: 05-worker-kernelarg-hpav
spec:
  config:
    ignition:
      version: 3.1.0
    kernelArguments:
      - rd.dasd=800-805
  # ...
```



NOTE

You must modify the **rd.dasd** arguments to fit the device IDs.

3. Create the MCO profiles by entering the following commands:

```
$ oc create -f 05-master-kernelarg-hpav.yaml
```

```
$ oc create -f 05-worker-kernelarg-hpav.yaml
```

4. To deactivate the MCO profiles, enter the following commands:

```
$ oc delete -f 05-master-kernelarg-hpav.yaml
```

```
$ oc delete -f 05-worker-kernelarg-hpav.yaml
```

Additional resources

- [Using HyperPAV for ECKD DASD](#)
- [Scaling HyperPAV alias devices on Linux guests on z/VM](#)

9.6. RHEL KVM ON IBM Z HOST RECOMMENDATIONS

To optimize Kernel-based Virtual Machine (KVM) performance on IBM Z, apply host recommendations.

Optimizing a KVM virtual server environment strongly depends on the workloads of the virtual servers and on the available resources. The same action that enhances performance in one environment can have adverse effects in another. Finding the best balance for a particular setting can be a challenge and often involves experimentation.

The following sections introduces some best practices when using OpenShift Container Platform with RHEL KVM on IBM Z® and IBM® LinuxONE environments.

9.6.1. Use I/O threads for your virtual block devices

To make virtual block devices use I/O threads, you must configure one or more I/O threads for the virtual server and each virtual block device to use one of these I/O threads.

The following example specifies `<iothreads>3</iothreads>` to configure three I/O threads, with consecutive decimal thread IDs 1, 2, and 3. The `iothread="2"` parameter specifies the driver element of the disk device to use the I/O thread with ID 2.

Sample I/O thread specification

```
...
<domain>
  <iothreads>3</iothreads>
  ...
  <devices>
    ...
    <disk type="block" device="disk">
      <driver ... iothread="2"/>
    </disk>
  ...
```

```

    </devices>
    ...
</domain>

```

where:

iothreads

Specifies the number of I/O threads.

disk

Specifies the driver element of the disk device.

Threads can increase the performance of I/O operations for disk devices, but they also use memory and CPU resources. You can configure multiple devices to use the same thread. The best mapping of threads to devices depends on the available resources and the workload.

Start with a small number of I/O threads. Often, a single I/O thread for all disk devices is sufficient. Do not configure more threads than the number of virtual CPUs, and do not configure idle threads.

You can use the **virsh iothreadadd** command to add I/O threads with specific thread IDs to a running virtual server.

9.6.2. Avoid virtual SCSI devices

Configure virtual SCSI devices only if you need to address the device through SCSI-specific interfaces. Configure disk space as virtual block devices rather than virtual SCSI devices, regardless of the backing on the host.

However, you might need SCSI-specific interfaces for:

- A logical unit number (LUN) for a SCSI-attached tape drive on the host.
- A DVD ISO file on the host file system that is mounted on a virtual DVD drive.

9.6.3. Configure guest caching for disk

To ensure that the guest manages caching instead of the host, configure your disk devices.

Ensure that the driver element of the disk device includes the **cache="none"** and **io="native"** parameters.

Example configuration

```

<disk type="block" device="disk">
  <driver name="qemu" type="raw" cache="none" io="native" iothread="1"/>
  ...
</disk>

```

9.6.4. Excluding the memory balloon device

Unless you need a dynamic memory size, do not define a memory balloon device and ensure that libvirt does not create one for you. Include the **memballoon** parameter as a child of the devices element in your domain configuration file.

Procedure

- To disable the memory balloon driver, add the following configuration setting to your domain configuration file:

```
<memballoon model="none"/>
```

9.6.5. Tuning the CPU migration algorithm of the host scheduler

You can tune the CPU migration algorithm of the host scheduler to meet the demands of your production system.



IMPORTANT

Do not change the scheduler settings unless you are an expert who understands the implications. Do not apply changes to production systems without testing them and confirming that they have the intended effect.

The **kernel.sched_migration_cost_ns** parameter specifies a time interval in nanoseconds. After the last execution of a task, the CPU cache is considered to have useful content until this interval expires. Increasing this interval results in fewer task migrations. The default value is **500000** ns.

If the CPU idle time is higher than expected when there are runnable processes, try reducing this interval. If tasks bounce between CPUs or nodes too often, try increasing it.

Procedure

- To dynamically set the interval to **60000** ns, enter the following command:

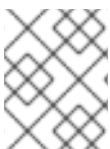
```
# sysctl kernel.sched_migration_cost_ns=60000
```

- To persistently change the value to **60000** ns, add the following entry to **/etc/sysctl.conf**:

```
kernel.sched_migration_cost_ns=60000
```

9.6.6. Disabling the cpuset cgroup controller

You can disable the cpuset cgroup controller. Disabling the controller requires a restart of the libvirtd daemon.



NOTE

This setting applies only to KVM hosts with cgroups version 1. To enable CPU hotplug on the host, disable the cgroup controller.

Procedure

1. Open **/etc/libvirt/qemu.conf** with an editor of your choice.
2. Go to the **cgroup_controllers** line.
3. Duplicate the entire line and remove the leading number sign (#) from the copy.

4. Remove the **cpuset** entry, as follows:

```
cgroup_controllers = [ "cpu", "devices", "memory", "blkio", "cpuacct" ]
```

5. For the new setting to take effect, you must restart the libvirtd daemon:

- a. Stop all virtual machines.
- b. Run the following command:

```
# systemctl restart libvirtd
```

- c. Restart the virtual machines.
This setting persists across host reboots.

9.6.7. Tuning the polling period for idle virtual CPUs

When a virtual CPU becomes idle, KVM polls for wakeup conditions for the virtual CPU before allocating the host resource. You can specify the time interval, during which polling takes place in sysfs at **/sys/module/kvm/parameters/halt_poll_ns**.

During the specified time, polling reduces the wakeup latency for the virtual CPU at the expense of resource usage. Depending on the workload, a longer or shorter time for polling can be beneficial. The time interval is specified in nanoseconds. The default is **50000** ns.

Procedure

- To optimize for low CPU consumption, enter a small value or write **0** to disable polling:

```
# echo 0 > /sys/module/kvm/parameters/halt_poll_ns
```

- To optimize for low latency, for example for transactional workloads, enter a large value:

```
# echo 80000 > /sys/module/kvm/parameters/halt_poll_ns
```

Additional resources

- [Linux on IBM Z® Performance Tuning for KVM](#)
- [Getting started with virtualization on IBM Z®](#)

CHAPTER 10. USING THE NODE TUNING OPERATOR

Learn about the Node Tuning Operator and how you can use it to manage node-level tuning by orchestrating the tuned daemon.

10.1. ABOUT THE NODE TUNING OPERATOR

The Node Tuning Operator helps you manage node-level tuning by orchestrating the Tuned daemon and achieves low latency performance by using the Performance Profile controller. The majority of high-performance applications require some level of kernel tuning. The Node Tuning Operator provides a unified management interface to users of node-level sysctls and more flexibility to add custom tuning specified by user needs.

The Operator manages the containerized Tuned daemon for OpenShift Container Platform as a Kubernetes daemon set. It ensures the custom tuning specification is passed to all containerized Tuned daemons running in the cluster in the format that the daemons understand. The daemons run on all nodes in the cluster, one per node.

Node-level settings applied by the containerized Tuned daemon are rolled back on an event that triggers a profile change or when the containerized Tuned daemon is terminated gracefully by receiving and handling a termination signal.

The Node Tuning Operator uses the Performance Profile controller to implement automatic tuning to achieve low latency performance for OpenShift Container Platform applications.

The cluster administrator configures a performance profile to define node-level settings such as the following:

- Updating the kernel to kernel-rt.
- Choosing CPUs for housekeeping.
- Choosing CPUs for running workloads.

The Node Tuning Operator is part of a standard OpenShift Container Platform installation in version 4.1 and later.



NOTE

In earlier versions of OpenShift Container Platform, the Performance Addon Operator was used to implement automatic tuning to achieve low latency performance for OpenShift applications. In OpenShift Container Platform 4.11 and later, this functionality is part of the Node Tuning Operator.

10.2. ACCESSING AN EXAMPLE NODE TUNING OPERATOR SPECIFICATION

Use this process to access an example Node Tuning Operator specification.

Procedure

- Run the following command to access an example Node Tuning Operator specification:

```
oc get tuned.tuned.openshift.io/default -o yaml -n openshift-cluster-node-tuning-operator
```

The default CR is meant for delivering standard node-level tuning for the OpenShift Container Platform platform and it can only be modified to set the Operator Management state. Any other custom changes to the default CR will be overwritten by the Operator. For custom tuning, create your own Tuned CRs. Newly created CRs will be combined with the default CR and custom tuning applied to OpenShift Container Platform nodes based on node or pod labels and profile priorities.



WARNING

While in certain situations the support for pod labels can be a convenient way of automatically delivering required tuning, this practice is discouraged and strongly advised against, especially in large-scale clusters. The default Tuned CR ships without pod label matching. If a custom profile is created with pod label matching, then the functionality will be enabled at that time. The pod label functionality will be deprecated in future versions of the Node Tuning Operator.

10.3. DEFAULT PROFILES SET ON A CLUSTER

The following are the default profiles set on a cluster.

```
apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: default
  namespace: openshift-cluster-node-tuning-operator
spec:
  profile:
    - data: |
        [main]
        summary=Optimize systems running OpenShift (provider specific parent profile)
        include=-provider-${f:exec:cat:/var/lib/ocp-tuned/provider},openshift
        name: openshift
      recommend:
        - profile: openshift-control-plane
          priority: 30
        match:
          - label: node-role.kubernetes.io/master
          - label: node-role.kubernetes.io/infra
        - profile: openshift-node
          priority: 40
```

Starting with OpenShift Container Platform 4.9, all OpenShift Tuned profiles are shipped with the Tuned package. You can use the **oc exec** command to view the contents of these profiles:

```
$ oc exec $tuned_pod -n openshift-cluster-node-tuning-operator -- find /usr/lib/tuned/openshift{-control-plane,-node} -name tuned.conf -exec grep -H ^ {} \;
```

10.4. VERIFYING THAT THE TUNED PROFILES ARE APPLIED

Verify the TuneD profiles that are applied to your cluster node.

Procedure

1. Run the following command to verify the TuneD profiles that are applied to your cluster node:

```
$ oc get profile.tuned.openshift.io -n openshift-cluster-node-tuning-operator
```

Example output

NAME	TUNED	APPLIED	DEGRADED	AGE
master-0	openshift-control-plane	True	False	6h33m
master-1	openshift-control-plane	True	False	6h33m
master-2	openshift-control-plane	True	False	6h33m
worker-a	openshift-node	True	False	6h28m
worker-b	openshift-node	True	False	6h28m

where:

- **NAME:** Name of the Profile object. There is one Profile object per node and their names match.
 - **TUNED:** Name of the desired TuneD profile to apply.
 - **APPLIED:** **True** if the TuneD daemon applied the desired profile. (**True/False/Unknown**).
 - **DEGRADED:** **True** if any errors were reported during application of the TuneD profile (**True/False/Unknown**).
 - **AGE:** Time elapsed since the creation of Profile object.
2. Run the following command to get status information about the **ClusterOperator/node-tuning** object:

```
$ oc get co/node-tuning -n openshift-cluster-node-tuning-operator
```

Example output

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED	SINCE	MESSAGE
node-tuning	4.22.1	True	False	True	60m	1/5 Profiles with bootcmdline conflict

The **ClusterOperator/node-tuning** object also contains useful information about the Operator and its node agents' health. For example, Operator misconfiguration is reported by **ClusterOperator/node-tuning** status messages.

If either the **ClusterOperator/node-tuning** or a profile object's status is **DEGRADED**, additional information is provided in the Operator or operand logs.

10.5. CUSTOM TUNING SPECIFICATION

The custom resource (CR) for the Operator has two major sections. The first section, **profile:**, is a list of TuneD profiles and their names. The second, **recommend:**, defines the profile selection logic.

Multiple custom tuning specifications can co-exist as multiple CRs in the Operator's namespace. The existence of new CRs or the deletion of old CRs is detected by the Operator. All existing custom tuning specifications are merged and appropriate objects for the containerized TuneD daemons are updated.

Management state

The Operator Management state is set by adjusting the default Tuned CR. By default, the Operator is in the Managed state and the **spec.managementState** field is not present in the default Tuned CR. Valid values for the Operator Management state are as follows:

- Managed: the Operator will update its operands as configuration resources are updated
- Unmanaged: the Operator will ignore changes to the configuration resources
- Removed: the Operator will remove its operands and resources the Operator provisioned

Profile data

The **profile:** section lists TuneD profiles and their names.

```
profile:
- name: tuned_profile_1
  data: |
    # TuneD profile specification
    [main]
    summary=Description of tuned_profile_1 profile

    [sysctl]
    net.ipv4.ip_forward=1
    # ... other sysctl's or other TuneD daemon plugins supported by the containerized TuneD

# ...

- name: tuned_profile_n
  data: |
    # TuneD profile specification
    [main]
    summary=Description of tuned_profile_n profile

    # tuned_profile_n profile settings
```

Recommended profiles

The **profile:** selection logic is defined by the **recommend:** section of the CR. The **recommend:** section is a list of items to recommend the profiles based on a selection criteria.

```
recommend:
<recommend-item-1>
# ...
<recommend-item-n>
```

The individual items of the list:

```
- machineConfigLabels:
  <mcLabels>
```

```

match:
  <match>
priority: <priority>
profile: <tuned_profile_name>
operand:
  debug: <bool>
  tunedConfig:
    reapply_sysctl: <bool>

```

where:

machineConfigLabels

Optional.

<mcLabels>

A dictionary of key/value **MachineConfig** labels. The keys must be unique.

match

If omitted, profile match is assumed unless a profile with a higher priority matches first or **machineConfigLabels** is set.

<match>

An optional list.

<priority>

Profile ordering priority. Lower numbers mean higher priority (**0** is the highest priority).

<tuned_profile_name>

A TuneD profile to apply on a match. For example **tuned_profile_1**.

operand

Optional operand configuration.

debug

Turn debugging on or off for the TuneD daemon. Options are **true** for on or **false** for off. The default is **false**.

reapply_sysctl

Turn **reapply_sysctl** functionality on or off for the TuneD daemon. Options are **true** for on and **false** for off.

The **<match>** parameter is an optional list recursively defined as follows:

```

- label: <label_name>
  value: <label_value>
  type: <label_type>
  <match>

```

where:

<label_name>

Node or pod label name.

<label_value>

Optional node or pod label value. If omitted, the presence of **<label_name>** is enough to match.

<label_type>

Optional object type (**node** or **pod**). If omitted, **node** is assumed.

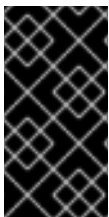
<match>

An optional <match> list.

If <match> is not omitted, all nested <match> sections must also evaluate to **true**. Otherwise, **false** is assumed and the profile with the respective <match> section will not be applied or recommended. Therefore, the nesting (child <match> sections) works as logical AND operator. Conversely, if any item of the <match> list matches, the entire <match> list evaluates to **true**. Therefore, the list acts as logical OR operator.

If **machineConfigLabels** is defined, machine config pool based matching is turned on for the given **recommend:** list item. <mcLabels> specifies the labels for a machine config. The machine config is created automatically to apply host settings, such as kernel boot parameters, for the profile <tuned_profile_name>. This involves finding all machine config pools with machine config selector matching <mcLabels> and setting the profile <tuned_profile_name> on all nodes that are assigned the found machine config pools. To target nodes that have both master and worker roles, you must use the master role.

The list items **match** and **machineConfigLabels** are connected by the logical OR operator. The **match** item is evaluated first in a short-circuit manner. Therefore, if it evaluates to **true**, the **machineConfigLabels** item is not considered.



IMPORTANT

When using machine config pool based matching, it is advised to group nodes with the same hardware configuration into the same machine config pool. Not following this practice might result in TuneD operands calculating conflicting kernel parameters for two or more nodes sharing the same machine config pool.

Example: Node or pod label based matching

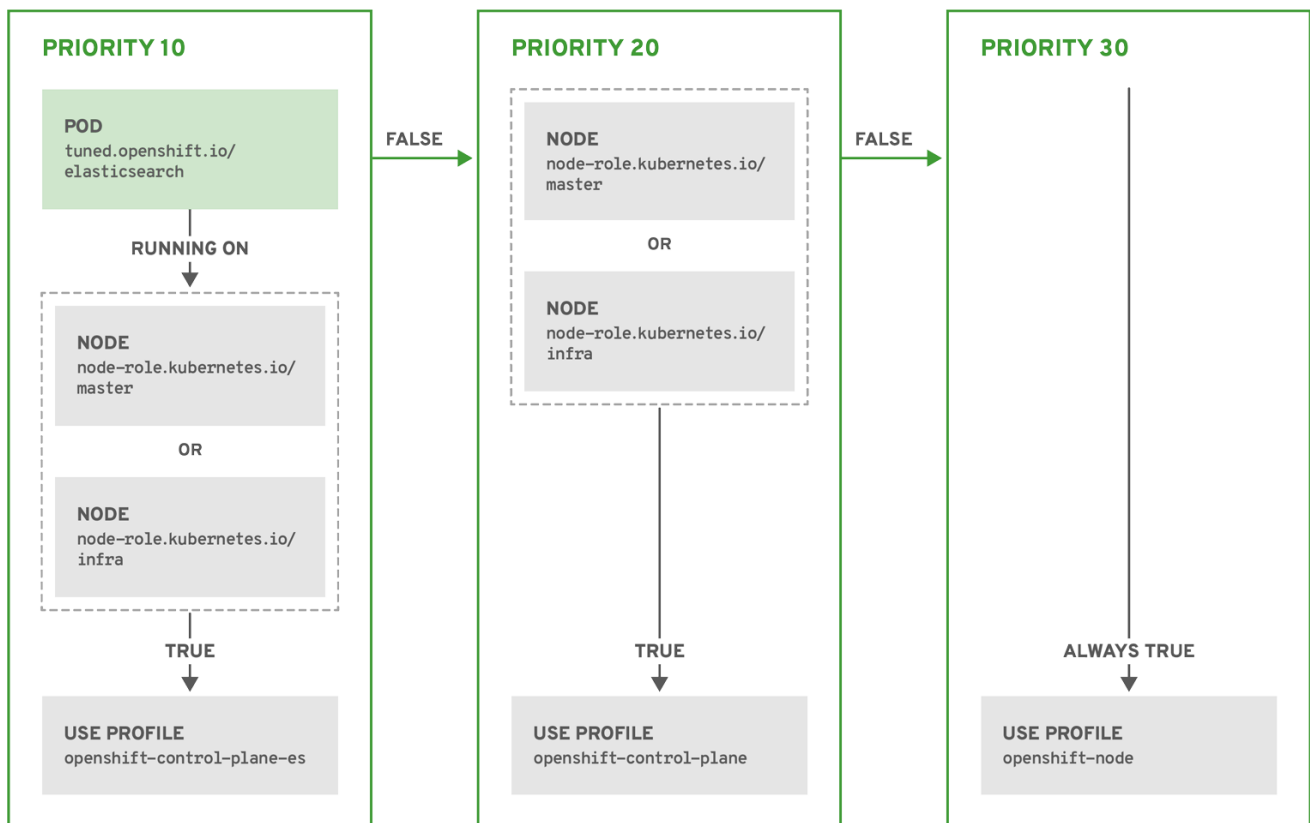
```
- match:
  - label: tuned.openshift.io/elasticsearch
    match:
      - label: node-role.kubernetes.io/master
      - label: node-role.kubernetes.io/infra
    type: pod
  priority: 10
  profile: openshift-control-plane-es
- match:
  - label: node-role.kubernetes.io/master
  - label: node-role.kubernetes.io/infra
  priority: 20
  profile: openshift-control-plane
- priority: 30
  profile: openshift-node
```

The CR above is translated for the containerized TuneD daemon into its **recommend.conf** file based on the profile priorities. The profile with the highest priority (**10**) is **openshift-control-plane-es** and, therefore, it is considered first. The containerized TuneD daemon running on a given node looks to see if there is a pod running on the same node with the **tuned.openshift.io/elasticsearch** label set. If not, the

entire **<match>** section evaluates as **false**. If there is such a pod with the label, in order for the **<match>** section to evaluate to **true**, the node label also needs to be **node-role.kubernetes.io/master** or **node-role.kubernetes.io/infra**.

If the labels for the profile with priority **10** matched, **openshift-control-plane-es** profile is applied and no other profile is considered. If the node/pod label combination did not match, the second highest priority profile (**openshift-control-plane**) is considered. This profile is applied if the containerized Tuned pod runs on a node with labels **node-role.kubernetes.io/master** or **node-role.kubernetes.io/infra**.

Finally, the profile **openshift-node** has the lowest priority of **30**. It lacks the **<match>** section and, therefore, will always match. It acts as a profile catch-all to set **openshift-node** profile, if no other profile with higher priority matches on a given node.



OPENSIFT_10_0319

Example: Machine config pool based matching

```

apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: openshift-node-custom
  namespace: openshift-cluster-node-tuning-operator
spec:
  profile:
    - data: |
        [main]
        summary=Custom OpenShift node profile with an additional kernel parameter
        include=openshift-node
        [bootloader]
        cmdline_openshift_node_custom=+skew_tick=1
  
```

```

name: openshift-node-custom

recommend:
- machineConfigLabels:
    machineconfiguration.openshift.io/role: "worker-custom"
priority: 20
profile: openshift-node-custom

```

To minimize node reboots, label the target nodes with a label the machine config pool's node selector will match, then create the Tuned CR above and finally create the custom machine config pool itself.

Cloud provider-specific Tuned profiles

With this functionality, all Cloud provider-specific nodes can conveniently be assigned a Tuned profile specifically tailored to a given Cloud provider on a OpenShift Container Platform cluster. This can be accomplished without adding additional node labels or grouping nodes into machine config pools.

This functionality takes advantage of **spec.providerID** node object values in the form of **<cloud-provider>://<cloud-provider-specific-id>** and writes the file **/var/lib/ocp-tuned/provider** with the value **<cloud-provider>** in NTO operand containers. The content of this file is then used by Tuned to load **provider-<cloud-provider>** profile if such profile exists.

The **openshift** profile that both **openshift-control-plane** and **openshift-node** profiles inherit settings from is now updated to use this functionality through the use of conditional profile loading. Neither NTO nor Tuned currently include any Cloud provider-specific profiles. However, it is possible to create a custom profile **provider-<cloud-provider>** that will be applied to all Cloud provider-specific cluster nodes.

Example GCE Cloud provider profile

```

apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: provider-gce
  namespace: openshift-cluster-node-tuning-operator
spec:
  profile:
    - data: |
        [main]
        summary=GCE Cloud provider-specific profile
        # Your tuning for GCE Cloud provider goes here.
    name: provider-gce

```



NOTE

Due to profile inheritance, any setting specified in the **provider-<cloud-provider>** profile will be overwritten by the **openshift** profile and its child profiles.

10.6. CUSTOM TUNING EXAMPLES

The following examples demonstrate custom tuning configurations for OpenShift Container Platform nodes using the Node Tuning Operator.

Using Tuned profiles from the default CR

The following CR applies custom node-level tuning for OpenShift Container Platform nodes with label **tuned.openshift.io/ingress-node-label** set to any value.

Example: custom tuning using the openshift-control-plane Tuned profile

```
apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: ingress
  namespace: openshift-cluster-node-tuning-operator
spec:
  profile:
    - data: |
        [main]
        summary=A custom OpenShift ingress profile
        include=openshift-control-plane
        [sysctl]
        net.ipv4.ip_local_port_range="1024 65535"
        net.ipv4.tcp_tw_reuse=1
        name: openshift-ingress
  recommend:
    - match:
        - label: tuned.openshift.io/ingress-node-label
    priority: 10
    profile: openshift-ingress
```



IMPORTANT

Custom profile writers are strongly encouraged to include the default Tuned daemon profiles shipped within the default Tuned CR. The example above uses the default **openshift-control-plane** profile to accomplish this.

Using built-in Tuned profiles

Given the successful rollout of the NTO-managed daemon set, the Tuned operands all manage the same version of the Tuned daemon. To list the built-in Tuned profiles supported by the daemon, query any Tuned pod in the following way:

```
$ oc exec $tuned_pod -n openshift-cluster-node-tuning-operator -- find /usr/lib/tuned/ -name
tuned.conf -printf '%h\n' | sed 's|^.*|/'
```

You can use the profile names retrieved by this in your custom tuning specification.

Example: using built-in hpc-compute Tuned profile

```
apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: openshift-node-hpc-compute
  namespace: openshift-cluster-node-tuning-operator
spec:
  profile:
    - data: |
        [main]
```

```

summary=Custom OpenShift node profile for HPC compute workloads
include=openshift-node,hpc-compute
name: openshift-node-hpc-compute

recommend:
- match:
  - label: tuned.openshift.io/openshift-node-hpc-compute
priority: 20
profile: openshift-node-hpc-compute

```

In addition to the built-in **hpc-compute** profile, the example above includes the **openshift-node** TuneD daemon profile shipped within the default TuneD CR to use OpenShift-specific tuning for compute nodes.

Overriding host-level sysctls

Various kernel parameters can be changed at runtime by using `/run/sysctl.d/`, `/etc/sysctl.d/`, and `/etc/sysctl.conf` host configuration files. OpenShift Container Platform adds several host configuration files which set kernel parameters at runtime; for example, **net.ipv[4-6].**, **fs.inotify.**, and **vm.max_map_count**. These runtime parameters provide basic functional tuning for the system prior to the kubelet and the Operator start.

The Operator does not override these settings unless the **reapply_sysctl** option is set to **false**. Setting this option to **false** results in **TuneD** not applying the settings from the host configuration files after it applies its custom profile.

Example: overriding host-level sysctls

```

apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: openshift-no-reapply-sysctl
  namespace: openshift-cluster-node-tuning-operator
spec:
  profile:
    - data: |
        [main]
        summary=Custom OpenShift profile
        include=openshift-node
        [sysctl]
        vm.max_map_count=>524288
      name: openshift-no-reapply-sysctl
  recommend:
    - match:
      - label: tuned.openshift.io/openshift-no-reapply-sysctl
    priority: 15
    profile: openshift-no-reapply-sysctl
    operand:
      tunedConfig:
        reapply_sysctl: false

```

10.7. DEFERRING APPLICATION OF TUNING CHANGES

As an administrator, use the Node Tuning Operator (NTO) to update custom resources (CRs) on a

running system and make tuning changes. For example, they can update or add a `sysctl` parameter to the `[sysctl]` section of the tuned object. When administrators apply a tuning change, the NTO prompts Tuned to reprocess all configurations, causing the tuned process to roll back all tuning and then reapply it.

Latency-sensitive applications may not tolerate the removal and reapplication of the tuned profile, as it can briefly disrupt performance. This is particularly critical for configurations that partition CPUs and manage process or interrupt affinity using the performance profile. To avoid this issue, OpenShift Container Platform introduced new methods for applying tuning changes. Before OpenShift Container Platform 4.17, the only available method, `immediate`, applied changes instantly, often triggering a tuned restart.

The following additional methods are supported:

- **always:** Every change is applied at the next node restart.
- **update:** When a tuning change modifies a tuned profile, it is applied immediately by default and takes effect as soon as possible. When a tuning change does not cause a tuned profile to change and its values are modified in place, it is treated as `always`.

Enable this feature by adding the annotation **`tuned.openshift.io/deferred`**. The following table summarizes the possible values for the annotation:

Annotation value	Description
missing	The change is applied immediately.
always	The change is applied at the next node restart.
update	The change is applied immediately if it causes a profile change, otherwise at the next node restart.

The following example demonstrates how to apply a change to the **`kernel.shmmni`** `sysctl` parameter by using the **`always`** method:

Example

```
apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: performance-patch
  namespace: openshift-cluster-node-tuning-operator
  annotations:
    tuned.openshift.io/deferred: "always"
spec:
  profile:
    - name: performance-patch
      data: |
        [main]
        summary=Configuration changes profile inherited from performance created tuned
        include=openshift-node-performance-performance
        [sysctl]
        kernel.shmmni=8192
```

recommend:

- machineConfigLabels:
machineconfiguration.openshift.io/role: worker-cnf
- priority: 19
- profile: performance-patch

where:

- The **include** directive is used to inherit the **openshift-node-performance-performance** profile. This is a best practice to ensure that the profile is not missing any required settings.
- The **kernel.shmmni** sysctl parameter is being changed to **8192**.
- The **machineConfigLabels** field is used to target the **worker-cnf** role. Configure a **MachineConfigPool** resource to ensure the profile is applied only to the correct nodes.



NOTE

You can use Topology Aware Lifecycle Manager to perform a controlled reboot across a fleet of spoke clusters to apply a deferred tuning change. For more information about coordinated reboots, see "Coordinating reboots for configuration changes".

Additional resources

- [Coordinating reboots for configuration changes](#)

10.7.1. Deferring application of tuning changes: An example

The following worked example describes how to defer the application of tuning changes by using the Node Tuning Operator.

Prerequisites

- You have **cluster-admin** role access.
- You have applied a performance profile to your cluster.
- A **MachineConfigPool** resource, for example, **worker-cnf** is configured to ensure that the profile is only applied to the designated nodes.

Procedure

1. Check what profiles are currently applied to your cluster by running the following command:

```
$ oc -n openshift-cluster-node-tuning-operator get tuned
```

Example output

```
NAME                                AGE
default                             63m
openshift-node-performance-performance 21m
```

2. Check the machine config pools in your cluster by running the following command:

■

```
$ oc get mcp
```

Example output

NAME	CONFIG	UPDATED	UPDATING	DEGRADED
MACHINECOUNT	READYMACHINECOUNT	UPDATEDMACHINECOUNT		
DEGRADEDMACHINECOUNT	AGE			
master	rendered-master-79a26af9f78ced61fa8ccd309d3c859c	True	False	
False	3	3	3	0
				157m
worker	rendered-worker-d9352e91a1b14de7ef453fa54480ce0e	True	False	
False	2	2	2	0
				157m
worker-cnf	rendered-worker-cnf-f398fc4fcb2b20104a51e744b8247272	True	False	
False	1	1	1	0
				92m

3. Describe the current applied performance profile by running the following command:

```
$ oc describe performanceprofile performance | grep Tuned
```

Example output

```
Tuned:          openshift-cluster-node-tuning-operator/openshift-node-performance-
performance
```

4. Verify the existing value of the **kernel.shmmni** sysctl parameter:

- a. Run the following command to display the node names:

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ip-10-0-26-151.ec2.internal	Ready	worker,worker-cnf	116m	v1.30.6
ip-10-0-46-60.ec2.internal	Ready	worker	115m	v1.30.6
ip-10-0-52-141.ec2.internal	Ready	control-plane,master	123m	v1.30.6
ip-10-0-6-97.ec2.internal	Ready	control-plane,master	121m	v1.30.6
ip-10-0-86-145.ec2.internal	Ready	worker	117m	v1.30.6
ip-10-0-92-228.ec2.internal	Ready	control-plane,master	123m	v1.30.6

- b. Run the following command to display the current value of the **kernel.shmmni** sysctl parameters on the node **ip-10-0-32-74.ec2.internal**:

```
$ oc debug node/ip-10-0-26-151.ec2.internal -q -- chroot host sysctl kernel.shmmni
```

Example output

```
kernel.shmmni = 4096
```

5. Create a profile patch, for example, **perf-patch.yaml** that changes the **kernel.shmmni** sysctl parameter to **8192**. Defer the application of the change to a new manual restart by using the **always** method by applying the following configuration:

■

```

apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: performance-patch
  namespace: openshift-cluster-node-tuning-operator
  annotations:
    tuned.openshift.io/deferred: "always"
spec:
  profile:
    - name: performance-patch
      data: |
        [main]
        summary=Configuration changes profile inherited from performance created tuned
        include=openshift-node-performance-performance
        [sysctl]
        kernel.shmmni=8192
  recommend:
    - machineConfigLabels:
        machineconfiguration.openshift.io/role: worker-cnf
      priority: 19
      profile: performance-patch

```

where:

- The **include** directive is used to inherit the **openshift-node-performance-performance** profile. This is a best practice to ensure that the profile is not missing any required settings.
- The **kernel.shmmni** sysctl parameter is being changed to **8192**.
- The **machineConfigLabels** field is used to target the **worker-cnf** role.

6. Apply the profile patch by running the following command:

```
$ oc apply -f perf-patch.yaml
```

7. Run the following command to verify that the profile patch is waiting for the next node restart:

```
$ oc -n openshift-cluster-node-tuning-operator get profile
```

Example output

NAME	TUNED	APPLIED	DEGRADED	MESSAGE
AGE				
ip-10-0-26-151.ec2.internal	performance-patch	False	True	The TuneD daemon profile is waiting for the next node restart: performance-patch 126m
ip-10-0-46-60.ec2.internal	openshift-node	True	False	TuneD profile applied. 125m
ip-10-0-52-141.ec2.internal	openshift-control-plane	True	False	TuneD profile applied. 130m
ip-10-0-6-97.ec2.internal	openshift-control-plane	True	False	TuneD profile applied. 130m
ip-10-0-86-145.ec2.internal	openshift-node	True	False	TuneD profile applied. 126m
ip-10-0-92-228.ec2.internal	openshift-control-plane	True	False	TuneD profile applied. 130m

8. Confirm the value of the **kernel.shmmni** sysctl parameter remain unchanged before a restart:
 - a. Run the following command to confirm that the application of the **performance-patch** change to the **kernel.shmmni** sysctl parameter on the node **ip-10-0-26-151.ec2.internal** is not applied:

```
$ oc debug node/ip-10-0-26-151.ec2.internal -q -- chroot host sysctl kernel.shmmni
```

Example output

```
kernel.shmmni = 4096
```

9. Restart the node **ip-10-0-26-151.ec2.internal** to apply the required changes by running the following command:

```
$ oc debug node/ip-10-0-26-151.ec2.internal -q -- chroot host reboot&
```

10. In another terminal window, run the following command to verify that the node has restarted:

```
$ watch oc get nodes
```

Wait for the node **ip-10-0-26-151.ec2.internal** to transition back to the **Ready** state.

11. Run the following command to verify that the profile patch is waiting for the next node restart:

```
$ oc -n openshift-cluster-node-tuning-operator get profile
```

Example output

NAME	TUNED	APPLIED	DEGRADED	MESSAGE
AGE				
ip-10-0-20-251.ec2.internal	performance-patch	True	False	TuneD profile applied.
		3h3m		
ip-10-0-30-148.ec2.internal	openshift-control-plane	True	False	TuneD profile applied.
		3h8m		
ip-10-0-32-74.ec2.internal	openshift-node	True	True	TuneD profile applied.
		179m		
ip-10-0-33-49.ec2.internal	openshift-control-plane	True	False	TuneD profile applied.
		3h8m		
ip-10-0-84-72.ec2.internal	openshift-control-plane	True	False	TuneD profile applied.
		3h8m		
ip-10-0-93-89.ec2.internal	openshift-node	True	False	TuneD profile applied.
		179m		

12. Check that the value of the **kernel.shmmni** sysctl parameter have changed after the restart:
 - a. Run the following command to verify that the **kernel.shmmni** sysctl parameter change has been applied on the node **ip-10-0-32-74.ec2.internal**:

```
$ oc debug node/ip-10-0-32-74.ec2.internal -q -- chroot host sysctl kernel.shmmni
```

Example output

```
■
```

 `kernel.shmmni = 8192`**NOTE**

An additional restart results in the restoration of the original value of the **kernel.shmmni** sysctl parameter.

10.8. SUPPORTED TUNED DAEMON PLUGINS

Excluding the **[main]** section, the following Tuned plugins are supported when using custom profiles defined in the **profile:** section of the Tuned CR:

- audio
- cpu
- disk
- eeepc_she
- modules
- mounts
- net
- scheduler
- scsi_host
- selinux
- sysctl
- sysfs
- usb
- video
- vm
- bootloader

There is some dynamic tuning functionality provided by some of these plugins that is not supported. The following Tuned plugins are currently not supported:

- script
- systemd

**NOTE**

The Tuned bootloader plugin only supports Red Hat Enterprise Linux CoreOS (RHCOS) worker nodes.

Additional resources

- [Available TuneD Plugins](#)
- [Getting Started with TuneD](#)

10.9. CONFIGURING NODE TUNING IN A HOSTED CLUSTER

To set node-level tuning on the nodes in your hosted cluster, you can use the Node Tuning Operator. In hosted control planes, you can configure node tuning by creating config maps that contain **Tuned** objects and referencing those config maps in your node pools.

Procedure

1. Create a config map that contains a valid tuned manifest, and reference the manifest in a node pool. In the following example, a **Tuned** manifest defines a profile that sets **vm.dirty_ratio** to 55 on nodes that contain the **tuned-1-node-label** node label with any value. Save the following **ConfigMap** manifest in a file named **tuned-1.yaml**:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: tuned-1
  namespace: clusters
data:
  tuning: |
    apiVersion: tuned.openshift.io/v1
    kind: Tuned
    metadata:
      name: tuned-1
      namespace: openshift-cluster-node-tuning-operator
    spec:
      profile:
        - data: |
            [main]
            summary=Custom OpenShift profile
            include=openshift-node
            [sysctl]
            vm.dirty_ratio="55"
          name: tuned-1-profile
      recommend:
        - priority: 20
          profile: tuned-1-profile
```

NOTE

If you do not add any labels to an entry in the **spec.recommend** section of the Tuned spec, node-pool-based matching is assumed, so the highest priority profile in the **spec.recommend** section is applied to nodes in the pool. Although you can achieve more fine-grained node-label-based matching by setting a label value in the Tuned **.spec.recommend.match** section, node labels will not persist during an upgrade unless you set the **.spec.management.upgradeType** value of the node pool to **InPlace**.

2. Create the **ConfigMap** object in the management cluster:

```
$ oc --kubeconfig="$MGMT_KUBECONFIG" create -f tuned-1.yaml
```

3. Reference the **ConfigMap** object in the **spec.tuningConfig** field of the node pool, either by editing a node pool or creating one. In this example, assume that you have only one **NodePool**, named **nodepool-1**, which contains 2 nodes.

```
apiVersion: hypershift.openshift.io/v1alpha1
kind: NodePool
metadata:
  ...
  name: nodepool-1
  namespace: clusters
  ...
spec:
  ...
  tuningConfig:
    - name: tuned-1
  status:
    ...
```



NOTE

You can reference the same config map in multiple node pools. In hosted control planes, the Node Tuning Operator appends a hash of the node pool name and namespace to the name of the Tuned CRs to distinguish them. Outside of this case, do not create multiple TuneD profiles of the same name in different Tuned CRs for the same hosted cluster.

Verification

Now that you have created the **ConfigMap** object that contains a **Tuned** manifest and referenced it in a **NodePool**, the Node Tuning Operator syncs the **Tuned** objects into the hosted cluster. You can verify which **Tuned** objects are defined and which TuneD profiles are applied to each node.

1. List the **Tuned** objects in the hosted cluster:

```
$ oc --kubeconfig="$HC_KUBECONFIG" get tuned.tuned.openshift.io \
  -n openshift-cluster-node-tuning-operator
```

Example output

```
NAME      AGE
default   7m36s
rendered  7m36s
tuned-1    65s
```

2. List the **Profile** objects in the hosted cluster:

```
$ oc --kubeconfig="$HC_KUBECONFIG" get profile.tuned.openshift.io \
  -n openshift-cluster-node-tuning-operator
```

Example output

NAME	TUNED	APPLIED	DEGRADED	AGE
nodepool-1-worker-1	tuned-1-profile	True	False	7m43s
nodepool-1-worker-2	tuned-1-profile	True	False	7m14s

**NOTE**

If no custom profiles are created, the **openshift-node** profile is applied by default.

- To confirm that the tuning was applied correctly, start a debug shell on a node and check the `sysctl` values:

```
$ oc --kubeconfig="$HC_KUBECONFIG" \
  debug node/nodepool-1-worker-1 -- chroot /host sysctl vm.dirty_ratio
```

Example output

```
vm.dirty_ratio = 55
```

10.10. ADVANCED NODE TUNING FOR HOSTED CLUSTERS BY SETTING KERNEL BOOT PARAMETERS

For more advanced tuning in hosted control planes, which requires setting kernel boot parameters, you can also use the Node Tuning Operator. The following example shows how you can create a node pool with huge pages reserved.

Procedure

- Create a **ConfigMap** object that contains a **Tuned** object manifest for creating 10 huge pages that are 2 MB in size. Save this **ConfigMap** manifest in a file named **tuned-hugepages.yaml**:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: tuned-hugepages
  namespace: clusters
data:
  tuning: |
    apiVersion: tuned.openshift.io/v1
    kind: Tuned
    metadata:
      name: hugepages
      namespace: openshift-cluster-node-tuning-operator
    spec:
      profile:
        - data: |
            [main]
            summary=Boot time configuration for hugepages
            include=openshift-node
            [bootloader]
```

```
cmdline_openshift_node_hugepages=hugepagesz=2M hugepages=50
name: openshift-node-hugepages
recommend:
- priority: 20
profile: openshift-node-hugepages
```



NOTE

The **.spec.recommend.match** field is intentionally left blank. In this case, this **Tuned** object is applied to all nodes in the node pool where this **ConfigMap** object is referenced. Group nodes with the same hardware configuration into the same node pool. Otherwise, TuneD operands can calculate conflicting kernel parameters for two or more nodes that share the same node pool.

2. Create the **ConfigMap** object in the management cluster:

```
$ oc --kubeconfig="<management_cluster_kubeconfig>" create -f tuned-hugepages.yaml
```

Replace **<management_cluster_kubeconfig>** with the name of your management cluster **kubeconfig** file.

3. Create a **NodePool** manifest YAML file, customize the upgrade type of the **NodePool**, and reference the **ConfigMap** object that you created in the **spec.tuningConfig** section. Create the **NodePool** manifest and save it in a file named **hugepages-nodepool.yaml** by using the **hcp** CLI:

```
$ hcp create nodepool aws \
  --cluster-name <hosted_cluster_name> \
  --name <nodepool_name> \
  --node-count <nodepool_replicas> \
  --instance-type <instance_type> \
  --render > hugepages-nodepool.yaml
```

where:

- **<hosted_cluster_name>**: The name of your hosted cluster.
- **<nodepool_name>**: The name of your node pool.
- **<nodepool_replicas>**: The number of your node pool replicas, for example, **2**.
- **<instance_type>**: The instance type, for example, **m5.2xlarge**.



NOTE

The **--render** flag in the **hcp create** command does not render the secrets. To render the secrets, you must use both the **--render** and the **--render-sensitive** flags in the **hcp create** command.

4. In the **hugepages-nodepool.yaml** file, set **.spec.management.upgradeType** to **InPlace**, and set **.spec.tuningConfig** to reference the **tuned-hugepages ConfigMap** object that you created.

```

apiVersion: hypershift.openshift.io/v1alpha1
kind: NodePool
metadata:
  name: hugepages-nodepool
  namespace: clusters
  ...
spec:
  management:
    ...
    upgradeType: InPlace
    ...
  tuningConfig:
    - name: tuned-hugepages

```



NOTE

To avoid the unnecessary re-creation of nodes when you apply the new **MachineConfig** objects, set **.spec.management.upgradeType** to **InPlace**. If you use the **Replace** upgrade type, nodes are fully deleted and new nodes can replace them when you apply the new kernel boot parameters that the TunedD operand calculated.

5. Create the **NodePool** in the management cluster:

```
$ oc --kubeconfig="<management_cluster_kubeconfig>" create -f hugepages-nodepool.yaml
```

Verification

After the nodes are available, the containerized TuneD daemon calculates the required kernel boot parameters based on the applied TuneD profile. After the nodes are ready and reboot once to apply the generated **MachineConfig** object, you can verify that the TuneD profile is applied and that the kernel boot parameters are set.

1. List the **Tuned** objects in the hosted cluster:

```
$ oc --kubeconfig="<hosted_cluster_kubeconfig>" get tuned.tuned.openshift.io \
-n openshift-cluster-node-tuning-operator
```

Example output

NAME	AGE
default	123m
hugepages-8dfb1fed	1m23s
rendered	123m

2. List the **Profile** objects in the hosted cluster:

```
$ oc --kubeconfig="<hosted_cluster_kubeconfig>" get profile.tuned.openshift.io \
-n openshift-cluster-node-tuning-operator
```

Example output

NAME	TUNED	APPLIED	DEGRADED	AGE
------	-------	---------	----------	-----

nodepool-1-worker-1	openshift-node	True	False	132m	
nodepool-1-worker-2	openshift-node	True	False	131m	
hugepages-nodepool-worker-1	openshift-node-hugepages	True	False	4m8s	
hugepages-nodepool-worker-2	openshift-node-hugepages	True	False	3m57s	

Both of the worker nodes in the new **NodePool** have the **openshift-node-hugepages** profile applied.

3. To confirm that the tuning was applied correctly, start a debug shell on a node and check **/proc/cmdline**.

```
$ oc --kubeconfig="<hosted_cluster_kubeconfig>" \
  debug node/nodepool-1-worker-1 -- chroot /host cat /proc/cmdline
```

Example output

```
BOOT_IMAGE=(hd0,gpt3)/ostree/rhcos-... hugepagesz=2M hugepages=50
```

CHAPTER 11. USING CPU MANAGER AND TOPOLOGY MANAGER

CPU Manager manages groups of CPUs and constrains workloads to specific CPUs.

CPU Manager is useful for workloads that have some of these attributes:

- Require as much CPU time as possible.
- Are sensitive to processor cache misses.
- Are low-latency network applications.
- Coordinate with other processes and benefit from sharing a single processor cache.

Topology Manager collects hints from the CPU Manager, Device Manager, and other Hint Providers to align pod resources, such as CPU, SR-IOV VFs, and other device resources, for all Quality of Service (QoS) classes on the same non-uniform memory access (NUMA) node.

Topology Manager uses topology information from the collected hints to decide if a pod can be accepted or rejected on a node, based on the configured Topology Manager policy and pod resources requested.

Topology Manager is useful for workloads that use hardware accelerators to support latency-critical execution and high throughput parallel computation.

To use Topology Manager you must configure CPU Manager with the **static** policy.

11.1. SETTING UP CPU MANAGER

To configure CPU manager, create a **KubeletConfig** custom resource (CR) and apply it to the required set of nodes.

Procedure

1. Label a node by running the following command:

```
# oc label node perf-node.example.com cpumanager=true
```

2. To enable CPU Manager for all compute nodes, edit the CR by running the following command:

```
# oc edit machineconfigpool worker
```

3. Add the **custom-kubelet: cpumanager-enabled** label to **metadata.labels** section.

```
metadata:
  creationTimestamp: 2020-xx-xxx
  generation: 3
  labels:
    custom-kubelet: cpumanager-enabled
```

4. Create a **KubeletConfig**, **cpumanager-kubeletconfig.yaml**, custom resource (CR). Refer to the label created in the previous step to have the correct nodes updated with the new kubelet config. See the **machineConfigPoolSelector** section:

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: cpumanager-enabled
spec:
  machineConfigPoolSelector:
    matchLabels:
      custom-kubelet: cpumanager-enabled
  kubeletConfig:
    cpuManagerPolicy: static
    cpuManagerReconcilePeriod: 5s
```

- **cpuManagerPolicy** specifies a policy:
 - **none**. This policy explicitly enables the existing default CPU affinity scheme, providing no affinity beyond what the scheduler does automatically. This is the default policy.
 - **static**. This policy allows containers in guaranteed pods with integer CPU requests. It also limits access to exclusive CPUs on the node. If **static**, you must use a lowercase **s**.
- **cpuManagerReconcilePeriod** is optional. Specify the CPU Manager reconcile frequency. The default is **5s**.

5. Create the dynamic kubelet config by running the following command:

```
# oc create -f cpumanager-kubeletconfig.yaml
```

This adds the CPU Manager feature to the kubelet config and, if needed, the Machine Config Operator (MCO) reboots the node. To enable CPU Manager, a reboot is not needed.

6. Check for the merged kubelet config by running the following command:

```
# oc get machineconfig 99-worker-XXXXXX-XXXXX-XXXX-XXXXX-kubelet -o json | grep
ownerReference -A7
```

Example output

```
"ownerReferences": [
  {
    "apiVersion": "machineconfiguration.openshift.io/v1",
    "kind": "KubeletConfig",
    "name": "cpumanager-enabled",
    "uid": "7ed5616d-6b72-11e9-aae1-021e1ce18878"
  }
]
```

7. Check the compute node for the updated **kubelet.conf** file by running the following command:

```
# oc debug node/perf-node.example.com
sh-4.2# cat /host/etc/kubernetes/kubelet.conf | grep cpuManager
```


The following is example output:

```
cpuManagerPolicy: static
cpuManagerReconcilePeriod: 5s
```

- **cpuManagerPolicy** is defined when you create the **KubeletConfig** CR.
- **cpuManagerReconcilePeriod** is defined when you create the **KubeletConfig** CR.

8. Create a project by running the following command:

```
$ oc new-project <project_name>
```

9. Create a pod that requests a core or multiple cores. Both limits and requests must have their CPU value set to a whole integer. That is the number of cores that will be dedicated to this pod:

```
# cat cpumanager-pod.yaml
```

Example output

```
apiVersion: v1
kind: Pod
metadata:
  generateName: cpumanager-
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: cpumanager
    image: gcr.io/google_containers/pause:3.2
    resources:
      requests:
        cpu: 1
        memory: "1G"
      limits:
        cpu: 1
        memory: "1G"
    securityContext:
      allowPrivilegeEscalation: false
      capabilities:
        drop: [ALL]
  nodeSelector:
    cpumanager: "true"
```

10. Create the pod:

```
# oc create -f cpumanager-pod.yaml
```

Verification

1. Verify that the pod is scheduled to the node that you labeled by running the following command:

```
# oc describe pod cpumanager
```

Example output

```
Name:          cpumanager-6cqz7
Namespace:     default
Priority:       0
PriorityClassName: <none>
Node: perf-node.example.com/xxx.xx.xx.xxx
...
Limits:
  cpu: 1
  memory: 1G
Requests:
  cpu: 1
  memory: 1G
...
QoS Class:     Guaranteed
Node-Selectors: cpumanager=true
```

2. Verify that a CPU has been exclusively assigned to the pod by running the following command:

```
# oc describe node --selector='cpumanager=true' | grep -i cpumanager- -B2
```

Example output

NAMESPACE	NAME	CPU Requests	CPU Limits	Memory Requests	Memory Limits	Age
cpuman	cpumanager-mlrrz	1 (28%)	1 (28%)	1G (13%)	1G (13%)	27m

3. Verify that the **cgroups** are set up correctly. Get the process ID (PID) of the **pause** process by running the following commands:

```
# oc debug node/perf-node.example.com
```

```
sh-4.2# systemctl status | grep -B5 pause
```



NOTE

If the output returns multiple pause process entries, you must identify the correct pause process.

Example output

```
# |—init.scope
|   |—1 /usr/lib/systemd/systemd --switched-root --system --deserialize 17
|   |—kubepods.slice
```

```

├──kubepods-pod69c01f8e_6b74_11e9_ac0f_0a2b62178a22.slice
│   ├──crio-b5437308f1a574c542bdf08563b865c0345c8f8c0b0a655612c.scope
│   └──32706 /pause

```

4. Verify that pods of quality of service (QoS) tier **Guaranteed** are placed within the **kubepods.slice** subdirectory by running the following commands:

```
# cd /sys/fs/cgroup/kubepods.slice/kubepods-
pod69c01f8e_6b74_11e9_ac0f_0a2b62178a22.slice/crio-
b5437308f1ad1a7db0574c542bdf08563b865c0345c86e9585f8c0b0a655612c.scope
```

```
# for i in `ls cpuset.cpus cgroup.procs` ; do echo -n "$i "; cat $i ; done
```



NOTE

Pods of other QoS tiers end up in child **cgroups** of the parent **kubepods**.

Example output

```
cpuset.cpus 1
tasks 32706
```

5. Check the allowed CPU list for the task by running the following command:

```
# grep ^Cpus_allowed_list /proc/32706/status
```

Example output

```
Cpus_allowed_list: 1
```

6. Verify that another pod on the system cannot run on the core allocated for the **Guaranteed** pod. For example, to verify the pod in the **besteffort** QoS tier, run the following commands:

```
# cat /sys/fs/cgroup/kubepods.slice/kubepods-besteffort.slice/kubepods-besteffort-
podc494a073_6b77_11e9_98c0_06bba5c387ea.slice/crio-
c56982f57b75a2420947f0afc6cafe7534c5734efc34157525fa9abbf99e3849.scope/cpuset.cpus
```

```
# oc describe node perf-node.example.com
```

Example output

```
...
Capacity:
attachable-volumes-aws-ebs: 39
cpu: 2
ephemeral-storage: 124768236Ki
hugepages-1Gi: 0
hugepages-2Mi: 0
memory: 8162900Ki
pods: 250
Allocatable:
```

```

attachable-volumes-aws-ebs: 39
cpu:                          1500m
ephemeral-storage:           124768236Ki
hugepages-1Gi:               0
hugepages-2Mi:               0
memory:                      7548500Ki
pods:                        250
-----
-
  default          cpumanager-6cqz7          1 (66%)    1 (66%)    1G (12%)
1G (12%)    29m

```

Allocated resources:
(Total limits may be over 100 percent, i.e., overcommitted.)

Resource	Requests	Limits
cpu	1440m (96%)	1 (66%)

This VM has two CPU cores. The **system-reserved** setting reserves 500 millicores, meaning that half of one core is subtracted from the total capacity of the node to arrive at the **Node Allocatable** amount. You can see that **Allocatable CPU** is 1500 millicores. This means you can run one of the CPU Manager pods since each will take one whole core. A whole core is equivalent to 1000 millicores. If you try to schedule a second pod, the system will accept the pod, but it will never be scheduled:

NAME	READY	STATUS	RESTARTS	AGE
cpumanager-6cqz7	1/1	Running	0	33m
cpumanager-7qc2t	0/1	Pending	0	11s

11.2. TOPOLOGY MANAGER POLICIES

Topology Manager aligns **Pod** resources of all Quality of Service (QoS) classes by collecting topology hints from Hint Providers, such as CPU Manager and Device Manager, and using the collected hints to align the **Pod** resources.

Topology Manager supports four allocation policies, which you assign in the **KubeletConfig** custom resource (CR) named **cpumanager-enabled**:

none policy

This is the default policy and does not perform any topology alignment.

best-effort policy

For each container in a pod with the **best-effort** topology management policy, kubelet tries to align all the required resources on a NUMA node according to the preferred NUMA node affinity for that container. Even if the allocation is not possible due to insufficient resources, the Topology Manager still admits the pod but the allocation is shared with other NUMA nodes.

restricted policy

For each container in a pod with the **restricted** topology management policy, kubelet determines the theoretical minimum number of NUMA nodes that can fulfill the request. If the actual allocation requires more than the that number of NUMA nodes, the Topology Manager rejects the admission, placing the pod in a **Terminated** state. If the number of NUMA nodes can fulfill the request, the Topology Manager admits the pod and the pod starts running.

single-numa-node policy

For each container in a pod with the **single-numa-node** topology management policy, kubelet admits the pod if all the resources required by the pod can be allocated on the same NUMA node. If a single NUMA node affinity is not possible, the Topology Manager rejects the pod from the node. This results in a pod in a **Terminated** state with a pod admission failure.

11.3. SETTING UP TOPOLOGY MANAGER

To use Topology Manager, you must configure an allocation policy in the **KubeletConfig** custom resource (CR) named **cpumanager-enabled**. This file might exist if you have set up CPU Manager. If the file does not exist, you can create the file.

Prerequisites

- Configure the CPU Manager policy to be **static**.

Procedure

1. To activate Topology Manager, configure the Topology Manager allocation policy in the custom resource.

```
$ oc edit KubeletConfig cpumanager-enabled
```

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: cpumanager-enabled
spec:
  machineConfigPoolSelector:
    matchLabels:
      custom-kubelet: cpumanager-enabled
  kubeletConfig:
    cpuManagerPolicy: static
    cpuManagerReconcilePeriod: 5s
    topologyManagerPolicy: single-numa-node
```

- **cpuManagerPolicy** must be **static** with a lowercase **s**.
- **topologyManagerPolicy** specifies your selected Topology Manager allocation policy. In this example, the policy is **single-numa-node**. Acceptable values are: **default**, **best-effort**, **restricted**, **single-numa-node**.

11.4. POD INTERACTIONS WITH TOPOLOGY MANAGER POLICIES

The example **Pod** specs illustrate pod interactions with Topology Manager.

The following pod runs in the **BestEffort** QoS class because no resource requests or limits are specified.

```
spec:
  containers:
  - name: nginx
    image: nginx
```

The next pod runs in the **Burstable** QoS class because requests are less than limits.

```
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      limits:
        memory: "200Mi"
      requests:
        memory: "100Mi"
```

If the selected policy is anything other than **none**, Topology Manager would process all the pods and it enforces resource alignment only for the **Guaranteed QoS Pod** specification. When the Topology Manager policy is set to **none**, the relevant containers are pinned to any available CPU without considering NUMA affinity. This is the default behavior and it does not optimize for performance-sensitive workloads. Other values enable the use of topology awareness information from device plugins core resources, such as CPU and memory. The Topology Manager attempts to align the CPU, memory, and device allocations according to the topology of the node when the policy is set to other values than **none**. For more information about the available values, see *Topology Manager policies*.

The following example pod runs in the **Guaranteed** QoS class because requests are equal to limits.

```
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      limits:
        memory: "200Mi"
        cpu: "2"
        example.com/device: "1"
      requests:
        memory: "200Mi"
        cpu: "2"
        example.com/device: "1"
```

Topology Manager would consider this pod. The Topology Manager would consult the Hint Providers, which are the CPU Manager, the Device Manager, and the Memory Manager, to get topology hints for the pod.

Topology Manager will use this information to store the best topology for this container. In the case of this pod, CPU Manager and Device Manager will use this stored information at the resource allocation stage.

Additional resources

- [Topology Manager policies](#)

CHAPTER 12. SCHEDULING NUMA-AWARE WORKLOADS

To deploy high performance workloads with optimal efficiency, use NUMA-aware scheduling. This feature aligns pods with the underlying hardware topology in your OpenShift Container Platform cluster, minimizing latency and maximizing resource utilization.

By using the NUMA Resources Operator, you can schedule high-performance workloads in the same NUMA zone. The Operator deploys a node resources exporting agent that reports on available cluster node NUMA resources, and a secondary scheduler that manages the workloads.

12.1. ABOUT NUMA

To reduce latency in multiprocessor systems, Non-Uniform Memory Access (NUMA) architecture allows CPUs to access local memory faster than remote memory. This design optimizes performance by prioritizing memory resources that are physically closer to the processor.

A CPU with multiple memory controllers can use any available memory across CPU complexes, regardless of where the memory is located. However, this increased flexibility comes at the expense of performance.

NUMA resource topology refers to the physical locations of CPUs, memory, and PCI devices relative to each other in a *NUMA zone*. In a NUMA architecture, a NUMA zone is a group of CPUs that has its own processors and memory. Colocated resources are said to be in the same NUMA zone, and CPUs in a zone have faster access to the same local memory than CPUs outside of that zone.

A CPU processing a workload using memory that is outside its NUMA zone is slower than a workload processed in a single NUMA zone. For I/O-constrained workloads, the network interface on a distant NUMA zone slows down how quickly information can reach the application.

Applications can achieve better performance by containing data and processing within the same NUMA zone. For high-performance workloads and applications, such as telecommunications workloads, the cluster must process pod workloads in a single NUMA zone so that the workload can operate to specification.

12.2. ABOUT NUMA-AWARE SCHEDULING

To process latency-sensitive or high-performance workloads efficiently, use NUMA-aware scheduling. This feature aligns cluster compute resources, such as CPUs, memory, and devices, in the same NUMA zone, optimizing resource efficiency and improving pod density per compute node.

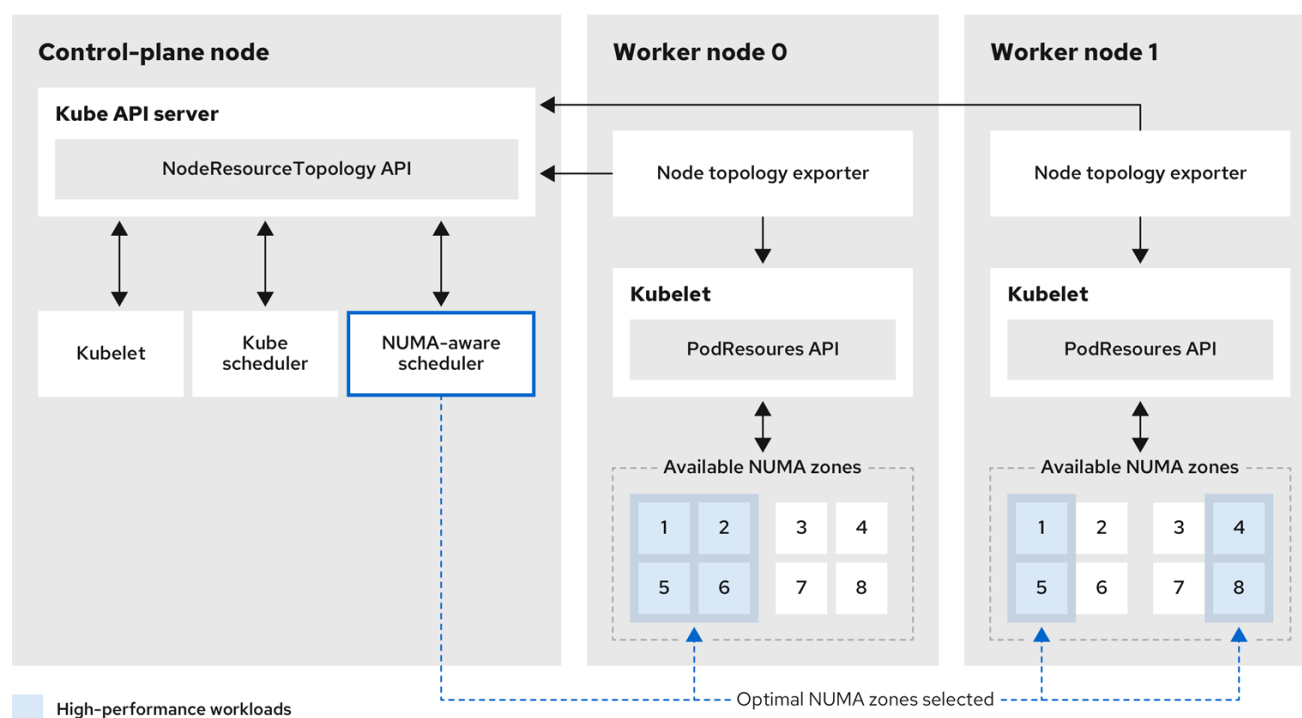
By integrating the performance profile of the Node Tuning Operator with NUMA-aware scheduling, you can further configure CPU affinity to optimize performance for latency-sensitive workloads.

The default OpenShift Container Platform pod scheduler scheduling logic considers the available resources of the entire compute node, not individual NUMA zones. If the most restrictive resource alignment is requested in the kubelet topology manager, error conditions can occur when admitting the pod to a node.

Conversely, if the most restrictive resource alignment is not requested, the pod can be admitted to the node without proper resource alignment, leading to worse or unpredictable performance. For example, runaway pod creation with **Topology Affinity Error** statuses can occur when the pod scheduler makes suboptimal scheduling decisions for guaranteed pod workloads without knowing if the pod's requested resources are available. Scheduling mismatch decisions can cause indefinite pod startup delays. Also, depending on the cluster state and resource allocation, poor pod scheduling decisions can cause extra load on the cluster because of failed startup attempts.

The NUMA Resources Operator deploys a custom NUMA resources secondary scheduler and other resources to mitigate against the shortcomings of the default OpenShift Container Platform pod scheduler. The following diagram provides a high-level overview of NUMA-aware pod scheduling.

Figure 12.1. NUMA-aware scheduling overview



216_OpenShift_0222

NodeResourceTopology API

The **NodeResourceTopology** API describes the available NUMA zone resources in each compute node.

NUMA-aware scheduler

The NUMA-aware secondary scheduler receives information about the available NUMA zones from the **NodeResourceTopology** API and schedules high-performance workloads on a node where it can be optimally processed.

Node topology exporter

The node topology exporter exposes the available NUMA zone resources for each compute node to the **NodeResourceTopology** API. The node topology exporter daemon tracks the resource allocation from the kubelet by using the **PodResources** API.

PodResources API

The **PodResources** API is local to each node and exposes the resource topology and available resources to the kubelet.



NOTE

The **List** endpoint of the **PodResources** API exposes exclusive CPUs allocated to a particular container. The API does not expose CPUs that belong to a shared pool.

The **GetAllocatableResources** endpoint exposes allocatable resources available on a node.

12.3. NUMA RESOURCE SCHEDULING STRATEGIES

To optimize the placement of high-performance workloads, the secondary scheduler uses NUMA-aware scoring strategies to select the most suitable compute nodes. This process assigns workloads based on resource availability while allowing local node managers to handle final resource pinning.

When scheduling high-performance workloads, the secondary scheduler determines which compute node is best suited for the task based on its internal NUMA resource distribution. While the scheduler uses NUMA-level data to score and select a compute node, the actual resource pinning within that node is managed by the local Topology Manager and CPU Manager.

When a high-performance workload is scheduled in a NUMA-aware cluster, the following steps occur:

1. **Node filtering:** The scheduler first filters the entire cluster to find a shortlist of *feasible* nodes. A node is only kept if the node meets all requirements, such as matching labels, respecting taints and tolerations, and, importantly, having sufficient available resources within its specific NUMA zones. If a node cannot satisfy the NUMA affinity of the workload, the node is filtered out at this stage.
2. **Node selection:** When a shortlist of suitable nodes is established, the scheduler evaluates them to find the best fit. The scheduler applies a NUMA-aware scoring strategy to rank these candidates based on their resource distribution. The node with the highest score is then selected for the workload.
3. **Local Allocation:** When the pod is assigned to a compute node, the node-level components (CPU, memory, device, and topology managers) perform the authoritative allocation of specific CPUs and memory. The scheduler does not influence this final selection.

The following table summarizes the different OpenShift Container Platform strategies and their outcomes:

Table 12.1. Scoring strategy summary

Strategy	Description	Outcome
LeastAllocated	Favors compute nodes that contain NUMA zones with the most available resources.	Distributes workloads across the cluster to nodes with the highest available headroom.
MostAllocated	Favors compute nodes where the requested resources fit into NUMA zones that are already highly utilized.	Consolidates workloads on already utilized nodes, potentially leaving other nodes idle.
BalancedAllocation	Favors compute nodes with the most balanced CPU and memory usage across NUMA zones.	Prevents skewed usage patterns where one resource type, such as CPU, is exhausted while another, such as memory, remains idle.

Additional resources

- [Scheduling pods using a secondary scheduler](#)
- [Changing where high-performance workloads run](#)

12.4. INSTALLING THE NUMA RESOURCES OPERATOR

NUMA Resources Operator deploys resources that allow you to schedule NUMA-aware workloads and deployments. You can install the NUMA Resources Operator using the OpenShift Container Platform CLI or the web console.

12.4.1. Installing the NUMA Resources Operator using the CLI

To enable NUMA-aware scheduling for high-performance workloads, install the NUMA Resources Operator by using the OpenShift CLI (**oc**). As a cluster administrator, you can deploy the Operator efficiently without using the web console.

Prerequisites

- Installed the OpenShift CLI (**oc**).
- Logged in as a user with **cluster-admin** privileges.

Procedure

1. Create a namespace for the NUMA Resources Operator:
 - a. Save the following YAML in the **nro-namespace.yaml** file:

```
apiVersion: v1
kind: Namespace
metadata:
  name: openshift-numaresources
# ...
```

- b. Create the **Namespace** CR by running the following command:

```
$ oc create -f nro-namespace.yaml
```

2. Create the Operator group for the NUMA Resources Operator:
 - a. Save the following YAML in the **nro-operatorgroup.yaml** file:

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: numaresources-operator
  namespace: openshift-numaresources
spec:
  targetNamespaces:
    - openshift-numaresources
# ...
```

- b. Create the **OperatorGroup** CR by running the following command:

```
$ oc create -f nro-operatorgroup.yaml
```

3. Create the subscription for the NUMA Resources Operator:

- a. Save the following YAML in the **nro-sub.yaml** file:

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: numaresources-operator
  namespace: openshift-numaresources
spec:
  channel: "4.22"
  name: numaresources-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
# ...
```

- b. Create the **Subscription** CR by running the following command:

```
$ oc create -f nro-sub.yaml
```

Verification

1. Verify that the installation succeeded by inspecting the CSV resource in the **openshift-numaresources** namespace. Run the following command:

```
$ oc get csv -n openshift-numaresources
```

Example output

NAME	DISPLAY	VERSION	REPLACES	PHASE
numaresources-operator.v4.22.2	numaresources-operator	4.22.2		Succeeded

12.4.2. Installing the NUMA Resources Operator using the web console

To enable NUMA-aware scheduling for high-performance workloads, install the NUMA Resources Operator by using the web console. As a cluster administrator, you can deploy the Operator through the graphical interface.

Procedure

1. Create a namespace for the NUMA Resources Operator:
 - a. In the OpenShift Container Platform web console, click **Administration** → **Namespaces**.
 - b. Click **Create Namespace**, enter **openshift-numaresources** in the **Name** field, and then click **Create**.
2. Install the NUMA Resources Operator:
 - a. In the OpenShift Container Platform web console, click **Ecosystem** → **Software Catalog**.
 - b. Choose **numaresources-operator** from the list of available Operators, and then click **Install**.
 - c. In the **Installed Namespaces** field, select the **openshift-numaresources** namespace, and then click **Install**.

3. Optional: Verify that the NUMA Resources Operator installed successfully:
 - a. Switch to the **Ecosystem → Installed Operators** page.
 - b. Ensure that **NUMA Resources Operator** is listed in the **openshift-numaresources** namespace with a **Status** of **InstallSucceeded**.



NOTE

During installation an Operator might display a **Failed** status. If the installation later succeeds with an **InstallSucceeded** message, you can ignore the **Failed** message.

If the Operator does not appear as installed, to troubleshoot further:

- Go to the **Ecosystem → Installed Operators** page and inspect the **Operator Subscriptions** and **Install Plans** tabs for any failure or errors under **Status**.
- Go to the **Workloads → Pods** page and check the logs for pods in the **default** project.

12.5. CONFIGURING A SINGLE NUMA NODE POLICY

To enable the NUMA Resources Operator, configure a single NUMA node policy on your cluster. You can implement this policy by creating a performance profile or by configuring a **KubeletConfig** custom resource (CR).



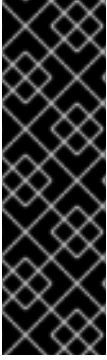
NOTE

The preferred way to configure a single NUMA node policy is to apply a performance profile. You can use the Performance Profile Creator (PPC) tool to create the performance profile. If a performance profile is created on the cluster, the PPC tool automatically creates other tuning components like **KubeletConfig** and the **tuned** profile.

For more information about creating a performance profile, see "About the Performance Profile Creator" in the "Additional resources" section.

12.5.1. Managing high availability (HA) for the NUMA-aware scheduler

To ensure high availability for the NUMA-aware secondary scheduler, the NUMA Resources Operator automatically creates scheduler replicas on control plane nodes. The Operator manages this configuration by using the **spec.replicas** field in the **NUMAResourcesScheduler** custom resource (CR).



IMPORTANT

Managing high availability is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

By default, the NUMA Resources Operator automatically enables HA mode by creating one scheduler replica for each control plane node, with a maximum of three replicas.

The following manifest demonstrates the default behavior. To automatically enable replica detection, omit the **replicas** field.

```
apiVersion: nodetopology.openshift.io/v1
kind: NUMAResourcesScheduler
metadata:
  name: example-auto-ha
spec:
  imageSpec: 'registry.redhat.io/openshift4/noderesourcetopology-scheduler-rhel9:v4.22'
  # The 'replicas' field is not included, enabling auto-detection.
```

You can control scheduler behavior by using one of the following options:

- Customizing the number of replicas.
- Disabling NUMA-aware scheduling.

12.5.1.1. Customizing scheduler replicas

You can set a specific number of scheduler replicas by updating the **spec.replicas** field in the **NUMAResourcesScheduler** custom resource. This configuration overrides the default HA behavior.

Procedure

1. Create the **NUMAResourcesScheduler** CR with the following YAML named for example **custom-ha.yaml** that sets the number of replicas to 2:

```
apiVersion: nodetopology.openshift.io/v1
kind: NUMAResourcesScheduler
metadata:
  name: example-custom
spec:
  imageSpec: 'registry.redhat.io/openshift4/noderesourcetopology-scheduler-rhel9:v4.22'
  replicas: 2
  # ...
```

2. Deploy the NUMA-aware pod scheduler by running the following command:

```
$ oc apply -f custom-ha.yaml
```

12.5.1.2. Disabling NUMA-aware scheduling

You can disable the NUMA-aware scheduler to stop all running scheduler pods and preventing new ones from starting.

Procedure

1. Save the following minimal required YAML in the **nro-disable-scheduler.yaml** file. Disable the scheduler by setting the **spec.replicas** field to **0**.

```
apiVersion: nodetopology.openshift.io/v1
kind: NUMAResourcesScheduler
metadata:
  name: example-disable
spec:
  imageSpec: 'registry.redhat.io/openshift4/noderesourcetopology-scheduler-rhel9:v4.22'
  replicas: 0
# ...
```

2. Disable the NUMA-aware pod scheduler by running the following command:

```
$ oc apply -f nro-disable-scheduler.yaml
```

12.5.1.3. Verifying scheduler high availability (HA) status

You can verify the status of the NUMA-aware scheduler to ensure the scheduler is running with the expected number of replicas based on your configuration.

Procedure

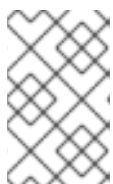
1. List only the scheduler pods by running the following command:

```
$ oc get pods -n openshift-numaresources -l app=secondary-scheduler
```

Expected output

NAME	READY	STATUS	RESTARTS	AGE
secondary-scheduler-5b8c9d479d-2r4p5	1/1	Running	0	5m
secondary-scheduler-5b8c9d479d-k2f3p	1/1	Running	0	5m
secondary-scheduler-5b8c9d479d-q8c7b	1/1	Running	0	5m

Using the default HA mode, the number of pods equals the number of control-plane nodes. A standard HA OpenShift Container Platform cluster typically has three control-plane nodes, and therefore displays three pods. If you **customized the replicas**, the number of pods matches the value you set. If you **disabled the scheduler**, there are no running pods with this label.



NOTE

A limit of 3 replicas is enforced for the NUMA-aware scheduler. On a hosted control planes cluster, the scheduler pods run on the compute nodes of the hosted-cluster.

2. Verify the number of replicas and their status by running the following command:

```
$ oc get deployment secondary-scheduler -n openshift-numaresources
```

Example output

```
NAME                READY  UP-TO-DATE  AVAILABLE  AGE
secondary-scheduler  3/3    3           3          5m
```

In this output, 3/3 means 3 replicas are ready out of an expected 3 replicas.

3. For more detailed information run the following command:

```
$ oc describe deployment secondary-scheduler -n openshift-numaresources
```

Example output

```
Replicas:      3 desired | 3 updated | 3 total | 3 available | 0 unavailable
```

The **Replicas** line shows a deployment configured for 3 replicas, with all 3 updated and available.

Additional resources

- [Configuring image registry repository mirroring](#)
- [About the Performance Profile Creator](#)

12.5.2. Sample performance profile

Reference an example YAML to understand how to use the performance profile creator (PPC) tool to create a performance profile.

```
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: performance
spec:
  cpu:
    isolated: "3"
    reserved: 0-2
  machineConfigPoolSelector:
    pools.operator.machineconfiguration.openshift.io/worker: ""
  nodeSelector:
    node-role.kubernetes.io/worker: ""
  numa:
    topologyPolicy: single-numa-node
  realTimeKernel:
    enabled: true
  workloadHints:
    highPowerConsumption: true
    perPodPowerManagement: false
    realTime: true
```

where:

spec.pools.operator.machineconfiguration.openshift.io/worker

Specifies the value that must match the **MachineConfigPool** value that you want to configure the NUMA Resources Operator on. For example, you might create a **MachineConfigPool** object named **worker-cnf** that designates a set of nodes that run telecommunications workloads. The value for **MachineConfigPool** must match the **machineConfigPoolSelector** value in the **NUMAResourcesOperator** CR that you configure later in "Creating the NUMAResourcesOperator custom resource".

spec.numa.topologyPolicy

Specifies that the **topologyPolicy** field is set to **single-numa-node** by setting the **topology-manager-policy** argument to **single-numa-node** when you run the PPC tool.

**NOTE**

For hosted control plane clusters, the **machineConfigPoolSelector** does not have any functional effect. Node association is instead determined by the specified **NodePool** object.

12.5.3. Creating a KubeletConfig CR

To configure a single NUMA node policy, create and apply a KubeletConfig custom resource (CR). While applying a performance profile is recommended, you can use the alternative method to manually manage the configuration on your cluster.

Procedure

1. Create the **KubeletConfig** custom resource (CR) that configures the pod admittance policy for the machine profile:
 - a. Save the following YAML in the **nro-kubeletconfig.yaml** file:

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: worker-tuning
spec:
  machineConfigPoolSelector:
    matchLabels:
      pools.operator.machineconfiguration.openshift.io/worker: ""
  kubeletConfig:
    cpuManagerPolicy: "static"
    cpuManagerReconcilePeriod: "5s"
    reservedSystemCPUs: "0,1"
    memoryManagerPolicy: "Static"
    evictionHard:
      memory.available: "100Mi"
    kubeReserved:
      memory: "512Mi"
    reservedMemory:
      - numaNode: 0
        limits:
          memory: "1124Mi"
```



```
systemReserved:
  memory: "512Mi"
  topologyManagerPolicy: "single-numa-node"
```

where:

spec.machineConfigPoolSelector.matchLabels.pools.operator.machineconfiguration.openshift.io/worker

Specifies that this label matches the **machineConfigPoolSelector** setting in the **NUMAResourcesOperator** CR that you configure later in "Creating the NUMAResourcesOperator custom resource".

spec.kubeletConfig.cpuManagerPolicy

Specifies the **static** value. You must use a lowercase **s**.

spec.kubeletConfig.reservedSystemCPUs

Adjust the field based on the CPU on your nodes.

spec.kubeletConfig.memoryManagerPolicy

Specifies **Static**. You must use an uppercase **S**.

spec.kubeletConfig.topologyManagerPolicy

Specifies the value as **single-numa-node**.



NOTE

For hosted control plane clusters, the **machineConfigPoolSelector** setting does not have any functional effect. Node association is instead determined by the specified **NodePool** object. To apply a **KubeletConfig** for hosted control plane clusters, you must create a **ConfigMap** that contains the configuration, and then reference that **ConfigMap** within the **spec.config** field of a **NodePool**.

- b. Create the **KubeletConfig** CR by running the following command:

```
$ oc create -f nro-kubeletconfig.yaml
```



NOTE

Applying performance profile or **KubeletConfig** automatically triggers rebooting of the nodes. If no reboot is triggered, you can troubleshoot the issue by looking at the labels in **KubeletConfig** that address the node group.

12.6. SCHEDULING NUMA-AWARE WORKLOADS

To process latency-sensitive and high-performance workloads efficiently, configure your OpenShift Container Platform cluster for NUMA-aware scheduling. This process aligns pods with specific NUMA zones to minimize network delays and maximize compute resource utilization.

Clusters running latency-sensitive workloads typically feature performance profiles that help to minimize workload latency and optimize performance. The NUMA-aware scheduler deploys workloads based on available node NUMA resources and with respect to any performance profile settings applied

to the node. The combination of NUMA-aware deployments, and the performance profile of the workload, ensures that workloads are scheduled in a way that maximizes performance.

For the NUMA Resources Operator to be fully operational, you must deploy the **NUMAResourcesOperator** custom resource and the NUMA-aware secondary pod scheduler.

12.6.1. Creating the NUMAResourcesOperator custom resource

After you have installed the NUMA Resources Operator, you can create the **NUMAResourcesOperator** custom resource (CR). This CR instructs the NUMA Resources Operator to install all the cluster infrastructure that is needed to support the NUMA-aware scheduler, including daemon sets and APIs.

Prerequisites

- Installed the OpenShift CLI (**oc**).
- Logged in as a user with **cluster-admin** privileges.
- Installed the NUMA Resources Operator.

Procedure

1. Create the **NUMAResourcesOperator** custom resource:
 - a. Save the following minimal required YAML file example as **nrop.yaml**:

```
apiVersion: nodetopology.openshift.io/v1
kind: NUMAResourcesOperator
metadata:
  name: numaresourcesoperator
spec:
  nodeGroups:
  - machineConfigPoolSelector:
      matchLabels:
        pools.operator.machineconfiguration.openshift.io/worker: ""
# ...
```

pools.operator.machineconfiguration.openshift.io/worker: Specifies a value that must match the **MachineConfigPool** resource that you want to configure the NUMA Resources Operator on. For example, you might have created a **MachineConfigPool** resource named **worker-cnf** that designates a set of nodes expected to run telecommunications workloads. When configuring the **nodeGroups** spec, ensure that each **MachineConfigPool** resource you reference targets nodes with a unique **nodeSelector** label. This **nodeSelector** label should be applied exclusively to that specific node set. A node you want to manage with topology-aware scheduling must be associated with a single **MachineConfigPool** resource. Consequently, each **nodeGroup** should match exactly one **MachineConfigPool** resource, as configurations matching multiple pools are not supported.

- b. Create the **NUMAResourcesOperator** CR by running the following command:

```
$ oc create -f nrop.yaml
```

2. Optional: To enable NUMA-aware scheduling for multiple machine config pools (MCPs), define a separate **NodeGroup** for each pool. For example, define three **NodeGroups** for **worker-cnf**,

worker-ht, and **worker-other**, in the **NUMAResourcesOperator** CR as shown in the following example:

Example YAML definition for a **NUMAResourcesOperator** CR with multiple **NodeGroups**

```
apiVersion: nodetopology.openshift.io/v1
kind: NUMAResourcesOperator
metadata:
  name: numaresourcesoperator
spec:
  logLevel: Normal
  nodeGroups:
    - machineConfigPoolSelector:
        matchLabels:
          machineconfiguration.openshift.io/role: worker-ht
    - machineConfigPoolSelector:
        matchLabels:
          machineconfiguration.openshift.io/role: worker-cnf
    - machineConfigPoolSelector:
        matchLabels:
          machineconfiguration.openshift.io/role: worker-other
# ...
```

Verification

1. Verify that the NUMA Resources Operator deployed successfully by running the following command:

```
$ oc get numaresourcesoperators.nodetopology.openshift.io
```

Example output

```
NAME                AGE
numaresourcesoperator 27s
```

2. After a few minutes, run the following command to verify that the required resources deployed successfully:

```
$ oc get all -n openshift-numaresources
```

Example output

```
NAME                                READY STATUS  RESTARTS  AGE
pod/numaresources-controller-manager-7d9d84c58d-qk2mr  1/1   Running  0         12m
pod/numaresourcesoperator-worker-7d96r                2/2   Running  0         97s
pod/numaresourcesoperator-worker-crsht                2/2   Running  0         97s
pod/numaresourcesoperator-worker-jp9mw                2/2   Running  0         97s
```

12.6.2. Creating the **NUMAResourcesOperator** custom resource for hosted control planes

After you install the NUMA Resources Operator, create the **NUMAResourcesOperator** custom resource (CR). The CR instructs the NUMA Resources Operator to install all the cluster infrastructure that is needed to support the NUMA-aware scheduler on hosted control planes, including daemon sets and APIs.



IMPORTANT

Creating the NUMAResourcesOperator custom resource for hosted control planes is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Prerequisites

- Installed the OpenShift CLI (**oc**).
- Logged in as a user with **cluster-admin** privileges.
- Installed the NUMA Resources Operator.

Procedure

1. Export the management cluster kubeconfig file by running the following command:

```
$ export KUBECONFIG=<path-to-management-cluster-kubeconfig>
```

2. Find the **node-pool-name** for your cluster by running the following command:

```
$ oc --kubeconfig="$MGMT_KUBECONFIG" get np -A
```

Example output

NAMESPACE	NAME	CLUSTER	DESIRED NODES	CURRENT NODES
AUTOSCALING	AUTOREPAIR	VERSION	UPDATINGVERSION	UPDATINGCONFIG
MESSAGE				
clusters	democluster-us-east-1a	democluster	1	1
4.22.0	False	False	False	False

The **node-pool-name** is the **NAME** field in the output. In this example, the **node-pool-name** is **democluster-us-east-1a**.

3. Create a YAML file named **nrop-hcp.yaml** with at least the following content:

```
apiVersion: nodetopology.openshift.io/v1
kind: NUMAResourcesOperator
metadata:
  name: numaresourcesoperator
spec:
```

```
nodeGroups:
- poolName: democluster-us-east-1a
# ...
```

- **spec.nodeGroups.poolName**: Specifies the **poolName**. The example shows the **node-pool-name** pool name that was retrieved from a previous step.

4. On the management cluster, run the following command to list the available secrets:

```
$ oc get secrets -n clusters
```

Example output

NAME	TYPE	DATA	AGE
builder-dockercfg-25qpp	kubernetes.io/dockercfg	1	128m
default-dockercfg-mkvlz	kubernetes.io/dockercfg	1	128m
democluster-admin-kubeconfig	Opaque	1	127m
democluster-etcd-encryption-key	Opaque	1	128m
democluster-kubeadmin-password	Opaque	1	126m
democluster-pull-secret	Opaque	1	128m
deployer-dockercfg-8lfpd	kubernetes.io/dockercfg	1	128m

5. Extract the **kubeconfig** file for the hosted cluster by running the following command:

```
$ oc get secret <SECRET_NAME> -n clusters -o jsonpath='{.data.kubeconfig}' | base64 -d >
hosted-cluster-kubeconfig
```

Example

```
$ oc get secret democluster-admin-kubeconfig -n clusters -o jsonpath='{.data.kubeconfig}' |
base64 -d > hosted-cluster-kubeconfig
```

6. Export the hosted cluster **kubeconfig** file by running the following command:

```
$ export HC_KUBECONFIG=<path_to_hosted-cluster-kubeconfig>
```

7. Create the **NUMAResourcesOperator** CR by running the following command on the hosted cluster:

```
$ oc create -f nrop-hcp.yaml
```

Verification

1. Verify that the NUMA Resources Operator deployed successfully by running the following command:

```
$ oc get numaresourcesoperators.nodetopology.openshift.io
```

Example output

NAME	AGE
numaresourcesoperator	27s

2. After a few minutes, run the following command to verify that the required resources deployed successfully:

```
$ oc get all -n openshift-numaresources
```

Example output

```
NAME                                READY STATUS RESTARTS AGE
pod/numaresources-controller-manager-7d9d84c58d-qk2mr 1/1 Running 0 12m
pod/numaresourcesoperator-democluster-7d96r           2/2 Running 0 97s
pod/numaresourcesoperator-democluster-crsht           2/2 Running 0 97s
pod/numaresourcesoperator-democluster-jp9mw           2/2 Running 0 97s
```

Additional resources

- [Creating a performance profile](#)

12.6.3. Deploying the NUMA-aware secondary pod scheduler

To optimize the placement of high-performance workloads, deploy the NUMA-aware secondary pod scheduler. This component aligns pods with specific NUMA zones to ensure efficient resource utilization in your cluster.

Procedure

1. Create the **NUMAResourcesScheduler** custom resource that deploys the NUMA-aware custom pod scheduler:
 - a. Save the following minimal required YAML in the **nro-scheduler.yaml** file:

```
apiVersion: nodetopology.openshift.io/v1
kind: NUMAResourcesScheduler
metadata:
  name: numaresourcesscheduler
spec:
  imageSpec: "registry.redhat.io/openshift4/noderesourcetopology-scheduler-rhel9:v4.22"
  # ...
```

- **spec.imageSpec:** In a disconnected environment, make sure to configure the resolution of this image by either:
 - b. Create an **ImageTagMirrorSet** custom resource (CR). For more information, see "Configuring image registry repository mirroring" in the "Additional resources" section.
 - c. Set the URL to the disconnected registry.
 - d. Create the **NUMAResourcesScheduler** CR by running the following command:

```
$ oc create -f nro-scheduler.yaml
```

**NOTE**

In a hosted control plane cluster, run this command on the hosted control plane node.

- After a few seconds, run the following command to confirm the successful deployment of the required resources:

```
$ oc get all -n openshift-numaresources
```

Example output

```
NAME                                READY STATUS  RESTARTS  AGE
pod/numaresources-controller-manager-7d9d84c58d-qk2mr  1/1   Running  0         12m
pod/numaresourcesoperator-worker-7d96r                2/2   Running  0         97s
pod/numaresourcesoperator-worker-crsht                2/2   Running  0         97s
pod/numaresourcesoperator-worker-jp9mw                2/2   Running  0         97s
pod/secondary-scheduler-847cb74f84-9whlm              1/1   Running  0         10m

NAME                                DESIRED  CURRENT  READY  UP-TO-DATE
AVAILABLE  NODE SELECTOR  AGE
daemonset.apps/numaresourcesoperator-worker  3        3        3      3        3      node-
role.kubernetes.io/worker= 98s

NAME                                READY  UP-TO-DATE  AVAILABLE  AGE
deployment.apps/numaresources-controller-manager  1/1    1          1         12m
deployment.apps/secondary-scheduler              1/1    1          1         10m

NAME                                DESIRED  CURRENT  READY  AGE
replicaset.apps/numaresources-controller-manager-7d9d84c58d  1        1        1      12m
replicaset.apps/secondary-scheduler-847cb74f84              1        1        1      10m
```

Additional resources

- [Configuring image registry repository mirroring](#)

12.6.4. Scheduling workloads with the NUMA-aware scheduler

To schedule workloads with the NUMA-aware scheduler, use deployment CRs that specify the minimum required resources. This ensures your cluster processes the workloads efficiently.

Before you schedule workloads with the NUMA-aware scheduler, ensure that you previously installed the **topo-aware-scheduler**, you applied the **NUMAResourcesOperator** and **NUMAResourcesScheduler** CRs, and that your cluster has a matching performance profile or **kubeletconfig**.

The example in the procedure uses NUMA-aware scheduling for a sample workload.

Prerequisites

- Installed the OpenShift CLI (**oc**).
- Logged in as a user with **cluster-admin** privileges.

Procedure

1. Get the name of the NUMA-aware scheduler that is deployed in the cluster by running the following command:

```
$ oc get numaresourcesschedulers.nodetopology.openshift.io numaresourcesscheduler -o json | jq '.status.schedulerName'
```

Example output

```
"topo-aware-scheduler"
```

2. Create a **Deployment** CR that uses scheduler named **topo-aware-scheduler**, for example:
 - a. Save the following YAML in the **nro-deployment.yaml** file:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: numa-deployment-1
  namespace: openshift-numaresources
spec:
  replicas: 1
  selector:
    matchLabels:
      app: test
  template:
    metadata:
      labels:
        app: test
    spec:
      schedulerName: topo-aware-scheduler
      containers:
        - name: ctnr
          image: quay.io/openshifttest/hello-openshift:openshift
          imagePullPolicy: IfNotPresent
          resources:
            limits:
              memory: "100Mi"
              cpu: "10"
            requests:
              memory: "100Mi"
              cpu: "10"
        - name: ctnr2
          image: registry.access.redhat.com/rhel:latest
          imagePullPolicy: IfNotPresent
          command: ["/bin/sh", "-c"]
          args: [ "while true; do sleep 1h; done;" ]
          resources:
            limits:
              memory: "100Mi"
              cpu: "8"
            requests:
              memory: "100Mi"
              cpu: "8"
```


spec.schedulerName: Specifies the scheduler name that must match the name of the NUMA-aware scheduler that is deployed in your cluster, such as **topo-aware-scheduler**.

- b. Create the **Deployment** CR by running the following command:

```
$ oc create -f nro-deployment.yaml
```

Verification

1. Verify that the deployment was successful:

```
$ oc get pods -n openshift-numaresources
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
numa-deployment-1-6c4f5bdb84-wgn6g	2/2	Running	0	5m2s
numaresources-controller-manager-7d9d84c58d-4v65j	1/1	Running	0	18m
numaresourcesoperator-worker-7d96r	2/2	Running	4	43m
numaresourcesoperator-worker-crsht	2/2	Running	2	43m
numaresourcesoperator-worker-jp9mw	2/2	Running	2	43m
secondary-scheduler-847cb74f84-fpncj	1/1	Running	0	18m

2. Verify that the **topo-aware-scheduler** is scheduling the deployed pod by running the following command:

```
$ oc describe pod numa-deployment-1-6c4f5bdb84-wgn6g -n openshift-numaresources
```

Example output

```
Events:
  Type    Reason      Age   From              Message
  ----    -
  Normal  Scheduled   4m45s topo-aware-scheduler Successfully assigned openshift-numaresources/numa-deployment-1-6c4f5bdb84-wgn6g to worker-1
```



NOTE

Deployments that request more resources than is available for scheduling will fail with a **MinimumReplicasUnavailable** error. The deployment succeeds when the required resources become available. Pods remain in the **Pending** state until the required resources are available.

3. Verify that the expected allocated resources are listed for the node.
 - a. Identify the node that is running the deployment pod by running the following command:

```
$ oc get pods -n openshift-numaresources -o wide
```

Example output

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
------	-------	--------	----------	-----	----	------

NOMINATED NODE READINESS GATES

```
numa-deployment-1-6c4f5bdb84-wgn6g 0/2 Running 0      82m 10.128.2.50
worker-1 <none> <none>
```

- b. Run the following command with the name of that node that is running the deployment pod.

```
$ oc describe noderesourcetopologies.topology.node.k8s.io worker-1
```

Example output

```
...
Zones:
Costs:
  Name: node-0
  Value: 10
  Name: node-1
  Value: 21
Name: node-0
Resources:
  Allocatable: 39
  Available: 21
  Capacity: 40
  Name: cpu
  Allocatable: 6442450944
  Available: 6442450944
  Capacity: 6442450944
  Name: hugepages-1Gi
  Allocatable: 134217728
  Available: 134217728
  Capacity: 134217728
  Name: hugepages-2Mi
  Allocatable: 262415904768
  Available: 262206189568
  Capacity: 270146007040
  Name: memory
Type: Node
```

Resources.Available: Specifies the **Available** capacity that is reduced because of the resources that have been allocated to the guaranteed pod. Resources consumed by guaranteed pods are subtracted from the available node resources listed under **noderesourcetopologies.topology.node.k8s.io**.

4. Resource allocations for pods with a **Best-effort** or **Burstable** quality of service (**qosClass**) are not reflected in the NUMA node resources under **noderesourcetopologies.topology.node.k8s.io**. If a pod's consumed resources are not reflected in the node resource calculation, verify that the pod has **qosClass** of **Guaranteed** and the CPU request is an integer value, not a decimal value. You can verify that the pod has a **qosClass** of **Guaranteed** by running the following command:

```
$ oc get pod numa-deployment-1-6c4f5bdb84-wgn6g -n openshift-numaresources -o
jsonpath="{ .status.qosClass }"
```

Example output

12.7. NUMA RESOURCES OPERATOR SUPPORT FOR SCHEDULABLE CONTROL-PLANE NODES

You can enable schedulable control plane nodes to run user-defined pods, effectively turning the nodes into hybrid control plane and compute nodes. This configuration is especially beneficial in resource-constrained environments, such as compact clusters.

When enabled, the NUMA Resources Operator can apply its topology-aware scheduling to the nodes for guaranteed workloads, ensuring pods are placed according to the best NUMA affinity.

Traditionally, control plane nodes in OpenShift Container Platform are dedicated to running critical cluster services. Enabling schedulable control plane nodes allows user-defined Pods to be scheduled on the nodes.

You can make control plane nodes schedulable by setting the **mastersSchedulable** field to true in the **schedulers.config.openshift.io** resource.



NOTE

When you enable schedulable control plane nodes, enabling workload partitioning is strongly recommended to safeguard critical infrastructure pods from resource starvation. This process restricts infrastructure components, like the **ovnkube-node** process, to dedicated, reserved CPUs. However, the OVS dynamic pinning feature relies on **ovnkube-node** having access to the CPUs designated for burstable/best-effort pods to correctly identify and use non-pinned CPUs. When workload partitioning configures the **ovnkube-node** process with CPU affinity for reserved CPUs, this dynamic pinning mechanism breaks.

The NUMA Resources Operator provides topology-aware scheduling for workloads that need a specific NUMA affinity. When control plane nodes are made schedulable, the management capabilities of the Operator can be applied to them, just as they are to compute nodes. This ensures that NUMA-aware pods are placed on a node with the best NUMA topology, whether it is a control plane or compute node.

When configuring the NUMA Resources Operator, its management scope is determined by the **nodeGroups** field in its custom resource (CR). This principle applies to both compact and multi-node clusters.

Compact clusters

In a compact cluster, all nodes are configured as schedulable control plane nodes. The NUMA Resources Operator can be configured to manage all nodes in the cluster. Follow the deployment instructions for more details on the process.

Multi-Node OpenShift (MNO) clusters

In a Multi-Node OpenShift Container Platform cluster, control plane nodes are made schedulable in addition to existing compute nodes. To manage these nodes, you can configure the NUMA Resources Operator by defining separate **nodeGroups** in the **NUMAResourcesOperator** CR for the control plane and compute nodes. This ensures that the NUMA Resources Operator correctly schedules pods on both sets of nodes based on resource availability and NUMA topology.



NOTE

Modifying a performance profile often triggers control plane node reboots. Due to stricter Pod Disruption Budgets (PDBs) on control plane nodes, the cluster's resilience mechanisms are activated. These mechanisms prevent the forced eviction of protected but unhealthy pods such as those in **CrashLoopBackOff**, which causes the Machine Config Pool (MCP) to stall during the reboot process.

If the MCP becomes stuck due to this behavior, intervention is required to resolve the issue and allow the control plane upgrade to complete.

To resolve this, administrators have two options:

1. Temporarily relax the PDB restrictions to allow the required eviction.
2. Manually delete the unhealthy pods to force the MCP to reconcile and continue the drain process.

12.7.1. Configuring NUMA Resources Operator on schedulable control plane nodes

To run workloads on control plane nodes, configure the NUMA Resources Operator (NROP) to manage them as schedulable. This configuration is ideal for compact clusters and multi-node OpenShift (MNO) environments where control plane nodes also function as compute nodes.

Prerequisites

- Install the OpenShift CLI (**oc**).
- Log in as a user with **cluster-admin** privileges.
- Install the NUMA Resources Operator.

Procedure

1. To enable Topology Aware Scheduling (TAS) on control plane nodes, configure the nodes to be schedulable first. This allows the NUMA Resources Operator to deploy and manage pods on them. Without this action, the operator cannot deploy the pods required to gather NUMA topology information from these nodes. Follow these steps to make the control plane nodes schedulable:

- a. Edit the **schedulers.config.openshift.io** resource by running the following command:

```
$ oc edit schedulers.config.openshift.io cluster
```

- b. In the editor, set the **mastersSchedulable** field to **true**, then save and exit the editor.

```
apiVersion: config.openshift.io/v1
kind: Scheduler
metadata:
  creationTimestamp: "2019-09-10T03:04:05Z"
  generation: 1
  name: cluster
  resourceVersion: "433"
  selfLink: /apis/config.openshift.io/v1/schedulers/cluster
  uid: a636d30a-d377-11e9-88d4-0a60097bee62
```

```
spec:
  mastersSchedulable: true
status: {}
#...
```

2. To configure the NUMA Resources Operator, you must create a single `NUMAResourcesOperator` custom resource (CR) on the cluster. The **nodeGroups** configuration within this CR specifies the node pools the Operator must manage.



NOTE

Before configuring **nodeGroups**, ensure the specified node pool meets all prerequisites detailed in Section 12.5, "Configuring a single NUMA node policy." The NUMA Resources Operator requires all nodes within a group to be identical. Non-compliant nodes prevent the NUMA Resources Operator from performing the expected topology-aware scheduling for the entire pool.

You can specify multiple non-overlapping node sets for the NUMA Resources Operator to manage. Each of these sets should correspond to a different machine config pool (MCP). The NUMA Resources Operator then manages the schedulable control plane nodes within these specified node groups.

- a. For a compact cluster, the compact cluster's master nodes are also the schedulable nodes, so specify only the master pool. Create the following **nodeGroups** configuration in the **NUMAResourcesOperator** CR:

```
apiVersion: nodetopology.openshift.io/v1
kind: NUMAResourcesOperator
metadata:
  name: numaresourcesoperator
spec:
  nodeGroups:
    - poolName: master
# ...
```



NOTE

Configuring a compact cluster with a worker pool in addition to the **master** pool should be avoided. While this setup does not break the cluster or affect operator functionality, it can lead to redundant or duplicate pods and create unnecessary noise in the system. The worker pool is essentially a pointless, empty MCP in this context and serves no purpose.

- b. For an MNO cluster where both control plane and compute nodes are schedulable, you have the option to configure the NUMA Resources Operator to manage multiple **nodeGroups**. You can specify which nodes to include by adding their corresponding MCPs to the **nodeGroups** list in the **NUMAResourcesOperator** CR. The configuration depends entirely on your specific requirements. For example, to manage both the **master** and **worker-cnf** pools, create the following **nodeGroups** configuration in the **NUMAResourcesOperator** CR:

```
apiVersion: nodetopology.openshift.io/v1
kind: NUMAResourcesOperator
metadata:
```

```

name: numaresourcesoperator
spec:
  nodeGroups:
    - poolName: master
    - poolName: worker-cnf
# ...

```



NOTE

You can customize this list to include any combination of nodeGroups for management with Topology-Aware Scheduling. To prevent duplicate, pending pods, you must ensure that each **poolName** in the configuration corresponds to a MachineConfigPool (MCP) with a unique node selector label. The label must be applied only to the nodes within that specific pool and must not overlap with labels on any other nodes in the cluster. The **worker-cnf** MCP designates a set of nodes that run telecommunications workloads.

- c. After you update the **nodeGroups** field in the **NUMAResourcesOperator** CR to reflect your cluster's configuration, apply the changes by running the following command:

```
$ oc apply -f <filename>.yaml
```



NOTE

Replace **<filename>.yaml** with the name of your configuration file.

Verification

After applying the configuration, verify that the NUMA Resources Operator is correctly managing the schedulable control plane nodes by performing the following checks:

1. Confirm that the control plane nodes have the worker role and are schedulable by running the following command:

```
$ oc get nodes
```

Example output:

NAME	STATUS	ROLES	AGE	VERSION
worker-0	Ready	worker,worker-cnf	100m	v1.35.4
worker-1	Ready	worker	93m	v1.35.4
master-0	Ready	control-plane,master,worker	108m	v1.35.4
master-1	Ready	control-plane,master,worker	107m	v1.35.4
master-2	Ready	control-plane,master,worker	107m	v1.35.4
worker-2	Ready	worker	100m	v1.35.4

2. Verify that the NUMA Resources Operator's pods are running on the intended nodes by running the following command. You should see a numaresourcesoperator pod for each node group you specified in the CR:

```
$ oc get pods -n openshift-numaresources -o wide
```

Example output:

NAME	READY	STATUS	RESTARTS	AGE	IP
NODE	NOMINATED	NODE	READINESS	GATES	
numaresources-controller-manager-bdbdd574-xx6bw	1/1	Running	0	49m	
10.130.0.17	master-0	<none>	<none>		
numaresourcesoperator-master-lprrh	2/2	Running	0	20m	10.130.0.20
master-0	<none>	2/2			
numaresourcesoperator-master-qk6k4	2/2	Running	0	20m	10.129.0.50
master-2	<none>	2/2			
numaresourcesoperator-master-zm79n	2/2	Running	0	20m	10.128.0.44
master-1	<none>	2/2			
numaresourcesoperator-worker-cnf-gqlmd	2/2	Running	0	4m27s	
10.128.2.21	worker-0	<none>	2/2		

- Confirm that the NUMA Resources Operator has collected and reported the NUMA topology data for all nodes in the specified groups by running the following command:

```
$ oc get noderesourcetopologies.topology.node.k8s.io
```

Example output:

NAME	AGE
worker-0	6m11s
master-0	22m
master-1	21m
master-2	21m

The presence of a **NodeResourceTopology** resource for a node confirms that the NUMA Resources Operator was able to schedule a pod on it to collect the data, enabling topology-aware scheduling.

- Inspect a single Node Resource Topology by running the following command:

```
$ oc get noderesourcetopologies <master_node_name> -o yaml
```

Example output:

```
apiVersion: topology.node.k8s.io/v1alpha2
attributes:
- name: nodeTopologyPodsFingerprint
  value: pfp0v001ef46db3751d8e999
- name: nodeTopologyPodsFingerprintMethod
  value: with-exclusive-resources
- name: topologyManagerScope
  value: container
- name: topologyManagerPolicy
  value: single-numa-node
kind: NodeResourceTopology
metadata:
  annotations:
    k8stopoawareschedwg/rte-update: periodic
    topology.node.k8s.io/fingerprint: pfp0v001ef46db3751d8e999
  creationTimestamp: "2025-09-23T10:18:34Z"
```

```

generation: 1
name: master-0
resourceVersion: "58173"
uid: 35c0d27e-7d9f-43d3-bab9-2ebc0d385861
zones:
- costs:
  - name: node-0
    value: 10
  name: node-0
resources:
- allocatable: "3"
  available: "2"
  capacity: "4"
  name: cpu
- allocatable: "1476189952"
  available: "1378189952"
  capacity: "1576189952"
  name: memory
type: Node
# ...

```

The presence of this resource for a node with a master role proves that the NUMA Resources Operator was able to deploy its discovery pods onto that node. These pods are what gather the NUMA topology data, and they can only be scheduled on nodes that are considered schedulable.

The output confirms that the procedure to make the master nodes schedulable was successful, as the NUMA Resources Operator has now collected and reported the NUMA-related information for that specific control plane node.

12.8. CONFIGURING POLLING OPERATIONS FOR NUMA RESOURCES UPDATES

As an optional task, you can improve scheduling behavior and troubleshoot suboptimal scheduling decisions by configuring the **spec.nodeGroups** specification in the **NUMAResourcesOperator** custom resource (CR). This configuration fine-tunes how daemons poll for available NUMA resources, providing advanced control over your polling operations.

The configuration options are listed as follows:

- **infoRefreshMode:** Determines the trigger condition for polling the kubelet. The NUMA Resources Operator reports the resulting information to the API server.
- **infoRefreshPeriod:** Determines the duration between polling updates.
- **PodsFingerprinting:** Determines if point-in-time information for the current set of pods running on a node is exposed in polling updates.



NOTE

The default value for **podsFingerprinting** is **EnabledExclusiveResources**. To optimize scheduler performance, set **podsFingerprinting** to either **EnabledExclusiveResources** or **Enabled**. Additionally, configure the **cacheResyncPeriod** in the **NUMAResourcesScheduler** custom resource (CR) to a value greater than 0. The **cacheResyncPeriod** specification helps to report more exact resource availability by monitoring pending resources on nodes.

Prerequisites

- Installed the OpenShift CLI (**oc**).
- Logged in as a user with **cluster-admin** privileges.
- Installed the NUMA Resources Operator.

Procedure

- Configure the **spec.nodeGroups** specification in your **NUMAResourcesOperator** CR:

```
apiVersion: nodetopology.openshift.io/v1
kind: NUMAResourcesOperator
metadata:
  name: numaresourcesoperator
spec:
  nodeGroups:
  - config:
    infoRefreshMode: Periodic
    infoRefreshPeriod: 10s
    podsFingerprinting: Enabled
    name: worker
# ...
```

where:

spec.nodeGroups.config.infoRefreshMode

Valid values are **Periodic**, **Events**, **PeriodicAndEvents**. Use **Periodic** to poll the kubelet at intervals that you define in **infoRefreshPeriod**. Use **Events** to poll the kubelet at every pod lifecycle event. Use **PeriodicAndEvents** to enable both methods.

spec.nodeGroups.config.infoRefreshPeriod

Specifies the polling interval for **Periodic** or **PeriodicAndEvents** refresh modes. The field is ignored if the refresh mode is **Events**.

spec.nodeGroups.config.podsFingerprinting

Valid values are **Enabled**, **Disabled**, and **EnabledExclusiveResources**. Setting to **Enabled** or **EnabledExclusiveResources** is a requirement for the **cacheResyncPeriod** specification in the **NUMAResourcesScheduler**.

Verification

1. After you deploy the NUMA Resources Operator, verify that the node group configurations were applied by running the following command:

—

```
$ oc get numaresop numaresourcesoperator -o json | jq '.status'
```

Example output

```
...
  "config": {
    "infoRefreshMode": "Periodic",
    "infoRefreshPeriod": "10s",
    "podsFingerprinting": "Enabled"
  },
  "name": "worker"
...
```

12.9. TOPOLOGY-AWARE SCHEDULER SCALABILITY

You can scale the NUMA-aware secondary scheduler to support clusters with up to 500 nodes. Understanding how the scheduler consumes resources at scale helps you size your control plane correctly and avoid resource exhaustion during cluster growth.

The NUMA-aware secondary scheduler relies on the **NodeResourceTopology** custom resource (CR) to track per-node NUMA zone availability. As the number of nodes in a cluster increases, the scheduler must process a larger set of **NodeResourceTopology** objects during each scheduling cycle. This relationship between node count, cache refresh interval, and scheduling latency determines the scalability profile of the scheduler.

From OpenShift Container Platform 4.22, the NUMA-aware scheduler pod defaults to **Burstable** quality of service (QoS), which reduces baseline resource consumption while allowing the scheduler to scale up in larger clusters. Switching to **Guaranteed** QoS is generally not recommended because it mandates a higher resource commitment that can unnecessarily constrain the control plane.

When high availability (HA) mode is enabled, the NUMA Resources Operator deploys multiple scheduler replicas across the control plane nodes to ensure redundancy.

12.9.1. Optimization strategies for large-cluster NUMA-aware scheduling

You can optimize the NUMA-aware secondary scheduler for clusters with 200 or more nodes by tuning cache resync intervals, polling configurations, and the scheduler QoS profile. These optimization points help you balance scheduling accuracy, API server load, and control plane resource consumption for your specific cluster size and workload profile.

To get the best performance, coordinate the **NUMAResourcesOperator** and **NUMAResourcesScheduler** settings. The operator controls how frequently topology data is exported, and the scheduler controls how frequently it consumes that data. Aligning these intervals ensures accurate scheduling decisions while minimizing API server traffic.

12.9.1.1. NUMAResourcesOperator settings

In the **NUMAResourcesOperator** CR, configure the following parameters to optimize for large clusters:

- For NUMA-sensitive workloads with minimal overhead, set **podsFingerprinting** to **EnabledExclusiveResources**, which is the default.

- For higher accuracy, set **podsFingerprinting** to **Enabled**. This increases the API load.
- Set **infoRefreshMode** to **Periodic** for large clusters. **PeriodicAndEvents** provides the highest accuracy but increases API load.
- Set **infoRefreshPeriod** to control how often the exporter updates topology data. This value must be shorter than the scheduler **cacheResyncPeriod** so that the cache always pulls fresh data. For example, set **infoRefreshPeriod** to 5 seconds.

12.9.1.2. NUMAResourcesScheduler settings

In the **NUMAResourcesScheduler** CR, configure the following parameters:

- Set **cacheResyncPeriod** to control how often the scheduler refreshes its cache from the exported topology data. This value must be longer than **infoRefreshPeriod**. For example, set **cacheResyncPeriod** to 10 seconds.
- Choose a QoS profile that matches your operational requirements:
- **Burstable** QoS (default): The recommended setting from OpenShift Container Platform 4.22. Minimizes baseline resource usage while allowing the scheduler to scale up during peak loads.
- **Guaranteed** QoS (opt-in): Ensures the scheduler pod is protected from eviction. Use this profile for clusters where control plane memory is frequently overcommitted or under pressure. Apply the Guaranteed QoS profile by annotating the **NUMAResourcesScheduler** custom resource with the following command:

```
$ oc patch numaresourcesscheduler numaresourcesscheduler --type='merge'
-p '{"metadata":{"annotations":{"config.numa-operator.openshift.io/scheduler-qos-request":"guaranteed"}}}'
```



NOTE

This option is expected to be removed in a later OpenShift Container Platform release.

12.9.1.3. Best practices for large clusters

When you deploy the NUMA-aware scheduler in clusters with 200 or more nodes:

- Enable HA mode to distribute replicas across control plane nodes.
- Use **oc adm top pod** in the **openshift-numaresources** namespace to monitor actual resource consumption.
- Align your refresh intervals so the exporter provides updates faster than the scheduler cache consumes them.

12.10. TROUBLESHOOTING NUMA-AWARE SCHEDULING

To resolve common problems with NUMA-aware pod scheduling, troubleshoot your cluster configuration. Identifying and fixing these issues ensures that your pods are optimally aligned with underlying hardware for high-performance workloads.

Prerequisites

- Installed the OpenShift CLI (**oc**).
- Logged in as a user with cluster-admin privileges.
- Installed the NUMA Resources Operator and deploy the NUMA-aware secondary scheduler.

Procedure

1. Verify that the **noderesourcetopologies** CRD is deployed in the cluster by running the following command:

```
$ oc get crd | grep noderesourcetopologies
```

Example output

NAME	CREATED AT
noderesourcetopologies.topology.node.k8s.io	2022-01-18T08:28:06Z

2. Check that the NUMA-aware scheduler name matches the name specified in your NUMA-aware workloads by running the following command:

```
$ oc get numaresourcesschedulers.nodeTopology.openshift.io numaresourcesscheduler -o json | jq '.status.schedulerName'
```

Example output

```
topo-aware-scheduler
```

3. Verify that NUMA-aware schedulable nodes have the **noderesourcetopologies** CR applied to them. Run the following command:

```
$ oc get noderesourcetopologies.topology.node.k8s.io
```

Example output

NAME	AGE
compute-0.example.com	17h
compute-1.example.com	17h



NOTE

The number of nodes should equal the number of worker nodes that are configured by the machine config pool (**mcp**) worker definition.

4. Verify the NUMA zone granularity for all schedulable nodes by running the following command:

```
$ oc get noderesourcetopologies.topology.node.k8s.io -o yaml
```

Example output

```

apiVersion: v1
items:
- apiVersion: topology.node.k8s.io/v1
  kind: NodeResourceTopology
  metadata:
    annotations:
      k8stopoawareschedwg/rte-update: periodic
    creationTimestamp: "2022-06-16T08:55:38Z"
    generation: 63760
    name: worker-0
    resourceVersion: "8450223"
    uid: 8b77be46-08c0-4074-927b-d49361471590
  topologyPolicies:
    - SingleNUMANodeContainerLevel
  zones:
    - costs:
        - name: node-0
          value: 10
        - name: node-1
          value: 21
      name: node-0
      resources:
        - allocatable: "38"
          available: "38"
          capacity: "40"
          name: cpu
        - allocatable: "134217728"
          available: "134217728"
          capacity: "134217728"
          name: hugepages-2Mi
        - allocatable: "262352048128"
          available: "262352048128"
          capacity: "270107316224"
          name: memory
        - allocatable: "6442450944"
          available: "6442450944"
          capacity: "6442450944"
          name: hugepages-1Gi
      type: Node
    - costs:
        - name: node-0
          value: 21
        - name: node-1
          value: 10
      name: node-1
      resources:
        - allocatable: "268435456"
          available: "268435456"
          capacity: "268435456"
          name: hugepages-2Mi
        - allocatable: "269231067136"
          available: "269231067136"
          capacity: "270573244416"
          name: memory
        - allocatable: "40"
          available: "40"

```

```

    capacity: "40"
    name: cpu
  - allocatable: "1073741824"
    available: "1073741824"
    capacity: "1073741824"
    name: hugepages-1Gi
    type: Node
- apiVersion: topology.node.k8s.io/v1
  kind: NodeResourceTopology
  metadata:
    annotations:
      k8stopoawareschedwg/rte-update: periodic
    creationTimestamp: "2022-06-16T08:55:37Z"
    generation: 62061
    name: worker-1
    resourceVersion: "8450129"
    uid: e8659390-6f8d-4e67-9a51-1ea34bba1cc3
  topologyPolicies:
  - SingleNUMANodeContainerLevel
  zones:
  - costs:
    - name: node-0
      value: 10
    - name: node-1
      value: 21
    name: node-0
  resources:
  - allocatable: "38"
    available: "38"
    capacity: "40"
    name: cpu
  - allocatable: "6442450944"
    available: "6442450944"
    capacity: "6442450944"
    name: hugepages-1Gi
  - allocatable: "134217728"
    available: "134217728"
    capacity: "134217728"
    name: hugepages-2Mi
  - allocatable: "262391033856"
    available: "262391033856"
    capacity: "270146301952"
    name: memory
  type: Node
- costs:
  - name: node-0
    value: 21
  - name: node-1
    value: 10
  name: node-1
  resources:
  - allocatable: "40"
    available: "40"
    capacity: "40"
    name: cpu
  - allocatable: "1073741824"

```

```

    available: "1073741824"
    capacity: "1073741824"
    name: hugepages-1Gi
  - allocatable: "268435456"
    available: "268435456"
    capacity: "268435456"
    name: hugepages-2Mi
  - allocatable: "269192085504"
    available: "269192085504"
    capacity: "270534262784"
    name: memory
  type: Node
kind: List
metadata:
  resourceVersion: ""
  selfLink: ""
# ...

```

- **zones:** Each stanza under **zones** describes the resources for a single NUMA zone.
- **costs.resources:** Specifies the current state of the NUMA zone resources. Check that resources listed under **items.zones.resources.available** correspond to the exclusive NUMA zone resources allocated to each guaranteed pod.

12.10.1. Reporting more exact resource availability

To report more exact resource availability and minimize Topology Affinity Errors, enable the **cacheResyncPeriod** specification for the NUMA Resources Operator. This configuration monitors pending resources on nodes and synchronizes them in the scheduler cache, though lower intervals increase network load.

The lower the interval, the greater the network load. The **cacheResyncPeriod** specification is disabled by default.

Prerequisites

- Installed the OpenShift CLI (**oc**).
- You are logged in as a user with **cluster-admin** privileges.

Procedure

1. Delete the currently running **NUMAResourcesScheduler** resource:
 - a. Get the active **NUMAResourcesScheduler** by running the following command:

```
$ oc get NUMAResourcesScheduler
```

Example output

```

NAME                AGE
numaresourcesscheduler 92m

```

- b. Delete the secondary scheduler resource by running the following command:

```
$ oc delete NUMAResourcesScheduler numaresourcesscheduler
```

Example output

```
numaresourcesscheduler.nodetopology.openshift.io "numaresourcesscheduler" deleted
```

2. Save the following YAML in the file **nro-scheduler-cacheresync.yaml**. This example changes the log level to **Debug**:

```
apiVersion: nodetopology.openshift.io/v1
kind: NUMAResourcesScheduler
metadata:
  name: numaresourcesscheduler
spec:
  imageSpec: "registry.redhat.io/openshift4/noderesourcetopology-scheduler-container-
rhel8:v4.22"
  cacheResyncPeriod: "5s"
```

- **spec.cacheResyncPeriod**: Enter an interval value in seconds for synchronization of the scheduler cache. A value of **5s** is typical for most implementations.

3. Create the updated **NUMAResourcesScheduler** resource by running the following command:

```
$ oc create -f nro-scheduler-cacheresync.yaml
```

Example output

```
numaresourcesscheduler.nodetopology.openshift.io/numaresourcesscheduler created
```

Verification

1. Check that the NUMA-aware scheduler was successfully deployed:
 - a. Run the following command to check that the CRD is created successfully:

```
$ oc get crd | grep numaresourcesschedulers
```

Example output

NAME	CREATED AT
numaresourcesschedulers.nodetopology.openshift.io	2022-02-25T11:57:03Z

- b. Check that the new custom scheduler is available by running the following command:

```
$ oc get numaresourcesschedulers.nodetopology.openshift.io
```

Example output

NAME	AGE
numaresourcesscheduler	3h26m

2. Check that the logs for the scheduler show the increased log level:

- a. Get the list of pods running in the **openshift-numaresources** namespace by running the following command:

```
$ oc get pods -n openshift-numaresources
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
numaresources-controller-manager-d87d79587-76mrm	1/1	Running	0	46h
numaresourcesoperator-worker-5wm2k	2/2	Running	0	45h
numaresourcesoperator-worker-pb75c	2/2	Running	0	45h
secondary-scheduler-7976c4d466-qm4sc	1/1	Running	0	21m

- b. Get the logs for the secondary scheduler pod by running the following command:

```
$ oc logs secondary-scheduler-7976c4d466-qm4sc -n openshift-numaresources
```

Example output

```
...
I0223 11:04:55.614788    1 reflector.go:535] k8s.io/client-go/informers/factory.go:134:
Watch close - *v1.Namespace total 11 items received
I0223 11:04:56.609114    1 reflector.go:535] k8s.io/client-go/informers/factory.go:134:
Watch close - *v1.ReplicationController total 10 items received
I0223 11:05:22.626818    1 reflector.go:535] k8s.io/client-go/informers/factory.go:134:
Watch close - *v1.StorageClass total 7 items received
I0223 11:05:31.610356    1 reflector.go:535] k8s.io/client-go/informers/factory.go:134:
Watch close - *v1.PodDisruptionBudget total 7 items received
I0223 11:05:31.713032    1 eventhandlers.go:186] "Add event for scheduled pod"
pod="openshift-marketplace/certified-operators-thtvq"
I0223 11:05:53.461016    1 eventhandlers.go:244] "Delete event for scheduled pod"
pod="openshift-marketplace/certified-operators-thtvq"
```

12.10.2. Changing where high-performance workloads run

To optimize the processing of high-performance workloads, change the default placement behavior of the NUMA-aware secondary scheduler. With this configuration, you can assign workloads to a specific NUMA node within a compute node instead of relying on default resource availability.

If you want to change where the workloads run, you can add the **scoringStrategy** setting to the **NUMAResourcesScheduler** custom resource and set its value to either **MostAllocated** or **BalancedAllocation**.

Prerequisites

- Installed the OpenShift CLI (**oc**).
- Logged in as a user with **cluster-admin** privileges.

Procedure

1. Delete the currently running **NUMAResourcesScheduler** resource by using the following steps:

- a. Get the active **NUMAResourcesScheduler** by running the following command:

```
$ oc get NUMAResourcesScheduler
```

Example output

```
NAME                AGE
numaresourcesscheduler 92m
```

- b. Delete the secondary scheduler resource by running the following command:

```
$ oc delete NUMAResourcesScheduler numaresourcesscheduler
```

Example output

```
numaresourcesscheduler.nodetopology.openshift.io "numaresourcesscheduler" deleted
```

2. Save the following YAML in the file **nro-scheduler-mostallocated.yaml**. This example changes the **scoringStrategy** to **MostAllocated**:

```
apiVersion: nodetopology.openshift.io/v1
kind: NUMAResourcesScheduler
metadata:
  name: numaresourcesscheduler
spec:
  imageSpec: "registry.redhat.io/openshift4/noderesourcetopology-scheduler-container-
rhel8:v{product-version}"
  scoringStrategy:
    type: "MostAllocated"
# ...
```

spec.imageSpec.scoringStrategy: If the **scoringStrategy** configuration is omitted, the default of **LeastAllocated** applies.

3. Create the updated **NUMAResourcesScheduler** resource by running the following command:

```
$ oc create -f nro-scheduler-mostallocated.yaml
```

Example output

```
numaresourcesscheduler.nodetopology.openshift.io/numaresourcesscheduler created
```

Verification

1. Check that the NUMA-aware scheduler was successfully deployed by using the following steps:

- a. Run the following command to check that the custom resource definition (CRD) is created successfully:

```
$ oc get crd | grep numaresourcesschedulers
```

Example output

NAME	CREATED AT
numaresourcesschedulers.nodetopology.openshift.io	2022-02-25T11:57:03Z

- b. Check that the new custom scheduler is available by running the following command:

```
$ oc get numaresourcesschedulers.nodetopology.openshift.io
```

Example output

NAME	AGE
numaresourcesscheduler	3h26m

2. Verify that the **ScoringStrategy** has been applied correctly by running the following command to check the relevant **ConfigMap** resource for the scheduler:

```
$ oc get -n openshift-numaresources cm topo-aware-scheduler-config -o yaml | grep scoring -A 1
```

Example output

```
scoringStrategy:
  type: MostAllocated
```

12.10.3. Checking the NUMA-aware scheduler logs

To troubleshoot problems with the NUMA-aware scheduler, review the scheduler logs. If necessary, increase the log level in the **NUMAResourcesScheduler** custom resource (CR) to capture more detailed diagnostic data.

Acceptable values are **Normal**, **Debug**, and **Trace**, with **Trace** being the most verbose option.

**NOTE**

To change the log level of the secondary scheduler, delete the running scheduler resource and re-deploy it with the changed log level. The scheduler is unavailable for scheduling new workloads during this downtime.

Prerequisites

- Installed the OpenShift CLI (**oc**).
- You are logged in as a user with **cluster-admin** privileges.

Procedure

1. Delete the currently running **NUMAResourcesScheduler** resource:
 - a. Get the active **NUMAResourcesScheduler** by running the following command:

```
$ oc get NUMAResourcesScheduler
```

-

Example output

```
NAME                      AGE
numaresourcesscheduler  90m
```

- b. Delete the secondary scheduler resource by running the following command:

```
$ oc delete NUMAResourcesScheduler numaresourcesscheduler
```

Example output

```
numaresourcesscheduler.nodetopology.openshift.io "numaresourcesscheduler" deleted
```

2. Save the following YAML in the file **nro-scheduler-debug.yaml**. This example changes the log level to **Debug**:

```
apiVersion: nodetopology.openshift.io/v1
kind: NUMAResourcesScheduler
metadata:
  name: numaresourcesscheduler
spec:
  imageSpec: "registry.redhat.io/openshift4/noderesourcetopology-scheduler-container-
rhel8:v4.22"
  logLevel: Debug
# ...
```

3. Create the updated **Debug** logging **NUMAResourcesScheduler** resource by running the following command:

```
$ oc create -f nro-scheduler-debug.yaml
```

Example output

```
numaresourcesscheduler.nodetopology.openshift.io/numaresourcesscheduler created
```

Verification

1. Check that the NUMA-aware scheduler was successfully deployed:
 - a. Run the following command to check that the CRD is created successfully:

```
$ oc get crd | grep numaresourcesschedulers
```

Example output

```
NAME                                                                CREATED AT
numaresourcesschedulers.nodetopology.openshift.io                 2022-02-25T11:57:03Z
```

- b. Check that the new custom scheduler is available by running the following command:

```
$ oc get numaresourcesschedulers.nodetopology.openshift.io
```

Example output

```
NAME                AGE
numaresourcesscheduler 3h26m
```

2. Check that the logs for the scheduler shows the increased log level:

- a. Get the list of pods running in the **openshift-numaresources** namespace by running the following command:

```
$ oc get pods -n openshift-numaresources
```

Example output

```
NAME                                READY STATUS  RESTARTS  AGE
numaresources-controller-manager-d87d79587-76mrm 1/1   Running  0         46h
numaresourcesoperator-worker-5wm2k              2/2   Running  0         45h
numaresourcesoperator-worker-pb75c              2/2   Running  0         45h
secondary-scheduler-7976c4d466-qm4sc            1/1   Running  0         21m
```

- b. Get the logs for the secondary scheduler pod by running the following command:

```
$ oc logs secondary-scheduler-7976c4d466-qm4sc -n openshift-numaresources
```

Example output

```
...
I0223 11:04:55.614788    1 reflector.go:535] k8s.io/client-go/informers/factory.go:134:
Watch close - *v1.Namespace total 11 items received
I0223 11:04:56.609114    1 reflector.go:535] k8s.io/client-go/informers/factory.go:134:
Watch close - *v1.ReplicationController total 10 items received
I0223 11:05:22.626818    1 reflector.go:535] k8s.io/client-go/informers/factory.go:134:
Watch close - *v1.StorageClass total 7 items received
I0223 11:05:31.610356    1 reflector.go:535] k8s.io/client-go/informers/factory.go:134:
Watch close - *v1.PodDisruptionBudget total 7 items received
I0223 11:05:31.713032    1 eventhandlers.go:186] "Add event for scheduled pod"
pod="openshift-marketplace/certified-operators-thtvq"
I0223 11:05:53.461016    1 eventhandlers.go:244] "Delete event for scheduled pod"
pod="openshift-marketplace/certified-operators-thtvq"
```

12.10.4. Troubleshooting the resource topology exporter

To resolve unexpected results in **noderesourcetopologies** objects, inspect the **resource-topology-exporter** logs. Reviewing this diagnostic data helps you identify and fix configuration issues within your cluster.



NOTE

Ensure that the NUMA resource topology exporter instances in the cluster are named for nodes they refer to. For example, a compute node with the name **worker** should have a corresponding **noderesourcetopologies** object called **worker**.

Prerequisites

- Install the OpenShift CLI (**oc**).
- Log in as a user with **cluster-admin** privileges.

Procedure

1. Get the daemonsets managed by the NUMA Resources Operator. Each daemonset has a corresponding **nodeGroup** in the **NUMAResourcesOperator** CR. Run the following command:

```
$ oc get numaresourcesoperators.nodetopology.openshift.io numaresourcesoperator -o jsonpath="{.status.daemonsets[0]}"
```

Example output

```
{"name":"numaresourcesoperator-worker","namespace":"openshift-numaresources"}
```

2. Get the label for the daemonset of interest using the value for **name** from the previous step:

```
$ oc get ds -n openshift-numaresources numaresourcesoperator-worker -o jsonpath="{.spec.selector.matchLabels}"
```

Example output

```
{"name":"resource-topology"}
```

3. Get the pods using the **resource-topology** label by running the following command:

```
$ oc get pods -n openshift-numaresources -l name=resource-topology -o wide
```

Example output

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
numaresourcesoperator-worker-5wm2k	2/2	Running	0	2d1h	10.135.0.64	compute-0.example.com
numaresourcesoperator-worker-pb75c	2/2	Running	0	2d1h	10.132.2.33	compute-1.example.com

4. Examine the logs of the **resource-topology-exporter** container running on the worker pod that corresponds to the node you are troubleshooting. Run the following command:

```
$ oc logs -n openshift-numaresources -c resource-topology-exporter numaresourcesoperator-worker-pb75c
```

Example output

```
I0221 13:38:18.334140    1 main.go:206] using sysinfo:
reservedCpus: 0,1
reservedMemory:
  "0": 1178599424
I0221 13:38:18.334370    1 main.go:67] === System information ===
```

```

I0221 13:38:18.334381    1 sysinfo.go:231] cpus: reserved "0-1"
I0221 13:38:18.334493    1 sysinfo.go:237] cpus: online "0-103"
I0221 13:38:18.546750    1 main.go:72]
cpus: allocatable "2-103"
hugepages-1Gi:
  numa cell 0 -> 6
  numa cell 1 -> 1
hugepages-2Mi:
  numa cell 0 -> 64
  numa cell 1 -> 128
memory:
  numa cell 0 -> 45758Mi
  numa cell 1 -> 48372Mi

```

12.10.5. Correcting a missing resource topology exporter config map

To correct a missing config map for the resource topology exporter (RTE), resolve misconfigured settings in your cluster. Fixing this issue ensures the NUMA Resources Operator functions properly when the logs of the RTE daemon set pods indicate missing configurations.

The following example log message indicates a missing configuration:

```
Info: couldn't find configuration in "/etc/resource-topology-exporter/config.yaml"
```

The previous log message indicates that the **kubeletconfig** with the required configuration was not properly applied in the cluster, resulting in a missing RTE **configmap**. For example, the following cluster is missing a **numaresourcesoperator-worker configmap** custom resource (CR):

```
$ oc get configmap
```

Example output:

NAME	DATA	AGE
0e2a6bd3.openshift-kni.io	0	6d21h
kube-root-ca.crt	1	6d21h
openshift-service-ca.crt	1	6d21h
topo-aware-scheduler-config	1	6d18h

In a correctly configured cluster, **oc get configmap** also returns a **numaresourcesoperator-worker configmap** CR.

Prerequisites

- Installed the OpenShift CLI (**oc**).
- Logged in as a user with cluster-admin privileges.
- Installed the NUMA Resources Operator and deploy the NUMA-aware secondary scheduler.

Procedure

1. Compare the values for **spec.machineConfigPoolSelector.matchLabels** in **kubeletconfig** and **metadata.labels** in the **MachineConfigPool (mcp)** worker CR using the following commands:

- a. Check the **kubeletconfig** labels by running the following command:

```
$ oc get kubeletconfig -o yaml
```

Example output

```
machineConfigPoolSelector:
  matchLabels:
    cnf-worker-tuning: enabled
```

- b. Check the **mcp** labels by running the following command:

```
$ oc get mcp worker -o yaml
```

Example output

```
labels:
  machineconfiguration.openshift.io/mco-built-in: ""
  pools.operator.machineconfiguration.openshift.io/worker: ""
```

The **cnf-worker-tuning: enabled** label is not present in the **MachineConfigPool** object.

2. Edit the **MachineConfigPool** CR to include the missing label, for example:

```
$ oc edit mcp worker -o yaml
```

Example output

```
labels:
  machineconfiguration.openshift.io/mco-built-in: ""
  pools.operator.machineconfiguration.openshift.io/worker: ""
  cnf-worker-tuning: enabled
```

3. Apply the label changes and wait for the cluster to apply the updated configuration.

Verification

- Check that the missing **numaresourcesoperator-worker configmap** CR is applied:

```
$ oc get configmap
```

Example output

NAME	DATA	AGE
0e2a6bd3.openshift-kni.io	0	6d21h
kube-root-ca.crt	1	6d21h
numaresourcesoperator-worker	1	5m
openshift-service-ca.crt	1	6d21h
topo-aware-scheduler-config	1	6d18h

12.10.6. Collecting NUMA Resources Operator data

You can use the **oc adm must-gather** CLI command to collect information about your cluster, including features and objects associated with the NUMA Resources Operator.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed the OpenShift CLI (**oc**).

Procedure

- To collect NUMA Resources Operator data with **must-gather**, you must specify the NUMA Resources Operator **must-gather** image.

```
$ oc adm must-gather --image=registry.redhat.io/openshift4/numaresources-must-gather-rhel9:v4.22
```

CHAPTER 13. SCALABILITY AND PERFORMANCE OPTIMIZATION

13.1. OPTIMIZING STORAGE

Optimizing storage helps to minimize storage use across all resources. As an administrator, you can optimize storage to ensure that existing storage resources are working in an efficient manner.

13.1.1. Available persistent storage options

To optimize your OpenShift Container Platform environment, review the available persistent storage options. By understanding these choices, you can select the appropriate storage configuration to meet your specific workload requirements.

Table 13.1. Available storage options

Storage type	Description	Examples
Block	- Presented to the operating system (OS) as a block device - Suitable for applications that need full control of storage and operate at a low level on files bypassing the file system. - Also referred to as a Storage Area Network (SAN). - Non-shareable, which means that only one client at a time can mount an endpoint of this type.	AWS EBS and VMware vSphere support dynamic persistent volume (PV) provisioning natively in OpenShift Container Platform.
File	- Presented to the OS as a file system export to be mounted - Also referred to as Network Attached Storage (NAS). - Concurrency, latency, file locking mechanisms, and other capabilities vary widely between protocols, implementations, vendors, and scales.	RHEL NFS, NetApp NFS, and Vendor NFS.
Object	- Accessible through a REST API endpoint. - Configurable for use in the OpenShift image registry - Applications must build their drivers into the application and/or container.	AWS S3.

- **File:** NetApp NFS supports dynamic PV provisioning when using the Trident plugin.

13.1.2. Recommended configurable storage technology

Review the recommended and configurable storage technologies for the given OpenShift Container Platform cluster application.

Table 13.2. Recommended and configurable storage technology

Storage type	Block	File	Object
ROX	Yes	Yes	Yes
RWX	No	Yes	Yes
Registry	Configurable	Configurable	Recommended
Scaled registry	Not configurable	Configurable	Recommended
Metrics	Recommended	Configurable	Not configurable
Elasticsearch Logging	Recommended	Configurable	Not supported
Loki Logging	Not configurable	Not configurable	Recommended
Apps	Recommended	Recommended	Not configurable

where:

ROX

Specifies **ReadOnlyMany** access mode.

ROX.Yes

Specifies that this access mode

RWX

Specifies **ReadWriteMany** access mode.

Metrics

Specifies Prometheus as the underlying technology used for metrics.

Metrics.Configurable

For metrics, using file storage with the **ReadWriteMany** (RWX) access mode is unreliable. If you use file storage, do not configure the RWX access mode on any persistent volume claims (PVCs) that are configured for use with metrics.

Elasticsearch Logging.Configurable

For logging, review the recommended storage solution in Configuring persistent storage for the log store section. Using NFS storage as a persistent volume or through NAS, such as Gluster, can corrupt the data. Therefore, NFS is not supported for Elasticsearch storage and LokiStack log store in OpenShift Container Platform Logging. You must use one persistent volume type per log store.

Apps.Not configurable

Specifies that object storage is not consumed through PVs or PVCs of OpenShift Container Platform. Apps must integrate with the object storage REST API.

**NOTE**

A scaled registry is an OpenShift image registry where two or more pod replicas are running.

13.1.2.1. Specific application storage recommendations

Review the specific storage recommendations for registries, scaled registries, metrics, logs, and applications to better understand the storage requirements for each of these entities.

**IMPORTANT**

Testing shows issues with using the NFS server on Red Hat Enterprise Linux (RHEL) as a storage backend for core services. This includes the OpenShift Container Registry and Quay, Prometheus for monitoring storage, and Elasticsearch for logging storage. Therefore, using RHEL NFS to back PVs used by core services is not recommended.

Other NFS implementations in the marketplace might not have these issues. Contact the individual NFS implementation vendor for more information on any testing that was possibly completed against these OpenShift Container Platform core components.

Registry

In a non-scaled/high-availability (HA) OpenShift image registry cluster deployment:

- The storage technology does not have to support RWX access mode.
- The storage technology must ensure read-after-write consistency.
- The preferred storage technology is object storage followed by block storage.
- File storage is not recommended for OpenShift image registry cluster deployment with production workloads.

Scaled registry

In a scaled/HA OpenShift image registry cluster deployment:

- The storage technology must support RWX access mode.
- The storage technology must ensure read-after-write consistency.
- The preferred storage technology is object storage.
- Red Hat OpenShift Data Foundation, Amazon Simple Storage Service (Amazon S3), Google Cloud Storage (GCS), Microsoft Azure Blob Storage, and OpenStack Swift are supported.
- Object storage should be S3 or Swift compliant.
- For non-cloud platforms, such as vSphere and bare-metal installations, the only configurable technology is file storage.
- Block storage is not configurable.

- The use of Network File System (NFS) storage with OpenShift Container Platform is supported. However, the use of NFS storage with a scaled registry can cause known issues. For more information, see the "Is NFS supported for OpenShift cluster internal components in Production?" Red Hat Knowledgebase solution.

Metrics

In an OpenShift Container Platform hosted metrics cluster deployment:

- The preferred storage technology is block storage.
- Object storage is not configurable.



IMPORTANT

It is not recommended to use file storage for a hosted metrics cluster deployment with production workloads.

Logging

In an OpenShift Container Platform hosted logging cluster deployment:

- Loki Operator:
 - The preferred storage technology is S3 compatible Object storage.
 - Block storage is not configurable.
- OpenShift Elasticsearch Operator:
 - The preferred storage technology is block storage.
 - Object storage is not supported.



NOTE

As of logging version 5.4.3 the OpenShift Elasticsearch Operator is deprecated and is planned to be removed in a future release. Red Hat will provide bug fixes and support for this feature during the current release lifecycle, but this feature will no longer receive enhancements and will be removed. As an alternative to using the OpenShift Elasticsearch Operator to manage the default log storage, you can use the Loki Operator.

Applications

Application use cases vary from application to application, as described in the following examples:

- Storage technologies that support dynamic PV provisioning have low mount time latencies, and are not tied to nodes to support a healthy cluster.
- Application developers are responsible for knowing and understanding the storage requirements for their application, and how it works with the provided storage to ensure that issues do not occur when an application scales or interacts with the storage layer.

Other specific application storage recommendations



IMPORTANT

Red Hat does not recommend using RAID configurations on **Write** intensive workloads, such as **etcd**. If you are running **etcd** with a RAID configuration, you might be at risk of encountering performance issues with your workloads.

- Red Hat OpenStack Platform (RHOSP) Cinder: RHOSP Cinder tends to be adept at ROX access mode use cases.
- Databases: Databases (RDBMSs, NoSQL DBs, etc.) tend to perform best with dedicated block storage.
- The etcd database must have enough storage and adequate performance capacity to enable a large cluster. Information about monitoring and benchmarking tools to establish ample storage and a high-performance environment is described in *Recommended etcd practices*.

13.1.3. Additional resources

- [Is NFS supported for OpenShift cluster internal components in Production?](#)

13.1.4. Data storage management

To effectively manage data storage in OpenShift Container Platform, review the main directories where components write data.

The following table summarizes the main directories that OpenShift Container Platform components write data to.

Table 13.3. Main directories for storing OpenShift Container Platform data

Directory	Notes	Sizing	Expected growth
<code>/var/lib/etcd</code>	Used for etcd storage when storing the database.	Less than 20 GB. Database can grow up to 8 GB.	Will grow slowly with the environment. Only storing metadata. Additional 20-25 GB for every additional 8 GB of memory.
<code>/var/lib/containers</code>	This is the mount point for the CRI-O runtime. Storage used for active container runtimes, including pods, and storage of local images. Not used for registry storage.	50 GB for a node with 16 GB memory. Note that this sizing should not be used to determine minimum cluster requirements. Additional 20-25 GB for every additional 8 GB of memory.	Growth is limited by capacity for running containers.

Directory	Notes	Sizing	Expected growth
<i>/var/lib/kubelet</i>	Ephemeral volume storage for pods. This includes anything external that is mounted into a container at runtime. Includes environment variables, kube secrets, and data volumes not backed by persistent volumes.	Varies	Minimal if pods requiring storage are using persistent volumes. If using ephemeral storage, this can grow quickly.
<i>/var/log</i>	Log files for all components.	10 to 30 GB.	Log files can grow quickly; size can be managed by growing disks or by using log rotate.

13.1.5. Optimizing storage performance for Microsoft Azure

To ensure optimal cluster performance on Microsoft Azure, configure faster storage for OpenShift Container Platform and Kubernetes. Red Hat recommends faster storage, particularly for etcd on the control plane nodes.

For production Azure clusters and clusters with intensive workloads, the virtual machine operating system disk for control plane machines should be able to sustain a tested and recommended minimum throughput of 5000 IOPS / 200 MBps. This throughput can be provided by having a minimum of 1 TiB Premium SSD (P30). In Azure and Azure Stack Hub, disk performance is directly dependent on SSD disk sizes. To achieve the throughput supported by a **Standard_D8s_v3** virtual machine, or other similar machine types, and the target of 5000 IOPS, at least a P30 disk is required.

Host caching must be set to **ReadOnly** for low latency and high IOPS and throughput when reading data. Reading data from the cache, which is present either in the VM memory or in the local SSD disk, is much faster than reading from the disk, which is in the blob storage.

13.2. OPTIMIZING ROUTING

You can scale or configure the OpenShift Container Platform HAProxy router to optimize routing performance.

13.2.1. Baseline Ingress Controller (router) performance

You can use the OpenShift Container Platform Ingress Controller, or router, as the ingress point for ingress traffic for applications and services that are configured by using routes and ingresses.

When evaluating a single HAProxy router performance in terms of HTTP requests handled per second, the performance varies depending on many factors. In particular:

- HTTP keep-alive/close mode
- Route type

- TLS session resumption client support
- Number of concurrent connections per target route
- Number of target routes
- Back end server page size
- Underlying infrastructure (network, CPU, and so on)

While performance in your specific environment will vary, Red Hat lab tests on a public cloud instance of size 4 vCPU/16GB RAM. A single HAProxy router handling 100 routes terminated by backends serving 1kB static pages is able to handle the following number of transactions per second.

In HTTP keep-alive mode scenarios:

Encryption	LoadBalancerService	HostNetwork
none	21515	29622
edge	16743	22913
passthrough	36786	53295
re-encrypt	21583	25198

In HTTP close (no keep-alive) scenarios:

Encryption	LoadBalancerService	HostNetwork
none	5719	8273
edge	2729	4069
passthrough	4121	5344
re-encrypt	2320	2941

The default Ingress Controller configuration was used with the **spec.tuningOptions.threadCount** field set to **4**. Two different endpoint publishing strategies were tested: Load Balancer Service and Host Network. TLS session resumption was used for encrypted routes. With HTTP keep-alive, a single HAProxy router is capable of saturating a 1 Gbit NIC at page sizes as small as 8 kB.

When running on bare metal with modern processors, you can expect roughly twice the performance of the public cloud instance above. This overhead is introduced by the virtualization layer in place on public clouds and holds mostly true for private cloud-based virtualization as well. The following table is a guide to how many applications to use behind the router:

Number of applications	Application type
5-10	static file/web server or caching proxy
100-1000	applications generating dynamic content

In general, HAProxy can support routes for up to 1000 applications, depending on the technology in use. Ingress Controller performance might be limited by the capabilities and performance of the applications behind it, such as language or static versus dynamic content.

Ingress, or router, sharding should be used to serve more routes towards applications and help horizontally scale the routing tier.

13.2.2. Additional resources

- [Configuring Ingress Controller sharding by using route labels](#)
- [Configuring Ingress Controller sharding by using namespace labels](#)
- [Setting Ingress Controller thread count](#)
- [Ingress Controller configuration parameters](#)

13.2.3. Configuring Ingress Controller liveness, readiness, and startup probes

As a cluster administrator, you can configure the timeout values for the kubelet liveness, readiness, and startup probes for router deployments that are managed by the OpenShift Container Platform Ingress Controller (router). The ability to set larger timeout values can reduce the risk of unnecessary and unwanted restarts.

The liveness and readiness probes of the router use the default timeout value of 1 second, which is too brief when networking or runtime performance is severely degraded. Probe timeouts can cause unwanted router restarts that interrupt application connections.

You can update the **timeoutSeconds** value on the **livenessProbe**, **readinessProbe**, and **startupProbe** parameters of the router container.

Parameter	Description
livenessProbe	The livenessProbe reports to the kubelet whether a pod is dead and needs to be restarted.
readinessProbe	The readinessProbe reports whether a pod is healthy or unhealthy. When the readiness probe reports an unhealthy pod, then the kubelet marks the pod as not ready to accept traffic. Subsequently, the endpoints for that pod are marked as not ready, and this status propagates to the kube-proxy. On cloud platforms with a configured load balancer, the kube-proxy communicates to the cloud load-balancer not to send traffic to the node with that pod.

Parameter	Description
startupProbe	The startupProbe gives the router pod up to 2 minutes to initialize before the kubelet begins sending the router liveness and readiness probes. This initialization time can prevent routers with many routes or endpoints from prematurely restarting.



IMPORTANT

The timeout configuration option is an advanced tuning technique that can be used to work around issues. However, these issues should eventually be diagnosed and possibly a support case or [Jira issue](#) opened for any issues that cause probes to time out.

The following example demonstrates how you can directly patch the default router deployment to set a 5-second timeout for the liveness and readiness probes:

```
$ oc -n openshift-ingress patch deploy/router-default --type=strategic --patch='{"spec":{"template":{"spec":{"containers":[{"name":"router","livenessProbe":{"timeoutSeconds":5},"readinessProbe":{"timeoutSeconds":5}]}}}}'
```

Verification

```
$ oc -n openshift-ingress describe deploy/router-default | grep -e Liveness: -e Readiness:
Liveness: http-get http://:1936/healthz delay=0s timeout=5s period=10s #success=1 #failure=3
Readiness: http-get http://:1936/healthz/ready delay=0s timeout=5s period=10s #success=1 #failure=3
```

13.2.4. Configuring HAProxy reload interval

You can configure the HAProxy reload interval, so that when HAProxy reloads it generates a new process that handles new connections by using the updated configuration.

When you update a route or an endpoint associated with a route, the OpenShift Container Platform router updates the configuration for HAProxy. The HAProxy then reloads the updated configuration for those changes to take effect.

HAProxy keeps the old process running to handle existing connections until those connections are all closed. When old processes have long-lived connections, these processes can accumulate and consume resources.

The default minimum HAProxy reload interval is 5 seconds. You can configure an Ingress Controller using its **spec.tuningOptions.reloadInterval** field to set a longer minimum reload interval.



WARNING

Setting a large value for the minimum HAProxy reload interval can cause latency in observing updates to routes and their endpoints. To lessen the risk, avoid setting a value larger than the tolerable latency for updates.

Procedure

- Change the minimum HAProxy reload interval of the default Ingress Controller to 15 seconds by running the following command:

```
$ oc -n openshift-ingress-operator patch ingresscontrollers/default --type=merge --
patch='{"spec":{"tuningOptions":{"reloadInterval":"15s"}}}'
```

13.3. OPTIMIZING NETWORKING

To tunnel traffic between nodes, use Generic Network Virtualization Encapsulation (Geneve). You can tune the network by using network interface controller (NIC) offloads.

Geneve provides benefits over VLANs, such as an increase in networks from 4096 to over 16 million, and layer 2 connectivity across physical networks. This allows for all pods behind a service to communicate with each other, even if they are running on different systems.

Cloud, virtual, and bare-metal environments running OpenShift Container Platform can use a high percentage of the capabilities of a network interface card (NIC) with minimal tuning. Production clusters using OVN-Kubernetes with Geneve tunneling can handle high-throughput traffic effectively and scale up (for example, utilizing 100 Gbps NICs) and scale out (for example, adding more NICs) without requiring special configuration.

In some high-performance scenarios where maximum efficiency is critical, targeted performance tuning can help optimize CPU usage, reduce overhead, and ensure that you are making full use of the NIC's capabilities.

For environments where maximum throughput and CPU efficiency are critical, you can further optimize performance with the following strategies:

- Validate network performance by using tools such as **iPerf3** and **k8s-netperf**. By using these tools, you can benchmark throughput, latency, and packets-per-second (PPS) across pod and node interfaces.
- Evaluate OVN-Kubernetes User Defined Networking (UDN) routing techniques, such as border gateway protocol (BGP).
- Use Geneve-offload capable network adapters. Geneve-offload moves the packet checksum calculation and associated CPU overhead off of the system CPU and onto dedicated hardware on the network adapter. This frees up CPU cycles for use by pods and applications, so that users can use the full bandwidth of their network infrastructure.

13.3.1. Additional resources

- [OVN-Kubernetes](#)

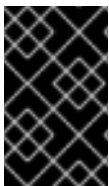
13.3.2. Optimizing the MTU for your network

You can optimize the MTU value of your network so that your network is optimized for throughput or low latency.

There are two important maximum transmission units (MTUs): the network interface controller (NIC) MTU and the cluster network MTU.

The NIC MTU is configured at the time of OpenShift Container Platform installation, and you can also change the MTU of a cluster as a postinstallation task. For more information, see "Changing cluster network MTU".

For a cluster that uses the OVN-Kubernetes plugin, the MTU must be at least **100** bytes less than the maximum supported value of the NIC of your network. If you are optimizing for throughput, choose the largest possible value, such as **8900**. If you are optimizing for lowest latency, choose a lower value.



IMPORTANT

If your cluster uses the OVN-Kubernetes plugin and the network uses a NIC to send and receive unfragmented jumbo frame packets over the network, you must specify **9000** bytes as the MTU value for the NIC so that pods do not fail.

Additional resources

- [Changing cluster network MTU](#)

13.3.3. Recommended practices for installing large-scale clusters

When installing large clusters or scaling the cluster to larger node counts, set the cluster network **cidr** accordingly in your **install-config.yaml** file before you install the cluster.

Example install-config.yaml file with a network configuration for a cluster with a large node count

```
apiVersion: v1
metadata:
  name: cluster-name
# ...
networking:
  clusterNetwork:
    - cidr: 10.128.0.0/14
      hostPrefix: 23
  machineNetwork:
    - cidr: 10.0.0.0/16
  networkType: OVNKubernetes
  serviceNetwork:
    - 172.30.0.0/16
# ...
```

- The default cluster network **cidr** **10.128.0.0/14** cannot be used if the cluster size is more than 500 nodes. The **cidr** must be set to **10.128.0.0/12** or **10.128.0.0/10** to support larger node counts beyond 500 nodes.

13.3.4. Impact of IPsec

Encrypting and decrypting node hosts uses CPU power so performance is affected both in throughput and CPU usage on the nodes when encryption is enabled, regardless of the IP security system being used. To account for performance overhead, review the impact of enabling IPsec.

IPSec encrypts traffic at the IP payload level, before it hits the NIC, protecting fields that would otherwise be used for NIC offloading. This means that some NIC acceleration features might not be usable when IPSec is enabled. This situation leads to decreased throughput and increased CPU usage.

13.3.5. Additional resources

- [Specifying advanced network configuration](#)
- [Cluster Network Operator configuration](#)
- [Improving cluster stability in high latency environments using worker latency profiles](#)

13.4. OPTIMIZING CPU USAGE WITH MOUNT NAMESPACE ENCAPSULATION

You can optimize CPU usage in OpenShift Container Platform clusters by using mount namespace encapsulation to provide a private namespace for kubelet and CRI-O processes. This reduces the cluster CPU resources used by systemd with no difference in functionality.



IMPORTANT

Mount namespace encapsulation is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

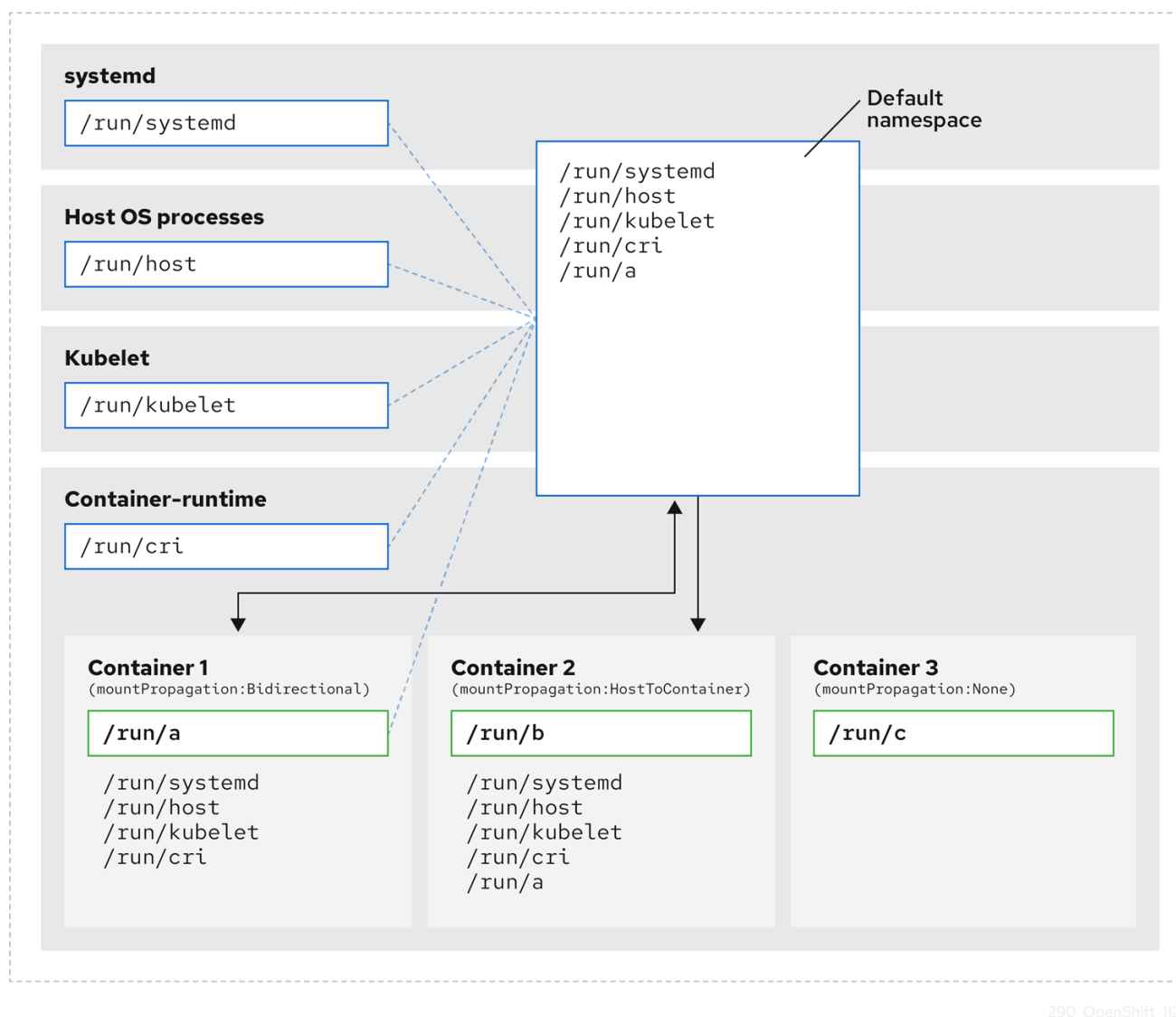
13.4.1. Encapsulating mount namespaces

You can encapsulate mount namespaces to isolate mount points so that processes in different namespaces cannot view each others' files. Encapsulation is the process of moving Kubernetes mount namespaces to an alternative location where they do not get constantly scanned by the host operating system.

The host operating system uses systemd to constantly scan all mount namespaces: both the standard Linux mounts and the numerous mounts that Kubernetes uses to operate. The current implementation of kubelet and CRI-O both use the top-level namespace for all container runtime and kubelet mount points. However, encapsulating these container-specific mount points in a private namespace reduces systemd overhead with no difference in functionality. Using a separate mount namespace for both CRI-O and kubelet can encapsulate container-specific mounts from any systemd or other host operating system interaction.

This ability to potentially achieve major CPU optimization is now available to all OpenShift Container Platform administrators. Encapsulation can also improve security by storing Kubernetes-specific mount points in a location safe from inspection by unprivileged users.

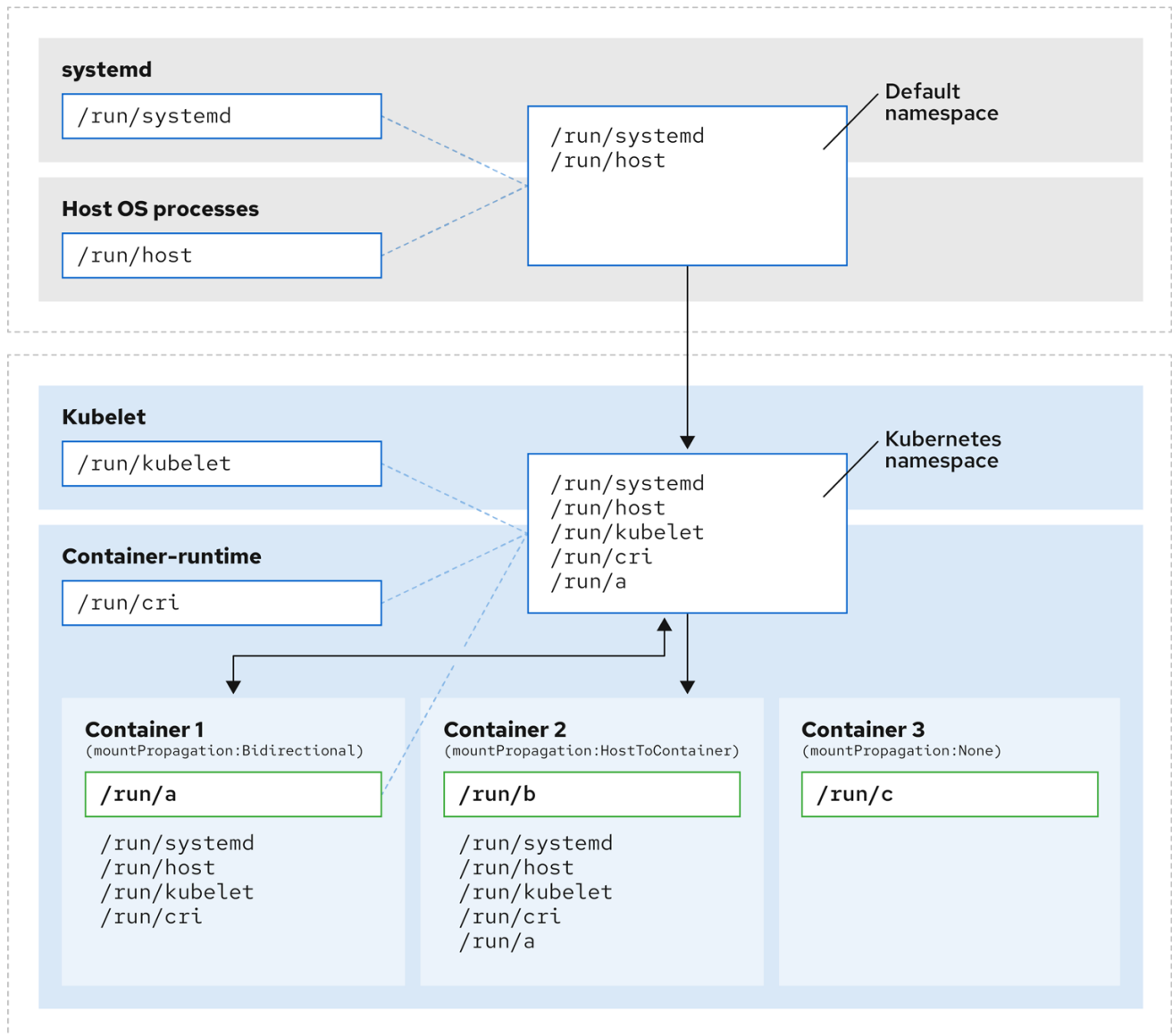
The following diagrams illustrate a Kubernetes installation before and after encapsulation. Both scenarios show example containers which have mount propagation settings of bidirectional, host-to-container, and none.



The diagram shows systemd, host operating system processes, kubelet, and the container runtime sharing a single mount namespace.

- systemd, host operating system processes, kubelet, and the container runtime each have access to and visibility of all mount points.
- Container 1, configured with bidirectional mount propagation, can access systemd and host mounts, kubelet and CRI-O mounts. A mount originating in Container 1, such as **/run/a** is visible to systemd, host operating system processes, kubelet, container runtime, and other containers with host-to-container or bidirectional mount propagation configured (as in Container 2).
- Container 2, configured with host-to-container mount propagation, can access systemd and host mounts, kubelet and CRI-O mounts. A mount originating in Container 2, such as **/run/b**, is not visible to any other context.
- Container 3, configured with no mount propagation, has no visibility of external mount points. A mount originating in Container 3, such as **/run/c**, is not visible to any other context.

The following diagram illustrates the system state after encapsulation.



290_OpenShift_1122

- The main systemd process is no longer devoted to unnecessary scanning of Kubernetes-specific mount points. It only monitors systemd-specific and host mount points.
- The host operating system processes can access only the systemd and host mount points.
- Using a separate mount namespace for both CRI-O and kubelet completely separates all container-specific mounts away from any systemd or other host operating system interaction whatsoever.
- The behavior of Container 1 is unchanged, except a mount it creates such as `/run/a` is no longer visible to systemd or host operating system processes. It is still visible to kubelet, CRI-O, and other containers with host-to-container or bidirectional mount propagation configured (like Container 2).
- The behavior of Container 2 and Container 3 is unchanged.

13.4.2. Configuring mount namespace encapsulation

You can configure mount namespace encapsulation so that a cluster runs with less resource overhead.



NOTE

Mount namespace encapsulation is a Technology Preview feature and the feature is disabled by default. To use the feature, you must enable the feature manually.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.

Procedure

1. Create a file called **mount_namespace_config.yaml** with the following YAML:

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: master
  name: 99-kubens-master
spec:
  config:
    ignition:
      version: 3.2.0
    systemd:
      units:
        - enabled: true
          name: kubens.service
  ---
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: worker
  name: 99-kubens-worker
spec:
  config:
    ignition:
      version: 3.2.0
    systemd:
      units:
        - enabled: true
          name: kubens.service
```

2. Apply the mount namespace **MachineConfig** CR by running the following command:

```
$ oc apply -f mount_namespace_config.yaml
```

Example output

```
machineconfig.machineconfiguration.openshift.io/99-kubens-master created
machineconfig.machineconfiguration.openshift.io/99-kubens-worker created
```


3. The **MachineConfig** CR can take up to thirty minutes to finish being applied in the cluster. You can check the status of the **MachineConfig** CR by running the following command:

```
$ oc get mcp
```

Example output

```
NAME      CONFIG                                UPDATED  UPDATING  DEGRADED
MACHINECOUNT READYMACHINECOUNT UPDATEDMACHINECOUNT
DEGRADEDMACHINECOUNT AGE
master    rendered-master-03d4bc4befb0f4ed3566a2c8f7636751  False    True      False
3         0         0         0         45m
worker    rendered-worker-10577f6ab0117ed1825f8af2ac687ddf  False    True      False
3         1         1
```

4. Wait for the **MachineConfig** CR to be applied successfully across all control plane and worker nodes after running the following command:

```
$ oc wait --for=condition=Updated mcp --all --timeout=30m
```

Example output

```
machineconfigpool.machineconfiguration.openshift.io/master condition met
machineconfigpool.machineconfiguration.openshift.io/worker condition met
```

Verification

1. Open a debug shell to the cluster host:

```
$ oc debug node/<node_name>
```

2. Open a **chroot** session:

```
sh-4.4# chroot /host
```

3. Check the systemd mount namespace:

```
sh-4.4# readlink /proc/1/ns/mnt
```

Example output

```
mnt:[4026531953]
```

4. Check kubelet mount namespace:

```
sh-4.4# readlink /proc/$(pgrep kubelet)/ns/mnt
```

Example output

```
mnt:[4026531840]
```

5. Check the CRI-O mount namespace:

```
sh-4.4# readlink /proc/$(pgrep crio)/ns/mnt
```

Example output

```
mnt:[4026531840]
```

These commands return the mount namespaces associated with `systemd`, `kubelet`, and the container runtime. In OpenShift Container Platform, the container runtime is CRI-O.

Encapsulation is in effect if `systemd` is in a different mount namespace from `kubelet` and CRI-O as in the previous output example. Encapsulation is not in effect if all three processes are in the same mount namespace.

13.4.3. Inspecting encapsulated namespaces

You can inspect Kubernetes-specific mount points in the cluster host operating system for debugging or auditing purposes by using the **kubensenter** script that is available in Red Hat Enterprise Linux CoreOS (RHCOS).

SSH shell sessions to the cluster host are in the default namespace. To inspect Kubernetes-specific mount points in an SSH shell prompt, you need to run the **kubensenter** script as root. The **kubensenter** script is aware of the state of the mount encapsulation, and the script is safe to run even if encapsulation is not enabled.



NOTE

oc debug remote shell sessions start inside the Kubernetes namespace by default. You do not need to run **kubensenter** to inspect mount points when you use **oc debug**.

If the encapsulation feature is not enabled, the **kubensenter findmnt** and **findmnt** commands return the same output, regardless of whether they are run in an **oc debug** session or in an SSH shell prompt.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.
- You have configured SSH access to the cluster host.

Procedure

1. Open a remote SSH shell to the cluster host. For example:

```
$ ssh core@<node_name>
```

2. Run commands using the provided **kubensenter** script as the root user. To run a single command inside the Kubernetes namespace, provide the command and any arguments to the **kubensenter** script. For example, to run the **findmnt** command inside the Kubernetes namespace, run the following command:

```
■
```

```
[core@control-plane-1 ~]$ sudo kubensenter findmnt
```

Example output

```
kubensenter: Autodetect: kubens.service namespace found at /run/kubens/mnt
TARGET          SOURCE          FSTYPE  OPTIONS
/
/dev/sda4[/ostree/dep.../rhcos/dep.../32074f0e8e5ec453e56f5a8a7bc9347eaa4172349ceab9c22b709d9d71a3f4b0.0]
|
rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,prjquota
shm              tmpfs
...
```

3. To start a new interactive shell inside the Kubernetes namespace, run the **kubensenter** script without any arguments:

```
[core@control-plane-1 ~]$ sudo kubensenter
```

Example output

```
kubensenter: Autodetect: kubens.service namespace found at /run/kubens/mnt
```

13.4.4. Running additional services in the encapsulated namespace

You can adapt any existing tools by using the **kubensenter** script that is provided with OpenShift Container Platform to execute an additional command inside the Kubernetes mount point.

Any monitoring tool that relies on the ability to run in the host operating system and have visibility of mount points created by kubelet, CRI-O, or containers themselves, must enter the container mount namespace to see these mount points.

The **kubensenter** script is aware of the state of the mount encapsulation feature status, and is safe to run even if encapsulation is not enabled. In that case the script executes the provided command in the default mount namespace.

For example, if a systemd service needs to run inside the new Kubernetes mount namespace, edit the service file and use the **ExecStart=** command line with **kubensenter**.

```
[Unit]
Description=Example service
[Service]
ExecStart=/usr/bin/kubensenter /path/to/original/command arg1 arg2
```

13.4.5. Additional resources

- [What are namespaces](#)
- [Manage containers in namespaces by using nsenter](#)
- [MachineConfig](#)

CHAPTER 14. MANAGING BARE-METAL HOSTS

You can configure bare-metal hosts directly within OpenShift Container Platform. To provision and manage nodes in a bare-metal cluster, use **Machine** and **MachineSet** custom resources (CRs).

14.1. ABOUT BARE METAL HOSTS AND NODES

To provision a Red Hat Enterprise Linux CoreOS (RHCOS) bare-metal host as a node in your cluster, first create a **MachineSet** custom resource (CR) object that corresponds to bare-metal host hardware.

Bare-metal host compute machine sets describe infrastructure components specific to your configuration. You apply specific Kubernetes labels to these compute machine sets and then update the infrastructure components to run on only those machines.

When you scale up the relevant **MachineSet** CR that contains a **metal3.io/autoscale-to-hosts** annotation, **Machine** CRs are created automatically. OpenShift Container Platform uses **Machine** CRs to provision the bare-metal node that corresponds to the host as specified in the **MachineSet** CR.

14.2. MAINTAINING BARE METAL HOSTS

You can maintain the details of the bare metal hosts in your cluster from the OpenShift Container Platform web console.

Procedure

1. From the web console, complete the following steps:
 - a. Navigate to **Compute → Bare Metal Hosts**
 - b. Select a task from the **Actions** drop-down menu.
 - c. Manage items such as baseboard management controller (BMC) details, boot MAC address for the host, enable power management, and so on. You can also review the details of the network interfaces and drives for the host.
2. Move a bare-metal host into maintenance mode. When you move a host into maintenance mode, the scheduler moves all managed workloads off the corresponding bare-metal node. No new workloads are scheduled while in maintenance mode.
3. Deprovision a bare-metal host in the web console. Deprovisioning a host does the following actions:
 - a. Annotates the bare-metal host CR with **cluster.k8s.io/delete-machine: true**.
 - b. Scales down the related compute machine set.



NOTE

Powering off the host without first moving the daemon set and unmanaged static pods to another node can cause service disruption and loss of data.

Additional resources

- [Adding compute machines to bare metal](#)

14.2.1. Adding a bare metal host to the cluster using the web console

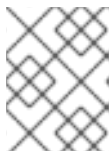
You can add bare-metal hosts to the cluster by using the web console.

Prerequisites

- Install an RHCOS cluster on bare metal.
- Log in as a user with **cluster-admin** privileges.

Procedure

1. In the web console, navigate to **Compute → Bare Metal Hosts**.
2. Select **Add Host → New with Dialog**.
3. Specify a unique name for the new bare-metal host.
4. Set the **Boot MAC address**.
5. Set the **Baseboard Management Console (BMC) Address**.
6. Enter the user credentials for the baseboard management controller (BMC) of the host.
7. Select to power on the host after creation, and select **Create**.
8. Scale up the number of replicas to match the number of available bare metal hosts. Navigate to **Compute → MachineSets**, and increase the number of machine replicas in the cluster by selecting **Edit Machine count** from the **Actions** drop-down menu.



NOTE

You can also manage the number of bare-metal nodes by using the **oc scale** command and the appropriate bare-metal compute machine set.

14.2.2. Adding a bare-metal host to the cluster using YAML in the web console

You can add bare-metal hosts to the cluster in the web console by using a YAML file that describes the bare-metal host.

Prerequisites

- Install a RHCOS compute machine on bare-metal infrastructure for use in the cluster.
- Log in as a user with **cluster-admin** privileges.
- Create a **Secret** CR for the bare-metal host.

Procedure

1. In the web console, navigate to **Compute → Bare Metal Hosts**.
2. Select **Add Host → New from YAML**.
3. Copy and paste the below YAML, modifying the relevant fields with the details of your host:

```

apiVersion: metal3.io/v1alpha1
kind: BareMetalHost
metadata:
  name: <bare_metal_host_name>
spec:
  online: true
  bmc:
    address: <bmc_address>
    credentialsName: <secret_credentials_name>
    disableCertificateVerification: True
    bootMACAddress: <host_boot_mac_address>
# ...

```

where:

spec.bmc.credentialsName

Specifies a reference to a valid **Secret** CR. The Bare Metal Operator cannot manage the bare-metal host without a valid **Secret** referenced in the **credentialsName**. For more information about secrets and how to create them, see "Understanding secrets".

spec.bmc.disableCertificateVerification

Specifies whether to require TLS host validation between the cluster and the baseboard management controller (BMC). When this field is set to **true**, TLS host validation is disabled.

4. Select **Create** to save the YAML and create the new bare-metal host.
5. Scale up the number of replicas to match the number of available bare-metal hosts. Navigate to **Compute** → **MachineSets**, and increase the number of machines in the cluster by selecting **Edit Machine count** from the **Actions** drop-down menu.



NOTE

You can also manage the number of bare-metal nodes by using the **oc scale** command and the appropriate bare-metal compute machine set.

Additional resources

- [Understanding secrets](#)

14.2.3. Automatically scaling machines to the number of available bare-metal hosts

To automatically create the number of **Machine** objects that matches the number of available **BareMetalHost** objects, add a **metal3.io/autoscale-to-hosts** annotation to the **MachineSet** object.

Prerequisites

- Install RHCOS bare-metal compute machines for use in the cluster, and create corresponding **BareMetalHost** objects.
- Install the OpenShift CLI (**oc**).
- Log in as a user with **cluster-admin** privileges.

Procedure

1. To configure automatic scaling for a compute machine set, annotate the compute machine set by running the following command:

```
$ oc annotate machineset <machineset> -n openshift-machine-api 'metal3.io/autoscale-to-hosts=<any_value>'
```

- **<machineset>**: Specifies the name of the compute machine set that you want to configure for automatic scaling.
- **<any_value>** Specifies is a value, such as **true** or **""**.

2. Wait for the new scaled machines to start.



NOTE

The **BareMetalHost** object continues to be counted against the **MachineSet** that the **Machine** object was created from when the following conditions are met:

- You use a **BareMetalHost** object to create a machine in the cluster.
- You subsequently change labels or selectors on the **BareMetalHost**.

14.2.4. Removing bare-metal hosts from the provisioner node

In certain circumstances, you might want to temporarily remove bare-metal hosts from the provisioner node. For example, to prevent the management of the number of **Machine** objects that matches the number of available **BareMetalHost** objects, add a **baremetalhost.metal3.io/detached** annotation to the **MachineSet** object.

Consider an example during provisioning when a bare-metal host reboot is triggered by using the OpenShift Container Platform administration console or as a result of a Machine Config Pool update. In this case, OpenShift Container Platform logs into the integrated Dell Remote Access Controller (iDRAC) and issues a delete of the job queue.



NOTE

This annotation has an effect for only **BareMetalHost** objects that are in either **Provisioned**, **ExternallyProvisioned**, or **Ready/Available** states.

Prerequisites

- Install RHCOS bare-metal compute machines for use in the cluster and create corresponding **BareMetalHost** objects.
- Install the OpenShift CLI (**oc**).
- Log in as a user with **cluster-admin** privileges.

Procedure

1. To configure automatic scaling for a compute machine set, annotate the compute machine set by running the following command:

```
$ oc annotate machineset <machineset> -n openshift-machine-api
'baremetalhost.metal3.io/detached'
```

Wait for the new machines to start.



NOTE

When you use a **BareMetalHost** object to create a machine in the cluster and labels or selectors are subsequently changed on the **BareMetalHost**, the **BareMetalHost** object continues to be counted against the **MachineSet** that the **Machine** object was created from.

2. In the provisioning use case, remove the annotation after the reboot is complete by using the following command:

```
$ oc annotate machineset <machineset> -n openshift-machine-api
'baremetalhost.metal3.io/detached-'
```

Additional resources

- [Expanding the cluster](#)
- [MachineHealthChecks on bare metal](#)

14.2.5. Powering off bare-metal hosts by using the web console

You can power off bare-metal cluster hosts in the web console. Before you power off a host, mark the node as unschedulable and drain all pods and workloads from the node.

Prerequisites

- You have installed a RHCOS compute machine on bare-metal infrastructure for use in the cluster.
- You have logged in as a user with **cluster-admin** privileges.
- You have configured the host to be managed and have added Baseboard Management Console credentials for the cluster host. You can add BMC credentials by applying a **Secret** custom resource (CR) in the cluster or by logging in to the web console and configuring the bare-metal host to be managed.

Procedure

1. Navigate to **Nodes** and select the node that you want to power off. Expand the **Actions** menu and select **Mark as unschedulable**.
2. Manually delete or relocate running pods on the node by adjusting the pod deployments or scaling down workloads on the node to zero. Wait for the drain process to complete.
3. Navigate to **Compute → Bare Metal Hosts**
4. Expand the **Options menu** for the bare-metal host that you want to power off, and select **Power Off**.

5. Select **Immediate power off**.

14.2.6. Powering off bare-metal hosts by using the CLI

You can power off bare-metal cluster hosts by applying a patch in the cluster by using the OpenShift CLI (**oc**). Before you power off a host, mark the node as unschedulable and drain all pods and workloads from the node.

Prerequisites

- You have installed a RHCOS compute machine on bare-metal infrastructure for use in the cluster.
- You have logged in as a user with **cluster-admin** privileges.
- You have configured the host to be managed and have added Baseboard Management Console credentials for the cluster host. You can add BMC credentials by applying a **Secret** custom resource (CR) in the cluster or by logging in to the web console and configuring the bare-metal host to be managed.

Procedure

1. Get the name of the managed bare-metal host by entering the following command:

```
$ oc get baremetalhosts -n openshift-machine-api -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.provisioning.state}{"\n"}{end}'
```

Example output

```
master-0.example.com managed
master-1.example.com managed
master-2.example.com managed
worker-0.example.com managed
worker-1.example.com managed
worker-2.example.com managed
```

2. Mark the node as unschedulable by entering the following command:

```
$ oc adm cordon <bare_metal_host>
```

- **<bare_metal_host>**: Specifies the name of the host that you want to shut down. For example, **worker-2.example.com**.

3. Drain all pods on the node by entering the following command:

```
$ oc adm drain <bare_metal_host> --force=true
```

Pods that are backed by replication controllers are rescheduled to other available nodes in the cluster.

4. Safely power off the bare-metal host by entering the following command:

```
$ oc patch <bare_metal_host> --type json -p '[{"op": "replace", "path": "/spec/online", "value": false}]'
```

-
- 5. After you power on the host, make the node schedulable for workloads by entering the following command:

```
$ oc adm uncordon <bare_metal_host>
```

CHAPTER 15. OPTIMIZING MEMORY MANAGEMENT FOR WORKLOADS BY USING HUGE PAGES

To optimize memory management for specific workloads, configure huge pages.

15.1. WHAT HUGE PAGES DO

To optimize memory mapping efficiency, understand the function of huge pages. Unlike standard 4Ki blocks, huge pages are larger memory segments that reduce the tracking load on the translation lookaside buffer (TLB) hardware cache.

Memory is managed in blocks known as pages. On most systems, a page is 4Ki; 1Mi of memory is equal to 256 pages; 1Gi of memory is 256,000 pages, and so on. CPUs have a built-in memory management unit that manages a list of these pages in hardware. The translation lookaside buffer (TLB) is a small hardware cache of virtual-to-physical page mappings. If the virtual address passed in a hardware instruction can be found in the TLB, the mapping can be determined quickly. If not, a TLB miss occurs, and the system falls back to slower, software-based address translation, resulting in performance issues. Since the size of the TLB is fixed, the only way to reduce the chance of a TLB miss is to increase the page size.

A huge page is a memory page that is larger than 4Ki. On x86_64 architectures, there are two common huge page sizes: 2Mi and 1Gi. Sizes vary on other architectures. To use huge pages, code must be written so that applications are aware of them. Transparent huge pages (THP) attempt to automate the management of huge pages without application knowledge, but they have limitations. In particular, they are limited to 2Mi page sizes. THP can lead to performance degradation on nodes with high memory utilization or fragmentation because of defragmenting efforts of THP, which can lock memory pages. For this reason, some applications might be designed to or recommend usage of pre-allocated huge pages instead of THP.

In OpenShift Container Platform, applications in a pod can allocate and consume pre-allocated huge pages.

15.2. HOW HUGE PAGES ARE CONSUMED BY APPS

You must ensure that nodes pre-allocate huge pages in order for the node to report its huge page capacity. A node can only pre-allocate huge pages for a single size.

Huge pages can be consumed through container-level resource requirements by using the resource name **hugepages-*<size>***, where *size* is the most compact binary notation by using integer values supported on a particular node. For example, if a node supports 2048 KiB page sizes, the node exposes a schedulable resource **hugepages-2Mi**. Unlike CPU or memory, huge pages do not support over-commitment.

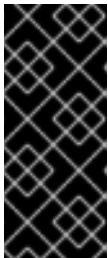
```
apiVersion: v1
kind: Pod
metadata:
  generateName: hugepages-volume-
spec:
  containers:
  - securityContext:
      privileged: true
    image: rhel7:latest
    command:
    - sleep
```

```

- inf
name: example
volumeMounts:
- mountPath: /dev/hugepages
  name: hugepage
resources:
  limits:
    hugepages-2Mi: 100Mi
    memory: "1Gi"
    cpu: "1"
  volumes:
- name: hugepage
  emptyDir:
    medium: HugePages

```

- **spec.containers.resources.limits.hugepages-2Mi**: Specifies the amount of memory for **hugepages** as the exact amount to be allocated.



IMPORTANT

Do not specify this value as the amount of memory for **hugepages** multiplied by the size of the page. For example, given a huge page size of 2 MB, if you want to use 100 MB of huge-page-backed RAM for your application, then you would allocate 50 huge pages. OpenShift Container Platform handles the math for you. As in the above example, you can specify **100MB** directly.

15.2.1. Allocating huge pages of a specific size

Some platforms support multiple huge page sizes. To allocate huge pages of a specific size, precede the huge pages boot command parameters with a huge page size selection parameter **hugepagesz=<size>**. The **<size>** value must be specified in bytes with an optional scale suffix [**kKmMgG**]. The default huge page size can be defined with the **default_hugepagesz=<size>** boot parameter.

15.2.2. Huge page requirements

- Huge page requests must equal the limits. This is the default if limits are specified, but requests are not.
- Huge pages are isolated at a pod scope. Container isolation is planned in a future iteration.
- **EmptyDir** volumes backed by huge pages must not consume more huge page memory than the pod request.
- Applications that consume huge pages via **shmget()** with **SHM_HUGETLB** must run with a supplemental group that matches **proc/sys/vm/hugetlb_shm_group**.

15.3. CONSUMING HUGE PAGES RESOURCES USING THE DOWNWARD API

To inject information about the huge pages resources consumed by a container, use the Downward API.

You can inject the resource allocation as environment variables, a volume plugin, or both. Applications that you develop and run in the container can determine the resources that are available by reading the environment variables or files in the specified volumes.

Procedure

1. Create a **hugepages-volume-pod.yaml** file that is similar to the following example:

```

apiVersion: v1
kind: Pod
metadata:
  generateName: hugepages-volume-
  labels:
    app: hugepages-example
spec:
  containers:
  - securityContext:
      capabilities:
        add: [ "IPC_LOCK" ]
    image: rhel7:latest
    command:
    - sleep
    - inf
    name: example
    volumeMounts:
    - mountPath: /dev/hugepages
      name: hugepage
    - mountPath: /etc/podinfo
      name: podinfo
  resources:
    limits:
      hugepages-1Gi: 2Gi
      memory: "1Gi"
      cpu: "1"
    requests:
      hugepages-1Gi: 2Gi
  env:
  - name: REQUESTS_HUGEPAGES_1Gi
    valueFrom:
      resourceFieldRef:
        containerName: example
        resource: requests.hugepages-1Gi
  volumes:
  - name: hugepage
    emptyDir:
      medium: HugePages
  - name: podinfo
  downwardAPI:
    items:
    - path: "hugepages_1G_request"
      resourceFieldRef:
        containerName: example
        resource: requests.hugepages-1Gi
        divisor: 1Gi

```

where:

spec.containers.securityContext.env.name

Specifies what resource to read and use from **requests.hugepages-1Gi** and expose the value as the **REQUESTS_HUGEPAGES_1Gi** environment variable.

spec.volumes.name.items.path

Specifies what resource to read and use from **requests.hugepages-1Gi** and expose the value as the file **/etc/podinfo/hugepages_1G_request**.

2. Create the pod from the **hugepages-volume-pod.yaml** file by entering the following command:

```
$ oc create -f hugepages-volume-pod.yaml
```

Verification

1. Check the value of the **REQUESTS_HUGEPAGES_1GI** environment variable:

```
$ oc exec -it $(oc get pods -l app=hugepages-example -o
jsonpath='{.items[0].metadata.name}') \
-- env | grep REQUESTS_HUGEPAGES_1GI
```

Example output

```
REQUESTS_HUGEPAGES_1GI=2147483648
```

2. Check the value of the **/etc/podinfo/hugepages_1G_request** file:

```
$ oc exec -it $(oc get pods -l app=hugepages-example -o
jsonpath='{.items[0].metadata.name}') \
-- cat /etc/podinfo/hugepages_1G_request
```

Example output

```
2
```

Additional resources

- [Allowing containers to consume Downward API objects](#)

15.4. CONFIGURING HUGE PAGES AT BOOT TIME

To ensure nodes in your OpenShift Container Platform cluster pre-allocate memory for specific workloads, reserve huge pages at boot time.

There are two ways of reserving huge pages: at boot time and at run time. Reserving at boot time increases the possibility of success because the memory has not yet been significantly fragmented. The Node Tuning Operator currently supports boot-time allocation of huge pages on specific nodes.

**NOTE**

The Tuned boot-loader plugin only supports Red Hat Enterprise Linux CoreOS (RHCOS) compute nodes.

Procedure

1. Label all nodes that need the same huge pages setting by a label by entering the following command:

```
$ oc label node <node_using_hugepages> node-role.kubernetes.io/worker-hp=
```

2. Create a file with the following content and name it **hugepages-tuned-boottime.yaml**:

```
apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: hugepages
  namespace: openshift-cluster-node-tuning-operator
spec:
  profile:
    - data: |
        [main]
        summary=Boot time configuration for hugepages
        include=openshift-node
        [bootloader]
        cmdline_openshift_node_hugepages=hugepagesz=2M hugepages=50
        name: openshift-node-hugepages

  recommend:
    - machineConfigLabels:
        machineconfiguration.openshift.io/role: "worker-hp"
      priority: 30
      profile: openshift-node-hugepages
# ...
```

where:

metadata.name

Specifies the **name** of the Tuned resource to **hugepages**.

spec.profile

Specifies the **profile** section to allocate huge pages.

spec.profile.data

Specifies the order of parameters. The order is important as some platforms support huge pages of various sizes.

spec.recommend.machineConfigLabels

Specifies the enablement of a machine config pool based matching.

3. Create the Tuned **hugepages** object by entering the following command:

```
$ oc create -f hugepages-tuned-boottime.yaml
```

4. Create a file with the following content and name it **hugepages-mcp.yaml**:

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  name: worker-hp
labels:
```

```

    worker-hp: ""
spec:
  machineConfigSelector:
    matchExpressions:
      - {key: machineconfiguration.openshift.io/role, operator: In, values: [worker,worker-hp]}
  nodeSelector:
    matchLabels:
      node-role.kubernetes.io/worker-hp: ""

```

5. Create the machine config pool by entering the following command:

```
$ oc create -f hugepages-mcp.yaml
```

Verification

- To check that enough non-fragmented memory exists and that all the nodes in the **worker-hp** machine config pool now have 50 2Mi huge pages allocated, enter the following command:

```
$ oc get node <node_using_hugepages> -o jsonpath="{.status.allocatable.hugepages-2Mi}"
100Mi
```

15.5. DISABLING TRANSPARENT HUGE PAGES

If your application can handle huge pages on its own, you can disable transparent huge pages (THP) to optimally handle huge pages for all types of workloads and avoid the performance regressions that THP can cause.

Disabling THP prevents them from attempting to automate most aspects of creating, managing, and using huge pages. You can disable THP by using the Node Tuning Operator (NTO).

Procedure

1. Create a file with the following content and name it **thp-disable-tuned.yaml**:

```

apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: thp-workers-profile
  namespace: openshift-cluster-node-tuning-operator
spec:
  profile:
    - data: |
        [main]
        summary=Custom tuned profile for OpenShift to turn off THP on worker nodes
        include=openshift-node

        [vm]
        transparent_hugepages=never
        name: openshift-thp-never-worker

  recommend:
    - match:
        - label: node-role.kubernetes.io/worker

```



```
priority: 25
profile: openshift-thp-never-worker
# ...
```

2. Create the Tuned object by entering the following command:

```
$ oc create -f thp-disable-tuned.yaml
```

3. Check the list of active profiles by entering the following command::

```
$ oc get profile -n openshift-cluster-node-tuning-operator
```

Verification

- Log in to one of the nodes and do a regular THP check to verify if the nodes applied the profile successfully:

```
$ cat /sys/kernel/mm/transparent_hugepage/enabled
```

Example output

```
always madvise [never]
```

CHAPTER 16. UNDERSTANDING LOW LATENCY TUNING FOR CLUSTER NODES

By first understanding low latency tuning for cluster nodes, you can then use edge computing as a key role in reducing latency and congestion problems and improving application performance for telco and 5G network applications.

Maintaining a network architecture with the lowest possible latency is key for meeting the network performance requirements of 5G. Compared to 4G technology, with an average latency of 50 ms, 5G is targeted to reach latency of 1 ms or less. This reduction in latency boosts wireless throughput by a factor of 10.

16.1. ABOUT LOW LATENCY

You can tune for zero packet loss so to mitigate the inherent issues that degrade network performance.

For example, many of the deployed applications in the Telco space require low latency that can only tolerate zero packet loss. For more information, see "Tuning for Zero Packet Loss in Red Hat OpenStack Platform (RHOSP)".

The Edge computing initiative also comes in to play for reducing latency rates. Think of it as being on the edge of the cloud and closer to the user. This greatly reduces the distance between the user and distant data centers, resulting in reduced application response times and performance latency.

Administrators must be able to manage their many Edge sites and local services in a centralized way so that all of the deployments can run at the lowest possible management cost. They also need an easy way to deploy and configure certain nodes of their cluster for real-time low latency and high-performance purposes. Low latency nodes are useful for applications such as Cloud-native Network Functions (CNF) and Data Plane Development Kit (DPDK).

OpenShift Container Platform currently provides mechanisms to tune software on an OpenShift Container Platform cluster for real-time running and low latency (around <20 microseconds reaction time). This includes tuning the kernel and OpenShift Container Platform set values, installing a kernel, and reconfiguring the machine. But this method requires setting up four different Operators and performing many configurations that, when done manually, is complex and could be prone to mistakes.

OpenShift Container Platform uses the Node Tuning Operator to implement automatic tuning to achieve low latency performance for OpenShift Container Platform applications. The cluster administrator uses this performance profile configuration that makes it easier to make these changes in a more reliable way. The administrator can specify whether to update the kernel to kernel-rt, reserve CPUs for cluster and operating system housekeeping duties, including pod infra containers, and isolate CPUs for application containers to run the workloads.

OpenShift Container Platform also supports workload hints for the Node Tuning Operator that can tune the **PerformanceProfile** to meet the demands of different industry environments. Workload hints are available for **highPowerConsumption** (very low latency at the cost of increased power consumption) and **realTime** (priority given to optimum latency). A combination of **true/false** settings for these hints can be used to deal with application-specific workload profiles and requirements.

Workload hints simplify the fine-tuning of performance to industry sector settings. Instead of a "one size fits all" approach, workload hints can cater to usage patterns such as placing priority on:

- Low latency
- Real-time capability

- Efficient use of power

Ideally, all of the previously listed items are prioritized. Some of these items come at the expense of others however. The Node Tuning Operator is now aware of the workload expectations and better able to meet the demands of the workload. The cluster admin can now specify into which use case that workload falls. The Node Tuning Operator uses the **PerformanceProfile** to fine tune the performance settings for the workload.

The environment in which an application is operating influences its behavior. For a typical data center with no strict latency requirements, only minimal default tuning is needed that enables CPU partitioning for some high performance workload pods. For data centers and workloads where latency is a higher priority, measures are still taken to optimize power consumption. The most complicated cases are clusters close to latency-sensitive equipment such as manufacturing machinery and software-defined radios. This last class of deployment is often referred to as Far edge. For Far edge deployments, ultra-low latency is the ultimate priority, and is achieved at the expense of power management.

Additional resources

- [Tuning for Zero Packet Loss in Red Hat OpenStack Platform \(RHOSP\)](#)

16.2. ABOUT HYPER-THREADING FOR LOW LATENCY AND REAL-TIME APPLICATIONS

You can use Hyper-Threading, an Intel processor technology, to allow a physical CPU processor core to function as two logical cores, executing two independent threads simultaneously.

Hyper-Threading allows for better system throughput for certain workload types where parallel processing is beneficial. The default OpenShift Container Platform configuration expects Hyper-Threading to be enabled.

For telecommunications applications, design your application infrastructure to minimize latency as much as possible. Hyper-Threading can slow performance times and negatively affect throughput for compute-intensive workloads that require low latency. Disabling Hyper-Threading ensures predictable performance and can decrease processing times for these workloads.



NOTE

Hyper-Threading implementation and configuration differs depending on the hardware you are running OpenShift Container Platform on. Consult the relevant host hardware tuning information for more details of the Hyper-Threading implementation specific to that hardware. Disabling Hyper-Threading can increase the cost per core of the cluster.

Additional resources

- [Configuring Hyper-Threading for a cluster](#)

CHAPTER 17. TUNING NODES FOR LOW LATENCY WITH THE PERFORMANCE PROFILE

Tune nodes for low latency by using the cluster performance profile. You can restrict CPUs for infra and application containers, configure huge pages, Hyper-Threading, and configure CPU partitions for latency-sensitive processes.

17.1. CREATING A PERFORMANCE PROFILE

You can create a cluster performance profile by using the Performance Profile Creator (PPC) tool. The PPC is a function of the Node Tuning Operator.

The PPC combines information about your cluster with user-supplied configurations to generate a performance profile that is appropriate to your hardware, topology and use-case.



NOTE

Performance profiles are applicable only to bare-metal environments where the cluster has direct access to the underlying hardware resources. You can configure performance profiles for both single-node OpenShift and multi-node clusters.

The following is a high-level workflow for creating and applying a performance profile in your cluster:

- Create a machine config pool (MCP) for nodes that you want to target with performance configurations. In single-node OpenShift clusters, you must use the **master** MCP because there is only one node in the cluster.
- Gather information about your cluster using the **must-gather** command.
- Use the PPC tool to create a performance profile by using either of the following methods:
 - Run the PPC tool by using Podman as described in *Running the Performance Profile Creator using Podman*.
 - Run the PPC tool by using a wrapper script as described in *Running the Performance Profile Creator wrapper script*.
- Configure the performance profile for your use case and apply the performance profile to your cluster.

17.1.1. About the Performance Profile Creator

The Performance Profile Creator (PPC) is a command-line tool and is delivered with the Node Tuning Operator. You can use the PPC CLI to create a performance profile for your cluster.

Initially, you can use the PPC tool to process the **must-gather** data to display key performance configurations for your cluster, including the following information:

- NUMA cell partitioning with the allocated CPU IDs
- Hyper-Threading node configuration

You can use this information to help you configure the performance profile.

Specify performance configuration arguments to the PPC tool to generate a proposed performance profile that is appropriate for your hardware, topology, and use-case.

You can run the PPC by using one of the following methods:

- Run the PPC by using Podman
- Run the PPC by using the wrapper script



NOTE

Using the wrapper script abstracts some of the more granular Podman tasks into an executable script. For example, the wrapper script handles tasks such as pulling and running the required container image, mounting directories into the container, and providing parameters directly to the container through Podman. Both methods achieve the same result.

17.1.2. Creating a machine config pool to target nodes for performance tuning

For multi-node clusters, you can define a machine config pool (MCP) to identify the target nodes that you want to configure with a performance profile.

In single-node OpenShift clusters, you must use the **master** MCP because there is only one node in the cluster. You do not need to create a separate MCP for single-node OpenShift clusters.

Prerequisites

- You have **cluster-admin** role access.
- You installed the OpenShift CLI (**oc**).

Procedure

1. Label the target nodes for configuration by running the following command:

```
$ oc label node <node_name> node-role.kubernetes.io/worker-cnf=""
```

- **<node_name>**: Specifies the name of your node. This example applies the **worker-cnf** label.
2. Create a **MachineConfigPool** resource containing the target nodes:
 - a. Create a YAML file that defines the **MachineConfigPool** resource:

Example mcp-worker-cnf.yaml file

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  name: worker-cnf
  labels:
    machineconfiguration.openshift.io/role: worker-cnf
spec:
  machineConfigSelector:
    matchExpressions:
```

```
- {
  key: machineconfiguration.openshift.io/role,
  operator: In,
  values: [worker, worker-cnf],
}
paused: false
nodeSelector:
  matchLabels:
    node-role.kubernetes.io/worker-cnf: ""
```

where:

metadata.name

Specifies a name for the **MachineConfigPool** resource.

machineconfiguration.openshift.io/role

Specifies a unique label for the machine config pool.

node-role.kubernetes.io/worker-cnf

Specifies the nodes with the target label that you defined.

- b. Apply the **MachineConfigPool** resource by running the following command:

```
$ oc apply -f mcp-worker-cnf.yaml
```

Example output

```
machineconfigpool.machineconfiguration.openshift.io/worker-cnf created
```

Verification

- Check the machine config pools in your cluster by running the following command:

```
$ oc get mcp
```

Example output

NAME	CONFIG	UPDATED	UPDATING	DEGRADED
MACHINECOUNT	READYMACHINECOUNT	UPDATEDMACHINECOUNT		
DEGRADEDMACHINECOUNT	AGE			
master	rendered-master-58433c7c3c1b4ed5ffef95234d451490	True	False	
False 3	3	3	0	6h46m
worker	rendered-worker-168f52b168f151e4f853259729b6azc4	True	False	
False 2	2	2	0	6h46m
worker-cnf	rendered-worker-cnf-168f52b168f151e4f853259729b6azc4	True	False	
False 1	1	1	0	73s

17.1.3. Gathering data about your cluster for the PPC

The Performance Profile Creator (PPC) tool requires **must-gather** data. As a cluster administrator, run the **must-gather** command to capture information about your cluster.

Prerequisites

- Access to the cluster as a user with the **cluster-admin** role.
- You installed the OpenShift CLI (**oc**).
- You identified a target MCP that you want to configure with a performance profile.

Procedure

1. Navigate to the directory where you want to store the **must-gather** data.
2. Collect cluster information by running the following command:

```
$ oc adm must-gather
```

The command creates a folder with the **must-gather** data in your local directory with a naming format similar to the following: **must-gather.local.1971646453781853027**.

3. Optional: Create a compressed file from the **must-gather** directory:

```
$ tar cvaf must-gather.tar.gz <must_gather_folder>
```

- **<must_gather_folder>**: Specifies the name of the **must-gather** data folder.



NOTE

Compressed output is required if you are running the Performance Profile Creator wrapper script.

Additional resources

- [Gathering data about your cluster](#)

17.1.4. Running the Performance Profile Creator using Podman

As a cluster administrator, you can use Podman with the Performance Profile Creator (PPC) to create a performance profile.

For more information about the PPC arguments, see the section "Performance Profile Creator arguments".



IMPORTANT

The PPC uses the **must-gather** data from your cluster to create the performance profile. If you make any changes to your cluster, such as relabeling a node targeted for performance configuration, you must re-create the **must-gather** data before running PPC again.

Prerequisites

- Access to the cluster as a user with the **cluster-admin** role.
- A cluster installed on bare-metal hardware.
- You installed **podman** and the OpenShift CLI (**oc**).

- Access to the Node Tuning Operator image.
- You identified a machine config pool containing target nodes for configuration.
- You have access to the **must-gather** data for your cluster.

Procedure

1. Check the machine config pool by running the following command:

```
$ oc get mcp
```

Example output

```
NAME          CONFIG                                UPDATED      UPDATING  DEGRADED
MACHINECOUNT READYMACHINECOUNT UPDATEDMACHINECOUNT
DEGRADEDMACHINECOUNT AGE
master        rendered-master-58433c8c3c0b4ed5feef95434d455490      True      False
False 3          3          3          0          8h
worker        rendered-worker-668f56a164f151e4a853229729b6adc4      True      False
False 2          2          2          0          8h
worker-cnf    rendered-worker-cnf-668f56a164f151e4a853229729b6adc4  True      False
False 1          1          1          0          79m
```

2. Use Podman to authenticate to **registry.redhat.io** by running the following command:

```
$ podman login registry.redhat.io
```

```
Username: <user_name>
Password: <password>
```

3. Optional: Display help for the PPC tool by running the following command:

```
$ podman run --rm --entrypoint performance-profile-creator registry.redhat.io/openshift4/ose-cluster-node-tuning-rhel9-operator:v4.22 -h
```

Example output

A tool that automates creation of Performance Profiles

Available Commands:

completion Generate the autocompletion script for the specified shell

help Help about any command

info requires --must-gather-dir-path, ignores other arguments. [Valid values: log,json]

Usage:

performance-profile-creator [flags]

performance-profile-creator [command]

Flags:

--disable-ht Disable Hyperthreading

--enable-hardware-tuning Enable setting maximum cpu frequencies

-h, --help help for performance-profile-creator


```

--mcp-name string          MCP name corresponding to the target machines
(required)
--must-gather-dir-path string  Must gather directory path (default "must-gather")
--offlined-cpu-count int      Number of offlined CPUs
--per-pod-power-management    Enable Per Pod Power Management
--power-consumption-mode string  The power consumption mode. [Valid values:
default, low-latency, ultra-low-latency] (default "default")
--profile-name string         Name of the performance profile to be created (default
"performance")
--reserved-cpu-count int      Number of reserved CPUs (required)
--rt-kernel                  Enable Real Time Kernel (required)
--split-reserved-cpus-across-numa  Split the Reserved CPUs across NUMA nodes
--topology-manager-policy string  Kubelet Topology Manager Policy of the performance
profile to be created. [Valid values: single-numa-node, best-effort, restricted] (default
"restricted")
--user-level-networking       Run with User level Networking(DPDK) enabled

Use "performance-profile-creator [command] --help" for more information about a command.

```

4. To display information about the cluster, run the PPC tool with the **log** argument by running the following command:

```

$ podman run --entrypoint performance-profile-creator -v <path_to_must_gather>:/must-
gather:z registry.redhat.io/openshift4/ose-cluster-node-tuning-rhel9-operator:v4.22 info --
must-gather-dir-path /must-gather

```

- **--entrypoint performance-profile-creator** defines the performance profile creator as a new entry point to **podman**.
- **-v <path_to_must_gather>** specifies the path to either of the following components:
 - The directory containing the **must-gather** data.
 - An existing directory containing the **must-gather** decompressed .tar file.

Example output

```

level=info msg="Nodes names targeted by master pool are: "
level=info msg="Nodes names targeted by worker-cnf pool are: host2.example.com "
level=info msg="Nodes names targeted by worker pool are: host.example.com
host1.example.com "
level=info msg="Cluster info:"
level=info msg="MCP 'master' nodes:"
level=info msg=---
level=info msg="MCP 'worker' nodes:"
level=info msg="Node: host.example.com (NUMA cells: 1, HT: true)"
level=info msg="NUMA cell 0 : [0 1 2 3]"
level=info msg="CPU(s): 4"
level=info msg="Node: host1.example.com (NUMA cells: 1, HT: true)"
level=info msg="NUMA cell 0 : [0 1 2 3]"
level=info msg="CPU(s): 4"
level=info msg=---
level=info msg="MCP 'worker-cnf' nodes:"
level=info msg="Node: host2.example.com (NUMA cells: 1, HT: true)"

```

```
level=info msg="NUMA cell 0 : [0 1 2 3]"
level=info msg="CPU(s): 4"
level=info msg=---
```

5. Create a performance profile by running the following command. The example uses sample PPC arguments and values:

```
$ podman run --entrypoint performance-profile-creator -v <path_to_must_gather>:/must-gather:z registry.redhat.io/openshift4/ose-cluster-node-tuning-rhel9-operator:v4.22 --mcp-name=worker-cnf --reserved-cpu-count=1 --rt-kernel=true --split-reserved-cpus-across-numa=false --must-gather-dir-path /must-gather --power-consumption-mode=ultra-low-latency --offlined-cpu-count=1 > my-performance-profile.yaml
```

- **-v <path_to_must_gather>** specifies the path to either of the following components:
 - The directory containing the **must-gather** data.
 - The directory containing the **must-gather** decompressed .tar file.
- **--mcp-name=worker-cnf** specifies the **worker-cnf** machine config pool.
- **--reserved-cpu-count=1** specifies one reserved CPU.
- **--rt-kernel=true** enables the real-time kernel.
- **--split-reserved-cpus-across-numa=false** disables reserved CPUs splitting across NUMA nodes.
- **--power-consumption-mode=ultra-low-latency** specifies minimal latency at the cost of increased power consumption.
- **--offlined-cpu-count=1** specifies one offlined CPU.



NOTE

The **mcp-name** argument in this example is set to **worker-cnf** based on the output of the command **oc get mcp**. For single-node OpenShift use **--mcp-name=master**.

Example output

```
level=info msg="Nodes targeted by worker-cnf MCP are: [worker-2]"
level=info msg="NUMA cell(s): 1"
level=info msg="NUMA cell 0 : [0 1 2 3]"
level=info msg="CPU(s): 4"
level=info msg="1 reserved CPUs allocated: 0 "
level=info msg="2 isolated CPUs allocated: 2-3"
level=info msg="Additional Kernel Args based on configuration: []"
```

6. Review the created YAML file by running the following command:

```
$ cat my-performance-profile.yaml
```

Example output

```

---
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: performance
spec:
  cpu:
    isolated: 2-3
    offlined: "1"
    reserved: "0"
  machineConfigPoolSelector:
    machineconfiguration.openshift.io/role: worker-cnf
  net:
    userLevelNetworking: false
  nodeSelector:
    node-role.kubernetes.io/worker-cnf: ""
  numa:
    topologyPolicy: restricted
  realTimeKernel:
    enabled: true
  workloadHints:
    highPowerConsumption: true
    perPodPowerManagement: false
    realTime: true

```

7. Apply the generated profile:

```
$ oc apply -f my-performance-profile.yaml
```

Example output

```
performanceprofile.performance.openshift.io/performance created
```

17.1.5. Running the Performance Profile Creator wrapper script

The wrapper script simplifies the process of creating a performance profile with the Performance Profile Creator (PPC) tool. The script handles tasks such as pulling and running the required container image, mounting directories into the container, and providing parameters directly to the container through Podman.

For more information about the Performance Profile Creator arguments, see the section "Performance Profile Creator arguments".



IMPORTANT

The PPC uses the **must-gather** data from your cluster to create the performance profile. If you make any changes to your cluster, such as relabeling a node targeted for performance configuration, you must re-create the **must-gather** data before running PPC again.

Prerequisites

- Access to the cluster as a user with the **cluster-admin** role.

- A cluster installed on bare-metal hardware.
- You installed **podman** and the OpenShift CLI (**oc**).
- Access to the Node Tuning Operator image.
- You identified a machine config pool containing target nodes for configuration.
- Access to the **must-gather** tarball.

Procedure

1. Create a file on your local machine named, for example, **run-perf-profile-creator.sh**:

```
$ vi run-perf-profile-creator.sh
```

2. Paste the following code into the file:

```
#!/bin/bash

readonly CONTAINER_RUNTIME=${CONTAINER_RUNTIME:-podman}
readonly CURRENT_SCRIPT=$(basename "$0")
readonly CMD="${CONTAINER_RUNTIME} run --entrypoint performance-profile-creator"
readonly IMG_EXISTS_CMD="${CONTAINER_RUNTIME} image exists"
readonly IMG_PULL_CMD="${CONTAINER_RUNTIME} image pull"
readonly MUST_GATHER_VOL="/must-gather"

NTO_IMG="registry.redhat.io/openshift4/ose-cluster-node-tuning-rhel9-operator:v4.22"
MG_TARBALL=""
DATA_DIR=""

usage() {
    print "Wrapper usage:"
    print "  ${CURRENT_SCRIPT} [-h] [-p image] [-t path] -- [performance-profile-creator flags]"
    print ""
    print "Options:"
    print "  -h          help for ${CURRENT_SCRIPT}"
    print "  -p          Node Tuning Operator image"
    print "  -t          path to a must-gather tarball"

    ${IMG_EXISTS_CMD} "${NTO_IMG}" && ${CMD} "${NTO_IMG}" -h
}

function cleanup {
    [ -d "${DATA_DIR}" ] && rm -rf "${DATA_DIR}"
}
trap cleanup EXIT

exit_error() {
    print "error: $"
    usage
    exit 1
}

print() {
    echo "$*" >&2
}
```

```

}

check_requirements() {
    ${IMG_EXISTS_CMD} "${NTO_IMG}" || ${IMG_PULL_CMD} "${NTO_IMG}" || \
        exit_error "Node Tuning Operator image not found"

    [ -n "${MG_TARBALL}" ] || exit_error "Must-gather tarball file path is mandatory"
    [ -f "${MG_TARBALL}" ] || exit_error "Must-gather tarball file not found"

    DATA_DIR=$(mktemp -d -t "${CURRENT_SCRIPT}XXXX") || exit_error "Cannot create the
data directory"
    tar -zxvf "${MG_TARBALL}" --directory "${DATA_DIR}" || exit_error "Cannot decompress the
must-gather tarball"
    chmod a+rx "${DATA_DIR}"

    return 0
}

main() {
    while getopts 'hp:t:' OPT; do
        case "${OPT}" in
            h)
                usage
                exit 0
                ;;
            p)
                NTO_IMG="${OPTARG}"
                ;;
            t)
                MG_TARBALL="${OPTARG}"
                ;;
            ?)
                exit_error "invalid argument: ${OPTARG}"
                ;;
        esac
    done
    shift $((OPTIND - 1))

    check_requirements || exit 1

    ${CMD} -v "${DATA_DIR}:${MUST_GATHER_VOL}:z" "${NTO_IMG}" "$@" --must-gather-
dir-path "${MUST_GATHER_VOL}"
    echo "" 1>&2
}

main "$@"

```

3. Add execute permissions for everyone on this script:

```
$ chmod a+x run-perf-profile-creator.sh
```

4. Use Podman to authenticate to **registry.redhat.io** by running the following command:

```
$ podman login registry.redhat.io
```

```
Username: <user_name>
Password: <password>
```

- Optional: Display help for the PPC tool by running the following command:

```
$ ./run-perf-profile-creator.sh -h
```

Wrapper usage:

```
run-perf-profile-creator.sh [-h] [-p image][-t path] -- [performance-profile-creator flags]
```

Options:

```
-h          help for run-perf-profile-creator.sh
-p          Node Tuning Operator image
-t          path to a must-gather tarball
```

A tool that automates creation of Performance Profiles

Usage:

```
performance-profile-creator [flags]
```

Flags:

```
--disable-ht          Disable Hyperthreading
-h, --help            help for performance-profile-creator
--info string         Show cluster information; requires --must-gather-dir-path,
ignore the other arguments. [Valid values: log, json] (default "log")
--mcp-name string      MCP name corresponding to the target machines
(required)
--must-gather-dir-path string  Must gather directory path (default "must-gather")
--offlined-cpu-count int    Number of offlined CPUs
--per-pod-power-management  Enable Per Pod Power Management
--power-consumption-mode string  The power consumption mode. [Valid values:
default, low-latency, ultra-low-latency] (default "default")
--profile-name string      Name of the performance profile to be created (default
"performance")
--reserved-cpu-count int    Number of reserved CPUs (required)
--rt-kernel             Enable Real Time Kernel (required)
--split-reserved-cpus-across-numa  Split the Reserved CPUs across NUMA nodes
--topology-manager-policy string  Kubelet Topology Manager Policy of the performance
profile to be created. [Valid values: single-numa-node, best-effort, restricted] (default
"restricted")
--user-level-networking    Run with User level Networking(DPDK) enabled
--enable-hardware-tuning   Enable setting maximum CPU frequencies
```



NOTE

You can optionally set a path for the Node Tuning Operator image using the **-p** option. If you do not set a path, the wrapper script uses the default image: **registry.redhat.io/openshift4/ose-cluster-node-tuning-rhel9-operator:v4.22**.

- To display information about the cluster, run the PPC tool with the **log** argument by running the following command:

```
$ ./run-perf-profile-creator.sh -t /<path_to_must_gather_dir>/must-gather.tar.gz -- --info=log
```

- **-t /<path_to_must_gather_dir>/must-gather.tar.gz**: Specifies the path to directory containing the must-gather tarball. This is a required argument for the wrapper script.

Example output

```
level=info msg="Cluster info:"
level=info msg="MCP 'master' nodes:"
level=info msg=---
level=info msg="MCP 'worker' nodes:"
level=info msg="Node: host.example.com (NUMA cells: 1, HT: true)"
level=info msg="NUMA cell 0 : [0 1 2 3]"
level=info msg="CPU(s): 4"
level=info msg="Node: host1.example.com (NUMA cells: 1, HT: true)"
level=info msg="NUMA cell 0 : [0 1 2 3]"
level=info msg="CPU(s): 4"
level=info msg=---
level=info msg="MCP 'worker-cnf' nodes:"
level=info msg="Node: host2.example.com (NUMA cells: 1, HT: true)"
level=info msg="NUMA cell 0 : [0 1 2 3]"
level=info msg="CPU(s): 4"
level=info msg=---
```

7. Create a performance profile by running the following command. The example command uses sample PPC arguments and values.

```
$ ./run-perf-profile-creator.sh -t /path-to-must-gather/must-gather.tar.gz -- --mcp-
name=worker-cnf --reserved-cpu-count=1 --rt-kernel=true --split-reserved-cpus-across-
numa=false --power-consumption-mode=ultra-low-latency --offlined-cpu-count=1 > my-
performance-profile.yaml
```

- **--mcp-name=worker-cnf** specifies the **worker-cnf** machine config pool.
- **--reserved-cpu-count=1** specifies one reserved CPU.
- **--rt-kernel=true** enables the real-time kernel.
- **--split-reserved-cpus-across-numa=false** disables reserved CPUs splitting across NUMA nodes.
- **--power-consumption-mode=ultra-low-latency** specifies minimal latency at the cost of increased power consumption.
- **--offlined-cpu-count=1** specifies one offlined CPUs.



NOTE

The **mcp-name** argument in this example is set to **worker-cnf** based on the output of the command **oc get mcp**. For single-node OpenShift use **--mcp-name=master**.

8. Review the created YAML file by running the following command:

```
$ cat my-performance-profile.yaml
```

Example output

```

apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: performance
spec:
  cpu:
    isolated: 2-3
    offlined: "1"
    reserved: "0"
  machineConfigPoolSelector:
    machineconfiguration.openshift.io/role: worker-cnf
  nodeSelector:
    node-role.kubernetes.io/worker-cnf: ""
  numa:
    topologyPolicy: restricted
  realTimeKernel:
    enabled: true
  workloadHints:
    highPowerConsumption: true
    perPodPowerManagement: false
    realTime: true

```

9. Apply the generated profile:

```
$ oc apply -f my-performance-profile.yaml
```

Example output

```
performanceprofile.performance.openshift.io/performance created
```

17.1.6. Performance Profile Creator arguments

To customize the generation of performance profiles, review the arguments for the Performance Profile Creator.

Table 17.1. Required Performance Profile Creator arguments

Argument	Description
mcp-name	Name for MCP; for example, worker-cnf corresponding to the target machines.
must-gather-dir-path	The path of the must gather directory. This argument is only required if you run the PPC tool by using Podman. If you use the PPC with the wrapper script, do not use this argument. Instead, specify the directory path to the must-gather tarball by using the -t option for the wrapper script.
reserved-cpu-count	Number of reserved CPUs. Use a natural number greater than zero.

Argument	Description
rt-kernel	Enables real-time kernel. Possible values: true or false.

Table 17.2. Optional Performance Profile Creator arguments

Argument	Description
disable-ht	Disable Hyper-Threading. Possible values: true or false. Default: false.  WARNING If this argument is set to true you should not disable Hyper-Threading in the BIOS. Disabling Hyper-Threading is accomplished with a kernel command-line argument.
enable-hardware-tuning	Enable the setting of maximum CPU frequencies. To enable this feature, set the maximum frequency for applications running on isolated and reserved CPUs for both of the following fields: • spec.hardwareTuning.isolatedCpuFreq • spec.hardwareTuning.reservedCpuFreq This is an advanced feature. If you configure hardware tuning, the generated PerformanceProfile includes warnings and guidance on how to set frequency settings.
info	This captures cluster information. This argument also requires the must-gather-dir-path argument. If any other arguments are set they are ignored. Possible values: • log • JSON Default: log.

Argument	Description
offlined-cpu-count	Number of offlined CPUs.  NOTE Use a natural number greater than zero. If not enough logical processors are offlined, then error messages are logged. The messages are: Error: failed to compute the reserved and isolated CPUs: please ensure that reserved-cpu-count plus offlined-cpu-count should be in the range [0,1] Error: failed to compute the reserved and isolated CPUs: please specify the offlined CPU count in the range [0,1]
power-consumption-mode	The power consumption mode. Possible values: • default: Performance achieved through CPU partitioning only. • low-latency: Enhanced measures to improve latency. • ultra-low-latency: Priority given to optimal latency, at the expense of power management. Default: default.
per-pod-power-management	Enable per pod power management. You cannot use this argument if you configured ultra-low-latency as the power consumption mode. Possible values: true or false. Default: false.
profile-name	Name of the performance profile to create. Default: performance.
split-reserved-cpus-across-numa	Split the reserved CPUs across NUMA nodes. Possible values: true or false. Default: false.

Argument	Description
topology-manager-policy	Kubelet Topology Manager policy of the performance profile to be created. Possible values: • single-numa-node • best-effort • restricted Default: restricted.
user-level-networking	Run with user level networking (DPDK) enabled. Possible values: true or false. Default: false.

17.2. REFERENCE PERFORMANCE PROFILES

Use the following reference performance profiles as the basis to develop your own custom profiles.

17.2.1. Performance profile template for clusters that use OVS-DPDK on OpenStack

To maximize machine performance in a cluster that uses Open vSwitch with the Data Plane Development Kit (OVS-DPDK) on Red Hat OpenStack Platform (RHOSP), you can use a performance profile.

You can use the following performance profile template to create a profile for your deployment.

Performance profile template for clusters that use OVS-DPDK

```

apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: cnf-performanceprofile
spec:
  additionalKernelArgs:
    - nmi_watchdog=0
    - audit=0
    - mce=off
    - processor.max_cstate=1
    - idle=poll
    - intel_idle.max_cstate=0
    - default_hugepagesz=1GB
    - hugepagesz=1G
    - intel_iommu=on
  cpu:
    isolated: <CPU_ISOLATED>
    reserved: <CPU_RESERVED>
  hugepages:
    defaultHugepagesSize: 1G

```

```

pages:
  - count: <HUGEPAGES_COUNT>
    node: 0
    size: 1G
nodeSelector:
  node-role.kubernetes.io/worker: ""
realTimeKernel:
  enabled: false
  globallyDisableIrqLoadBalancing: true

```

Insert values that are appropriate for your configuration for the **CPU_ISOLATED**, **CPU_RESERVED**, and **HUGEPAGES_COUNT** keys.

17.2.2. Telco RAN DU reference design performance profile

You can use a pre-configured design performance profile that configures node-level performance settings for OpenShift Container Platform clusters on commodity hardware to host telco RAN DU workloads.

Telco RAN DU reference design performance profile

```

apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  # if you change this name make sure the 'include' line in TunedPerformancePatch.yaml
  # matches this name: include=openshift-node-performance-${PerformanceProfile.metadata.name}
  # Also in file 'validatorCRs/informDuValidator.yaml':
  # name: 50-performance-${PerformanceProfile.metadata.name}
  name: openshift-node-performance-profile
  annotations:
    ran.openshift.io/reference-configuration: "ran-du.redhat.com"
spec:
  additionalKernelArgs:
    - "rcupdate.rcu_normal_after_boot=0"
    - "efi=runtime"
    - "vfio_pci.enable_sriov=1"
    - "vfio_pci.disable_idle_d3=1"
    - "module_blacklist=irdma"
  cpu:
    isolated: $isolated
    reserved: $reserved
  hugepages:
    defaultHugepagesSize: $defaultHugepagesSize
  pages:
    - size: $size
      count: $count
      node: $node
  machineConfigPoolSelector:
    pools.operator.machineconfiguration.openshift.io/$mcp: ""
  nodeSelector:
    node-role.kubernetes.io/$mcp: ""
  numa:
    topologyPolicy: "restricted"
  # To use the standard (non-realtime) kernel, set enabled to false
  realTimeKernel:

```

```

enabled: true
workloadHints:
  # WorkloadHints defines the set of upper level flags for different type of workloads.
  # See https://github.com/openshift/cluster-node-tuning-
operator/blob/master/docs/performanceprofile/performance_profile.md#workloadhints
  # for detailed descriptions of each item.
  # The configuration below is set for a low latency, performance mode.
realTime: true
highPowerConsumption: false
perPodPowerManagement: false

```

17.2.3. Telco core reference design performance profile

You can use a pre-configured design performance profile that configures node-level performance settings for OpenShift Container Platform clusters on commodity hardware to host telco core workloads.

Telco core reference design performance profile

```

# required
# count: 1
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: $name
  annotations:
    # Some pods want the kernel stack to ignore IPv6 router Advertisement.
    kubeletconfig.experimental: |
      {"allowedUnsafeSysctls":["net.ipv6.conf.all.accept_ra"]}
spec:
  cpu:
    # node0 CPUs: 0-17,36-53
    # node1 CPUs: 18-34,54-71
    # siblings: (0,36), (1,37)...
    # we want to reserve the first Core of each NUMA socket
    #
    # no CPU left behind! all-cpus == isolated + reserved
    isolated: $isolated # eg 1-17,19-35,37-53,55-71
    reserved: $reserved # eg 0,18,36,54
    # Guaranteed QoS pods will disable IRQ balancing for cores allocated to the pod.
    # default value of globallyDisableIrqLoadBalancing is false
    globallyDisableIrqLoadBalancing: false
  hugepages:
    defaultHugepagesSize: 1G
  pages:
    # 32GB per numa node
    - count: $count # eg 64
      size: 1G
  #machineConfigPoolSelector: {}
  # pools.operator.machineconfiguration.openshift.io/worker: "
  nodeSelector: {}
  #node-role.kubernetes.io/worker: ""
  workloadHints:
    realTime: false
    highPowerConsumption: false

```

```

perPodPowerManagement: true
realTimeKernel:
  enabled: false
numa:
  # All guaranteed QoS containers get resources from a single NUMA node
  topologyPolicy: "single-numa-node"
net:
  userLevelNetworking: false

```

17.3. SUPPORTED PERFORMANCE PROFILE API VERSIONS

The Node Tuning Operator supports **v2**, **v1**, and **v1alpha1** for the performance profile **apiVersion** field. The v1 and v1alpha1 APIs are identical. The v2 API includes an optional boolean field **globallyDisableIrqLoadBalancing** with a default value of **false**.

Upgrading the performance profile to use device interrupt processing

When you upgrade the Node Tuning Operator performance profile custom resource definition (CRD) from v1 or v1alpha1 to v2, **globallyDisableIrqLoadBalancing** is set to **true** on existing profiles.



NOTE

globallyDisableIrqLoadBalancing toggles whether IRQ load balancing will be disabled for the Isolated CPU set. When the option is set to **true** it disables IRQ load balancing for the Isolated CPU set. Setting the option to **false** allows the IRQs to be balanced across all CPUs.

Upgrading Node Tuning Operator API from v1alpha1 to v1

When upgrading Node Tuning Operator API version from v1alpha1 to v1, the v1alpha1 performance profiles are converted on-the-fly using a "None" Conversion strategy and served to the Node Tuning Operator with API version v1.

Upgrading Node Tuning Operator API from v1alpha1 or v1 to v2

When upgrading from an older Node Tuning Operator API version, the existing v1 and v1alpha1 performance profiles are converted using a conversion webhook that injects the **globallyDisableIrqLoadBalancing** field with a value of **true**.

17.4. NODE POWER CONSUMPTION AND REALTIME PROCESSING WITH WORKLOAD HINTS

You can create a performance profile appropriate for the hardware and topology of an environment by using the Performance Profile Creator (PPC) tool.

The following table describes the possible values set for the **power-consumption-mode** flag associated with the PPC tool and the workload hint that is applied.

Table 17.3. Impact of combinations of power consumption and real-time settings on latency

Performance Profile creator setting	Hint	Environment	Description
-------------------------------------	------	-------------	-------------

Performance Profile creator setting	Hint	Environment	Description
Default	<code>workloadHints: highPowerConsumption: false realTime: false</code>	High throughput cluster without latency requirements	Performance achieved through CPU partitioning only.
Low-latency	<code>workloadHints: highPowerConsumption: false realTime: true</code>	Regional data-centers	Both energy savings and low-latency are desirable: compromise between power management, latency and throughput.
Ultra-low-latency	<code>workloadHints: highPowerConsumption: true realTime: true</code>	Far edge clusters, latency critical workloads	Optimized for absolute minimal latency and maximum determinism at the cost of increased power consumption.
Per-pod power management	<code>workloadHints: realTime: true highPowerConsumption: false perPodPowerManagement: true</code>	Critical and non-critical workloads	Allows for power management per pod.

The following configuration is commonly used in a telco RAN DU deployment:

```

apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: workload-hints
spec:
  ...
  workloadHints:
    realTime: true
    highPowerConsumption: false
    perPodPowerManagement: false

```

perPodPowerManagement

Specifies to disable some debugging and monitoring features that can affect system latency.

**NOTE**

When the **realTime** workload hint flag is set to **true** in a performance profile, add the **cpu-quota.crio.io: disable** annotation to every guaranteed pod with pinned CPUs. This annotation is necessary to prevent the degradation of the process performance within the pod. If the **realTime** workload hint is not explicitly set, it defaults to **true**.

For more information how combinations of power consumption and real-time settings impact latency, see "Understanding workload hints".

Additional resources

- [Understanding workload hints](#)

17.5. CONFIGURING POWER SAVING FOR NODES THAT RUN COLOCATED HIGH AND LOW PRIORITY WORKLOADS

You can enable power savings for a node that has low priority workloads that are colocated with high priority workloads without impacting the latency or throughput of the high priority workloads. Power saving is possible without modifications to the workloads themselves.

**IMPORTANT**

The feature is supported on Intel Ice Lake and later generations of Intel CPUs. The capabilities of the processor might impact the latency and throughput of the high priority workloads.

Prerequisites

- You enabled C-states and operating system controlled P-states in the BIOS

Procedure

1. Generate a **PerformanceProfile** with the **per-pod-power-management** argument set to **true**:

```
$ podman run --entrypoint performance-profile-creator -v \
/must-gather:/must-gather:z registry.redhat.io/openshift4/ose-cluster-node-tuning-rhel9-
operator:v4.22 \
--mcp-name=worker-cnf --reserved-cpu-count=20 --rt-kernel=true \
--split-reserved-cpus-across-numa=false --topology-manager-policy=single-numa-node \
--must-gather-dir-path /must-gather --power-consumption-mode=low-latency \
--per-pod-power-management=true > my-performance-profile.yaml
```

The **power-consumption-mode** argument must be **default** or **low-latency** when the **per-pod-power-management** argument is set to **true**.

Example PerformanceProfile with perPodPowerManagement

```
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: performance
spec:
```



```
[.....]
workloadHints:
  realTime: true
  highPowerConsumption: false
  perPodPowerManagement: true
# ...
```

- Set the default **cpufreq** governor as an additional kernel argument in the **PerformanceProfile** custom resource (CR):

```
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: performance
spec:
  ...
  additionalKernelArgs:
    - cpufreq.default_governor=schedutil
# ...
```

where:

cpufreq.default_governor=schedutil

Specifies using the **schedutil** governor. You can use other governors, such as the **ondemand** or **powersave** governors.

- Set the maximum CPU frequency in the **TunedPerformancePatch** CR:

```
spec:
  profile:
    - data: |
        [sysfs]
        /sys/devices/system/cpu/intel_pstate/max_perf_pct = <x>
```

where:

/sys/devices/system/cpu/intel_pstate/max_perf_pct

Specifies the **max_perf_pct** that controls the maximum frequency that the **cpufreq** driver is allowed to set as a percentage of the maximum supported cpu frequency. This value applies to all CPUs. You can check the maximum supported frequency in **/sys/devices/system/cpu/cpu0/cpufreq/cpuinfo_max_freq**. As a starting point, you can use a percentage that caps all CPUs at the **All Cores Turbo** frequency. The **All Cores Turbo** frequency is the frequency that all cores will run at when the cores are all fully occupied.

Additional resources

- [About the Performance Profile Creator](#)
- [Disabling power saving mode for high priority pods](#)
- [Managing device interrupt processing for guaranteed pod isolated CPUs](#)

17.6. CPUS FOR INFRA AND APPLICATION CONTAINERS

Generic housekeeping and workload tasks use CPUs in a way that might impact latency-sensitive processes. By default, the container runtime uses all online CPUs to run all containers together, which can result in context switches and spikes in latency.

Partitioning the CPUs prevents noisy processes from interfering with latency-sensitive processes by separating them from each other. The following table describes how processes run on a CPU after you have tuned the node using the Node Tuning Operator:

Table 17.4. Process' CPU assignments

Process type	Details
Burstable and BestEffort pods	Runs on any CPU except where low latency workload is running
Infrastructure pods	Runs on any CPU except where low latency workload is running
Interrupts	Redirects to reserved CPUs (optional in OpenShift Container Platform 4.7 and later)
Kernel processes	Pins to reserved CPUs
Latency-sensitive workload pods	Pins to a specific set of exclusive CPUs from the isolated pool
OS processes/systemd services	Pins to reserved CPUs

The allocatable capacity of cores on a node for pods of all QoS process types, **Burstable**, **BestEffort**, or **Guaranteed**, is equal to the capacity of the isolated pool. The capacity of the reserved pool is removed from the node's total core capacity for use by the cluster and operating system housekeeping duties.

Example 1

A node features a capacity of 100 cores. Using a performance profile, the cluster administrator allocates 50 cores to the isolated pool and 50 cores to the reserved pool. The cluster administrator assigns 25 cores to QoS **Guaranteed** pods and 25 cores for **BestEffort** or **Burstable** pods. This matches the capacity of the isolated pool.

Example 2

A node features a capacity of 100 cores. Using a performance profile, the cluster administrator allocates 50 cores to the isolated pool and 50 cores to the reserved pool. The cluster administrator assigns 50 cores to QoS **Guaranteed** pods and one core for **BestEffort** or **Burstable** pods. This exceeds the capacity of the isolated pool by one core. Pod scheduling fails because of insufficient CPU capacity.

The exact partitioning pattern to use depends on many factors like hardware, workload characteristics and the expected system load. Some sample use cases are as follows:

- If the latency-sensitive workload uses specific hardware, such as a network interface controller (NIC), ensure that the CPUs in the isolated pool are as close as possible to this hardware. At a minimum, you should place the workload in the same Non-Uniform Memory Access (NUMA) node.

- The reserved pool is used for handling all interrupts. When depending on system networking, allocate a sufficiently-sized reserve pool to handle all the incoming packet interrupts. In 4.22 and later versions, workloads can optionally be labeled as sensitive.

The decision regarding which specific CPUs should be used for reserved and isolated partitions requires detailed analysis and measurements. Factors like NUMA affinity of devices and memory play a role. The selection also depends on the workload architecture and the specific use case.



IMPORTANT

The reserved and isolated CPU pools must not overlap and together must span all available cores in the worker node.

To ensure that housekeeping tasks and workloads do not interfere with each other, specify two groups of CPUs in the **spec** section of the performance profile.

- **isolated** - Specifies the CPUs for the application container workloads. These CPUs have the lowest latency. Processes in this group have no interruptions and can, for example, reach much higher DPDK zero packet loss bandwidth.
- **reserved** - Specifies the CPUs for the cluster and operating system housekeeping duties. Threads in the **reserved** group are often busy. Do not run latency-sensitive applications in the **reserved** group. Latency-sensitive applications run in the **isolated** group.

17.7. PARTITIONING CPUS FOR INFRA AND APPLICATION CONTAINERS

By partitioning CPUs, you can prevent noisy processes from interfering with latency-sensitive processes by separating the processes from each other.

Procedure

1. Create a performance profile appropriate for the environment's hardware and topology. The following example adds the **reserved** and **isolated** parameters with the CPUs you want reserved and isolated for the infra and application containers:

```
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: infra-cpus
spec:
  cpu:
    reserved: "0-4,9"
    isolated: "5-8"
  nodeSelector:
    node-role.kubernetes.io/worker: ""
# ...
```

where:

spec.cpu.reserved

Specifies which CPUs are for infra containers to perform cluster and operating system housekeeping duties.

spec.cpu.isolated

Specifies which CPUs are for application containers to run workloads.

spec.nodeSelector

Specifies a node selector to apply the performance profile to specific nodes. Optional parameter.

17.8. CONFIGURING HYPER-THREADING FOR A CLUSTER

To configure Hyper-Threading for an OpenShift Container Platform cluster, set the CPU threads in the performance profile to the same cores that are configured for the reserved or isolated CPU pools.

**NOTE**

If you configure a performance profile, and subsequently change the Hyper-Threading configuration for the host, ensure that you update the CPU **isolated** and **reserved** fields in the **PerformanceProfile** YAML to match the new configuration.

**WARNING**

Disabling a previously enabled host Hyper-Threading configuration can cause the CPU core IDs listed in the **PerformanceProfile** YAML to be incorrect. This incorrect configuration can cause the node to become unavailable because the listed CPUs can no longer be found.

Prerequisites

- Access to the cluster as a user with the **cluster-admin** role.
- Install the OpenShift CLI (**oc**).

Procedure

1. Ascertain which threads are running on what CPUs for the host you want to configure. You can view which threads are running on the host CPUs by logging in to the cluster and running the following command:

```
$ lscpu --all --extended
```

Example output

```
CPU NODE SOCKET CORE L1d:L1i:L2:L3 ONLINE MAXMHZ  MINMHZ
0 0 0 0 0:0:0:0 yes 4800.0000 400.0000
1 0 0 1 1:1:1:0 yes 4800.0000 400.0000
2 0 0 2 2:2:2:0 yes 4800.0000 400.0000
3 0 0 3 3:3:3:0 yes 4800.0000 400.0000
4 0 0 0 0:0:0:0 yes 4800.0000 400.0000
5 0 0 1 1:1:1:0 yes 4800.0000 400.0000
6 0 0 2 2:2:2:0 yes 4800.0000 400.0000
```

```
7 0 0 3 3:3:3:0 yes 4800.0000 400.0000
```

In this example, there are eight logical CPU cores running on four physical CPU cores. CPU0 and CPU4 are running on physical Core0, CPU1 and CPU5 are running on physical Core 1, and so on. Alternatively, to view the threads that are set for a particular physical CPU core (**cpu0** in the example below), open a shell prompt and run the following:

```
$ cat /sys/devices/system/cpu/cpu0/topology/thread_siblings_list
```

Example output

```
0-4
```

2. Apply the isolated and reserved CPUs in the **PerformanceProfile** YAML. For example, you can set logical cores CPU0 and CPU4 as **isolated**, and logical cores CPU1 to CPU3 and CPU5 to CPU7 as **reserved**. When you configure reserved and isolated CPUs, the infra containers in pods use the reserved CPUs and the application containers use the isolated CPUs.

```
...
cpu:
  isolated: 0,4
  reserved: 1-3,5-7
...
```



NOTE

The reserved and isolated CPU pools must not overlap and together must span all available cores in the worker node.



IMPORTANT

Hyper-Threading is enabled by default on most Intel processors. If you enable Hyper-Threading, all threads processed by a particular core must be isolated or processed on the same core.

When Hyper-Threading is enabled, all guaranteed pods must use multiples of the simultaneous multi-threading (SMT) level to avoid a "noisy neighbor" situation that can cause the pod to fail. See [Static policy options](#) for more information.

17.9. DISABLING HYPER-THREADING FOR LOW LATENCY APPLICATIONS

When configuring clusters for low latency processing, consider whether you want to disable Hyper-Threading before you deploy the cluster.

To disable Hyper-Threading, perform the following steps:

Procedure

- Create a performance profile that is appropriate for your hardware and topology. The following example sets **nosmt** as an additional kernel argument:

Example performance profile

```

apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: example-performanceprofile
spec:
  additionalKernelArgs:
    - nmi_watchdog=0
    - audit=0
    - mce=off
    - processor.max_cstate=1
    - idle=poll
    - intel_idle.max_cstate=0
    - nosmt
  cpu:
    isolated: 2-3
    reserved: 0-1
  hugepages:
    defaultHugepagesSize: 1G
  pages:
    - count: 2
      node: 0
      size: 1G
  nodeSelector:
    node-role.kubernetes.io/performance: "
  realTimeKernel:
    enabled: true

```



NOTE

When you configure reserved and isolated CPUs, the infra containers in pods use the reserved CPUs and the application containers use the isolated CPUs.

17.10. MANAGING DEVICE INTERRUPT PROCESSING FOR GUARANTEED POD ISOLATED CPUS

The Node Tuning Operator can manage host CPUs by dividing them into reserved CPUs for cluster and operating system housekeeping duties, including pod infra containers, and isolated CPUs for application containers to run the workloads. By completing these tasks, you can set CPUs for low-latency workloads as isolated workloads.

Device interrupts are load balanced between all isolated and reserved CPUs to avoid CPUs being overloaded, with the exception of CPUs where there is a guaranteed pod running. Guaranteed pod CPUs are prevented from processing device interrupts when the relevant annotations are set for the pod.

In the performance profile, **globallyDisableIrqLoadBalancing** is used to manage whether device interrupts are processed or not. For certain workloads, the reserved CPUs are not always sufficient for dealing with device interrupts, and for this reason, device interrupts are not globally disabled on the isolated CPUs. By default, Node Tuning Operator does not disable device interrupts on isolated CPUs.

17.10.1. Finding the effective IRQ affinity setting for a node

Some IRQ controllers lack support for an IRQ affinity setting and might always expose all online CPUs as the IRQ mask. Because these IRQ controllers effectively run on CPU 0, you must find the effective IRQ affinity setting for a node.

The following are examples of drivers and hardware that Red Hat are aware lack support for IRQ affinity setting. The list is, by no means, exhaustive:

- Some RAID controller drivers, such as **megaraid_sas**
- Many non-volatile memory express (NVMe) drivers
- Some LAN on motherboard (LOM) network controllers
- The driver uses **managed_irqs**



NOTE

The reason they do not support IRQ affinity setting might be associated with factors such as the type of processor, the IRQ controller, or the circuitry connections in the motherboard.

If the effective affinity of any IRQ is set to an isolated CPU, it might be a sign of some hardware or driver not supporting IRQ affinity setting. To find the effective affinity, log in to the host and run the following command:

```
$ find /proc/irq -name effective_affinity -printf "%p: " -exec cat {} \;
```

Example output

```
/proc/irq/0/effective_affinity: 1
/proc/irq/1/effective_affinity: 8
/proc/irq/2/effective_affinity: 0
/proc/irq/3/effective_affinity: 1
/proc/irq/4/effective_affinity: 2
/proc/irq/5/effective_affinity: 1
/proc/irq/6/effective_affinity: 1
/proc/irq/7/effective_affinity: 1
/proc/irq/8/effective_affinity: 1
/proc/irq/9/effective_affinity: 2
/proc/irq/10/effective_affinity: 1
/proc/irq/11/effective_affinity: 1
/proc/irq/12/effective_affinity: 4
/proc/irq/13/effective_affinity: 1
/proc/irq/14/effective_affinity: 1
/proc/irq/15/effective_affinity: 1
/proc/irq/24/effective_affinity: 2
/proc/irq/25/effective_affinity: 4
/proc/irq/26/effective_affinity: 2
/proc/irq/27/effective_affinity: 1
/proc/irq/28/effective_affinity: 8
/proc/irq/29/effective_affinity: 4
/proc/irq/30/effective_affinity: 4
/proc/irq/31/effective_affinity: 8
```

```
/proc/irq/32/effective_affinity: 8
/proc/irq/33/effective_affinity: 1
/proc/irq/34/effective_affinity: 2
```

Some drivers use **managed_irqs**, whose affinity is managed internally by the kernel and userspace cannot change the affinity. In some cases, these IRQs might be assigned to isolated CPUs. For more information about **managed_irqs**, see "Affinity of managed interrupts cannot be changed even if they target isolated CPU".

Additional resources

- [Affinity of managed interrupts cannot be changed even if they target isolated CPU](#)

17.10.2. Configuring node interrupt affinity

Configure a cluster node for IRQ dynamic load balancing to control which cores can receive device interrupt requests (IRQ).

Prerequisites

- For core isolation, all server hardware components must support IRQ affinity. To check if the hardware components of your server support IRQ affinity, view the hardware specifications of the server or contact your hardware provider.

Procedure

1. Log in to the OpenShift Container Platform cluster as a user with cluster-admin privileges.
2. Set the performance profile **apiVersion** to use **performance.openshift.io/v2**.
3. Remove the **globallyDisableIrqLoadBalancing** field or set it to **false**.
4. Set the appropriate isolated and reserved CPUs. The following snippet illustrates a profile that reserves 2 CPUs. IRQ load-balancing is enabled for pods running on the **isolated** CPU set:

```
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: dynamic-irq-profile
spec:
  cpu:
    isolated: 2-5
    reserved: 0-1
  ...
```



NOTE

When you configure reserved and isolated CPUs, operating system processes, kernel processes, and systemd services run on reserved CPUs. Infrastructure pods run on any CPU except where the low latency workload is running. Low latency workload pods run on exclusive CPUs from the isolated pool. For more information, see "Partitioning CPUs for infra and application containers".

17.11. CONFIGURING MEMORY PAGE SIZES

By configuring memory page sizes, system administrators can implement more efficient memory management on a specific node to suit workload requirements. The Node Tuning Operator provides a method for configuring huge pages and kernel page sizes by using a performance profile.

17.11.1. Configuring kernel page sizes

Use the **kernelPageSize** specification in a performance profile to configure the kernel page size on a specific node. Specify larger kernel page sizes for memory-intensive, high-performance workloads.



NOTE

For nodes with an x86_64 or AMD64 architecture, you can only specify **4k** for the **kernelPageSize** specification. For nodes with an AArch64 architecture, you can specify **4k** or **64k** for the **kernelPageSize** specification. You must disable the realtime kernel before you can use the **64k** option. The default value is **4k**.

Prerequisites

- Access to the cluster as a user with the **cluster-admin** role.
- Install the OpenShift CLI (**oc**).

Procedure

1. Create a performance profile to target nodes where you want to configure the kernel page size by creating a YAML file that defines the **PerformanceProfile** resource:

Example pp-kernel-pages.yaml file

```
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: example-performance-profile
  #...
spec:
  kernelPageSize: "64k"
  realTimeKernel:
    enabled: false
  nodeSelector:
    node-role.kubernetes.io/worker: ""
```

where:

spec.kernelPageSize

Specifies a kernel page size of **64k**. You can only specify **64k** for nodes with an AArch64 architecture. The default value is **4k**.

spec.realTimeKernel.enabled:false

Specifies whether to disable the realtime kernel. A setting of **false** disables the kernel. You must disable the realtime kernel to use the **64k** kernel page size option.

spec.nodeSelector.node-role.kubernetes.io/worker

Specifies targets nodes with the **worker** role.

2. Apply the performance profile to the cluster:

```
$ oc create -f pp-kernel-pages.yaml
```

Example output

```
performanceprofile.performance.openshift.io/example-performance-profile created
```

Verification

1. Start a debug session on the node where you applied the performance profile by running the following command:

```
$ oc debug node/<node_name>
```

- **<node_name>**: Replace **<node_name>** with the name of the node with the performance profile applied.
2. Verify that the kernel page size is set to the value you specified in the performance profile by running the following command:

```
$ getconf PAGESIZE
```

Example output

```
65536
```

17.11.2. Configuring huge pages

Because nodes must pre-allocate huge pages used in an OpenShift Container Platform cluster, use the Node Tuning Operator to allocate huge pages on a specific node.

OpenShift Container Platform provides a method for creating and allocating huge pages. Node Tuning Operator provides an easier method for doing this using the performance profile.

Procedure

- In the **hugepages.pages** section of the performance profile, specify multiple blocks of **size**, **count**, and, optionally, **node**:

Example configuration

```
hugepages:
  defaultHugepagesSize: "1G"
  pages:
    - size: "1G"
      count: 4
      node: 0
  # ...
```

where:

hugepages.pages.node

Specifies the **node** that is the NUMA node in which the huge pages are allocated. If you omit **node**, the pages are evenly spread across all NUMA nodes.



NOTE

Wait for the relevant machine config pool status that indicates the update is finished.

These are the only configuration steps you need to do to allocate huge pages.

Verification

- To verify the configuration, see the **/proc/meminfo** file on the node:

```
$ oc debug node/ip-10-0-141-105.ec2.internal
```

```
# grep -i huge /proc/meminfo
```

Example output

```
AnonHugePages: ##### ##
ShmemHugePages: 0 kB
HugePages_Total: 2
HugePages_Free: 2
HugePages_Rsvd: 0
HugePages_Surp: 0
Hugepagesize: ##### ##
Hugetlb: ##### ##
```

- Use **oc describe** to report the new size:

```
$ oc describe node worker-0.ocp4poc.example.com | grep -i huge
```

Example output

```
hugepages-1g=true
hugepages-###: ###
hugepages-###: ###
```

17.11.3. Allocating multiple huge page sizes

You can request huge pages with different sizes under the same container. By doing this task, you can define more complicated pods consisting of containers with different huge page size needs.

The following example, shows you how to define sizes **1G** and **2M**. The Node Tuning Operator configures both sizes on the node.

Procedure

- Edit the **PerformanceProfile** object to define **1G** and **2M** sizes for the huge pages. The Node Tuning Operator configures both sizes on the node.

```

apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: example-performance-profile
#...
spec:
  hugepages:
    defaultHugepagesSize: 1G
  pages:
    - count: 1024
      node: 0
      size: 2M
    - count: 4
      node: 1
      size: 1G
# ...

```

17.12. REDUCING NIC QUEUES USING THE NODE TUNING OPERATOR

The Node Tuning Operator facilitates reducing NIC queues for enhanced performance. Adjustments are made using the performance profile, allowing customization of queues for different network devices.

17.12.1. Adjusting the NIC queues with the performance profile

You can use a performance profile to adjust the queue count for each network device. By using the Node Tuning Operator, you can reduce NIC queues for enhanced performance.

Supported network devices:

- Non-virtual network devices
- Network devices that support multiple queues (channels)

Unsupported network devices:

- Pure software network interfaces
- Block devices
- Intel DPDK virtual functions

Prerequisites

- Access to the cluster as a user with the **cluster-admin** role.
- Install the OpenShift CLI (**oc**).

Procedure

1. Log in to the OpenShift Container Platform cluster running the Node Tuning Operator as a user with **cluster-admin** privileges.
2. Create and apply a performance profile appropriate for your hardware and topology. For guidance on creating a profile, see the "Creating a performance profile" section.
3. Edit this created performance profile:

```
$ oc edit -f <your_profile_name>.yaml
```

4. Populate the **spec** field with the **net** object. The object list can contain two fields:
 - **userLevelNetworking** is a required field specified as a boolean flag. If **userLevelNetworking** is **true**, the queue count is set to the reserved CPU count for all supported devices. The default is **false**.
 - **devices** is an optional field specifying a list of devices that will have the queues set to the reserved CPU count. If the device list is empty, the configuration applies to all network devices. The configuration is as follows:
 - **interfaceName**: This field specifies the interface name, and it supports shell-style wildcards, which can be positive or negative.
 - Example wildcard syntax is as follows: **<string>.***
 - Negative rules are prefixed with an exclamation mark. To apply the net queue changes to all devices other than the excluded list, use **!<device>**, for example, **!eno1**.
 - **vendorID**: The network device vendor ID represented as a 16-bit hexadecimal number with a **0x** prefix.
 - **deviceID**: The network device ID (model) represented as a 16-bit hexadecimal number with a **0x** prefix.



NOTE

When a **deviceID** is specified, the **vendorID** must also be defined. A device that matches all of the device identifiers specified in a device entry **interfaceName**, **vendorID**, or a pair of **vendorID** plus **deviceID** qualifies as a network device. This network device then has its net queues count set to the reserved CPU count.

When two or more devices are specified, the net queues count is set to any net device that matches one of them.

5. Set the queue count to the reserved CPU count for all devices by using this example performance profile:

```
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: manual
spec:
  cpu:
```

```

isolated: 3-51,55-103
reserved: 0-2,52-54
net:
  userLevelNetworking: true
nodeSelector:
  node-role.kubernetes.io/worker-cnf: ""
# ...

```

6. Set the queue count to the reserved CPU count for all devices matching any of the defined device identifiers by using this example performance profile:

```

apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: manual
spec:
  cpu:
    isolated: 3-51,55-103
    reserved: 0-2,52-54
  net:
    userLevelNetworking: true
  devices:
    - interfaceName: "eth0"
    - interfaceName: "eth1"
    - vendorID: "0x1af4"
      deviceID: "0x1000"
  nodeSelector:
    node-role.kubernetes.io/worker-cnf: ""
# ...

```

7. Set the queue count to the reserved CPU count for all devices starting with the interface name **eth** by using this example performance profile:

```

apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: manual
spec:
  cpu:
    isolated: 3-51,55-103
    reserved: 0-2,52-54
  net:
    userLevelNetworking: true
  devices:
    - interfaceName: "eth*"
  nodeSelector:
    node-role.kubernetes.io/worker-cnf: ""
# ...

```

8. Set the queue count to the reserved CPU count for all devices with an interface named anything other than **eno1** by using this example performance profile:

```

apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:

```

```

name: manual
spec:
  cpu:
    isolated: 3-51,55-103
    reserved: 0-2,52-54
  net:
    userLevelNetworking: true
  devices:
    - interfaceName: "eno1"
  nodeSelector:
    node-role.kubernetes.io/worker-cnf: ""
# ...

```

9. Set the queue count to the reserved CPU count for all devices that have an interface name **eth0**, **vendorID** of **0x1af4**, and **deviceID** of **0x1000** by using this example performance profile:

```

apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: manual
spec:
  cpu:
    isolated: 3-51,55-103
    reserved: 0-2,52-54
  net:
    userLevelNetworking: true
  devices:
    - interfaceName: "eth0"
    - vendorID: "0x1af4"
      deviceID: "0x1000"
  nodeSelector:
    node-role.kubernetes.io/worker-cnf: ""
# ...

```

10. Apply the updated performance profile:

```
$ oc apply -f <your_profile_name>.yaml
```

17.12.2. Verifying the queue status

To ensure that your performance profile changes are active, verify the queue status.

By reviewing the provided examples, you can confirm that specific tuning configurations are successfully applied to your environment.

In this section, several examples illustrate different performance profiles and how to verify the changes are applied.

Example 1

Example 1 demonstrates that the net queue count that is set to the reserved CPU count (2) for *all* supported devices.

The relevant section from the performance profile is:

```
apiVersion: performance.openshift.io/v2
```

```

metadata:
  name: performance
spec:
  kind: PerformanceProfile
  spec:
    cpu:
      reserved: 0-1 #total = 2
      isolated: 2-8
    net:
      userLevelNetworking: true
# ...

```

The following command displays the status of the queues associated with a device:



NOTE

Run this command on the node where the performance profile was applied.

```
$ ethtool -l <device>
```

The following command verifies the queue status before the profile is applied:

```
$ ethtool -l ens4
```

Example output

```

Channel parameters for ens4:
Pre-set maximums:
RX:      0
TX:      0
Other:    0
Combined: 4
Current hardware settings:
RX:      0
TX:      0
Other:    0
Combined: 4

```

The following command verifies the queue status after the profile is applied:

```
$ ethtool -l ens4
```

Example output

```

Channel parameters for ens4:
Pre-set maximums:
RX:      0
TX:      0
Other:    0
Combined: 4
Current hardware settings:
RX:      0

```



```
TX:    0
Other:  0
Combined: 2
```

- **Combined:** Specifies the combined channel that shows the total count of reserved CPUs for *all* supported devices is 2. This matches what is configured in the performance profile.

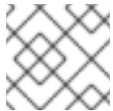
Example 2

Example 2 demonstrates that the net queue count is set to the reserved CPU count (2) for *all* supported network devices with a specific **vendorID**.

The relevant section from the performance profile is:

```
apiVersion: performance.openshift.io/v2
metadata:
  name: performance
spec:
  kind: PerformanceProfile
  spec:
    cpu:
      reserved: 0-1
      isolated: 2-8
    net:
      userLevelNetworking: true
      devices:
        - vendorID = 0x1af4
# ...
```

The following command displays the status of the queues associated with a device:



NOTE

Run this command on the node where the performance profile was applied.

```
$ ethtool -l <device>
```

The following command verifies the queue status after the profile is applied:

```
$ ethtool -l ens4
```

Example output

```
Channel parameters for ens4:
Pre-set maximums:
RX:    0
TX:    0
Other:  0
Combined: 4
Current hardware settings:
RX:    0
TX:    0
Other:  0
Combined: 2
```

- **Combined:** Specifies that the total count of reserved CPUs for all supported devices with **vendorID=0x1af4** is 2. For example, if there is another network device **ens2** with **vendorID=0x1af4** it will also have total net queues of 2. This matches what is configured in the performance profile.

Example 3

Example 3 shows that the net queue count is set to the reserved CPU count (2) for *all* supported network devices that match any of the defined device identifiers. The command **udevadm info** provides a detailed report on a device. In this example the devices are:

```
# udevadm info -p /sys/class/net/ens4
...
E: ID_MODEL_ID=0x1000
E: ID_VENDOR_ID=0x1af4
E: INTERFACE=ens4
...
```

```
# udevadm info -p /sys/class/net/eth0
...
E: ID_MODEL_ID=0x1002
E: ID_VENDOR_ID=0x1001
E: INTERFACE=eth0
...
```

Set the net queues to 2 for a device with **interfaceName** equal to **eth0** and any devices that have a **vendorID=0x1af4** with the following performance profile:

```
apiVersion: performance.openshift.io/v2
metadata:
  name: performance
spec:
  kind: PerformanceProfile
  spec:
    cpu:
      reserved: 0-1 #total = 2
      isolated: 2-8
    net:
      userLevelNetworking: true
      devices:
        - interfaceName = eth0
        - vendorID = 0x1af4
# ...
```

The following command verifies the queue status after the profile is applied:

```
$ ethtool -l ens4
```

Example output

```
Channel parameters for ens4:
Pre-set maximums:
RX:      0
TX:      0
```

```
Other:    0
Combined: 4
Current hardware settings:
RX:      0
TX:      0
Other:    0
Combined: 2
```

- **Combined:** Specifies that the total count of reserved CPUs for all supported devices with **vendorID=0x1af4** is set to 2.

For example, if there is another network device **ens2** with **vendorID=0x1af4**, it will also have the total net queues set to 2. Similarly, a device with **interfaceName** equal to **eth0** will have total net queues set to 2.

17.12.3. Logging associated with adjusting NIC queues

To verify NIC queue adjustments, review the Tuned daemon logs. These log messages detail the assigned devices that are recorded in the respective Tuned daemon logs.

The following messages might be recorded to the **/var/log/tuned/tuned.log** file:

- An **INFO** message is recorded detailing the successfully assigned devices:

```
INFO tuned.plugins.base: instance net_test (net): assigning devices ens1, ens2, ens3
```

- A **WARNING** message is recorded if none of the devices can be assigned:

```
WARNING tuned.plugins.base: instance net_test: no matching devices available
```

CHAPTER 18. TUNING HOSTED CONTROL PLANES FOR LOW LATENCY WITH THE PERFORMANCE PROFILE

Tune hosted control planes for low latency by applying a performance profile. With the performance profile, you can restrict CPUs for infrastructure and application containers and configure huge pages, Hyper-Threading, and CPU partitions for latency-sensitive processes.

18.1. CREATING A PERFORMANCE PROFILE FOR HOSTED CONTROL PLANES

You can create a cluster performance profile by using the Performance Profile Creator (PPC) tool. The PPC is a function of the Node Tuning Operator.

The PPC combines information about your cluster with user-supplied configurations to generate a performance profile that is appropriate to your hardware, topology, and use-case. The following high-level workflow creates and applies a performance profile in your cluster:

1. Gather information about your cluster by using the **must-gather** command.
2. Use the PPC tool to create a performance profile.
3. Apply the performance profile to your cluster.

18.1.1. Gathering data about your hosted control planes cluster for the PPC

The Performance Profile Creator (PPC) tool requires **must-gather** data. As a cluster administrator, run the **must-gather** command to capture information about your cluster.

Prerequisites

- You have **cluster-admin** role access to the management cluster.
- You installed the OpenShift CLI (**oc**).

Procedure

1. Export the management cluster **kubeconfig** file by running the following command:

```
$ export MGMT_KUBECONFIG=<path_to_mgmt_kubeconfig>
```

2. List all node pools across all namespaces by running the following command:

```
$ oc --kubeconfig="$MGMT_KUBECONFIG" get np -A
```

Example output

```

NAMESPACE NAME          CLUSTER  DESIRED NODES  CURRENT NODES
AUTOSCALING  AUTOREPAIR  VERSION  UPDATINGVERSION  UPDATINGCONFIG
MESSAGE
clusters  democluster-us-east-1a  democluster  1              1              False         False
4.17.0    False        True

```

- The output shows the namespace **clusters** in the management cluster where the **NodePool** resource is defined.
 - The name of the **NodePool** resource, for example **democluster-us-east-1a**.
 - The **HostedCluster** this **NodePool** belongs to. For example, **democluster**.
3. On the management cluster, run the following command to list available secrets:

```
$ oc get secrets -n clusters
```

Example output

NAME	TYPE	DATA	AGE
builder-dockercfg-25qpp	kubernetes.io/dockercfg	1	128m
default-dockercfg-mkviz	kubernetes.io/dockercfg	1	128m
democluster-admin-kubeconfig	Opaque	1	127m
democluster-etcd-encryption-key	Opaque	1	128m
democluster-kubeadmin-password	Opaque	1	126m
democluster-pull-secret	Opaque	1	128m
deployer-dockercfg-8lfpd	kubernetes.io/dockercfg	1	128m

4. Extract the **kubeconfig** file for the hosted cluster by running the following command:

```
$ oc get secret <secret_name> -n <cluster_namespace> -o jsonpath='{.data.kubeconfig}' |
base64 -d > hosted-cluster-kubeconfig
```

Example

```
$ oc get secret democluster-admin-kubeconfig -n clusters -o jsonpath='{.data.kubeconfig}' |
base64 -d > hosted-cluster-kubeconfig
```

5. To create a **must-gather** bundle for the hosted cluster, open a separate terminal window and run the following commands:
- Export the hosted cluster **kubeconfig** file:

```
$ export HC_KUBECONFIG=<path_to_hosted_cluster_kubeconfig>
```

Example

```
$ export HC_KUBECONFIG=~/.hostedcpkube/hosted-cluster-kubeconfig
```

- Navigate to the directory where you want to store the **must-gather** data.
- Gather the troubleshooting data for your hosted cluster:

```
$ oc --kubeconfig="$HC_KUBECONFIG" adm must-gather
```

- Create a compressed file from the **must-gather** directory that was just created in your working directory. For example, on a computer that uses a Linux operating system, run the following command:

■

```
$ tar -czvf must-gather.tar.gz must-gather.local.1203869488012141147
```

Additional resources

- [Gathering data about your cluster](#)

18.1.2. Running the Performance Profile Creator on a hosted cluster using Podman

As a cluster administrator, you can use Podman with the Performance Profile Creator (PPC) tool to create a performance profile.

For more information about PPC arguments, see "Performance Profile Creator arguments".

The PPC tool is designed to be hosted-cluster aware. When it detects a hosted cluster from the **must-gather** data it automatically takes the following actions:

- Recognizes that there is no machine config pool (MCP).
- Uses node pools as the source of truth for compute node configurations instead of MCPs.
- Does not require you to specify the **node-pool-name** value explicitly unless you want to target a specific pool.



IMPORTANT

The PPC uses the **must-gather** data from your hosted cluster to create the performance profile. If you make any changes to your cluster, such as relabeling a node targeted for performance configuration, you must re-create the **must-gather** data before running PPC again.

Prerequisites

- Access to the cluster as a user with the **cluster-admin** role.
- A hosted cluster is installed.
- Installation of Podman and the OpenShift CLI (**oc**).
- Access to the Node Tuning Operator image.
- Access to the **must-gather** data for your cluster.

Procedure

1. On the hosted cluster, use Podman to authenticate to **registry.redhat.io** by running the following command:

```
$ podman login registry.redhat.io
```

```
Username: <user_name>
```

```
Password: <password>
```

2. Create a performance profile on the hosted cluster, by running the following command. The example uses sample PPC arguments and values:

```
$ podman run --entrypoint performance-profile-creator \
-v /path/to/must-gather:/must-gather:z \
registry.redhat.io/openshift4/ose-cluster-node-tuning-rhel9-operator:v4.22 \
--must-gather-dir-path /must-gather \
--reserved-cpu-count=2 \
--rt-kernel=false \
--split-reserved-cpus-across-numa=false \
--topology-manager-policy=single-numa-node \
--node-pool-name=democluster-us-east-1a \
--power-consumption-mode=ultra-low-latency \
--offlined-cpu-count=1 \
> my-hosted-cp-performance-profile.yaml
```

where:

/path/to/must-gather:/must-gather:z

Specifies the local directory to mount where the output of an **oc adm must-gather** was created into the container.

reserved-cpu-count=2

Specifies two reserved CPUs.

rt-kernel=false

Specifies whether to disable the real-time kernel. A setting of **false** disables the kernel.

split-reserved-cpus-across-numa=false

Specifies whether to split CPUs across NUMA nodes. A setting of **false** disables the CPU-splitting.

topology-manager-policy=single-numa-node

Specifies the NUMA topology policy. If installing the NUMA Resources Operator, this must be set to **single-numa-node**.

power-consumption-mode=ultra-low-latency

Specifies minimal latency at the cost of increased power consumption.

offlined-cpu-count=1

Specifies one offlined CPU.

Example output

```
level=info msg="Nodes names targeted by democluster-us-east-1a pool are: ip-10-0-129-110.ec2.internal "
level=info msg="NUMA cell(s): 1"
level=info msg="NUMA cell 0 : [0 2 1 3]"
level=info msg="CPU(s): 4"
level=info msg="2 reserved CPUs allocated: 0,2 "
level=info msg="1 isolated CPUs allocated: 1"
level=info msg="Additional Kernel Args based on configuration: []"
```

3. Review the created YAML file by running the following command:

```
$ cat my-hosted-cp-performance-profile
```

Example output

```

---
apiVersion: v1
data:
  tuning: |
    apiVersion: performance.openshift.io/v2
    kind: PerformanceProfile
    metadata:
      creationTimestamp: null
      name: performance
    spec:
      cpu:
        isolated: "1"
        offlined: "3"
        reserved: 0,2
      net:
        userLevelNetworking: false
      nodeSelector:
        node-role.kubernetes.io/worker: ""
      numa:
        topologyPolicy: single-numa-node
      realTimeKernel:
        enabled: false
      workloadHints:
        highPowerConsumption: true
        perPodPowerManagement: false
        realTime: true
    status: {}
kind: ConfigMap
metadata:
  name: performance
  namespace: clusters

```

Additional resources

- [Performance Profile Creator arguments](#)

18.1.3. Configuring low-latency tuning in a hosted cluster

To set low latency with the performance profile on the nodes in your hosted cluster, you can use the Node Tuning Operator. In hosted control planes, you can configure low-latency tuning by creating config maps that contain **Tuned** objects and referencing those config maps in your node pools.

The tuned object in this case is a **PerformanceProfile** object that defines the performance profile you want to apply to the nodes in a node pool.

Procedure

1. Export the management cluster **kubeconfig** file by running the following command:

```
$ export MGMT_KUBECONFIG=<path_to_mgmt_kubeconfig>
```

2. Create the **ConfigMap** object in the management cluster by running the following command:


```
$ oc --kubeconfig="$MGMT_KUBECONFIG" apply -f my-hosted-cp-performance-profile.yaml
```

3. Edit the **NodePool** object in the **clusters** namespace adding the **spec.tuningConfig** field and the name of the created performance profile in that field by running the following command:

```
$ oc edit np -n clusters
```

```
apiVersion: hypershift.openshift.io/v1beta1
kind: NodePool
metadata:
  annotations:
    hypershift.openshift.io/nodePoolCurrentConfig: 2f752a2c
    hypershift.openshift.io/nodePoolCurrentConfigVersion: 998aa3ce
    hypershift.openshift.io/nodePoolPlatformMachineTemplate: democluster-us-east-1 a-
3dff55ec
  creationTimestamp: "2025-04-09T09:41:55Z"
  finalizers:
    - hypershift.openshift.io/finalizer
  generation: 1
  labels:
    hypershift.openshift.io/auto-created-for-infra: democluster
  name: democluster-us-east-1a
  namespace: clusters
  ownerReferences:
    - apiVersion: hypershift.openshift.io/v1beta1
      kind: HostedCluster
      name: democluster
      uid: af77e390-c289-433c-9d29-3aee8e5dc76f
  resourceVersion: "53056"
  uid: 11efa47c-5a7b-476c-85cf-a274f748a868
spec:
  tuningConfig:
    - name: performance
  arch: amd64
  clusterName: democluster
  management:
```



NOTE

You can reference the same profile in multiple node pools. In hosted control planes, the Node Tuning Operator appends a hash of the node pool name and namespace to the name of the **Tuned** custom resources to distinguish them. After you make the changes, the system detects that a configuration change is required and starts a rolling update of the nodes in that pool to apply the new configuration.

Verification

1. List all node pools across all namespaces by running the following command:

```
$ oc --kubeconfig="$MGMT_KUBECONFIG" get np -A
```

Example output

NAMESPACE	NAME	CLUSTER	DESIRED NODES	CURRENT NODES	AUTOSCALING	AUTOREPAIR	VERSION	UPDATINGVERSION	UPDATINGCONFIG	MESSAGE
clusters	democluster-us-east-1a	democluster	1	1	False	False	4.17.0	True		

**NOTE**

The **UPDATINGCONFIG** field indicates whether the node pool is in the process of updating its configuration. During this update, the **UPDATINGCONFIG** field in the node pool's status becomes **True**. The new configuration is considered fully applied only when the **UPDATINGCONFIG** field returns to **False**.

- List all config maps in the **clusters-democluster** namespace by running the following command:

```
$ oc --kubeconfig="$MGMT_KUBECONFIG" get cm -n clusters-democluster
```

Example output

NAME	DATA	AGE
aggregator-client-ca	1	69m
auth-config	1	68m
aws-cloud-config	1	68m
aws-ebs-csi-driver-trusted-ca-bundle	1	66m
...	1	67m
kubelet-client-ca	1	69m
kubeletconfig-performance-democluster-us-east-1a	1	22m
...		
ovnkube-identity-cm	2	66m
performance-democluster-us-east-1a	1	22m
...		
tuned-performance-democluster-us-east-1a	1	22m

The output shows a kubeletconfig **kubeletconfig-performance-democluster-us-east-1a** and a performance profile **performance-democluster-us-east-1a** has been created. The Node Tuning Operator syncs the **Tuned** objects into the hosted cluster. You can verify which **Tuned** objects are defined and which profiles are applied to each node.

- List available secrets on the management cluster by running the following command:

```
$ oc get secrets -n clusters
```

Example output

NAME	TYPE	DATA	AGE
builder-dockercfg-25qpp	kubernetes.io/dockercfg	1	128m
default-dockercfg-mkvlz	kubernetes.io/dockercfg	1	128m
democluster-admin-kubeconfig	Opaque	1	127m
democluster-etcd-encryption-key	Opaque	1	128m

```
democluster-kubeadmin-password Opaque 1 126m
democluster-pull-secret Opaque 1 128m
deployer-dockerconfig-8lfpd kubernetes.io/dockercfg 1 128m
```

4. Extract the **kubeconfig** file for the hosted cluster by running the following command:

```
$ oc get secret <secret_name> -n clusters -o jsonpath='{.data.kubeconfig}' | base64 -d >
hosted-cluster-kubeconfig
```

Example

```
$ oc get secret democluster-admin-kubeconfig -n clusters -o jsonpath='{.data.kubeconfig}' |
base64 -d > hosted-cluster-kubeconfig
```

5. Export the hosted cluster kubeconfig by running the following command:

```
$ export HC_KUBECONFIG=<path_to_hosted-cluster-kubeconfig>
```

6. Verify that the kubeletconfig is mirrored in the hosted cluster by running the following command:

```
$ oc --kubeconfig="$HC_KUBECONFIG" get cm -n openshift-config-managed | grep kubelet
```

Example output

```
kubelet-serving-ca 1 79m
kubeletconfig-performance-democluster-us-east-1a 1 15m
```

7. Verify that the **single-numa-node** policy is set on the hosted cluster by running the following command:

```
$ oc --kubeconfig="$HC_KUBECONFIG" get cm kubeletconfig-performance-democluster-us-
east-1a -o yaml -n openshift-config-managed | grep single
```

Example output

```
topologyManagerPolicy: single-numa-node
```

CHAPTER 19. PROVISIONING REAL-TIME AND LOW LATENCY WORKLOADS

If your organization needs high performance computing and low, predictable latency, especially in the financial and telecommunications industries, you can use the Node Tuning Operator to implement automatic tuning to achieve low latency performance and consistent response time for OpenShift Container Platform applications.

You use the performance profile configuration to make these changes.

You can update the kernel to kernel-rt, reserve CPUs for cluster and operating system housekeeping duties, including pod infra containers, isolate CPUs for application containers to run the workloads, and disable unused CPUs to reduce power consumption.



NOTE

When writing your applications, follow the general recommendations described in [RHEL for Real Time processes and threads](#).

Additional resources

- [Creating a performance profile](#)

19.1. SCHEDULING A LOW LATENCY WORKLOAD ONTO A COMPUTE NODE

You can schedule low latency workloads onto a compute node where a performance profile that configures real-time capabilities is applied.



NOTE

To schedule a workload on specific nodes, use label selectors in the **Pod** custom resource (CR). The label selectors must match the nodes that are attached to the machine config pool that was configured for low latency by the Node Tuning Operator.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.
- You have applied a performance profile in the cluster that tunes compute nodes for low latency workloads.

Procedure

1. Create a **Pod** CR for the low latency workload and apply it in the cluster, for example:

Example Pod spec configured to use real-time processing

```
apiVersion: v1
kind: Pod
metadata:
```

```

name: dynamic-low-latency-pod
annotations:
  cpu-quota.crio.io: "disable"
  cpu-load-balancing.crio.io: "disable"
  irq-load-balancing.crio.io: "disable"
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: dynamic-low-latency-pod
    image: "registry.redhat.io/openshift4/cnf-tests-rhel8:v4.22"
    command: ["sleep", "10h"]
    resources:
      requests:
        cpu: 2
        memory: "200M"
      limits:
        cpu: 2
        memory: "200M"
    securityContext:
      allowPrivilegeEscalation: false
      capabilities:
        drop: [ALL]
    nodeSelector:
      node-role.kubernetes.io/worker-cnf: ""
    runtimeClassName: performance-dynamic-low-latency-profile 1
# ...

```

where

metadata.annotations.cpu-quota.crio.io

Disables the CPU completely fair scheduler (CFS) quota at the pod run time.

metadata.annotations.cpu-load-balancing.crio.io

Disables CPU load balancing.

metadata.annotations.irq-load-balancing.crio.io

Opts the pod out of interrupt handling on the node.

spec.nodeSelector.node-role.kubernetes.io/worker-cnf

The **nodeSelector** label must match the label that you specify in the **Node** CR.

spec.runtimeClassName

runtimeClassName must match the name of the performance profile configured in the cluster.

2. Enter the pod **runtimeClassName** in the form `performance-<profile_name>`, where `<profile_name>` is the **name** from the **PerformanceProfile** YAML. In the previous YAML example, the **name** is **performance-dynamic-low-latency-profile**.
3. Ensure the pod is running correctly. Status should be **running**, and the correct **cnf-worker** node should be set.

```
$ oc get pod -o wide
```

Expected output

```
NAME                READY  STATUS   RESTARTS  AGE    IP           NODE
dynamic-low-latency-pod 1/1    Running  0         5h33m  10.131.0.10  cnf-
worker.example.com
```

4. Get the CPUs that the pod configured for IRQ dynamic load balancing runs on:

```
$ oc exec -it dynamic-low-latency-pod -- /bin/bash -c "grep Cpus_allowed_list /proc/self/status | awk '{print $2}'"
```

Expected output

```
Cpus_allowed_list: 2-3
```

Verification

1. Log in to the node to verify the configuration.

```
$ oc debug node/<node-name>
```

2. Verify that you can use the node file system:

```
sh-4.4# chroot /host
```

Expected output

```
sh-4.4#
```

3. Ensure the default system CPU affinity mask does not include the **dynamic-low-latency-pod** CPUs, for example, CPUs 2 and 3.

```
sh-4.4# cat /proc/irq/default_smp_affinity
```

Example output

```
33
```

4. Ensure the system IRQs are not configured to run on the **dynamic-low-latency-pod** CPUs:

```
sh-4.4# find /proc/irq/ -name smp_affinity_list -exec sh -c 'i="$1"; mask=$(cat $i); file=$(echo $i); echo $file: $mask' _ {} \;
```

Example output

```
/proc/irq/0/smp_affinity_list: 0-5
/proc/irq/1/smp_affinity_list: 5
/proc/irq/2/smp_affinity_list: 0-5
/proc/irq/3/smp_affinity_list: 0-5
/proc/irq/4/smp_affinity_list: 0
/proc/irq/5/smp_affinity_list: 0-5
```

```

/proc/irq/6/smp_affinity_list: 0-5
/proc/irq/7/smp_affinity_list: 0-5
/proc/irq/8/smp_affinity_list: 4
/proc/irq/9/smp_affinity_list: 4
/proc/irq/10/smp_affinity_list: 0-5
/proc/irq/11/smp_affinity_list: 0
/proc/irq/12/smp_affinity_list: 1
/proc/irq/13/smp_affinity_list: 0-5
/proc/irq/14/smp_affinity_list: 1
/proc/irq/15/smp_affinity_list: 0
/proc/irq/24/smp_affinity_list: 1
/proc/irq/25/smp_affinity_list: 1
/proc/irq/26/smp_affinity_list: 1
/proc/irq/27/smp_affinity_list: 5
/proc/irq/28/smp_affinity_list: 1
/proc/irq/29/smp_affinity_list: 0
/proc/irq/30/smp_affinity_list: 0-5

```



WARNING

When you tune nodes for low latency, the usage of execution probes in conjunction with applications that require guaranteed CPUs can cause latency spikes. Use other probes, such as a properly configured set of network probes, as an alternative.

Additional resources

- [Placing pods on specific nodes using node selectors](#)
- [Assigning pods to nodes](#)

19.2. CREATING A POD WITH A GUARANTEED QOS CLASS

You can create a pod with a quality of service (QoS) class of **Guaranteed** for high-performance workloads. Configuring a pod with a QoS class of **Guaranteed** ensures that the pod has priority access to the specified CPU and memory resources.

To create a pod with a QoS class of **Guaranteed**, you must apply the following specifications:

- Set identical values for the memory limit and memory request fields for each container in the pod.
- Set identical values for CPU limit and CPU request fields for each container in the pod.

In general, a pod with a QoS class of **Guaranteed** will not be evicted from a node. One exception is during resource contention caused by system daemons exceeding reserved resources. In this scenario, the **kubelet** might evict pods to preserve node stability, starting with the lowest priority pods.

Prerequisites

- Access to the cluster as a user with the **cluster-admin** role.
- The OpenShift CLI (**oc**).

Procedure

1. Create a namespace for the pod by running the following command:

```
$ oc create namespace qos-example
```

- `qos-example`: Specifies a **qos-example** example namespace.

Example output

```
namespace/qos-example created
```

2. Create the **Pod** resource:
 - a. Create a YAML file that defines the **Pod** resource:

Example `qos-example.yaml` file

```
apiVersion: v1
kind: Pod
metadata:
  name: qos-demo
  namespace: qos-example
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: qos-demo-ctr
    image: quay.io/openshifttest/hello-openshift:openshift
    resources:
      limits:
        memory: "200Mi"
        cpu: "1"
      requests:
        memory: "200Mi"
        cpu: "1"
    securityContext:
      allowPrivilegeEscalation: false
    capabilities:
      drop: [ALL]
```

where:

`spec.containers.image`

Specifies public image, such as the **hello-openshift** image.

`spec.containers.resources.limits.memory`

Specifies a memory limit of 200 MB.

spec.containers.resources.limits.cpu

Specifies a CPU limit of 1 CPU.

spec.containers.resources.requests.memory

Specifies a memory request of 200 MB.

spec.containers.resources.requests.cpu

Specifies a CPU request of 1 CPU.

**NOTE**

If you specify a memory limit for a container, but do not specify a memory request, OpenShift Container Platform automatically assigns a memory request that matches the limit. Similarly, if you specify a CPU limit for a container, but do not specify a CPU request, OpenShift Container Platform automatically assigns a CPU request that matches the limit.

- b. Create the **Pod** resource by running the following command:

```
$ oc apply -f qos-example.yaml --namespace=qos-example
```

Example output

```
pod/qos-demo created
```

Verification

- View the **qosClass** value for the pod by running the following command:

```
$ oc get pod qos-demo --namespace=qos-example --output=yaml | grep qosClass
```

Example output

```
qosClass: Guaranteed
```

19.3. DISABLING CPU LOAD BALANCING IN A POD

Functionality to disable or enable CPU load balancing is implemented on the CRI-O level. Before CRI-O disables or enables CPU load balancing, you must ensure certain prerequisites are met.

The pod must use the **performance-*<profile-name>*** runtime class. You can get the proper name by looking at the status of the performance profile, as shown here:

```
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
...
status:
  ...
  runtimeClass: performance-manual
```

The Node Tuning Operator is responsible for the creation of the high-performance runtime handler

config snippet under relevant nodes and for creation of the high-performance runtime class under the cluster. It will have the same content as the default runtime handler except that it enables the CPU load balancing configuration functionality.

To disable the CPU load balancing for the pod, the **Pod** specification must include the following fields:

```
apiVersion: v1
kind: Pod
metadata:
  #...
  annotations:
    #...
    cpu-load-balancing.crio.io: "disable"
    #...
  #...
spec:
  #...
  runtimeClassName: performance-<profile_name>
  #...
```



NOTE

Only disable CPU load balancing when the CPU manager static policy is enabled and for pods with guaranteed QoS that use whole CPUs. Otherwise, disabling CPU load balancing can affect the performance of other containers in the cluster.

19.4. DISABLING POWER SAVING MODE FOR HIGH PRIORITY PODS

To protect high priority workloads when using power saving configurations on a node, apply performance settings at the pod level. This ensures that the configuration applies to all cores used by the pod, maintaining performance stability.

By disabling P-states and C-states at the pod level, you can configure high priority workloads for best performance and lowest latency.

Table 19.1. Configuration for high priority workloads

Annotation	Possible Values	Description
cpu-c-states.crio.io :	• "enable" • "disable" • "max_latency:microseconds"	This annotation allows you to enable or disable C-states for each CPU. Alternatively, you can also specify a maximum latency in microseconds for the C-states. For example, enable C-states with a maximum latency of 10 microseconds with the setting cpu-c-states.crio.io: "max_latency:10" . Set the value to "disable" to provide the best performance for a pod.
cpu-freq-governor.crio.io:	Any supported cpufreq governor .	Sets the cpufreq governor for each CPU. The "performance" governor is recommended for high priority workloads.

Prerequisites

- You have configured power saving in the performance profile for the node where the high priority workload pods are scheduled.

Procedure

1. Add the required annotations to your high priority workload pods. The annotations override the **default** settings.

Example high priority workload annotation

```
apiVersion: v1
kind: Pod
metadata:
  #...
  annotations:
    #...
    cpu-c-states.crio.io: "disable"
    cpu-freq-governor.crio.io: "performance"
    #...
  #...
spec:
  #...
  runtimeClassName: performance-<profile_name>
  #...
```

2. Restart the pods to apply the annotation.

Additional resources

- [Configuring power saving for nodes that run colocated high and low priority workloads](#)

19.5. DISABLING CPU CFS QUOTA

To eliminate CPU throttling for pinned pods, create a pod with the **cpu-quota.crio.io: "disable"** annotation. This annotation disables the CPU completely fair scheduler (CFS) quota when the pod runs.

Procedure

- To eliminate CPU throttling for pinned pods, create a pod with the **cpu-quota.crio.io: "disable"** annotation. This annotation disables the CPU completely fair scheduler (CFS) quota when the pod runs.

Example pod specification with **cpu-quota.crio.io** disabled

```
apiVersion: v1
kind: Pod
metadata:
  annotations:
    cpu-quota.crio.io: "disable"
spec:
  runtimeClassName: performance-<profile_name>
  #...
```

**NOTE**

Only disable CPU CFS quota when the CPU manager static policy is enabled and for pods with guaranteed QoS that use whole CPUs. For example, pods that contain CPU-pinned containers. Otherwise, disabling CPU CFS quota can affect the performance of other containers in the cluster.

Additional resources

- [Recommended firmware configuration for vDU cluster hosts](#)

19.6. CONFIGURING INTERRUPT PROCESSING FOR INDIVIDUAL PODS

To achieve low latency for workloads, some containers require that the CPUs they are pinned to do not process device interrupts. You can use the **irq-load-balancing.crio.io** pod annotation to control whether device interrupts are processed on CPUs where the pinned containers are running.

The annotation supports the following values:

disable

Disables IRQ load balancing for all CPUs allocated to the container. Use this value for latency-sensitive workloads when you want to exclude container CPUs from interrupt handling.

housekeeping

Preserves IRQ handling on the first CPU that is allocated to the container, including that CPU's thread siblings. The subsequent CPUs allocated to the container are excluded from interrupt processing. This configuration also injects the **OPENSIFT_HOUSEKEEPING_CPUS** environment variable into the container. Use this variable to see which CPUs are designated for housekeeping tasks.

You can use the **housekeeping** value to reduce the overall CPU footprint by allowing a small subset of container CPUs to handle both application housekeeping work and system interrupts.

**NOTE**

When using the **housekeeping** value, the CPUs designated for housekeeping handle interrupts for the entire system.

Prerequisites

- You configured a performance profile for the node.
- You set the **globallyDisableIrqLoadBalancing** field to **false** in the performance profile.

Procedure

1. Create the **Pod** resource and configure the **irq-load-balancing.crio.io** annotation:

Example pod specification

```
apiVersion: v1
kind: Pod
metadata:
  name: dpdk-workload
```

```

annotations:
  irq-load-balancing.crio.io: "disable"
spec:
  runtimeClassName: performance-<profile_name>
  containers:
  - name: app
    image: example-image
    resources:
      requests:
        cpu: "8"
        memory: "4Gi"
      limits:
        cpu: "8"
        memory: "4Gi"

```

- **metadata.annotations irq-load-balancing.crio.io**: Specifies if device interrupts are processed on the container CPUs. Set to **disable** to prevent all container CPUs from handling IRQs, or set to **housekeeping** to allow the first allocated CPU and its thread siblings to handle IRQs while excluding the remaining CPUs from IRQ handling.
- **spec.runtimeClassName**: Specifies the runtime class for the performance profile. Replace **<profile_name>** with the name of your performance profile.

2. Apply the **Pod** resource by running the following command:

```
$ oc apply -f pod.yaml
```

Verification

1. Verify the CPUs assigned to the pod:

```
$ oc exec <pod_name> -- cat /sys/fs/cgroup/cpuset/cpus
```

2. For pods using the **housekeeping** annotation, verify the housekeeping CPU environment variable:

```
$ oc exec <pod_name> -- printenv OPENSIFT_HOUSEKEEPING_CPUS
```

Replace **<pod_name>** with the name of the pod.

3. On the worker node, verify the CPUs excluded from IRQ handling:

```
$ grep IRQBALANCE_BANNED_CPUS /etc/sysconfig/irqbalance
```

The output is a hexadecimal bitmask representing the CPUs excluded from IRQ handling.

Additional resources

- [Managing device interrupt processing for guaranteed pod isolated CPUs](#)

CHAPTER 20. DEBUGGING LOW LATENCY NODE TUNING STATUS

Use the **PerformanceProfile** custom resource (CR) status fields for reporting tuning status and debugging latency issues in a cluster node.

20.1. DEBUGGING LOW LATENCY CNF TUNING STATUS

To report tuning status and debug latency degradation issues, use the status fields in the **PerformanceProfile** custom resource (CR). These fields describe the conditions of the reconciliation functionality of an Operator, helping you verify the state of your configuration.

A typical issue can arise when the status of machine config pools that are attached to the performance profile are in a degraded state, causing the **PerformanceProfile** status to degrade. In this case, the machine config pool issues a failure message.

The Node Tuning Operator contains the **performanceProfile.spec.status.Conditions** status field:

```
Status:
Conditions:
  Last Heartbeat Time: 2020-06-02T10:01:24Z
  Last Transition Time: 2020-06-02T10:01:24Z
  Status:              True
  Type:                Available
  Last Heartbeat Time: 2020-06-02T10:01:24Z
  Last Transition Time: 2020-06-02T10:01:24Z
  Status:              True
  Type:                Upgradeable
  Last Heartbeat Time: 2020-06-02T10:01:24Z
  Last Transition Time: 2020-06-02T10:01:24Z
  Status:              False
  Type:                Progressing
  Last Heartbeat Time: 2020-06-02T10:01:24Z
  Last Transition Time: 2020-06-02T10:01:24Z
  Status:              False
  Type:                Degraded
```

The **Status** field contains **Conditions** that specify **Type** values that indicate the status of the performance profile:

Available

All machine configs and Tuned profiles have been created successfully and are available for cluster components, such as NTO, MCO, Kubelet, that are responsible to process them.

Upgradeable

Indicates whether the resources maintained by the Operator are in a state that is safe to upgrade.

Progressing

Indicates that the deployment process from the performance profile has started.

Degraded

Indicates an error if:

- Validation of the performance profile has failed.

- Creation of all relevant components did not complete successfully.

Each of these types contain the following fields:

Status

The state for the specific type (**true** or **false**).

Timestamp

The transaction timestamp.

Reason string

The machine readable reason.

Message string

The human readable reason describing the state and error details, if any.

20.2. MACHINE CONFIG POOLS

To apply performance profiles to specific nodes, associate them with a machine config pool (MCP). The MCP tracks the status of tuning updates, such as kernel arguments, huge pages, and real-time kernels, ensuring your cluster configurations are applied correctly.

The Performance Profile controller monitors changes in the MCP and updates the performance profile status accordingly.

The only conditions returned by the MCP to the performance profile status is when the MCP is **Degraded**, which leads to **performanceProfile.status.condition.Degraded = true**.

Procedure

1. Check the state of the associated machine config pool by entering the following command. The output example shows a performance profile with an associated machine config pool (**worker-cnf**) that is in a degraded state.

```
# oc get mcp
```

Example output

```
NAME      CONFIG                                UPDATED  UPDATING  DEGRADED
MACHINECOUNT  READYMACHINECOUNT  UPDATEDMACHINECOUNT
DEGRAEDMACHINECOUNT  AGE
master      rendered-master-2ee57a93fa6c9181b546ca46e1571d2d  True  False
False  3      3      3      0      2d21h
worker      rendered-worker-d6b2bdc07d9f5a59a6b68950acf25e5f  True  False
False  2      2      2      0      2d21h
worker-cnf  rendered-worker-cnf-6c838641b8a08fff08dbd8b02fb63f7c  False  True
True  2      1      1      1      2d20h
```

2. To check the reason for the degraded state, enter the following command, ensuring that you change the example machine config pool with your machine config pool. The **describe** section of the MCP shows the reason.

```
# oc describe mcp worker-cnf
```

Example output

```

Message:      Node node-worker-cnf is reporting: "prepping update:
machineconfig.machineconfiguration.openshift.io \"rendered-worker-cnf-
40b9996919c08e335f3ff230ce1d170\" not
found"
Reason:      1 nodes are reporting degraded status on sync

```

- Optional: You can also run the **oc describe** command against the performance profile to check the degraded state status. The example output shows the performance profile **status** field marked as **degraded = true**:

```
# oc describe performanceprofiles performance
```

Example output

```

Message: Machine config pool worker-cnf Degraded Reason: 1 nodes are reporting
degraded status on sync.
Machine config pool worker-cnf Degraded Message: Node yquinn-q8s5v-w-b-
z5lqn.c.openshift-gce-devel.internal is
reporting: "prepping update: machineconfig.machineconfiguration.openshift.io
\"rendered-worker-cnf-40b9996919c08e335f3ff230ce1d170\" not found". Reason:
MCPDegraded
Status: True
Type: Degraded

```

20.3. ABOUT THE MUST-GATHER TOOL

To debug issues in your cluster, use the **oc adm must-gather** CLI command. This tool collects the diagnostic information most likely needed for troubleshooting, ensuring that you have the necessary data for analysis.

The **oc adm must-gather** CLI command collects the following information from your cluster:

- Resource definitions
- Audit logs
- Service logs

You can specify one or more images when you run the command by including the **--image** argument. When you specify an image, the tool collects data related to that feature or product. When you run **oc adm must-gather**, a new pod is created on the cluster. The data is collected on that pod and saved in a new directory that starts with **must-gather.local**. This directory is created in your current working directory.

20.4. COLLECTING LOW LATENCY TUNING DEBUGGING DATA FOR RED HAT SUPPORT

To debug low latency setup issues when opening a support case, collect diagnostic information for Red Hat Support using the **must-gather** tool. This command gathers essential data, such as node tuning and NUMA topology, from your OpenShift Container Platform cluster.

For prompt support, supply diagnostic information for both OpenShift Container Platform and low latency tuning.

Use the **oc adm must-gather** CLI command to collect the following information about your cluster, including features and objects associated with low latency tuning:

- The Node Tuning Operator namespaces and child objects.
- **MachineConfigPool** and associated **MachineConfig** objects.
- The Node Tuning Operator and associated Tuned objects.
- Linux kernel command-line options.
- CPU and NUMA topology
- Basic PCI device information and NUMA locality.

Prerequisites

- Access to the cluster as a user with the **cluster-admin** role.
- The OpenShift Container Platform OpenShift CLI (**oc**) installed.

Procedure

1. Navigate to the directory where you want to store the **must-gather** data.
2. Collect debugging information by running the following command:

```
$ oc adm must-gather
```

Example output

```
[must-gather ] OUT Using must-gather plug-in image: quay.io/openshift-release
When opening a support case, bugzilla, or issue please include the following summary data
along with any other requested information:
ClusterID: 829er0fa-1ad8-4e59-a46e-2644921b7eb6
ClusterVersion: Stable at "<cluster_version>"
ClusterOperators:
All healthy and stable

[must-gather ] OUT namespace/openshift-must-gather-8fh4x created
[must-gather ] OUT clusterrolebinding.rbac.authorization.k8s.io/must-gather-rhlgc created
[must-gather-5564g] POD 2023-07-17T10:17:37.610340849Z Gathering data for
ns/openshift-cluster-version...
[must-gather-5564g] POD 2023-07-17T10:17:38.786591298Z Gathering data for ns/default...
[must-gather-5564g] POD 2023-07-17T10:17:39.117418660Z Gathering data for
ns/openshift...
[must-gather-5564g] POD 2023-07-17T10:17:39.447592859Z Gathering data for ns/kube-
system...
[must-gather-5564g] POD 2023-07-17T10:17:39.803381143Z Gathering data for
ns/openshift-etcd...
```

...

Reprinting Cluster State:

When opening a support case, bugzilla, or issue please include the following summary data along with any other requested information:

ClusterID: 829er0fa-1ad8-4e59-a46e-2644921b7eb6

ClusterVersion: Stable at "<cluster_version>"

ClusterOperators:

All healthy and stable

3. Create a compressed file from the **must-gather** directory that was created in your working directory. For example, on a computer that uses a Linux operating system, run the following command:

```
$ tar cvaf must-gather.tar.gz must-gather-local.5421342344627712289//
```

- **must-gather-local.5421342344627712289//**: Replace this value with the directory name created by the **must-gather** tool.

**NOTE**

Create a compressed file to attach the data to a support case or to use with the Performance Profile Creator wrapper script when you create a performance profile.

4. Attach the compressed file to your support case on the [Red Hat Customer Portal](#).

Additional resources

- [Gathering data about your cluster with the **must-gather** tool](#)
- [Managing nodes with MachineConfig and KubeletConfig CRs](#)
- [Using the Node Tuning Operator](#)
- [Configuring huge pages at boot time](#)
- [How huge pages are consumed by apps](#)

CHAPTER 21. PERFORMING LATENCY TESTS FOR PLATFORM VERIFICATION

You can use the Cloud-native Network Functions (CNF) tests image to run latency tests on a CNF-enabled OpenShift Container Platform cluster, where all the components required for running CNF workloads are installed. Run the latency tests to validate node tuning for your workload.

The **cnf-tests** container image is available at registry.redhat.io/openshift4/cnf-tests-rhel9:v4.22.

21.1. PREREQUISITES FOR RUNNING LATENCY TESTS

Your cluster must meet the following requirements before you can run the latency tests:

- You have applied all the required CNF configurations. This includes the **PerformanceProfile** cluster and other configuration according to the reference design specifications (RDS) or your specific requirements.
- You have logged in to **registry.redhat.io** with your Customer Portal credentials by using the **podman login** command.

Additional resources

- [Scheduling a workload onto a worker with real-time capabilities](#)

21.2. MEASURING LATENCY

To accurately measure system latency, use the **hwlatdetect**, **cyclictest**, and **oslat** tools provided in the **cnf-tests** image. Evaluating these metrics helps you identify and resolve performance delays in your environment.

Each tool has a specific use. Use the tools in sequence to achieve reliable test results.

hwlatdetect

Measures the baseline that the bare-metal hardware can achieve. Before proceeding with the next latency test, ensure that the latency reported by **hwlatdetect** meets the required threshold because you cannot fix hardware latency spikes by operating system tuning.

cyclictest

Verifies the real-time kernel scheduler latency after **hwlatdetect** passes validation. The **cyclictest** tool schedules a repeated timer and measures the difference between the desired and the actual trigger times. The difference can uncover basic issues with the tuning caused by interrupts or process priorities. The tool must run on a real-time kernel.

oslat

Behaves similarly to a CPU-intensive DPDK application and measures all the interruptions and disruptions to the busy loop that simulates CPU heavy data processing.

The tests introduce the following environment variables:

Table 21.1. Latency test environment variables

Environment variables	Description
LATENCY_TEST_DELAY	Specifies the amount of time in seconds after which the test starts running. You can use the variable to allow the CPU manager reconcile loop to update the default CPU pool. The default value is 0.
LATENCY_TEST_CPUS	Specifies the number of CPUs that the pod running the latency tests uses. If you do not set the variable, the default configuration includes all isolated CPUs.
LATENCY_TEST_RUNTIME	Specifies the amount of time in seconds that the latency test must run. The default value is 300 seconds.  NOTE To prevent the Ginkgo 2.0 test suite from timing out before the latency tests complete, set the -ginkgo.timeout flag to a value greater than LATENCY_TEST_RUNTIME + 2 minutes. If you also set a LATENCY_TEST_DELAY value then you must set -ginkgo.timeout to a value greater than LATENCY_TEST_RUNTIME + LATENCY_TEST_DELAY + 2 minutes. The default timeout value for the Ginkgo 2.0 test suite is 1 hour.
HWLATDETECT_MAXIMUM_LATENCY	Specifies the maximum acceptable hardware latency in microseconds for the workload and operating system. If you do not set the value of HWLATDETECT_MAXIMUM_LATENCY or MAXIMUM_LATENCY , the tool compares the default expected threshold (20µs) and the actual maximum latency in the tool itself. Then, the test fails or succeeds accordingly.
CYCLICTEST_MAXIMUM_LATENCY	Specifies the maximum latency in microseconds that all threads expect before waking up during the cyclictest run. If you do not set the value of CYCLICTEST_MAXIMUM_LATENCY or MAXIMUM_LATENCY , the tool skips the comparison of the expected and the actual maximum latency.
OSLAT_MAXIMUM_LATENCY	Specifies the maximum acceptable latency in microseconds for the oslat test results. If you do not set the value of OSLAT_MAXIMUM_LATENCY or MAXIMUM_LATENCY , the tool skips the comparison of the expected and the actual maximum latency.
MAXIMUM_LATENCY	Unified variable that specifies the maximum acceptable latency in microseconds. Applicable for all available latency tools.



NOTE

Variables that are specific to a latency tool take precedence over unified variables. For example, if **OSLAT_MAXIMUM_LATENCY** is set to 30 microseconds and **MAXIMUM_LATENCY** is set to 10 microseconds, the **oslat** test will run with maximum acceptable latency of 30 microseconds.

21.3. RUNNING THE LATENCY TESTS

Run the cluster latency tests to validate node tuning for your Cloud-native Network Functions (CNF) workload.



NOTE

When executing **podman** commands as a non-root or non-privileged user, mounting paths can fail with **permission denied** errors. Depending on your local operating system and SELinux configuration, you might also experience issues running these commands from your home directory. To make the **podman** commands work, run the commands from a folder that is not your home/<username> directory, and append **:Z** to the volumes creation. For example, **-v \$(pwd)/:/kubeconfig:Z**. This allows **podman** to do the proper SELinux relabeling.

The procedure runs the three individual tests **hwlatdetect**, **cyclictest**, and **oslat**. For details on these individual tests, see their individual sections.

Procedure

1. Open a shell prompt in the directory containing the **kubeconfig** file.
You provide the test image with a **kubeconfig** file in current directory and its related **\$KUBECONFIG** environment variable, mounted through a volume. This allows the running container to use the **kubeconfig** file from inside the container.



NOTE

In the following command, your local **kubeconfig** is mounted to **kubeconfig/kubeconfig** in the **cnf-tests** container, which allows access to the cluster.

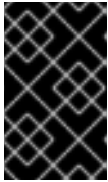
2. To run the latency tests, run the following command, substituting variable values as appropriate:

```
$ podman run -v $(pwd)/:/kubeconfig:Z -e KUBECONFIG=/kubeconfig/kubeconfig \
-e LATENCY_TEST_RUNTIME=600 \
-e MAXIMUM_LATENCY=20 \
registry.redhat.io/openshift4/cnf-tests-rhel9:v4.22 /usr/bin/test-run.sh \
--ginkgo.v --ginkgo.timeout="24h"
```

The **LATENCY_TEST_RUNTIME** is shown in seconds, in this case 600 seconds (10 minutes). The test runs successfully when the maximum observed latency is lower than **MAXIMUM_LATENCY** (20 μ s).

If the results exceed the latency threshold, the test fails.

3. Optional: Append **--ginkgo.dry-run** flag to run the latency tests in dry-run mode. This is useful for checking what commands the tests run.
4. Optional: Append **--ginkgo.v** flag to run the tests with increased verbosity.
5. Optional: Append **--ginkgo.timeout="24h"** flag to ensure the Ginkgo 2.0 test suite does not timeout before the latency tests complete.



IMPORTANT

During testing shorter time periods, as shown, can be used to run the tests. However, for final verification and valid results, the test should run for at least 12 hours (43200 seconds).

21.3.1. Running hwlatdetect

To measure hardware latency, run the **hwlatdetect** tool. This diagnostic utility is available in the **rt-kernel** package through your Red Hat Enterprise Linux (RHEL) 9.x subscription.



NOTE

When executing **podman** commands as a non-root or non-privileged user, mounting paths can fail with **permission denied** errors. Depending on your local operating system and SELinux configuration, you might also experience issues running these commands from your home directory. To make the **podman** commands work, run the commands from a folder that is not your home/<username> directory, and append **:Z** to the volumes creation. For example, **-v \$(pwd)/:/kubecfg:Z**. This allows **podman** to do the proper SELinux relabeling.

Prerequisites

- You have reviewed the prerequisites for running latency tests.

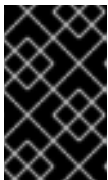
Procedure

- To run the **hwlatdetect** tests, run the following command, substituting variable values as appropriate:

```
$ podman run -v $(pwd)/:/kubecfg:Z -e KUBECFG=/kubecfg/kubecfg \
-e LATENCY_TEST_RUNTIME=600 -e MAXIMUM_LATENCY=20 \
registry.redhat.io/openshift4/cnf-tests-rhel9:v4.22 \
/usr/bin/test-run.sh --ginkgo.focus="hwlatdetect" --ginkgo.v --ginkgo.timeout="24h"
```

The **hwlatdetect** test runs for 10 minutes (600 seconds). The test runs successfully when the maximum observed latency is lower than **MAXIMUM_LATENCY** (20 µs).

If the results exceed the latency threshold, the test fails.



IMPORTANT

During testing shorter time periods, as shown, can be used to run the tests. However, for final verification and valid results, the test should run for at least 12 hours (43200 seconds).

Example failure output

```
running /usr/bin/cnftests -ginkgo.v -ginkgo.focus=hwlatdetect
I0908 15:25:20.023712    27 request.go:601] Waited for 1.046586367s due to client-side
throttling, not priority and fairness, request:
GET:https://api.hlxcl6.lab.eng.tlv2.redhat.com:6443/apis/imageregistry.operator.openshift.io/v1?
timeout=32s
```

Running Suite: CNF Features e2e integration tests

=====

Random Seed: 1662650718

Will run 1 of 3 specs

[...]

• Failure [283.574 seconds]

[performance] Latency Test

/remote-source/app/vendor/github.com/openshift/cluster-node-tuning-operator/test/e2e/performanceprofile/functests/4_latency/latency.go:62
with the hwlatdetect image

/remote-source/app/vendor/github.com/openshift/cluster-node-tuning-operator/test/e2e/performanceprofile/functests/4_latency/latency.go:228
should succeed [It]

/remote-source/app/vendor/github.com/openshift/cluster-node-tuning-operator/test/e2e/performanceprofile/functests/4_latency/latency.go:236

Log file created at: 2022/09/08 15:25:27

Running on machine: hwlatdetect-b6n4n

Binary: Built with gc go1.17.12 for linux/amd64

Log line format: [IWEF]mmdd hh:mm:ss.uuuuuu threadid file:line] msg I0908

15:25:27.160620 1 node.go:39] Environment information: /proc/cmdline: BOOT_IMAGE=(hd1,gpt3)/ostree/rhcos-

c6491e1eedf6c1f12ef7b95e14ee720bf48359750ac900b7863c625769ef5fb9/vmlinuz-4.18.0-372.19.1.el8_6.x86_64 random.trust_cpu=on console=tty0 console=ttyS0,115200n8

ignition.platform.id=metal

ostree=/ostree/boot.1/rhcos/c6491e1eedf6c1f12ef7b95e14ee720bf48359750ac900b7863c625769ef5fb9/0 ip=dhcp root=UUID=5f80c283-f6e6-4a27-9b47-a287157483b2 rw

rootflags=prjquota boot=UUID=773bf59a-bafd-48fc-9a87-f62252d739d3 skew_tick=1

nohz=on rcu_nocbs=0-3 tuned.non_isolcpus=0000ffff,ffffff,ffffff0

systemd.cpu_affinity=4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,57,58,59,

60,61,62,63,64,65,66,67,68,69,70,71,72,73,74,75,76,77,78,79 intel_iommu=on iommu=pt

isolcpus=managed_irq,0-3 nohz_full=0-3 tsc=nowatchdog nosoftlockup nmi_watchdog=0

mce=off skew_tick=1 rcutree.kthread_prio=11 + +

I0908 15:25:27.160830 1 node.go:46] Environment information: kernel version 4.18.0-372.19.1.el8_6.x86_64

I0908 15:25:27.160857 1 main.go:50] running the hwlatdetect command with arguments [/usr/bin/hwlatdetect --threshold 1 --hardlimit 1 --duration 100 --window 100000000us --width 950000us]

F0908 15:27:10.603523 1 main.go:53] failed to run hwlatdetect command; out: hwlatdetect: test duration 100 seconds

detector: tracer

parameters:

Latency threshold: 1us

Sample window: 100000000us

Sample width: 950000us

Non-sampling period: 9050000us

Output File: None

Starting test

test finished

Max Latency: 326us

Samples recorded: 5

Samples exceeding threshold: 5

```
ts: 1662650739.017274507, inner:6, outer:6
ts: 1662650749.257272414, inner:14, outer:326
ts: 1662650779.977272835, inner:314, outer:12
ts: 1662650800.457272384, inner:3, outer:9
ts: 1662650810.697273520, inner:3, outer:2
```

[...]

JUnit report was created: /junit.xml/cnftests-junit.xml

Summarizing 1 Failure:

[Fail] [performance] Latency Test with the hwlatdetect image [It] should succeed
/remote-source/app/vendor/github.com/openshift/cluster-node-tuning-
operator/test/e2e/performanceprofile/functests/4_latency/latency.go:476

```
Ran 1 of 194 Specs in 365.797 seconds
FAIL! -- 0 Passed | 1 Failed | 0 Pending | 2 Skipped
--- FAIL: TestTest (366.08s)
FAIL
```

- **Latency threshold:** You can configure the latency threshold by using the **MAXIMUM_LATENCY** or the **HWLATDETECT_MAXIMUM_LATENCY** environment variables.
- **Max Latency:** The maximum latency value measured during the test.

21.3.2. Example hwlatdetect test results

To track the impact of changes made during testing, capture the raw data from each run along with a combined set of your optimal configuration settings. Retaining these metrics provides a comprehensive history of your test results.

You can capture the following types of results:

- Rough results that are gathered after each run to create a history of impact on any changes made throughout the test.
- The combined set of the rough tests with the best results and configuration settings.

Example of good results

```
hwlatdetect: test duration 3600 seconds
detector: tracer
parameters:
Latency threshold: 10us
Sample window: 1000000us
Sample width: 950000us
Non-sampling period: 50000us
Output File: None
```

```
Starting test
test finished
Max Latency: Below threshold
Samples recorded: 0
```


The **hwlatdetect** tool only provides output if the sample exceeds the specified threshold.

Example of bad results

```
hwlatdetect: test duration 3600 seconds
detector: tracer
parameters:Latency threshold: 10usSample window: 1000000us
Sample width: 950000usNon-sampling period: 50000usOutput File: None
```

```
Starting tests:1610542421.275784439, inner:78, outer:81
ts: 1610542444.330561619, inner:27, outer:28
ts: 1610542445.332549975, inner:39, outer:38
ts: 1610542541.568546097, inner:47, outer:32
ts: 1610542590.681548531, inner:13, outer:17
ts: 1610543033.818801482, inner:29, outer:30
ts: 1610543080.938801990, inner:90, outer:76
ts: 1610543129.065549639, inner:28, outer:39
ts: 1610543474.859552115, inner:28, outer:35
ts: 1610543523.973856571, inner:52, outer:49
ts: 1610543572.089799738, inner:27, outer:30
ts: 1610543573.091550771, inner:34, outer:28
ts: 1610543574.093555202, inner:116, outer:63
```

The output of **hwlatdetect** shows that multiple samples exceed the threshold. However, the same output can indicate different results based on the following factors:

- The duration of the test
- The number of CPU cores
- The host firmware settings



WARNING

Before proceeding with the next latency test, ensure that the latency reported by **hwlatdetect** meets the required threshold. Fixing latencies introduced by hardware might require you to contact the system vendor support.

Not all latency spikes are hardware related. Ensure that you tune the host firmware to meet your workload requirements. For more information, see "Setting firmware parameters for system tuning".

Additional resources

- [Setting firmware parameters for system tuning](#)

21.3.3. Running `cyclictest`

To measure real-time kernel scheduler latency on specified CPUs, run the **cyclictest** tool. Evaluating these metrics helps you identify execution delays and optimize your system for high-performance operations.



NOTE

When executing **podman** commands as a non-root or non-privileged user, mounting paths can fail with **permission denied** errors. Depending on your local operating system and SELinux configuration, you might also experience issues running these commands from your home directory. To make the **podman** commands work, run the commands from a folder that is not your home/<username> directory, and append **:Z** to the volumes creation. For example, **-v \$(pwd)/:/kubeconfig:Z**. This allows **podman** to do the proper SELinux relabeling.

Prerequisites

- You have reviewed the prerequisites for running latency tests.

Procedure

- To perform the **cyclictest**, run the following command, substituting variable values as appropriate:

```
$ podman run -v $(pwd)/:/kubeconfig:Z -e KUBECONFIG=/kubeconfig/kubeconfig \
-e LATENCY_TEST_CPUS=10 -e LATENCY_TEST_RUNTIME=600 -e
MAXIMUM_LATENCY=20 \
registry.redhat.io/openshift4/cnf-tests-rhel9:v4.22 \
/usr/bin/test-run.sh --ginkgo.focus="cyclictest" --ginkgo.v --ginkgo.timeout="24h"
```

The command runs the **cyclictest** tool for 10 minutes (600 seconds). The test runs successfully when the maximum observed latency is lower than **MAXIMUM_LATENCY** (in this example, 20 μ s). Latency spikes of 20 μ s and above are generally not acceptable for telco RAN workloads.

If the results exceed the latency threshold, the test fails.



IMPORTANT

During testing shorter time periods, as shown, can be used to run the tests. However, for final verification and valid results, the test should run for at least 12 hours (43200 seconds).

Example failure output

```
running /usr/bin/cnftests -ginkgo.v -ginkgo.focus=cyclictest
I0908 13:01:59.193776    27 request.go:601] Waited for 1.046228824s due to client-side
throttling, not priority and fairness, request: GET:https://api.compute-
1.example.com:6443/apis/packages.operators.coreos.com/v1?timeout=32s
Running Suite: CNF Features e2e integration tests
=====
Random Seed: 1662642118
Will run 1 of 3 specs

[...]
```

Summarizing 1 Failure:

```
[Fail] [performance] Latency Test with the cyclicttest image [It] should succeed
/remotesource/app/vendor/github.com/openshift/cluster-node-tuning-
operator/test/e2e/performanceprofile/functests/4_latency/latency.go:220
```

```
Ran 1 of 194 Specs in 161.151 seconds
FAIL! -- 0 Passed | 1 Failed | 0 Pending | 2 Skipped
--- FAIL: TestTest (161.48s)
FAIL
```

21.3.4. Example cyclicttest results

To accurately interpret latency test results, evaluate the metrics against your specific workload requirements. Acceptable performance thresholds differ significantly depending on whether you are running 4G DU or 5G DU workloads.

The following example shows a spike up to 18μs that is acceptable for 4G DU workloads, but not for 5G DU workloads:

Example of good results

```
running cmd: cyclicttest -q -D 10m -p 1 -t 16 -a 2,4,6,8,10,12,14,16,54,56,58,60,62,64,66,68 -h 30 -i
1000 -m
# Histogram
000000 000000 000000 000000 000000 000000 000000 000000 000000 000000 000000
000000 000000 000000 000000 000000 000000
000001 000000 000000 000000 000000 000000 000000 000000 000000 000000 000000
000000 000000 000000 000000 000000 000000
000002 579506 535967 418614 573648 532870 529897 489306 558076 582350 585188
583793 223781 532480 569130 472250 576043
More histogram entries ...
# Total: 000600000 000600000 000600000 000599999 000599999 000599999 000599998
000599998 000599998 000599997 000599997 000599996 000599996 000599995 000599995
000599995
# Min Latencies: 00002 00002 00002 00002 00002 00002 00002 00002 00002 00002 00002 00002
00002 00002 00002 00002
# Avg Latencies: 00002 00002 00002 00002 00002 00002 00002 00002 00002 00002 00002 00002
00002 00002 00002 00002
# Max Latencies: 00005 00005 00004 00005 00004 00004 00005 00005 00006 00005 00004 00005
00004 00004 00005 00004
# Histogram Overflows: 00000 00000 00000 00000 00000 00000 00000 00000 00000 00000 00000
00000 00000 00000 00000 00000
# Histogram Overflow at cycle number:
# Thread 0:
# Thread 1:
# Thread 2:
# Thread 3:
# Thread 4:
# Thread 5:
# Thread 6:
# Thread 7:
# Thread 8:
# Thread 9:
# Thread 10:
```

```
# Thread 11:
# Thread 12:
# Thread 13:
# Thread 14:
# Thread 15:
```

Example of bad results

```
running cmd: cyclictst -q -D 10m -p 1 -t 16 -a 2,4,6,8,10,12,14,16,54,56,58,60,62,64,66,68 -h 30 -i
1000 -m
# Histogram
000000 000000 000000 000000 000000 000000 000000 000000 000000 000000 000000
000000 000000 000000 000000 000000 000000
000001 000000 000000 000000 000000 000000 000000 000000 000000 000000 000000
000000 000000 000000 000000 000000 000000
000002 564632 579686 354911 563036 492543 521983 515884 378266 592621 463547
482764 591976 590409 588145 589556 353518
More histogram entries ...
# Total: 000599999 000599999 000599999 000599997 000599997 000599998 000599998
000599997 000599997 000599996 000599995 000599996 000599995 000599995 000599995
000599993
# Min Latencies: 00002 00002 00002 00002 00002 00002 00002 00002 00002 00002 00002 00002
00002 00002 00002 00002
# Avg Latencies: 00002 00002 00002 00002 00002 00002 00002 00002 00002 00002 00002 00002
00002 00002 00002 00002
# Max Latencies: 00493 00387 00271 00619 00541 00513 00009 00389 00252 00215 00539 00498
00363 00204 00068 00520
# Histogram Overflows: 00001 00001 00001 00002 00002 00001 00000 00001 00001 00001 00002
00001 00001 00001 00001 00002
# Histogram Overflow at cycle number:
# Thread 0: 155922
# Thread 1: 110064
# Thread 2: 110064
# Thread 3: 110063 155921
# Thread 4: 110063 155921
# Thread 5: 155920
# Thread 6:
# Thread 7: 110062
# Thread 8: 110062
# Thread 9: 155919
# Thread 10: 110061 155919
# Thread 11: 155918
# Thread 12: 155918
# Thread 13: 110060
# Thread 14: 110060
# Thread 15: 110059 155917
```

21.3.5. Running oslat

To evaluate how your cluster handles CPU-heavy data processing, run the **oslat** test. This diagnostic tool simulates a CPU-intensive DPDK application to measure system interruptions and performance disruptions.



NOTE

When executing **podman** commands as a non-root or non-privileged user, mounting paths can fail with **permission denied** errors. Depending on your local operating system and SELinux configuration, you might also experience issues running these commands from your home directory. To make the **podman** commands work, run the commands from a folder that is not your home/<username> directory, and append **:Z** to the volumes creation. For example, **-v \$(pwd)/:/kubeconfig:Z**. This allows **podman** to do the proper SELinux relabeling.

Prerequisites

- You have reviewed the prerequisites for running latency tests.

Procedure

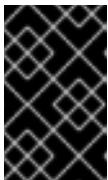
- To perform the **oslat** test, run the following command, substituting variable values as appropriate:

```
$ podman run -v $(pwd)/:/kubeconfig:Z -e KUBECONFIG=/kubeconfig/kubeconfig \
-e LATENCY_TEST_CPUS=10 -e LATENCY_TEST_RUNTIME=600 -e
MAXIMUM_LATENCY=20 \
registry.redhat.io/openshift4/cnf-tests-rhel9:v4.22 \
/usr/bin/test-run.sh --ginkgo.focus="oslat" --ginkgo.v --ginkgo.timeout="24h"
```

LATENCY_TEST_CPUS specifies the number of CPUs to test with the **oslat** command.

The command runs the **oslat** tool for 10 minutes (600 seconds). The test runs successfully when the maximum observed latency is lower than **MAXIMUM_LATENCY** (20 µs).

If the results exceed the latency threshold, the test fails.



IMPORTANT

During testing shorter time periods, as shown, can be used to run the tests. However, for final verification and valid results, the test should run for at least 12 hours (43200 seconds).

In this example failure output, the measured latency is outside the maximum allowed value.

```
running /usr/bin/cnftests -ginkgo.v -ginkgo.focus=oslat
I0908 12:51:55.999393    27 request.go:601] Waited for 1.044848101s due to client-side
throttling, not priority and fairness, request: GET:https://compute-
1.example.com:6443/apis/machineconfiguration.openshift.io/v1?timeout=32s
Running Suite: CNF Features e2e integration tests
=====
Random Seed: 1662641514
Will run 1 of 3 specs

[...]

• Failure [77.833 seconds]
[performance] Latency Test
remote-source/app/vendor/github.com/openshift/cluster-node-tuning-
```

```
operator/test/e2e/performanceprofile/functests/4_latency/latency.go:62
with the oslat image
/remote-source/app/vendor/github.com/openshift/cluster-node-tuning-
operator/test/e2e/performanceprofile/functests/4_latency/latency.go:128
should succeed [It]
/remote-source/app/vendor/github.com/openshift/cluster-node-tuning-
operator/test/e2e/performanceprofile/functests/4_latency/latency.go:153
```

The current latency 304 is bigger than the expected one 1 :

[...]

Summarizing 1 Failure:

```
[Fail] [performance] Latency Test with the oslat image [It] should succeed
/remote-source/app/vendor/github.com/openshift/cluster-node-tuning-
operator/test/e2e/performanceprofile/functests/4_latency/latency.go:177
```

```
Ran 1 of 194 Specs in 161.091 seconds
FAIL! -- 0 Passed | 1 Failed | 0 Pending | 2 Skipped
--- FAIL: TestTest (161.42s)
FAIL
```

21.4. GENERATING A LATENCY TEST FAILURE REPORT

To analyze test failures and troubleshoot performance issues, generate a JUnit latency test output and test failure report. Reviewing this diagnostic data helps you pinpoint exactly where your system is experiencing delays.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.

Procedure

- Create a test failure report with information about the cluster state and resources for troubleshooting by passing the **--report** parameter with the path to where the report is dumped:

```
$ podman run -v $(pwd)/:/kubeconfig:Z -v $(pwd)/reportdest:<report_folder_path> \
-e KUBECONFIG=/kubeconfig/kubeconfig registry.redhat.io/openshift4/cnf-tests-rhel9:v4.22 \
/usr/bin/test-run.sh --report <report_folder_path> --ginkgo.v
```

- **<report_folder_path>**: Specifies the path to the folder where the report is generated.

21.5. GENERATING A JUNIT LATENCY TEST REPORT

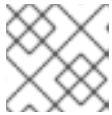
To analyze system performance and track execution delays, generate a JUnit latency test report. Reviewing this diagnostic output helps you identify configuration issues and performance bottlenecks within your cluster.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.

Procedure

- Create a JUnit-compliant XML report by passing the **--junit** parameter together with the path to where the report is dumped:



NOTE

You must create the **junit** folder before running this command.

```
$ podman run -v $(pwd)/:/kubeconfig:Z -v $(pwd)/junit:/junit \
-e KUBECONFIG=/kubeconfig/kubeconfig registry.redhat.io/openshift4/cnf-tests-rhel9:v4.22 \
/usr/bin/test-run.sh --ginkgo.junit-report junit/<file_name>.xml --ginkgo.v
```

where:

file_name

The name of the XML report file.

21.6. RUNNING LATENCY TESTS ON A SINGLE-NODE OPENSIFT CLUSTER

To validate node tuning and identify performance delays, run latency tests on your single-node OpenShift clusters. Evaluating these metrics ensures your environment is optimized for high-performance workloads.



NOTE

When executing **podman** commands as a non-root or non-privileged user, mounting paths can fail with **permission denied** errors. To make the **podman** command work, append **:Z** to the volumes creation; for example, **-v \$(pwd)/:/kubeconfig:Z**. This allows **podman** to do the proper SELinux relabeling.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.
- You have applied a cluster performance profile by using the Node Tuning Operator.

Procedure

- To run the latency tests on a single-node OpenShift cluster, run the following command:

```
$ podman run -v $(pwd)/:/kubeconfig:Z -e KUBECONFIG=/kubeconfig/kubeconfig \
-e LATENCY_TEST_RUNTIME=<time_in_seconds> registry.redhat.io/openshift4/cnf-tests-
rhel9:v4.22 \
/usr/bin/test-run.sh --ginkgo.v --ginkgo.timeout="24h"
```

**NOTE**

The default runtime for each test is 300 seconds. For valid latency test results, run the tests for at least 12 hours by updating the **LATENCY_TEST_RUNTIME** variable.

To run the buckets latency validation step, you must specify a maximum latency. For details on maximum latency variables, see the table in the "Measuring latency" section.

After running the test suite, all the dangling resources are cleaned up.

21.7. RUNNING LATENCY TESTS IN A DISCONNECTED CLUSTER

The CNF tests image can run tests in a disconnected cluster that is not able to reach external registries. This requires two steps:

1. Mirroring the **cnf-tests** image to the custom disconnected registry.
2. Instructing the tests to consume the images from the custom disconnected registry.

21.7.1. Mirroring the images to a custom registry accessible from the cluster

To make required images accessible from your cluster, mirror them to a custom registry. Performing this synchronization ensures that your deployment has the necessary container files, which is particularly useful in restricted or disconnected network environments.

A **mirror** executable is shipped in the image to provide the input required by **oc** to mirror the test image to a local registry.

Procedure

1. Run the following command from an intermediate machine that has access to the cluster and registry.redhat.io:

```
$ podman run -v $(pwd)/:/kubeconfig:Z -e KUBECONFIG=/kubeconfig/kubeconfig \
registry.redhat.io/openshift4/cnf-tests-rhel9:v4.22 \
/usr/bin/mirror -registry <disconnected_registry> | oc image mirror -f -
```

where:

<disconnected_registry>

Specifies the disconnected mirror registry you have configured, such as **my.local.registry:5000/**.

2. When you have mirrored the **cnf-tests** image into the disconnected registry, you must override the original registry used to fetch the images when running the tests by a command similar to the following example:

```
$ podman run -v $(pwd)/:/kubeconfig:Z -e KUBECONFIG=/kubeconfig/kubeconfig \
-e IMAGE_REGISTRY="<disconnected_registry>" \
-e CNF_TESTS_IMAGE="cnf-tests-rhel9:v4.22" \
```



```
-e LATENCY_TEST_RUNTIME=<time_in_seconds> \
<disconnected_registry>/cnf-tests-rhel9:v4.22 /usr/bin/test-run.sh --ginkgo.v --
ginkgo.timeout="24h"
```

Additional resources

- registry.redhat.io

21.7.2. Configuring the tests to consume images from a custom registry

You can run the latency tests by using a custom test image and image registry using **CNF_TESTS_IMAGE** and **IMAGE_REGISTRY** variables.

Procedure

- To configure the latency tests to use a custom test image and image registry, run a command similar to the following example:

```
$ podman run -v $(pwd)/:/kubeconfig:Z -e KUBECONFIG=/kubeconfig/kubeconfig \
-e IMAGE_REGISTRY="<custom_image_registry>" \
-e CNF_TESTS_IMAGE="<custom_cnf-tests_image>" \
-e LATENCY_TEST_RUNTIME=<time_in_seconds> \
registry.redhat.io/openshift4/cnf-tests-rhel9:v4.22 /usr/bin/test-run.sh --ginkgo.v --
ginkgo.timeout="24h"
```

where:

<custom_image_registry>

Specifies the custom image registry, for example, **custom.registry:5000/**.

<custom_cnf-tests_image>

Specifies the custom cnf-tests image, for example, **custom-cnf-tests-image:latest**.

21.7.3. Mirroring images to the cluster OpenShift image registry

To make container images locally available for your deployment, mirror them to the built-in OpenShift image registry. This integrated component runs as a standard workload on your OpenShift Container Platform cluster to ensure continuous access to required files.

Procedure

1. Gain external access to the registry by exposing the registry with a route. You can do this task by running a command similar to the following example:

```
$ oc patch configs.imageregistry.operator.openshift.io/cluster --patch '{"spec":
{"defaultRoute":true}}' --type=merge
```

2. Fetch the registry endpoint by running a command similar to the following example:

```
$ REGISTRY=$(oc get route default-route -n openshift-image-registry --template='{{
.spec.host }}')
```

3. Create a namespace for exposing the images by running a command similar to the following example:

```
$ oc create ns cnftests
```

4. Make the image stream available to all the namespaces used for tests. This is required to allow the tests namespaces to fetch the images from the **cnf-tests** image stream. Run commands similar to the following examples:

```
$ oc policy add-role-to-user system:image-puller system:serviceaccount:cnf-features-testing:default --namespace=cnftests
```

```
$ oc policy add-role-to-user system:image-puller system:serviceaccount:performance-addon-operators-testing:default --namespace=cnftests
```

5. Retrieve the docker secret name by running a command similar to the following example:

```
$ SECRET=$(oc -n cnftests get secret | grep builder-docker | awk {'print $1'})
```

6. Retrieve the docker auth token by running a command similar to the following example:

```
$ TOKEN=$(oc -n cnftests get secret $SECRET -o jsonpath="{.data[\".dockercfg\"]}" | base64 --decode | jq '["image-registry.openshift-image-registry.svc:5000"].auth')
```

7. Create a **dockerauth.json** file, for example:

```
$ echo "{\"auths\": { \"$REGISTRY\": { \"auth\": $TOKEN } }}" > dockerauth.json
```

8. Mirror the image by running a command similar to the following example:

```
$ podman run -v $(pwd)/:/kubeconfig:Z -e KUBECONFIG=/kubeconfig/kubeconfig \
registry.redhat.io/openshift4/cnf-tests-rhel9:v4.22 \
/usr/bin/mirror -registry $REGISTRY/cnftests | oc image mirror --insecure=true \
-a=$(pwd)/dockerauth.json -f -
```

9. Run the tests by running a command similar to the following example:

```
$ podman run -v $(pwd)/:/kubeconfig:Z -e KUBECONFIG=/kubeconfig/kubeconfig \
-e LATENCY_TEST_RUNTIME=<time_in_seconds> \
-e IMAGE_REGISTRY=image-registry.openshift-image-registry.svc:5000/cnftests cnf-tests-
local:latest /usr/bin/test-run.sh --ginkgo.v --ginkgo.timeout="24h"
```

21.7.4. Mirroring a different set of test images

You can optionally change the default upstream images that are mirrored for the latency tests.

Procedure

1. The **mirror** command tries to mirror the upstream images by default. This can be overridden by passing a file with the following format to the image:

```
[
```

```
{
  "registry": "public.registry.io:5000",
  "image": "imageforcnftests:4.22"
}
```

2. Pass the file to the **mirror** command, for example saving it locally as **images.json**. With the following command, the local path is mounted in **/kubeconfig** inside the container and that can be passed to the mirror command.

```
$ podman run -v $(pwd)/:/kubeconfig:Z -e KUBECONFIG=/kubeconfig/kubeconfig \
registry.redhat.io/openshift4/cnf-tests-rhel9:v4.22 /usr/bin/mirror \
--registry "my.local.registry:5000/" --images "/kubeconfig/images.json" \
| oc image mirror -f -
```

21.8. TROUBLESHOOTING ERRORS WITH THE CNF-TESTS CONTAINER

To troubleshoot errors when running latency tests, verify that your cluster is accessible from within the **cnf-tests** container. Ensuring this connectivity resolves common test execution failures.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.

Procedure

- Verify that the cluster is accessible from inside the **cnf-tests** container by running the following command:

```
$ podman run -v $(pwd)/:/kubeconfig:Z -e KUBECONFIG=/kubeconfig/kubeconfig \
registry.redhat.io/openshift4/cnf-tests-rhel9:v4.22 \
oc get nodes
```

If this command does not work, an error related to spanning across DNS, MTU size, or firewall access might be occurring.

CHAPTER 22. IMPROVING CLUSTER STABILITY IN HIGH LATENCY ENVIRONMENTS USING WORKER LATENCY PROFILES

To improve cluster stability in high latency environments, apply worker latency profiles.

If as a cluster administrator, you performed latency tests for platform verification, you might discover the need to adjust the operation of the cluster to ensure stability in cases of high latency.

As a cluster administrator, you need to change only one parameter, recorded in a file, which controls four parameters affecting how supervisory processes read status and interpret the health of the cluster. Changing only the one parameter provides cluster tuning in an easy, supportable manner.

The **Kubelet** process provides the starting point for monitoring cluster health. The **Kubelet** sets status values for all nodes in the OpenShift Container Platform cluster. The Kubernetes Controller Manager (**kube controller**) reads the status values every 10 seconds, by default. If the **kube controller** cannot read a node status value, it loses contact with that node after a configured period. The default behavior is:

1. The node controller on the control plane updates the node health to **Unhealthy** and marks the node **Ready** condition **Unknown**.
2. In response, the scheduler stops scheduling pods to that node.
3. The Node Lifecycle Controller adds a **node.kubernetes.io/unreachable** taint with a **NoExecute** effect to the node and schedules any pods on the node for eviction after five minutes, by default.

This behavior can cause problems if your network is prone to latency issues, especially if you have nodes at the network edge. In some cases, the Kubernetes Controller Manager might not receive an update from a healthy node due to network latency. The **Kubelet** evicts pods from the node even though the node is healthy.

To avoid this problem, you can use *worker latency profiles* to adjust the frequency that the **Kubelet** and the Kubernetes Controller Manager wait for status updates before taking action. These adjustments help to ensure that your cluster runs properly if network latency between the control plane and the worker nodes is not optimal.

These worker latency profiles contain three sets of parameters that are predefined with carefully tuned values to control the reaction of the cluster to increased latency. There is no need to experimentally find the best values manually.

You can configure worker latency profiles when installing a cluster or at any time you notice increased latency in your cluster network.

22.1. UNDERSTANDING WORKER LATENCY PROFILES

Review the following information to learn about worker latency profiles, which allow you to control the reaction of the cluster to latency issues without needing to determine the best values by using manual methods.

Worker latency profiles are four different categories of carefully-tuned parameters. The four parameters which implement these values are **node-status-update-frequency**, **node-monitor-grace-period**, **default-not-ready-toleration-seconds** and **default-unreachable-toleration-seconds**.



IMPORTANT

Setting these parameters manually is not supported. Incorrect parameter settings adversely affect cluster stability.

All worker latency profiles configure the following parameters:

node-status-update-frequency

Specifies how often the kubelet posts node status to the API server.

node-monitor-grace-period

Specifies the amount of time in seconds that the Kubernetes Controller Manager waits for an update from a kubelet before marking the node unhealthy and adding the **node.kubernetes.io/not-ready** or **node.kubernetes.io/unreachable** taint to the node.

default-not-ready-toleration-seconds

Specifies the amount of time in seconds after marking a node unhealthy that the Kube API Server Operator waits before evicting pods from that node.

default-unreachable-toleration-seconds

Specifies the amount of time in seconds after marking a node unreachable that the Kube API Server Operator waits before evicting pods from that node.

The following Operators monitor the changes to the worker latency profiles and respond accordingly:

- The Machine Config Operator (MCO) updates the **node-status-update-frequency** parameter on the compute nodes.
- The Kubernetes Controller Manager updates the **node-monitor-grace-period** parameter on the control plane nodes.
- The Kubernetes API Server Operator updates the **default-not-ready-toleration-seconds** and **default-unreachable-toleration-seconds** parameters on the control plane nodes.

Although the default configuration works in most cases, OpenShift Container Platform offers two other worker latency profiles for situations where the network is experiencing higher latency than usual. The three worker latency profiles are described in the following sections:

Default worker latency profile

With the **Default** profile, each **Kubelet** updates its status every 10 seconds (**node-status-update-frequency**). The **Kube Controller Manager** checks the statuses of **Kubelet** every 5 seconds. The Kubernetes Controller Manager waits 40 seconds (**node-monitor-grace-period**) for a status update from **Kubelet** before considering the **Kubelet** unhealthy. If no status is made available to the Kubernetes Controller Manager, it then marks the node with the **node.kubernetes.io/not-ready** or **node.kubernetes.io/unreachable** taint and evicts the pods on that node.

If a pod is on a node that has the **NoExecute** taint, the pod runs according to **tolerationSeconds**. If the node has no taint, it will be evicted in 300 seconds (**default-not-ready-toleration-seconds** and **default-unreachable-toleration-seconds** settings of the **Kube API Server**).

Profile	Component	Parameter	Value
Default	kubelet	node-status-update-frequency	10s

Profile	Component	Parameter	Value
	Kubelet Controller Manager	node-monitor-grace-period	40s
	Kubernetes API Server Operator	default-not-ready-toleration-seconds	300s
	Kubernetes API Server Operator	default-unreachable-toleration-seconds	300s

Medium worker latency profile

Use the **MediumUpdateAverageReaction** profile if the network latency is slightly higher than usual. The **MediumUpdateAverageReaction** profile reduces the frequency of kubelet updates to 20 seconds and changes the period that the Kubernetes Controller Manager waits for those updates to 2 minutes. The pod eviction period for a pod on that node is reduced to 60 seconds. If the pod has the **tolerationSeconds** parameter, the eviction waits for the period specified by that parameter.

The Kubernetes Controller Manager waits for 2 minutes to consider a node unhealthy. In another minute, the eviction process starts.

Profile	Component	Parameter	Value
MediumUpdateAverageReaction	kubelet	node-status-update-frequency	20s
	Kubelet Controller Manager	node-monitor-grace-period	2m
	Kubernetes API Server Operator	default-not-ready-toleration-seconds	60s
	Kubernetes API Server Operator	default-unreachable-toleration-seconds	60s

Low worker latency profile

Use the **LowUpdateSlowReaction** profile if the network latency is extremely high.

The **LowUpdateSlowReaction** profile reduces the frequency of kubelet updates to 1 minute and changes the period that the Kubernetes Controller Manager waits for those updates to 5 minutes. The pod eviction period for a pod on that node is reduced to 60 seconds. If the pod has the **tolerationSeconds** parameter, the eviction waits for the period specified by that parameter.

The Kubernetes Controller Manager waits for 5 minutes to consider a node unhealthy. In another minute, the eviction process starts.

Profile	Component	Parameter	Value
LowUpdateSlowReaction	kubelet	node-status-update-frequency	1m
	Kubelet Controller Manager	node-monitor-grace-period	5m
	Kubernetes API Server Operator	default-not-ready-toleration-seconds	60s
	Kubernetes API Server Operator	default-unreachable-toleration-seconds	60s

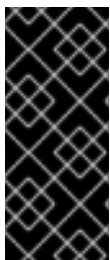


NOTE

The latency profiles do not support custom machine config pools, only the default worker machine config pools.

22.2. IMPLEMENTING WORKER LATENCY PROFILES AT CLUSTER CREATION

During cluster creation, you can implement worker latency profiles so that you can control the reaction of the cluster to latency issues without relying on manual methods to determine the best values.



IMPORTANT

To edit the configuration of the installation program, first use the command **openshift-install create manifests** to create the default node manifest and other manifest YAML files. This file structure must exist before you can add **workerLatencyProfile**. The platform on which you are installing might have varying requirements. Refer to the Installing section of the documentation for your specific platform.

Procedure

1. Create the manifest that is needed to build the cluster by using a folder name appropriate for your installation.
2. Create a YAML file to define **config.node**. The file must be in the **manifests** directory.
3. When defining **workerLatencyProfile** in the manifest for the first time, specify any of the profiles at cluster creation time: **Default**, **MediumUpdateAverageReaction** or **LowUpdateSlowReaction**.

Verification

- View the manifest file by running the following command. The output of the command should show the creation of the **spec.workerLatencyProfile Default** value in the manifest file.

```
$ openshift-install create manifests --dir=<cluster_install_dir>
```

- <cluster_install_dir>**: Specifies the directory where you installed your cluster.
- Edit the manifest and add the value by entering the following command. The following example command uses the **vi** editor to show an example manifest file with the "Default" **workerLatencyProfile** value added.

```
$ vi <cluster_install_dir>/manifests/config-node-default-profile.yaml
```

- <cluster_install_dir>**: Specifies the directory where you installed your cluster.

Example output

```
apiVersion: config.openshift.io/v1
kind: Node
metadata:
  name: cluster
spec:
  workerLatencyProfile: "Default"
# ...
```

22.3. USING AND CHANGING WORKER LATENCY PROFILES

You can change a worker latency profile to deal with network latency at any time by editing the **node.config** object. With this configuration, you can ensure that your cluster runs properly if network latency between the control plane and the compute nodes fluctuates.

You must move one worker latency profile at a time. For example, you cannot move directly from the **Default** profile to the **LowUpdateSlowReaction** worker latency profile. You must move from the **Default** worker latency profile to the **MediumUpdateAverageReaction** profile and then to the **LowUpdateSlowReaction** profile. Similarly, when returning to the **Default** profile, you must move from the low profile to the medium profile first, then to **Default**.



NOTE

You can also configure worker latency profiles upon installing an OpenShift Container Platform cluster.

Procedure

1. Move to the medium worker latency profile:

- a. Edit the **node.config** object:

```
$ oc edit nodes.config/cluster
```

- b. Add **spec.workerLatencyProfile: MediumUpdateAverageReaction:**

Example node.config object

```

apiVersion: config.openshift.io/v1
kind: Node
metadata:
  annotations:
    include.release.openshift.io/ibm-cloud-managed: "true"
    include.release.openshift.io/self-managed-high-availability: "true"
    include.release.openshift.io/single-node-developer: "true"
    release.openshift.io/create-only: "true"
  creationTimestamp: "2022-07-08T16:02:51Z"
  generation: 1
  name: cluster
  ownerReferences:
    - apiVersion: config.openshift.io/v1
      kind: ClusterVersion
      name: version
      uid: 36282574-bf9f-409e-a6cd-3032939293eb
    resourceVersion: "1865"
    uid: 0c0f7a4c-4307-4187-b591-6155695ac85b
  spec:
    workerLatencyProfile: MediumUpdateAverageReaction
# ...

```

where:

spec.workerLatencyProfile.MediumUpdateAverageReaction

Specifies that the medium worker latency policy should be used.

Scheduling on each compute node is disabled as the change is being applied.

2. Optional: Move to the low worker latency profile:

- a. Edit the **node.config** object:

```
$ oc edit nodes.config/cluster
```

- b. Change the **spec.workerLatencyProfile** value to **LowUpdateSlowReaction**:

Example node.config object

```

apiVersion: config.openshift.io/v1
kind: Node
metadata:
  annotations:
    include.release.openshift.io/ibm-cloud-managed: "true"
    include.release.openshift.io/self-managed-high-availability: "true"
    include.release.openshift.io/single-node-developer: "true"
    release.openshift.io/create-only: "true"
  creationTimestamp: "2022-07-08T16:02:51Z"
  generation: 1
  name: cluster
  ownerReferences:
    - apiVersion: config.openshift.io/v1

```

```

kind: ClusterVersion
name: version
uid: 36282574-bf9f-409e-a6cd-3032939293eb
resourceVersion: "1865"
uid: 0c0f7a4c-4307-4187-b591-6155695ac85b
spec:
  workerLatencyProfile: LowUpdateSlowReaction
# ...

```

where:

spec.workerLatencyProfile.LowUpdateSlowReaction

Specifies that the low worker latency policy should be used.

Scheduling on each compute node is disabled as the change is being applied.

Verification

- When all nodes return to the **Ready** condition, you can use the following command to look in the Kubernetes Controller Manager to ensure it was applied:

```
$ oc get KubeControllerManager -o yaml | grep -i workerlatency -A 5 -B 5
```

Example output

```

# ...
- lastTransitionTime: "2022-07-11T19:47:10Z"
  reason: ProfileUpdated
  status: "False"
  type: WorkerLatencyProfileProgressing
- lastTransitionTime: "2022-07-11T19:47:10Z"
  message: all static pod revision(s) have updated latency profile
  reason: ProfileUpdated
  status: "True"
  type: WorkerLatencyProfileComplete
- lastTransitionTime: "2022-07-11T19:20:11Z"
  reason: AsExpected
  status: "False"
  type: WorkerLatencyProfileDegraded
- lastTransitionTime: "2022-07-11T19:20:36Z"
  status: "False"
# ...

```

where:

status.message: all static pod revision(s) have updated latency profile

Specifies that the profile is applied and active.

To change the medium profile to default or change the default to medium, edit the **node.config** object and set the **spec.workerLatencyProfile** parameter to the appropriate value.

22.4. DISPLAYING RESULTING VALUES OF WORKER LATENCY PROFILE

You can run specific commands to display the values for the worker latency profile. You can then check the displayed values for information accuracy.

Procedure

1. Check the **default-not-ready-toleration-seconds** and **default-unreachable-toleration-seconds** fields output by the Kube API Server:

```
$ oc get KubeAPIServer -o yaml | grep -A 1 default-
```

Example output

```
default-not-ready-toleration-seconds:
- "300"
default-unreachable-toleration-seconds:
- "300"
```

2. Check the values of the **node-monitor-grace-period** field from the Kube Controller Manager:

```
$ oc get KubeControllerManager -o yaml | grep -A 1 node-monitor
```

Example output

```
node-monitor-grace-period:
- 40s
```

3. Check the **nodeStatusUpdateFrequency** value from the Kubelet by entering the following command. Set the directory **/host** as the root directory within the debug shell. By changing the root directory to **/host**, you can run binaries contained in the executable paths of the host.

```
$ oc debug node/<compute_node_name>
```

```
$ chroot /host
```

```
# cat /etc/kubernetes/kubelet.conf|grep nodeStatusUpdateFrequency
```

Example output

```
"nodeStatusUpdateFrequency": "10s"
```

These outputs validate the set of timing variables for the Worker Latency Profile.

CHAPTER 23. WORKLOAD PARTITIONING

Workload partitioning separates compute node CPU resources into distinct CPU sets. Ensure that you keep platform pods on the specified cores to avoid interrupting the CPUs the customer workloads are running on.

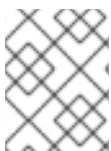
The minimum number of reserved CPUs required for the cluster management is four CPU Hyper-Threads (HTs).

In the context of enabling workload partitioning and managing CPU resources effectively, the cluster might not permit incorrectly configured nodes to join the cluster through a node admission webhook. When the workload partitioning feature is enabled, the machine config pools for control plane nodes and compute nodes get supplied with configurations for nodes to use. Adding new nodes to these pools ensures the pools correctly get configured before joining the cluster.

Currently, nodes must have uniform configurations per machine config pool to ensure that correct CPU affinity is set across all nodes within that pool. After admission, nodes within the cluster identify themselves as supporting a new resource type called **management.workload.openshift.io/cores** and accurately report their CPU capacity. Workload partitioning can be enabled during cluster installation only by adding the additional field **cpuPartitioningMode** to the **install-config.yaml** file.

When workload partitioning is enabled, the **management.workload.openshift.io/cores** resource allows the scheduler to correctly assign pods based on the **cpushares** capacity of the host, not just the default **cpuset**. This ensures more precise allocation of resources for workload partitioning scenarios.

Workload partitioning ensures that CPU requests and limits specified in the pod's configuration are respected. In OpenShift Container Platform 4.16 or later, accurate CPU usage limits are set for platform pods through CPU partitioning. As workload partitioning uses the custom resource type of **management.workload.openshift.io/cores**, the values for requests and limits are the same due to a requirement by Kubernetes for extended resources. However, the annotations modified by workload partitioning correctly reflect the desired limits.



NOTE

Extended resources cannot be overcommitted, so request and limit must be equal if both are present in a container spec.

23.1. ENABLING WORKLOAD PARTITIONING

To partition cluster management pods into a specified CPU affinity, enable workload partitioning. This configuration ensures that management pods operate within the reserved CPU limits defined in your Performance Profile.

Consider additional post-installation Operators that use workload partitioning when calculating how many reserved CPU cores to set aside for the platform.

Workload partitioning isolates user workloads from platform workloads using standard Kubernetes scheduling capabilities.



NOTE

You can enable workload partitioning only during cluster installation. You cannot disable workload partitioning post-installation. However, you can change the CPU configuration for **reserved** and **isolated** CPUs post-installation.

The procedure demonstrates enabling workload partitioning cluster-wide.

Procedure

- In the **install-config.yaml** file, add the additional field **cpuPartitioningMode** and set it to **AllNodes**.

```
apiVersion: v1
baseDomain: devcluster.openshift.com
cpuPartitioningMode: AllNodes
compute:
  - architecture: amd64
    hyperthreading: Enabled
    name: worker
    platform: {}
    replicas: 3
controlPlane:
  architecture: amd64
  hyperthreading: Enabled
  name: master
  platform: {}
  replicas: 3
```

- **cpuPartitioningMode**: Specifies the cluster to set up for CPU partitioning at install time. The default value is **None**, which ensures that no CPU partitioning is enabled at install time.

23.2. PERFORMANCE PROFILES AND WORKLOAD PARTITIONING

To enable workload partitioning, apply a performance profile.

An appropriately configured performance profile specifies the **isolated** and **reserved** CPUs. Create a performance profile by using the Performance Profile Creator (PPC) tool.

Sample performance profile configuration

```
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  # if you change this name make sure the 'include' line in TunedPerformancePatch.yaml
  # matches this name: include=openshift-node-performance-${PerformanceProfile.metadata.name}
  # Also in file 'validatorCRs/informDuValidator.yaml':
  # name: 50-performance-${PerformanceProfile.metadata.name}
  name: openshift-node-performance-profile
annotations:
  ran.openshift.io/reference-configuration: "ran-du.redhat.com"
spec:
  additionalKernelArgs:
    - "rcupdate.rcu_normal_after_boot=0"
    - "efi=runtime"
    - "vfio_pci.enable_sriov=1"
    - "vfio_pci.disable_idle_d3=1"
    - "module_blacklist=irdma"
  cpu:
    isolated: $isolated
    reserved: $reserved
```

```

hugepages:
  defaultHugepagesSize: $defaultHugepagesSize
pages:
  - size: $size
    count: $count
    node: $node
machineConfigPoolSelector:
  pools.operator.machineconfiguration.openshift.io/$mcp: ""
nodeSelector:
  node-role.kubernetes.io/$mcp: "
numa:
  topologyPolicy: "restricted"
# To use the standard (non-realtime) kernel, set enabled to false
realTimeKernel:
  enabled: true
workloadHints:
  # WorkloadHints defines the set of upper level flags for different type of workloads.
  # See https://github.com/openshift/cluster-node-tuning-
operator/blob/master/docs/performanceprofile/performance_profile.md#workloadhints
  # for detailed descriptions of each item.
  # The configuration below is set for a low latency, performance mode.
realTime: true
highPowerConsumption: false
perPodPowerManagement: false

```

Table 23.1. PerformanceProfile CR options for single-node OpenShift clusters

PerformanceProfile CR field	Description
metadata.name	Ensure that name matches the following fields set in related GitOps ZTP custom resources (CRs): ● include=openshift-node-performance-\${PerformanceProfile.metadata.name} in TunedPerformancePatch.yaml ● name: 50-performance-\${PerformanceProfile.metadata.name} in validatorCRs/informDuValidator.yaml
spec.additionalKernelArgs	"efi=runtime" Configures UEFI secure boot for the cluster host.

PerformanceProfile CR field	Description
spec.cpu.isolated	Set the isolated CPUs. Ensure all of the Hyper-Threading pairs match.  IMPORTANT The reserved and isolated CPU pools must not overlap and together must span all available cores. CPU cores that are not accounted for cause an undefined behaviour in the system.
spec.cpu.reserved	Set the reserved CPUs. When workload partitioning is enabled, system processes, kernel threads, and system container threads are restricted to these CPUs. All CPUs that are not isolated should be reserved.
spec.hugepages.pages	● Set the number of huge pages (count) ● Set the huge pages size (size). ● Set node to the NUMA node where the hugepages are allocated (node)
spec.realTimeKernel	Set enabled to true to use the realtime kernel.
spec.workloadHints	Use workloadHints to define the set of top level flags for different type of workloads. The example configuration configures the cluster for low latency and high performance.

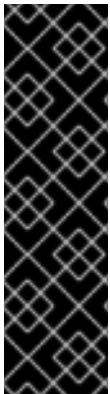
Additional resources

- [About the Performance Profile Creator](#)

CHAPTER 24. USING THE NODE OBSERVABILITY OPERATOR

The Node Observability Operator collects and stores CRI-O and Kubelet profiling or metrics from scripts of compute nodes.

With the Node Observability Operator, you can query the profiling data, enabling analysis of performance trends in CRI-O and Kubelet. It supports debugging performance-related issues and executing embedded scripts for network metrics by using the **run** field in the custom resource definition. To enable CRI-O and Kubelet profiling or scripting, you can configure the **type** field in the custom resource definition.



IMPORTANT

The Node Observability Operator is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

24.1. WORKFLOW OF THE NODE OBSERVABILITY OPERATOR

The following workflow outlines on how to query the profiling data using the Node Observability Operator:

1. Install the Node Observability Operator in the OpenShift Container Platform cluster.
2. Create a NodeObservability custom resource to enable the CRI-O profiling on the worker nodes of your choice.
3. Run the profiling query to generate the profiling data.

24.2. INSTALLING THE NODE OBSERVABILITY OPERATOR

The Node Observability Operator is not installed in OpenShift Container Platform by default. You can install the Node Observability Operator by using the OpenShift Container Platform CLI or the web console.

24.2.1. Installing the Node Observability Operator using the CLI

You can install the Node Observability Operator by using the OpenShift CLI (oc).

Prerequisites

- You have installed the OpenShift CLI (oc).
- You have access to the cluster with **cluster-admin** privileges.

Procedure

1. Confirm that the Node Observability Operator is available by running the following command:


```
$ oc get packagemanifests -n openshift-marketplace node-observability-operator
```

Example output

```
NAME                                CATALOG          AGE
node-observability-operator    Red Hat Operators    9h
```

2. Create the **node-observability-operator** namespace by running the following command:

```
$ oc new-project node-observability-operator
```

3. Create an **OperatorGroup** object YAML file:

```
cat <<EOF | oc apply -f -
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: node-observability-operator
  namespace: node-observability-operator
spec:
  targetNamespaces: []
EOF
```

4. Create a **Subscription** object YAML file to subscribe a namespace to an Operator:

```
cat <<EOF | oc apply -f -
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: node-observability-operator
  namespace: node-observability-operator
spec:
  channel: alpha
  name: node-observability-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
EOF
```

Verification

1. View the install plan name by running the following command:

```
$ oc -n node-observability-operator get sub node-observability-operator -o yaml | yq
'.status.installplan.name'
```

Example output

```
install-dt54w
```

2. Verify the install plan status by running the following command:

```
$ oc -n node-observability-operator get ip <install_plan_name> -o yaml | yq '.status.phase'
```

<install_plan_name> is the install plan name that you obtained from the output of the previous command.

Example output

```
COMPLETE
```

3. Verify that the Node Observability Operator is up and running:

```
$ oc get deploy -n node-observability-operator
```

Example output

```
NAME                                READY  UP-TO-DATE  AVAILABLE  AGE
node-observability-operator-controller-manager  1/1    1           1          40h
```

24.2.2. Installing the Node Observability Operator using the web console

You can install the Node Observability Operator from the OpenShift Container Platform web console.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have access to the OpenShift Container Platform web console.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. In the Administrator's navigation panel, select **Ecosystem → Software Catalog**.
3. In the **All items** field, enter **Node Observability Operator** and select the **Node Observability Operator** tile.
4. Click **Install**.
5. On the **Install Operator** page, configure the following settings:
 - a. In the **Update channel** area, click **alpha**.
 - b. In the **Installation mode** area, click **A specific namespace on the cluster**.
 - c. From the **Installed Namespace** list, select **node-observability-operator** from the list.
 - d. In the **Update approval** area, select **Automatic**.
 - e. Click **Install**.

Verification

1. In the Administrator's navigation panel, expand **Ecosystem → Installed Operators**.
2. Verify that the Node Observability Operator is listed in the Operators list.

24.3. REQUESTING CRI-O AND KUBELET PROFILING DATA USING THE NODE OBSERVABILITY OPERATOR

Creating a Node Observability custom resource to collect CRI-O and Kubelet profiling data.

24.3.1. Creating the Node Observability custom resource

You must create and run the **NodeObservability** custom resource (CR) before you run the profiling query. When you run the **NodeObservability** CR, it creates the necessary machine config and machine config pool CRs to enable the CRI-O profiling on the worker nodes matching the **nodeSelector**.



IMPORTANT

If CRI-O profiling is not enabled on the worker nodes, the **NodeObservabilityMachineConfig** resource gets created. Worker nodes matching the **nodeSelector** specified in **NodeObservability** CR restarts. This might take 10 or more minutes to complete.



NOTE

Kubelet profiling is enabled by default.

The CRI-O unix socket of the node is mounted on the agent pod, which allows the agent to communicate with CRI-O to run the pprof request. Similarly, the **kubelet-serving-ca** certificate chain is mounted on the agent pod, which allows secure communication between the agent and node's kubelet endpoint.

Prerequisites

- You have installed the Node Observability Operator.
- You have installed the OpenShift CLI (oc).
- You have access to the cluster with **cluster-admin** privileges.

Procedure

1. Log in to the OpenShift Container Platform CLI by running the following command:

```
$ oc login -u kubeadmin https://<HOSTNAME>:6443
```

2. Switch back to the **node-observability-operator** namespace by running the following command:

```
$ oc project node-observability-operator
```

3. Create a CR file named **nodeobservability.yaml** that contains the following text:

```
apiVersion: nodeobservability.olm.openshift.io/v1alpha2
kind: NodeObservability
metadata:
  name: cluster
```

```
spec:
  nodeSelector:
    kubernetes.io/hostname: <node_hostname>
  type: crio-kubelet
```

where:

cluster

You must specify the name as **cluster** because there should be only one **NodeObservability** CR per cluster.

<node_hostname>

Specify the nodes on which the Node Observability agent must be deployed.

4. Run the **NodeObservability** CR:

```
oc apply -f nodeobservability.yaml
```

Example output

```
nodeobservability.olm.openshift.io/cluster created
```

5. Review the status of the **NodeObservability** CR by running the following command:

```
$ oc get nob/cluster -o yaml | yq '.status.conditions'
```

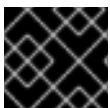
Example output

```
conditions:
  conditions:
    - lastTransitionTime: "2022-07-05T07:33:54Z"
      message: 'DaemonSet node-observability-ds ready: true NodeObservabilityMachineConfig
        ready: true'
      reason: Ready
      status: "True"
      type: Ready
```

NodeObservability CR run is completed when the reason is **Ready** and the status is **True**.

24.3.2. Running the profiling query

To run the profiling query, you must create a **NodeObservabilityRun** resource. The profiling query is a blocking operation that fetches CRI-O and Kubelet profiling data for a duration of 30 seconds. After the profiling query is complete, you must retrieve the profiling data inside the container file system **/run/node-observability** directory. The lifetime of data is bound to the agent pod through the **emptyDir** volume, so you can access the profiling data while the agent pod is in the **running** status.



IMPORTANT

You can request only one profiling query at any point of time.

Prerequisites

- You have installed the Node Observability Operator.
- You have created the **NodeObservability** custom resource (CR).
- You have access to the cluster with **cluster-admin** privileges.

Procedure

1. Create a **NodeObservabilityRun** resource file named **nodeobservabilityrun.yaml** that contains the following text:

```
apiVersion: nodeobservability.olm.openshift.io/v1alpha2
kind: NodeObservabilityRun
metadata:
  name: nodeobservabilityrun
spec:
  nodeObservabilityRef:
    name: cluster
```

2. Trigger the profiling query by running the **NodeObservabilityRun** resource:

```
$ oc apply -f nodeobservabilityrun.yaml
```

3. Review the status of the **NodeObservabilityRun** by running the following command:

```
$ oc get nodeobservabilityrun nodeobservabilityrun -o yaml | yq '.status.conditions'
```

Example output

```
conditions:
- lastTransitionTime: "2022-07-07T14:57:34Z"
  message: Ready to start profiling
  reason: Ready
  status: "True"
  type: Ready
- lastTransitionTime: "2022-07-07T14:58:10Z"
  message: Profiling query done
  reason: Finished
  status: "True"
  type: Finished
```

The profiling query is complete once the status is **True** and type is **Finished**.

4. Retrieve the profiling data from the container's **/run/node-observability** path by running the following bash script:

```
for a in $(oc get nodeobservabilityrun nodeobservabilityrun -o yaml | yq
.status.agents[].name); do
  echo "agent ${a}"
  mkdir -p "/tmp/${a}"
  for p in $(oc exec "${a}" -c node-observability-agent -- bash -c "ls /run/node-
observability/*.pprof"); do
    f="$(basename ${p})"
    echo "copying ${f} to /tmp/${a}/${f}"
```

```
oc exec "${a}" -c node-observability-agent -- cat "${p}" > "/tmp/${a}/${f}"
done
done
```

24.4. NODE OBSERVABILITY OPERATOR SCRIPTING

Scripting allows you to run pre-configured bash scripts, using the current Node Observability Operator and Node Observability Agent.

These scripts monitor key metrics like CPU load, memory pressure, and worker node issues. They also collect sar reports and custom performance metrics.

24.4.1. Creating the Node Observability custom resource for scripting

You must create and run the **NodeObservability** custom resource (CR) before you run the scripting. When you run the **NodeObservability** CR, it enables the agent in scripting mode on the compute nodes matching the **nodeSelector** label.

Prerequisites

- You have installed the Node Observability Operator.
- You have installed the OpenShift CLI (**oc**).
- You have access to the cluster with **cluster-admin** privileges.

Procedure

1. Log in to the OpenShift Container Platform cluster by running the following command:

```
$ oc login -u kubeadmin https://<host_name>:6443
```

2. Switch to the **node-observability-operator** namespace by running the following command:

```
$ oc project node-observability-operator
```

3. Create a file named **nodeobservability.yaml** that contains the following content:

```
apiVersion: nodeobservability.olm.openshift.io/v1alpha2
kind: NodeObservability
metadata:
  name: cluster
spec:
  nodeSelector:
    kubernetes.io/hostname: <node_hostname>
  type: scripting
```

where:

cluster

You must specify the name as **cluster** because there should be only one **NodeObservability** CR per cluster.

<node_hostname>

Specify the nodes on which the Node Observability agent must be deployed.

scripting

To deploy the agent in scripting mode, you must set the type to **scripting**.

4. Create the **NodeObservability** CR by running the following command:

```
$ oc apply -f nodeobservability.yaml
```

Example output

```
nodeobservability.olm.openshift.io/cluster created
```

5. Review the status of the **NodeObservability** CR by running the following command:

```
$ oc get nob/cluster -o yaml | yq '.status.conditions'
```

Example output

```
conditions:
  conditions:
  - lastTransitionTime: "2022-07-05T07:33:54Z"
    message: 'DaemonSet node-observability-ds ready: true NodeObservabilityScripting
      ready: true'
    reason: Ready
    status: "True"
    type: Ready
```

The **NodeObservability** CR run is completed when the **reason** is **Ready** and **status** is **"True"**.

24.4.2. Configuring Node Observability Operator scripting

You can configure the Node Observability Operator to run pre-configured scripts for collecting metrics data from compute nodes.

Prerequisites

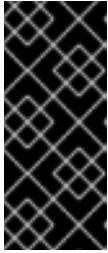
- You have installed the Node Observability Operator.
- You have created the **NodeObservability** custom resource (CR).
- You have access to the cluster with **cluster-admin** privileges.

Procedure

1. Create a file named **nodeobservabilityrun-script.yaml** that contains the following content:

```
apiVersion: nodeobservability.olm.openshift.io/v1alpha2
kind: NodeObservabilityRun
metadata:
  name: nodeobservabilityrun-script
  namespace: node-observability-operator
```

```
spec:
  nodeObservabilityRef:
    name: cluster
    type: scripting
```



IMPORTANT

You can request only the following scripts:

- **metrics.sh**
- **network-metrics.sh** (uses **monitor.sh**)

2. Trigger the scripting by creating the **NodeObservabilityRun** resource with the following command:

```
$ oc apply -f nodeobservabilityrun-script.yaml
```

3. Review the status of the **NodeObservabilityRun** scripting by running the following command:

```
$ oc get nodeobservabilityrun nodeobservabilityrun-script -o yaml | yq '.status.conditions'
```

Example output

```
Status:
Agents:
  Ip: 10.128.2.252
  Name: node-observability-agent-n2fpm
  Port: 8443
  Ip: 10.131.0.186
  Name: node-observability-agent-wcc8p
  Port: 8443
Conditions:
  Conditions:
    Last Transition Time: 2023-12-19T15:10:51Z
    Message: Ready to start profiling
    Reason: Ready
    Status: True
    Type: Ready
    Last Transition Time: 2023-12-19T15:11:01Z
    Message: Profiling query done
    Reason: Finished
    Status: True
    Type: Finished
  Finished Timestamp: 2023-12-19T15:11:01Z
  Start Timestamp: 2023-12-19T15:10:51Z
```

The scripting is complete once **Status** is **True** and **Type** is **Finished**.

4. Retrieve the scripting data from the root path of the container by running the following bash script:

```
#!/bin/bash
```



```

RUN=$(oc get nodeobservabilityrun --no-headers | awk '{print $1}')

for a in $(oc get nodeobservabilityruns.nodeobservability.olm.openshift.io/${RUN} -o json | jq
.status.agents[].name); do
  echo "agent ${a}"
  agent=$(echo ${a} | tr -d "\"\\`")
  base_dir=$(oc exec "${agent}" -c node-observability-agent -- bash -c "ls -t | grep node-
observability-agent" | head -1)
  echo "${base_dir}"
  mkdir -p "/tmp/${agent}"
  for p in $(oc exec "${agent}" -c node-observability-agent -- bash -c "ls ${base_dir}"); do
    f="/${base_dir}/${p}"
    echo "copying ${f} to /tmp/${agent}/${p}"
    oc exec "${agent}" -c node-observability-agent -- cat ${f} > "/tmp/${agent}/${p}"
  done
done

```

24.5. ADDITIONAL RESOURCES

- [Collecting worker metrics using the Node Observability Operator](#)